Topic 3

Ethereum and Smart Contract

Bitcoin System

Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto
satoshin@gmx.com
www.bitcoin.org

Abstract. A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work, forming a record that cannot be changed without redoing the proof-of-work. The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power. As long as a majority of CPU power is controlled by nodes that are not cooperating to attack the network, they'll generate the longest chain and outpace attackers. The network itself requires minimal structure. Messages are broadcast on a best effort basis, and nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone.

Published online in 2008

The first coin implemented in 2009

First *decentralized* digital currency

Bitcoin System

A decentralized payment system must possess the following properties:

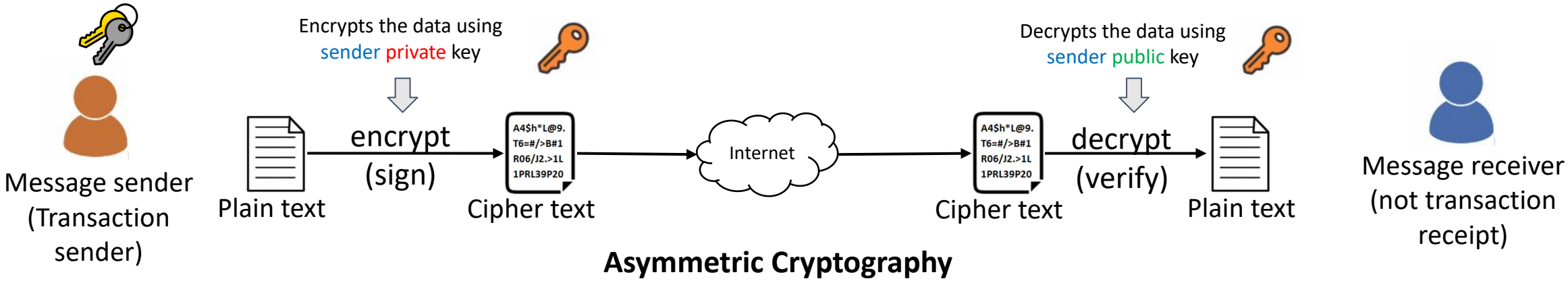
1. Each user is assigned a unique identifier, which they can use to transfer value to the intended recipient.
2. Only the transaction sender has the access to create the related transaction, preventing other users from impersonating and creating identical transactions.
3. Members within this system can easily verify whether a transaction has been created by the designated sender and not by any other user.

One of the solutions is to use **asymmetric cryptography (asymmetric key pair).**

Bitcoin System

Transaction

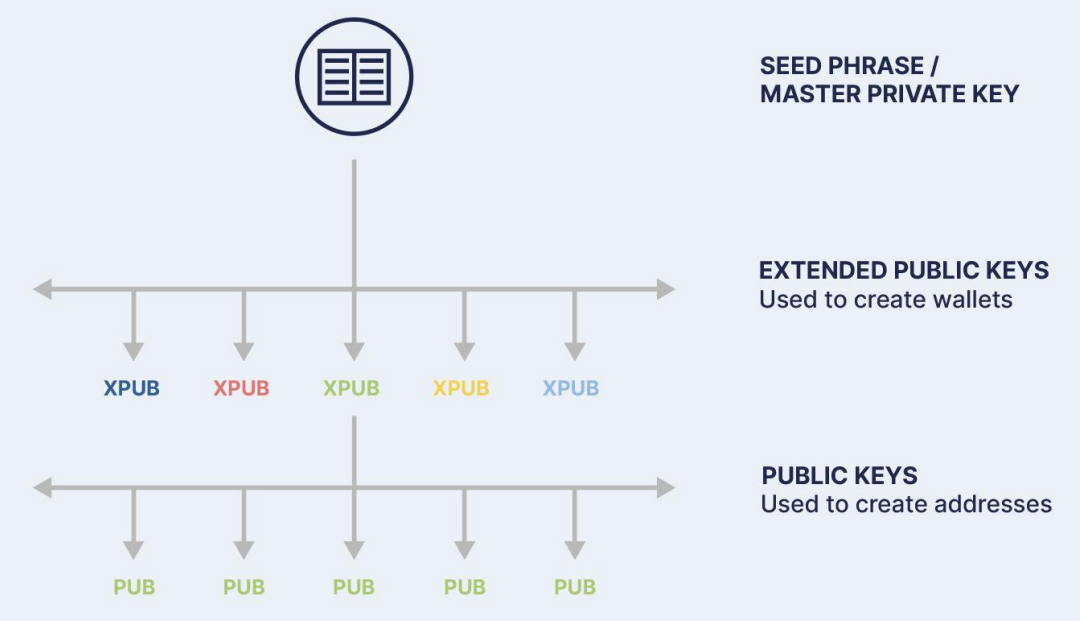
- In order to transfer value, sender signs a message (transaction) using his **private key**.
  This transaction is then **broadcast** to **the rest nodes in Bitcoin network** where nodes verify that sender’s private key is able to access the inputs (by checking that sender’s encrypted message can decrypted by the **public key (address)** he is claiming to own).



Bitcoin System

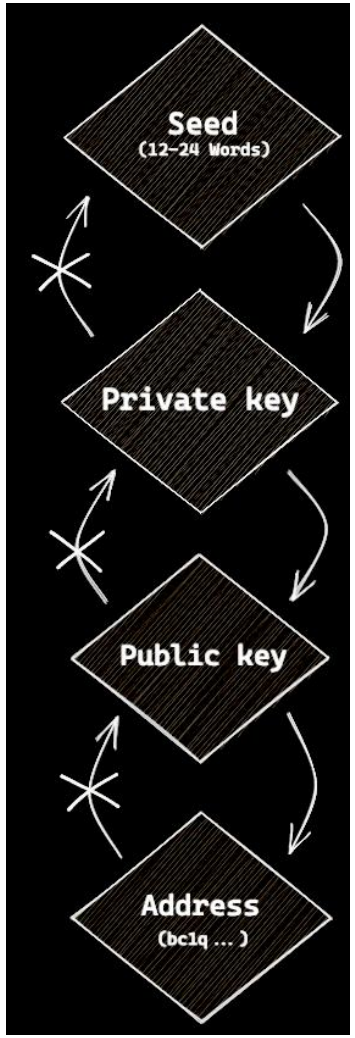
Address

- A bitcoin address is a **unique identifier** that represents a virtual location that cryptocurrencies can be used to send or receive bitcoin.
- Every address is the public key of an **asymmetric key pair**.
- Bitcoin addresses typically consist of a string of 26 to 35 alphanumeric characters.
- One private key can generate multi public keys.
- Examples:
  - 1Lbcfr7sAHTD9CgdQo3HTMTkV8LK4ZnX71
  - 3E13MQrZvPHqSSTsdQaZzZiYPzjEDT5VKE



Reference:
https://unchained.com/blog/what-is-a-bitcoin-address/

How is a Bitcoin address created?



A seed is a randomly generated list 12-24 words.

Example: resource plastic opinion divorce since play neglect domain park apart edge vague world mango slim over wreck icon artist pudding drill orphan camp fish

A private key can be thought of as the key to one of your individual bitcoin addresses.

Example:
L5UxSZahnxwzrNU3kv6gxatYcC3TXffv6sxQkebxUxYFxxdSmva1

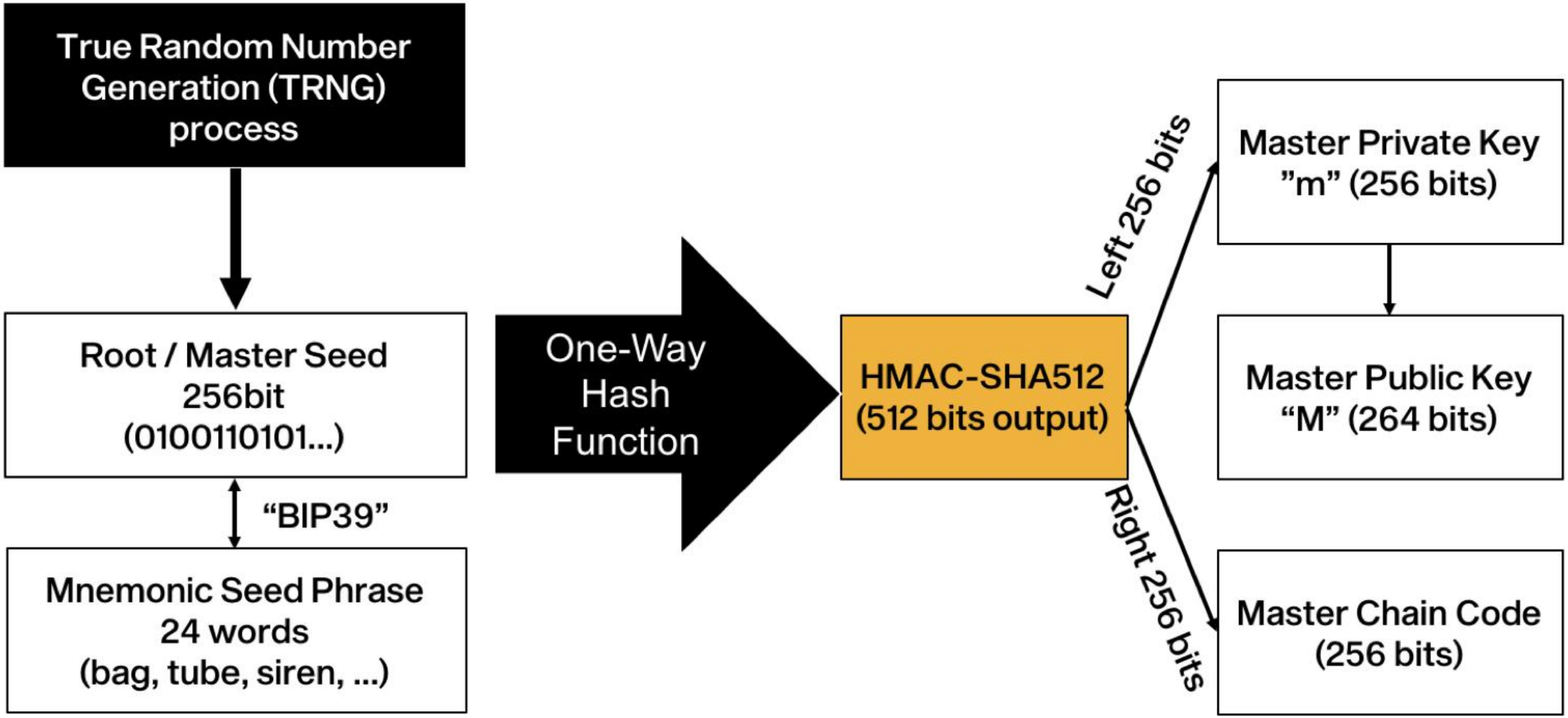
A public key is needed to identify a digital signature.

Example:
020be9666e50552e0f10c15fa0a7691197144313d30a61c9129ec40375748e28ed

An address includes the hash of the public key.

Example: bc1q6zjfr73jywxck3stj0rzl62ga94hyxgw9tfu8g

https://shiftcrypto.ch/blog/how-is-a-bitcoin-address-created/


True Random Number Generation (TRNG) process
Root / Master Seed 256bit (0100110101...)
"BIP39"
Mnemonic Seed Phrase 24 words (bag, tube, siren, ...)
One-Way Hash Function
HMAC-SHA512 (512 bits output)
Left 256 bits
Right 256 bits
Master Private Key "m" (256 bits)
Master Public Key "M" (264 bits)
Master Chain Code (256 bits)

NGRAVE

The Seed Phrase

The ‘master key’ to your wallet

Generally represented by a list of 12-24 words

Translated to a string made up of lots of ones and zeroes

You should never pick random words from the wordlist yourself!

Private Key

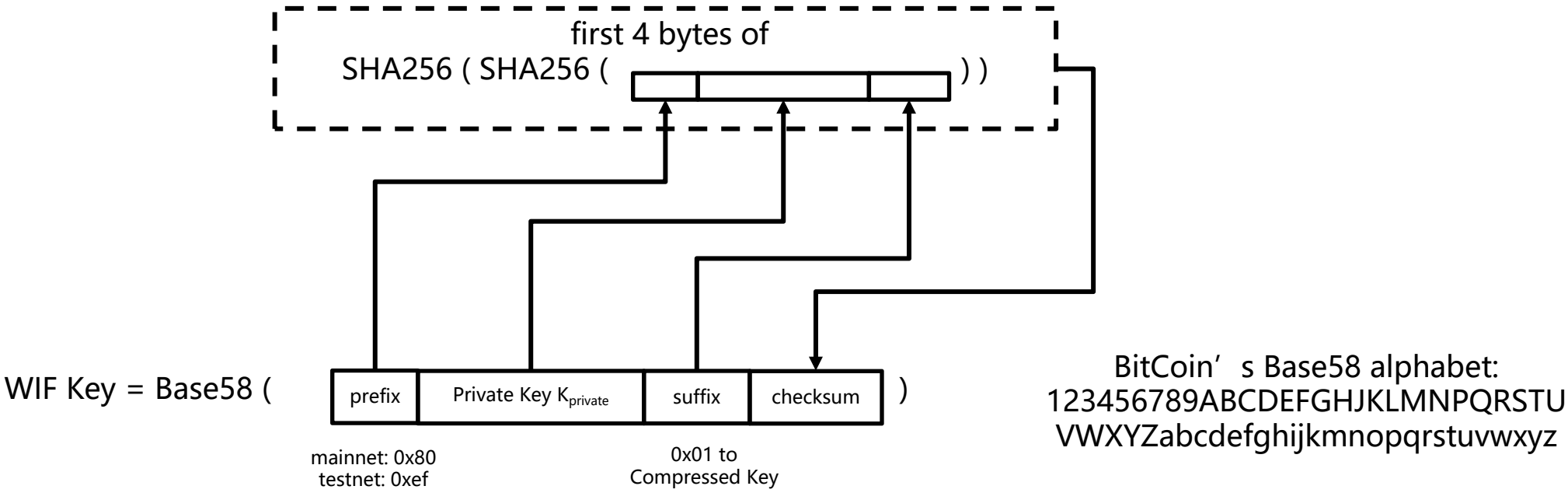
SHA256(Random Seed)

- A Random Number between 1 and $1.158 * 10^{77}$-1 slightly less than 2^{256}
- 2^{256} is an unfathomably large number
  - Approximately 10^{77} in decimal
  - Number of Atoms in Visible Universe is Estimated to 10^{80}

Bitcoin uses the OS' s random number generators to produce 256 bits of entropy (randomness)

Wallet Import Format (WIF) Private Key

A way of encoding a private key so as to make it easier to copy



BitCoin’ s Base58 alphabet:
123456789ABCDEFGHJKLMNPQRSTU
VWXYZabcdefghijkmnopqrstuvwxyz

Bitcoin Base58 Encoding Example

- Base58 encode the word: Cat

Character	ASCII Dec		Decimal Presentation
C	67	$67 * 2^{2*8}$	4390912
a	97	$97 * 2^{1*8}$	24832
t	116	$116 * 2^{0*8}$	116
			4415860

Calculate	Equals	Remainder
4415860/58	76135	30
76135 / 58	1312	39
1312 / 58	22	36
22 / 58	0	22

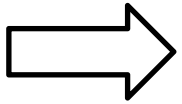
Remainder values:
22, 36, 39, 30

Bitcoin Base58 Encoding Example

- Converts to alphanumeric text
- Similar to Base64 encoding
- Skipping (because they look similar)
  - O (zero)
  - O (capital o)
  - I (capital i)
  - l (lowercase L)
  - + (plus)
  - / (slash)

Value	Character	Value	Character	Value	Character	Value	Character
0	1	1	2	2	3	3	4
4	5	5	6	6	7	7	8
8	9	9	A	10	B	11	C
12	D	13	E	14	F	15	G
16	H	17	J	18	K	19	L
20	M	21	N	22	P	23	Q
24	R	25	S	26	T	27	U
28	V	29	W	30	X	31	Y
32	Z	33	a	34	b	35	c
36	d	37	e	38	f	39	g
40	h	41	i	42	j	43	k
44	m	45	n	46	o	47	p
48	q	49	r	50	s	51	t
52	u	53	v	54	w	55	x
56	y	57	z				

Remainder values:
22, 36, 39, 30



Base58 code:
PdgX

WIF Private Key

WIF Key = Base58 (

)

mainnet: 0x80
testnet: 0xef

0x01 to
Compressed Key

Result:
Compressed WIF starts with the letter K or L (52 Characters)
WIF-encoded (non-compressed) keys starts with “5” (51 Characters)

Compression is on public key

Public Key

for the owner to prove that he owns the associated private key

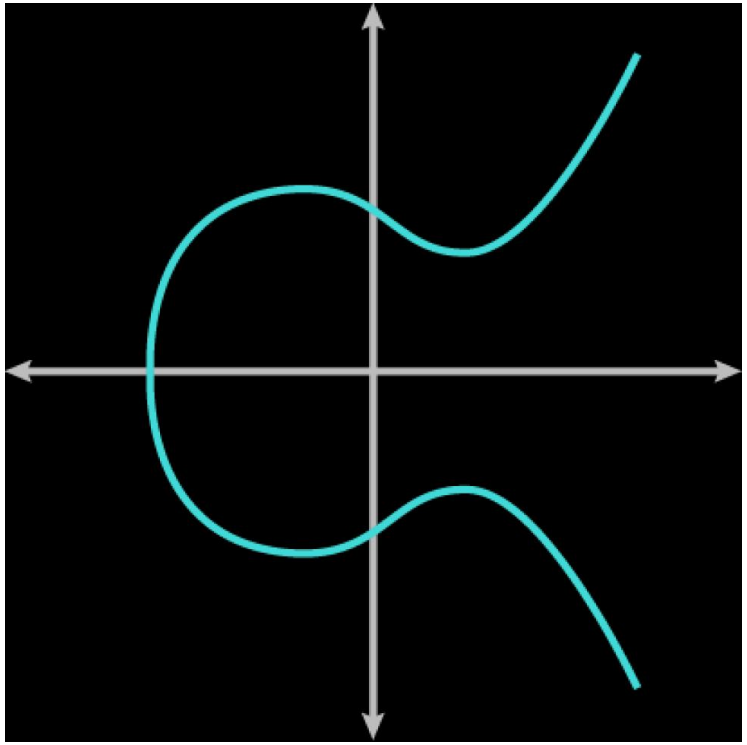
defined as a point $K_{public} = (x,y)$

$K_{public} = K_{private} * G$

- G is a constant point (x,y) called the generator point
- G is always the same for all keys in bitcoin
- G is specified as part of the secp256k1 standard

secp256k1: A Elliptic Curve Equation

An Example of An Elliptic Curve



$$y^2 \bmod p = (x^3 + 7) \bmod p$$

secp256k1: bitcoin

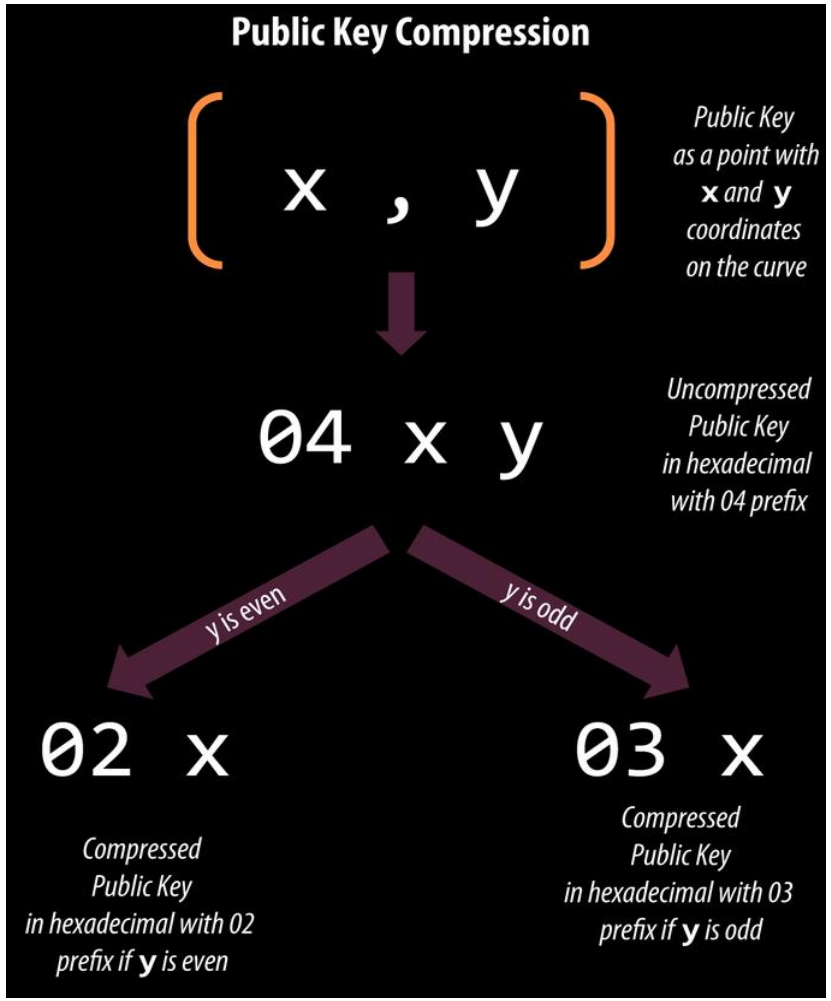
sec: standards for efficient cryptography

$p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$

$y^2 = x^3 + 7$

G = (
79BE667E F9DCBBAC 55A06295 CE870B07 029BFCDB
2DCE28D9 59F2815B 16F81798,
483ADA77 26A3C465 5DA4FBFC 0E1108A8 FD17B448
A6855419 9C47D08F FB10D4B8
)

Public Key Compression



G = (
79BE667E F9DCBBAC 55A06295 CE870B07 029BFCDB
2DCE28D9 59F2815B 16F81798,
483ADA77 26A3C465 5DA4FBFC 0E1108A8 FD17B448
A6855419 9C47D08F FB10D4B8
)

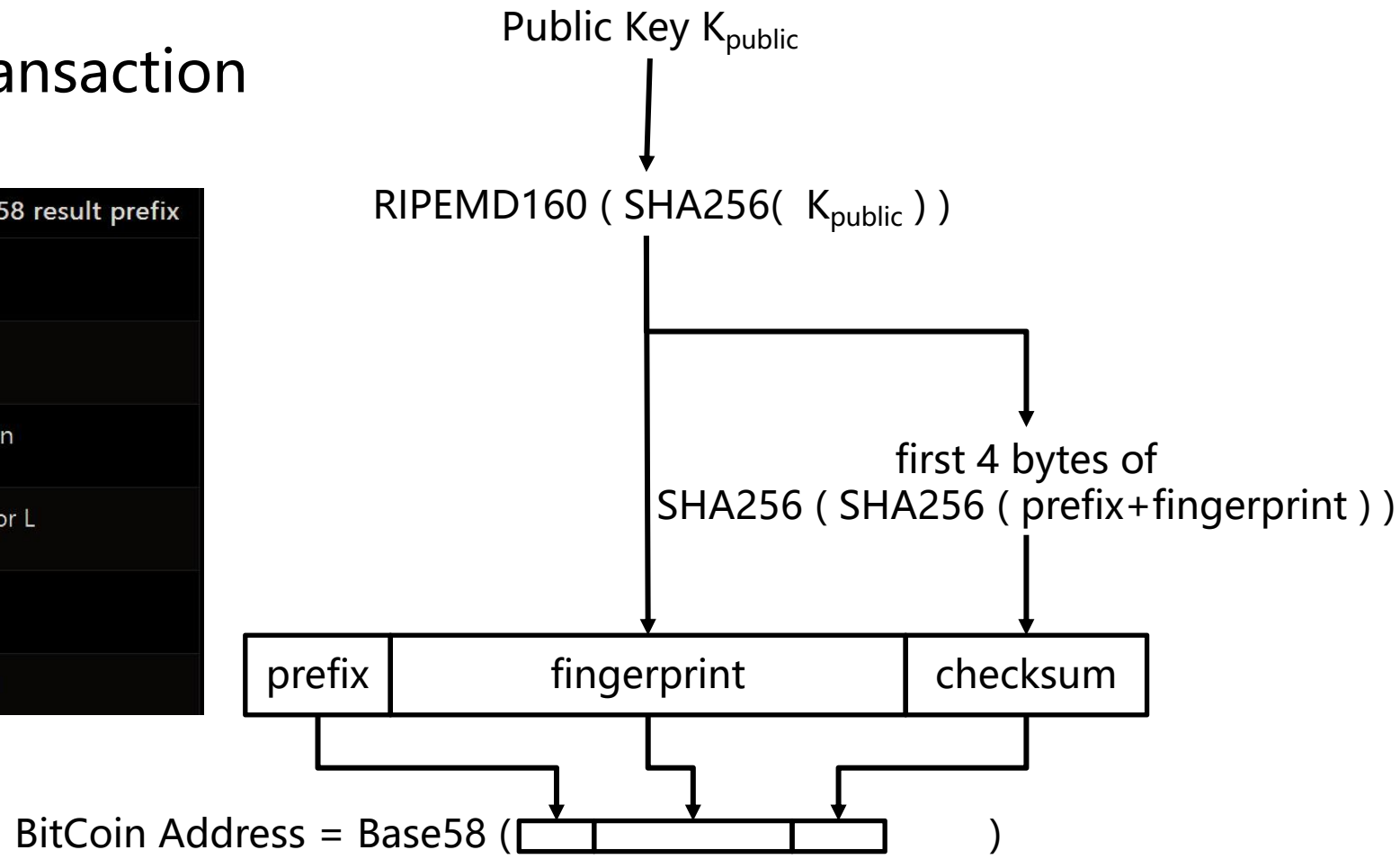
G = 04 79BE667E F9DCBBAC 55A06295 CE870B07 029BFCDB 2DCE28D9 59F2815B 16F81798 483ADA77 26A3C465 5DA4FBFC 0E1108A8 FD17B448 A6855419 9C47D08F FB10D4B8

G = 02 9BE667E F9DCBBAC 55A06295 CE870B07 029BFCDB 2DCE28D9 59F2815B 16F81798

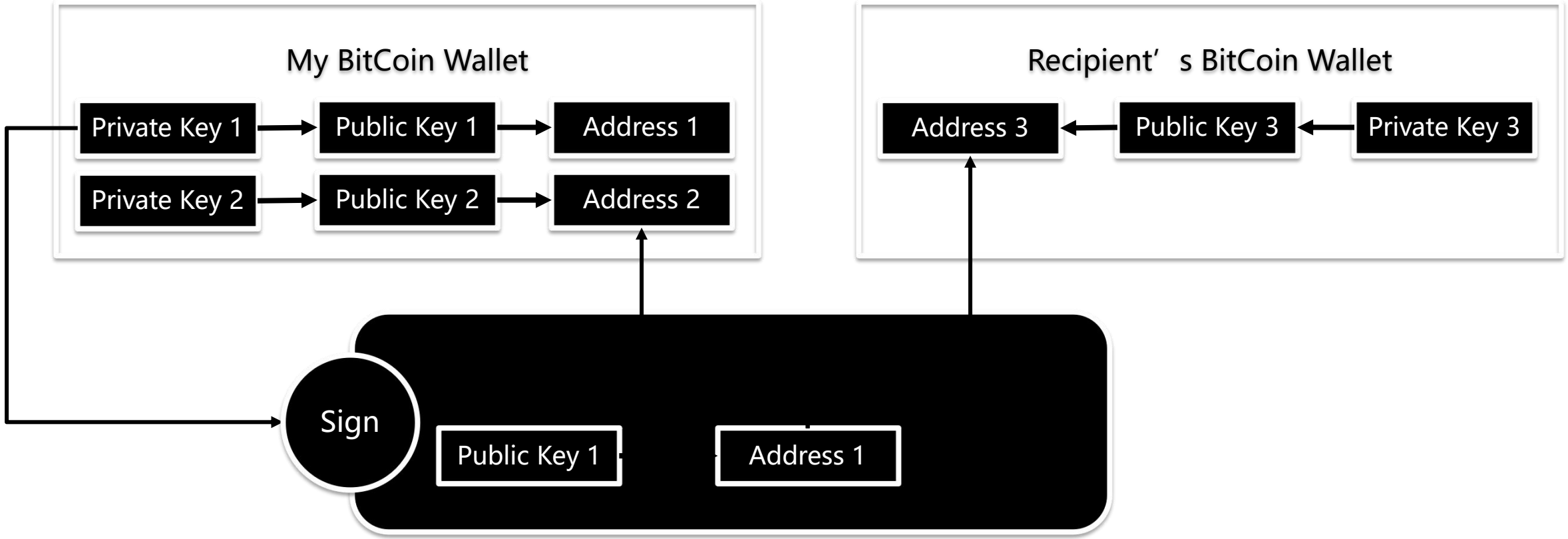
BitCoin Address

The “recipient” in a Transaction

Type	Version prefix (hex)	Base58 result prefix
Bitcoin Address	0x00	1
Pay-to-Script-Hash Address	0x05	3
Bitcoin Testnet Address	0x6F	m or n
Private Key WIF	0x80	5, K, or L
BIP-38 Encrypted Private Key	0x0142	6P
BIP-32 Extended Public Key	0x0488B21E	xpub



Why bother to use Address but not Public Key?



Better Security!

Generate Keys with Bitcoin Core Client

$ bitcoind getnewaddress
1J7mdg5rbQyUHENYdx39WVWK7fsLpEoXZy

$ bitcoind dumpprivkey 1J7mdg5rbQyUHENYdx39WVWK7fsLpEoXZy
KxFC1jmwwCoACiCAWZ3eXa96mBM6tb3TYzGmf6YwgdGWZgawvrtJ

Taproot Address

- An upgraded version of Segwit introduced in November 2021
- P2TR, pay-to-Taproot
- bech32m: encoded using a modified version of bech32
  - begin with the characters *bc1p*
  - example: bc1p8denc9m4sqe9hluasrvxkkdqgkydrk5ctxre5nkk4qwdvefn0sdsc6eqxe
- more efficient transactions with lower fees
- improved privacy as adoption increases

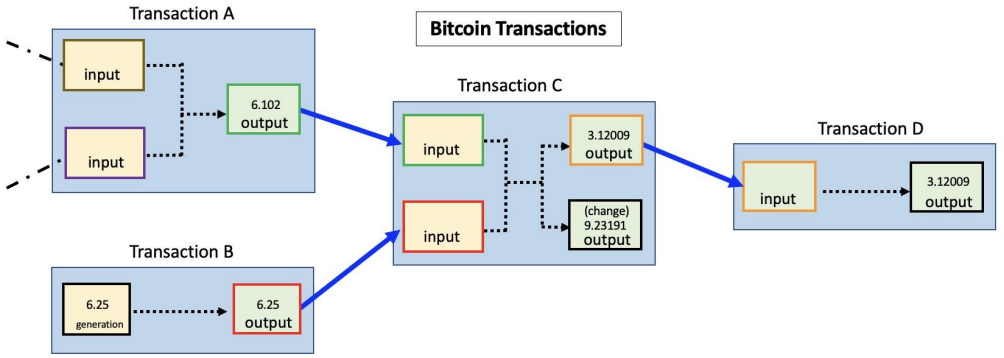
Reference:

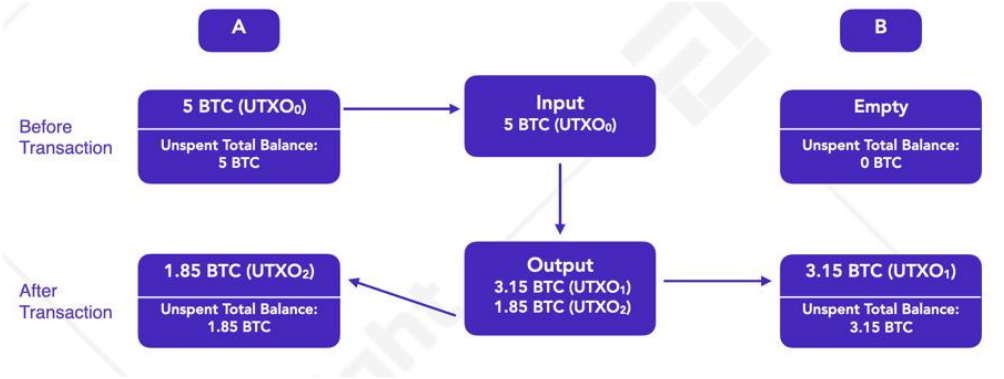
- https://shiftcrypto.ch/blog/taproot-planting-the-seeds-for-bitcoin-applications/

Bitcoin System

Unspent Transaction Output (UTXO) Model

- An Unspent Transaction Output (UTXO) is an unused token.
- Every time a transaction is executed, the input UTXOs are deleted and the output UTXOs are generated. Any output of the transaction as a new UTXO that can be later spent in a new transaction.
- The main differences between the cash and the UTXO model is that bitcoin isn’ t bound by set amounts, i.e. $5, $10, $20, etc. Any amount of bitcoin (up to eight decimal places) can be an UTXO.
- Double Spent is impossible in this model as every UTXO will have the state whether it is spent already in the same global ledger.





Bitcoin System

Transaction Format

- A transaction is a transfer message record of bitcoin value on the bitcoin system, from sender address to receiver address.
- At a high level, a transaction has 2 main parts:
  - **Input:**

    (1) Previous UTXO’ s Transaction ID and index in this transaction (previous output)

    (2) input script (action)
  - **Output:**

    (1) new UTXO’ s amount

    (2) Receipt’ s address

Bitcoin System

Real Transaction Example:

Tx Hash(ID) = 0627052b6f28912f2703066a912ea577f2ce4da4caa5a5fbd8a57286c345c2f2

address: 1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK → address 1GdK9UzpHBzqzX2A9JFP3Di4weBwqgmoQA

transfer amount = 0.015 BTC

Transaction View information about a bitcoin transaction

0627052b6f28912f2703066a912ea577f2ce4da4caa5a5fbd8a57286c345c2f2

1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK (0.1 BTC - Output)

1GdK9UzpHBzqzX2A9JFP3Di4weBwqgmoQA - (Unspent) 0.015 BTC

1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK - (Unspent) 0.0845 BTC

97 Confirmations | 0.0995 BTC

Summary	
Size	258 (bytes)
Received Time	2013-12-27 23:03:05
Included In Blocks	277316 (2013-12-27 23:11:54 +9 minutes)

Inputs and Outputs	
Total Input	0.1 BTC
Total Output	0.0995 BTC
Fees	0.0005 BTC
Estimated BTC Transacted	0.015 BTC


```
{
  "txid": "0627052b6f28912f2703066a912ea577f2ce4da4caa5a5fbd8a57286c345c2f2",
  "size": 258,
  "version": 1,
  "locktime": 0,
  "fee": 50000,
  "inputs": [
    {
      "coinbase": false,
      "txid": "7957a35fe64f80d234d76d83a2a8f1a0d8149a41d81de548f0a65a8a999f6f18",
      "output": 0,
      "sigscript":
"483045022100884d142d86652a3f47ba4746ec719bbfbd040a570b1deccbb6498c75c4ae24cb02204b9f039ff08df09cbe9f6add
ac960298cad530a863ea8f53982c09db8f6e381301410484ecc0d46f1918b30928fa0e4ed99f16a0fb4fde0735e7ade8416ab9fe4
23cc54123376376789d172787ec3457eee41c04f4938de5cc17b4a10fa336a8d752adf",
      "sequence": 4294967295,
      "pkscript": "76a9147f9b1a7fb68d60c536c2fd8aeaa53a8f3cc025a888ac",
      "value": 10000000,
      "address": "1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK",
      "witness": []
    }
  ],
  "outputs": [
    {
      "address": "1GdK9UzpHBzqzX2A9JFP3Di4weBwqgmoQA",
      "pkscript": "76a914ab68025513c3dbd2f7b92a94e0581f5d50f654e788ac",
      "value": 1500000,
      "spent": false,
      "spender": null
    },
    {
      "address": "1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK",
      "pkscript": "76a9147f9b1a7fb68d60c536c2fd8aeaa53a8f3cc025a888ac",
      "value": 8450000,
      "spent": false,
      "spender": null
    }
  ],
```


UTXO index 0

UTXO index 1

Transaction Structure

Size	Field	Description
4 bytes	Version	Specifies which rules this transaction follows
1–9 bytes (VarInt)	Input Counter	How many inputs are included
Variable	Inputs	One or more transaction inputs
1–9 bytes (VarInt)	Output Counter	How many outputs are included
Variable	Outputs	One or more transaction outputs
4 bytes	Locktime a.k.a nLockTime	The earliest time that a transaction is valid and can be relayed on the network or added to the blockchain. Zero in most transactions. Like postdating a paper check.

Find UTXO of A Specific Address

```
import json
import requests

# example address
address = '1Dorian4RoXcnBv9hnQ4Y2C1an6NJ4UrjX'

resp = requests.get('https://blockchain.info/unspent?active=%s' % address)
utxo_set = json.loads(resp.text)["unspent_outputs"]

for utxo in utxo_set:
    print "%s:%d - %ld Satoshis" % (utxo['tx_hash'], utxo['tx_output_n'], utxo['value'])
```

Select from Available UTXO to Pay

Greedy Algorithm

1. Find UTXOs greater than target value
2. Use smallest one of greater UTXOs
3. Otherwise, find UTXOs less than target value
4. Accumulate less UTXOs to pay

```
def select_outputs_greedy(unspent, value):

    if not unspent:    # Fail if empty.
        return None

    lessers = [utxo for utxo in unspent if utxo.value < value]
    greaters = [utxo for utxo in unspent if utxo.value >= value]    # Partition into 2 lists.
    key_func = lambda utxo: utxo.value

    if greaters:                                    # Not-empty. Find the smallest greater.
        min_greater = min(greaters)
        change = min_greater.value - value
        return [min_greater], change


    # Not found in greaters. Try several lessers instead.
    lessers.sort(key=key_func, reverse=True)       # Sort from biggest to smallest.
    result = []
    accum = 0
    for utxo in lessers:          # amount of inputs as possible.
        result.append(utxo)
        accum += utxo.value
        if accum >= value:
            change = accum - value
            return result, "Change: %d Satoshis" % change

    return None, 0        # No results found.
```

Bitcoin System

Solutions for Consistency in the Bitcoin System

Q. How to confirm the global ledger having same transactions across all nodes of the bitcoin network (keep consistency) and ensure its scalability for millions of participants?

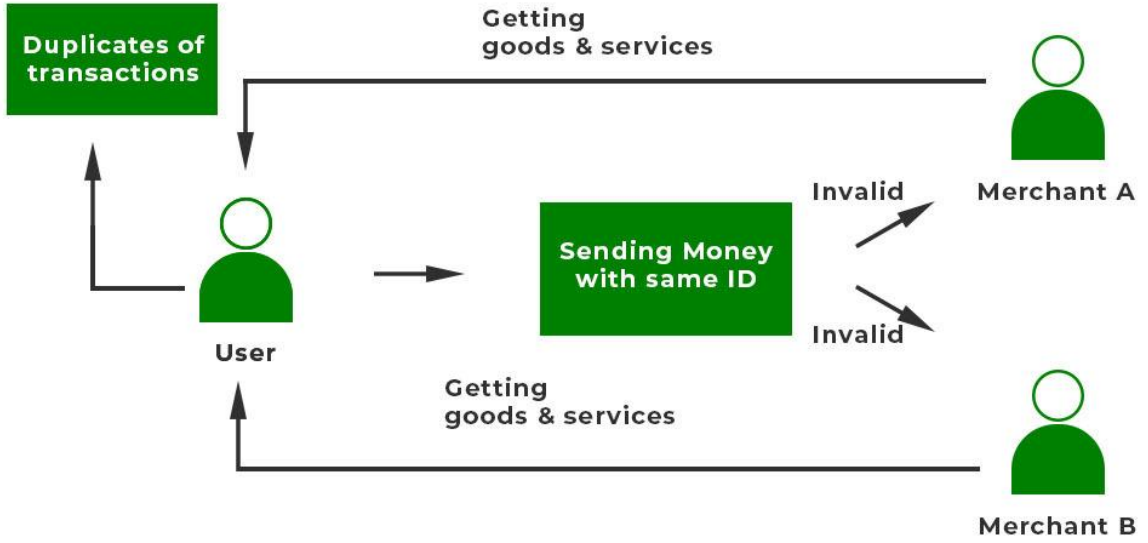
Example: Double-spending problem

- Sender writes and broadcasts message 1 that it transfer a UTXO to receiver A.
- Meanwhile, sender also writes and broadcasts message 2 that it transfer the same UTXO to receiver B.

How to handle or avoid this problem?

Bitcoin solutions:

1. Miner
2. Proof-of-Work (PoW) Consensus
3. Block & Blockchain
4. Miner Reward Mechanism

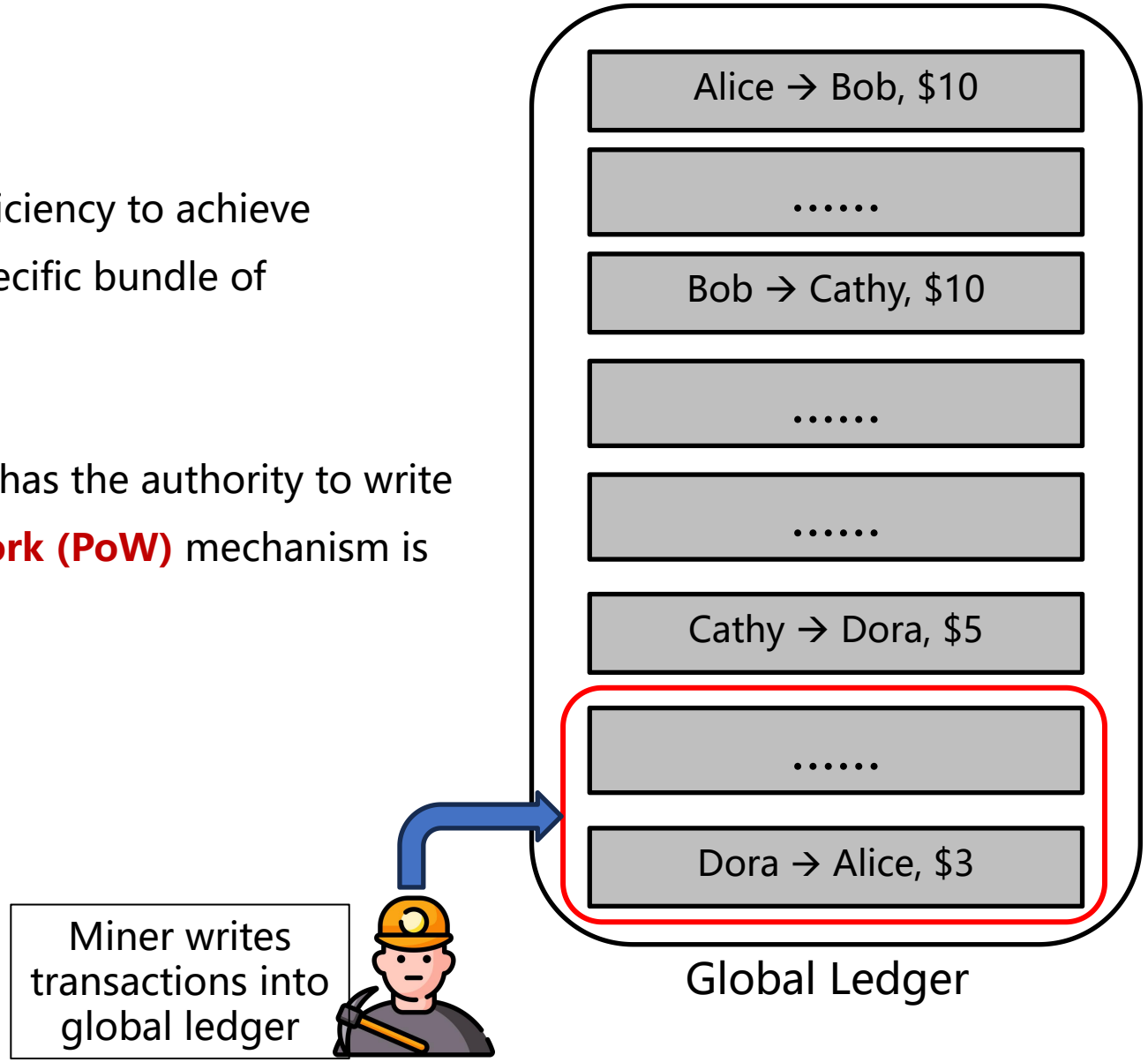


Bitcoin System

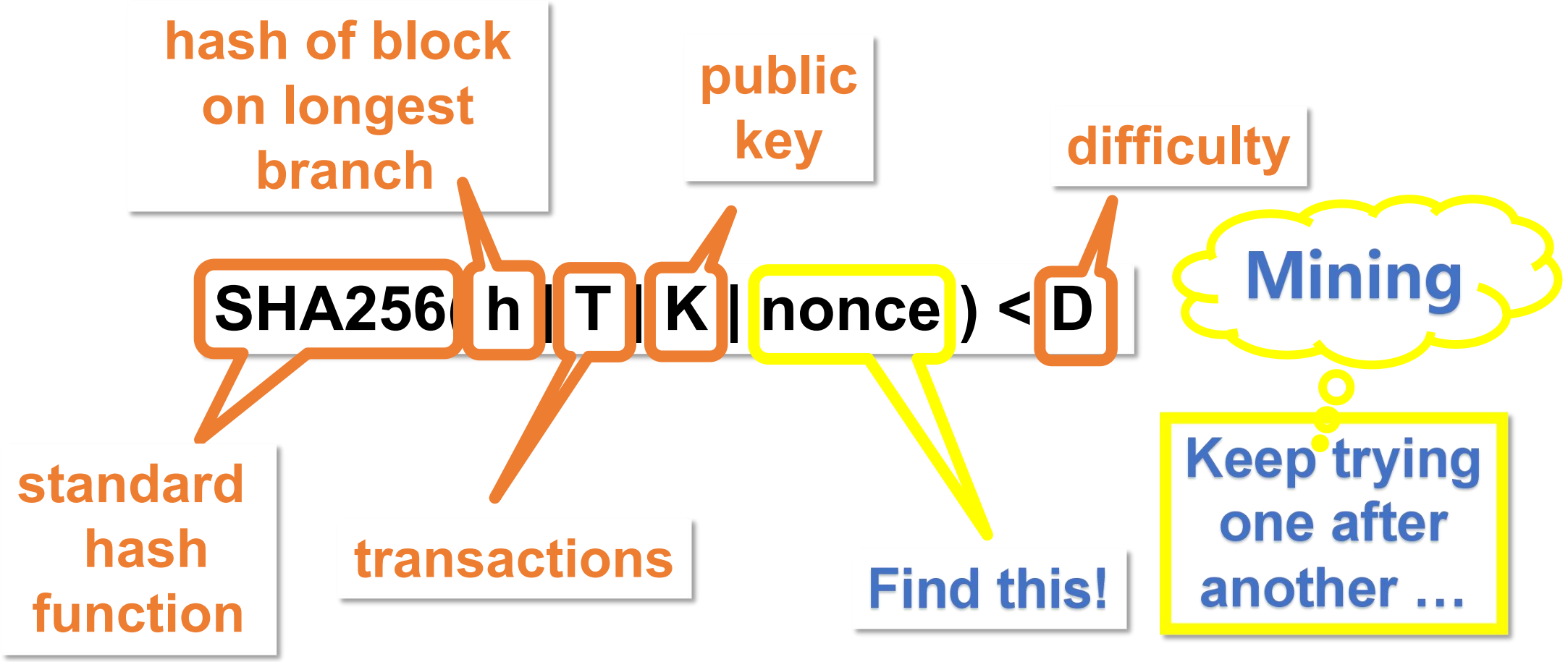
Miner

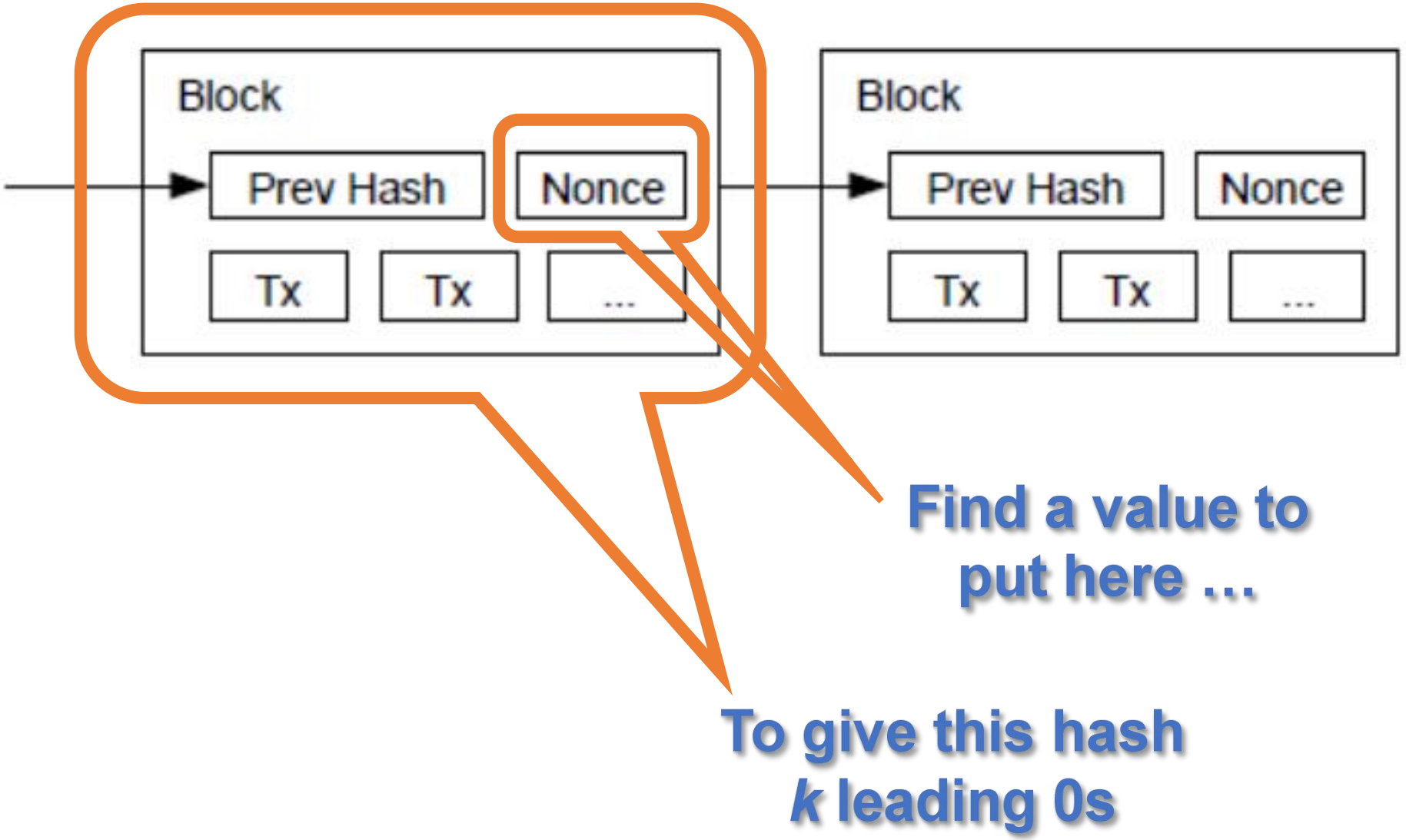
- To avoid conflicts in data recording and improve efficiency to achieve consensus, **ONLY one node** is allowed to write a specific bundle of transactions. (dictatorship mechanism)

- A mechanism is required to determine which miner has the authority to write transactions. In the Bitcoin system, the **Proof-of-Work (PoW)** mechanism is utilized to confirm the specific miner.



Nakamoto PoW Consensus in One Line

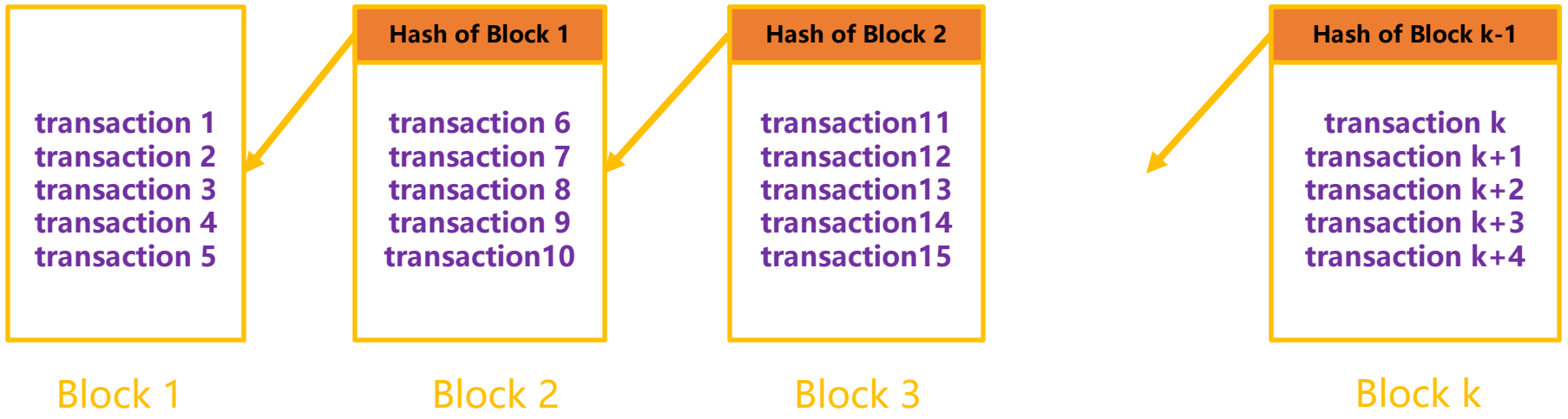



Block
Prev Hash
Nonce
Tx
Tx
...
Block
Prev Hash
Nonce
Tx
Tx
...
Find a value to put here …
To give this hash *k* leading 0s

Bitcoin System

Block & Blockchain

- In BitCoin ledger, we write *transactions* to *blocks*



Bitcoin System

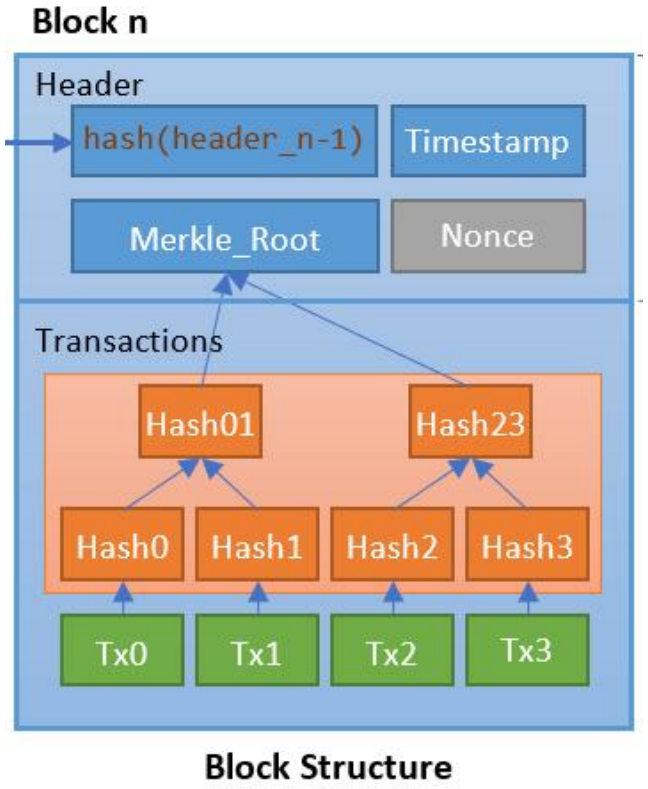
Block & Blockchain

A block's data structure is mainly divided into 2 parts:

- Transactions (It includes a bundle of transactions that need to be written into the global ledger.)
- Header (It includes extra information such as the previous block's hash, timestamp, nonce, and Merkle root for block validation and identification.)

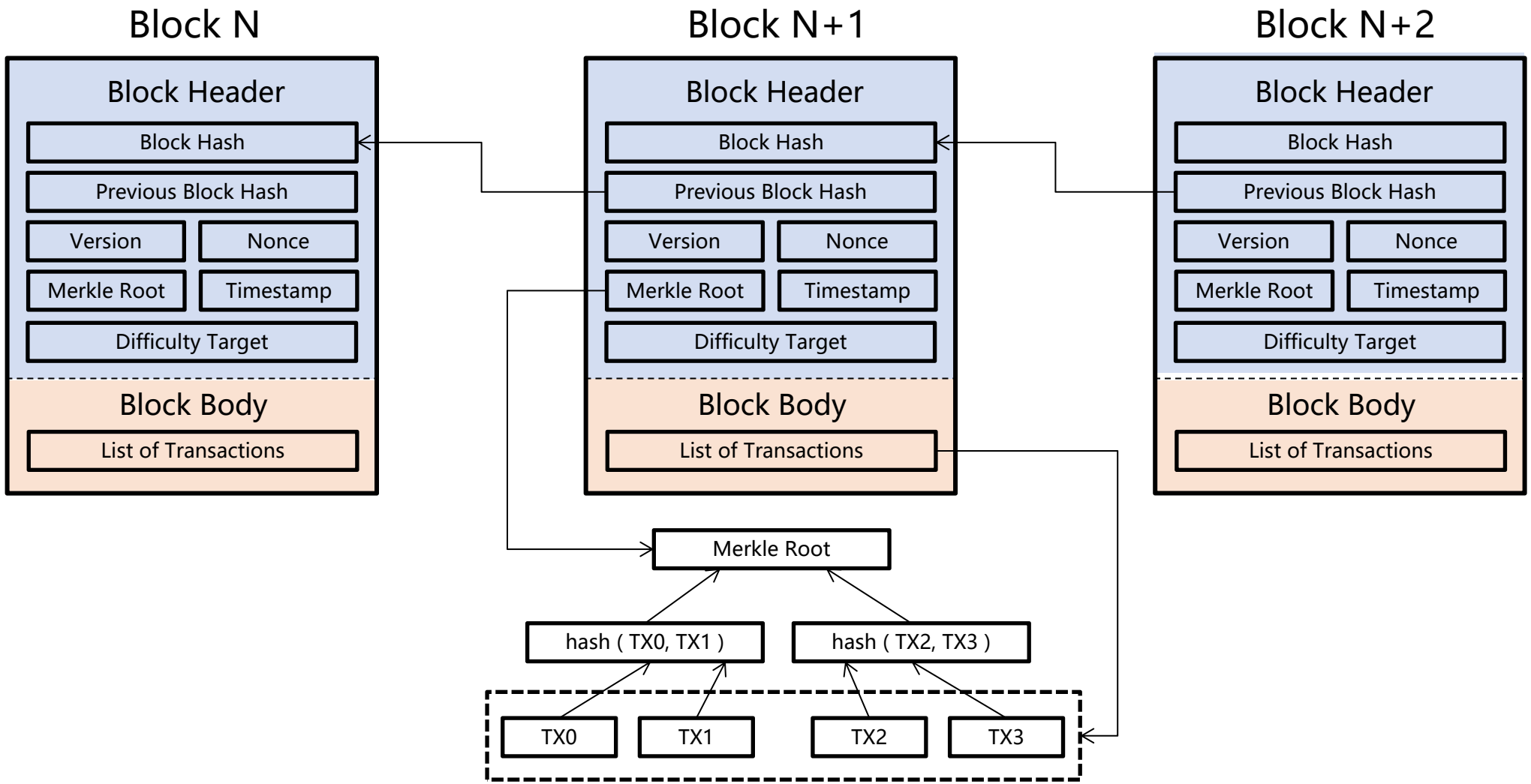
- The hash of a block is obtained by applying a **cryptographic hash function** to the header parameters of the block.

  $block_hash_n$ = Hash($block_hash_{n-1}$, Merkle root, Timestamp, Nonce)



Block Structure

Blockchain Structure



The Genesis Block

The First Block in the Blockchain

- https://blockchain.info/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f

Contains a hidden message within it:

"The Times 03/Jan/2009 Chancellor on brink of second bailout for banks."

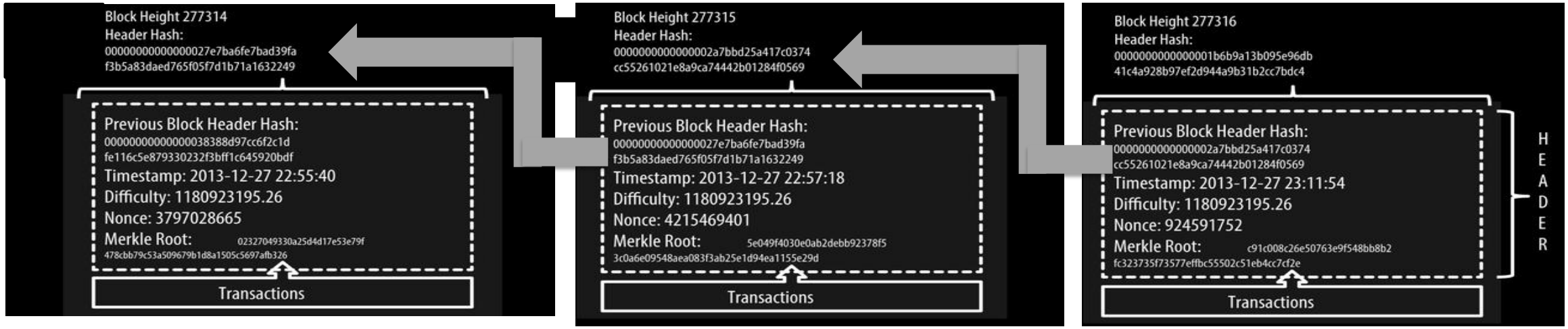
- offer proof of the earliest date this block was created by referencing the headline of The Times
- a reminder of the importance of an independent monetary system

The Genesis Block


```
{
    "hash" : "000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f",
    "confirmations" : 308321,
    "size" : 285,
    "height" : 0,
    "version" : 1,
    "merkleroot" : "4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b",
    "tx" : [ "4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b" ],
    "time" : 1231006505,
    "nonce" : 2083236893,
    "bits" : "1d00ffff",
    "difficulty" : 1.00000000,
    "nextblockhash" : "00000000839a8e6886ab5951d76f411475428afc90947ee320161bbf18eb6048"
}
```

Linking Blocks in the Blockchain

- Bitcoin full nodes maintain an up-to-date local copy of the blockchain
- A node link incoming valid block to the existing blockchain



Block Information


```
{
    "size" : 43560,
    "version" : 2,
    "previousblockhash" :
"000000000000000027e7ba6fe7bad39faf3b5a83daed765f05f7d1b71a1632249",
    "merkleroot" : "5e049f4030e0ab2debb92378f53c0a6e09548aea083f3ab25e1d94ea1155e29d",
    "time" : 1388185038,
    "difficulty" : 1180923195.25802612,
    "nonce" : 4215469401,
    "tx" : [ "257e7497fb8bc68421eb2c7b699dbab234831600e7352f0d9e6522c7cf3f6c77",
     #[... many more transactions omitted ...]
        "05cfd38f6ae6aa83674cc99e4d75a1458c165b7ab84725eda41d018a09176634"
    ]
}
```

The Origins of Blockchain (cont.)

Improving the Efficiency and Reliability of Digital Time-Stamping

Dave Bayer*
Barnard College
Columbia University
New York, N.Y. 10027 U.S.A.
dab@math.columbia.edu

Stuart Haber
Bellcore
445 South Street
Morristown, N.J. 07960 U.S.A.
stuart@bellcore.com

W. Scott Stornetta
Bellcore
445 South Street
Morristown, N.J. 07960 U.S.A.
stornetta@bellcore.com

March 1992

Abstract

To establish that a document was created after a given moment in time, it is necessary to report events that could not have been predicted before they happened. To establish that a document was created before a given moment in time, it is necessary to cause an event based on the document, which can be observed by others. Cryptographic hash functions can be used both to report events succinctly, and to cause events based on documents without revealing their contents. Haber and Stornetta have proposed two schemes for digital time-stamping which rely on these principles [HaSt 91].

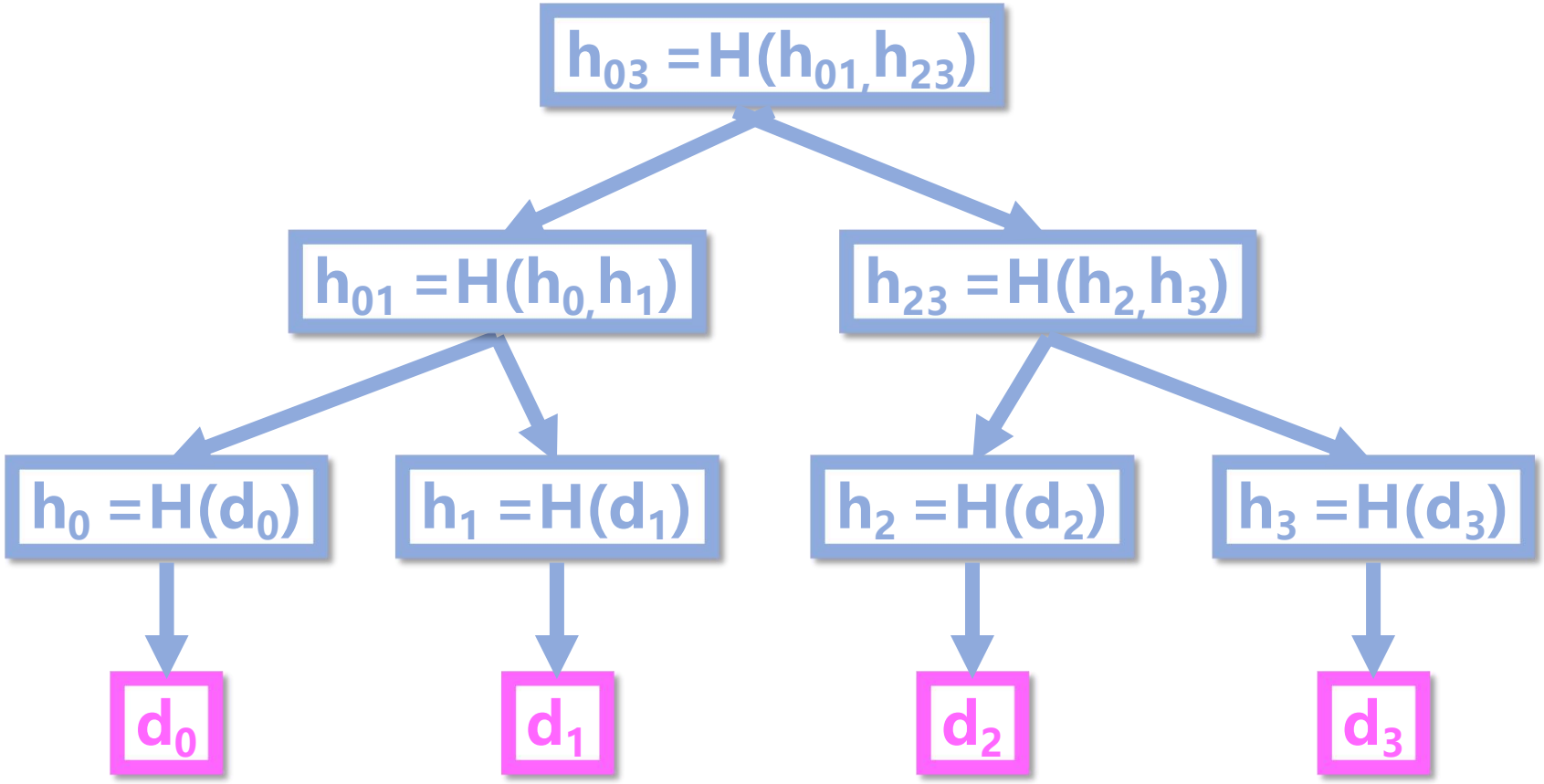
We reexamine one of those protocols, addressing the resource constraint required for storage and verification of time-stamp certificates. By using trees, we show how to achieve an exponential increase in the publicity obtained for each time-stamping event, while reducing the storage and the computation required in order to validate a given certificate.

We show how time-stamping can be used in certain circumstances to extend the useful lifetime of different kinds of cryptographic certifications of authenticity, in the event that the certifying protocol is compromised. This can be applied to digital signatures, or to time-stamping itself, making the digital time-stamping process renewable.

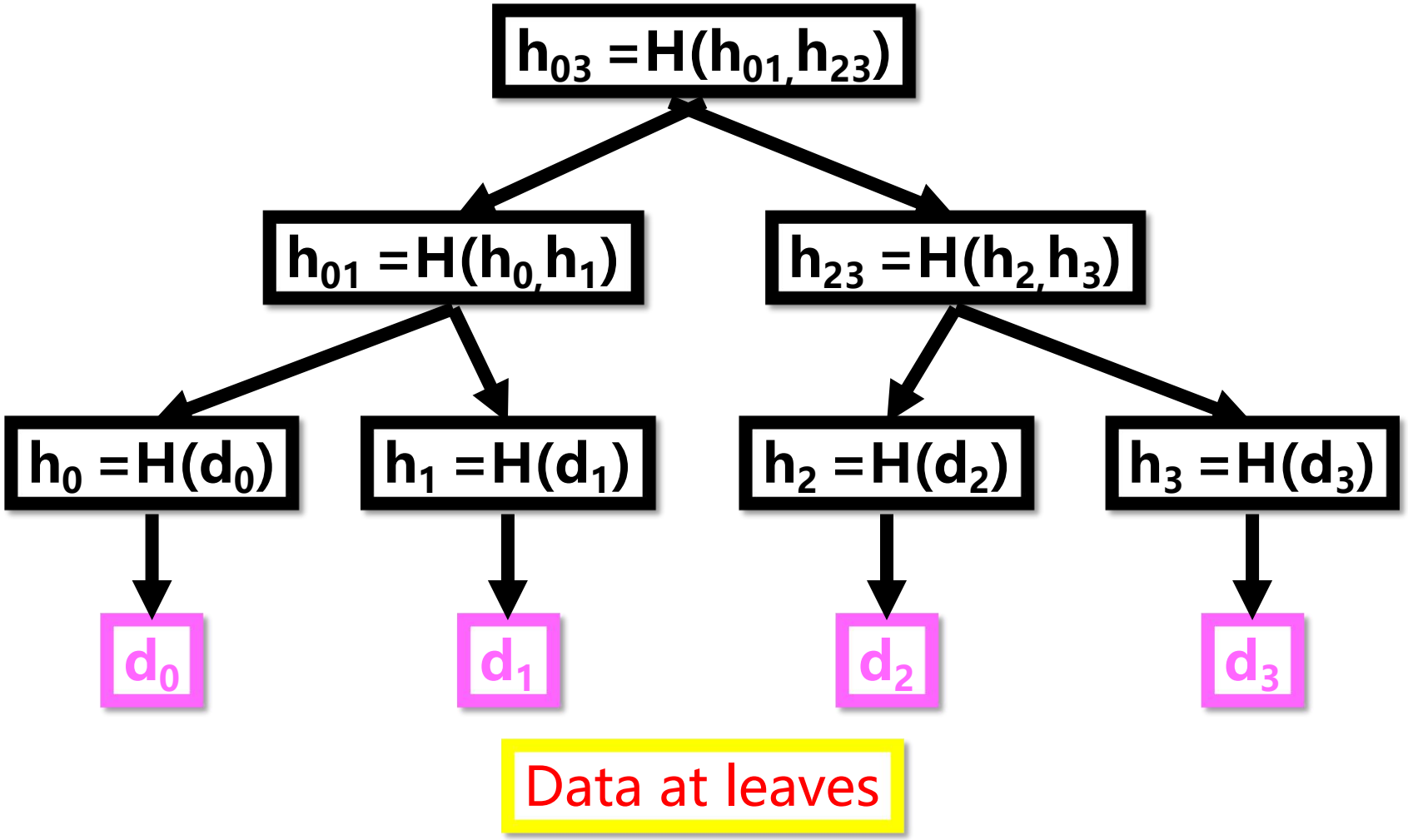
Published in Sequences II.
Springer, New York, NY (1993) pp 329-334

- incorporated *Merkle trees*
- to improved its efficiency
- by allowing several documents to be collected into one block

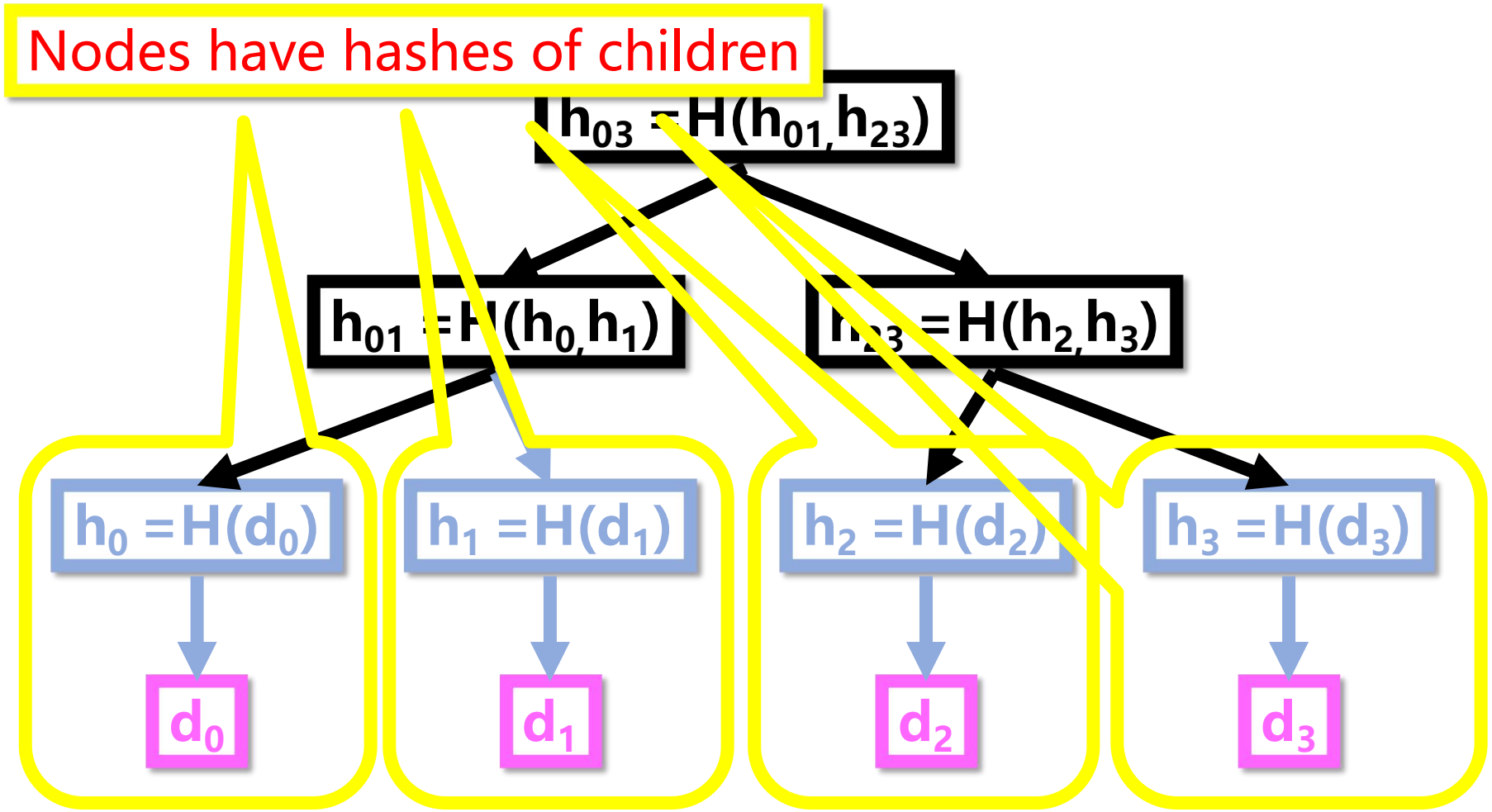
Merkle Tree


$h_{03} = H(h_{01}, h_{23})$
$h_{01} = H(h_0, h_1)$
$h_{23} = H(h_2, h_3)$
$h_0 = H(d_0)$
$h_1 = H(d_1)$
$h_2 = H(d_2)$
$h_3 = H(d_3)$
d_0
d_1
d_2
d_3

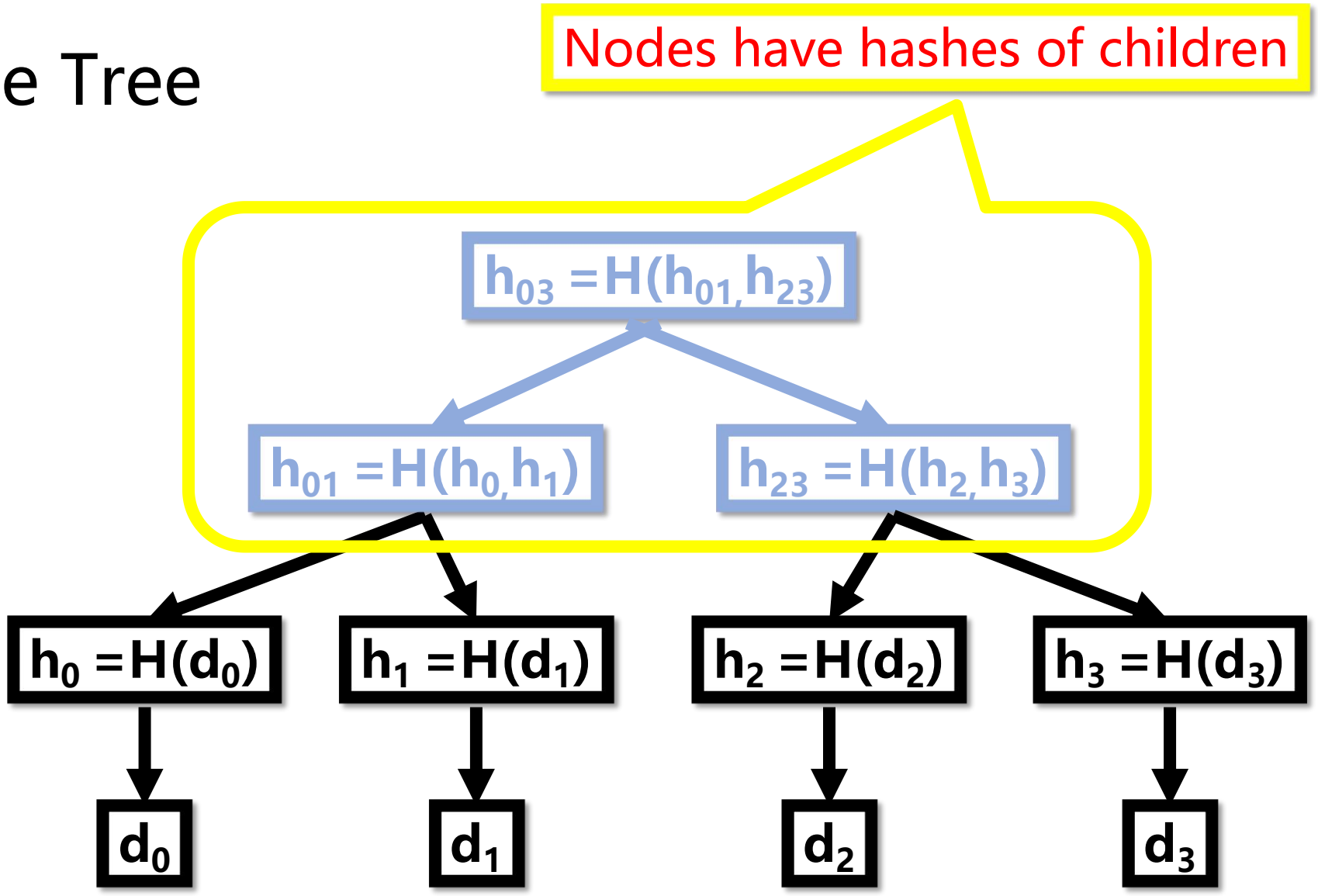
Merkle Tree


$h_{03} = H(h_{01}, h_{23})$
$h_{01} = H(h_0, h_1)$
$h_{23} = H(h_2, h_3)$
$h_0 = H(d_0)$
$h_1 = H(d_1)$
$h_2 = H(d_2)$
$h_3 = H(d_3)$
d_0
d_1
d_2
d_3
Data at leaves

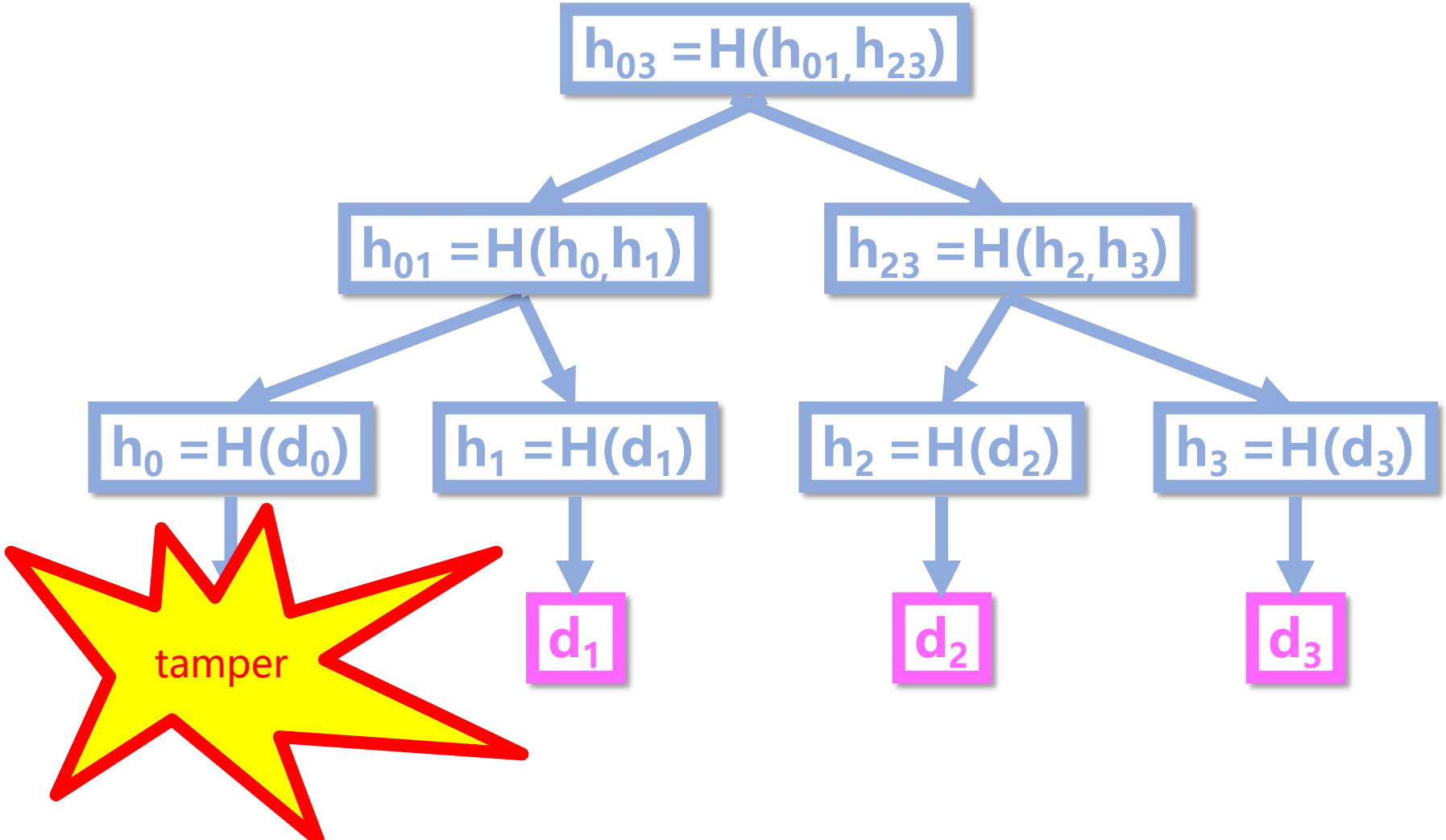
Merkle Tree


Nodes have hashes of children
$h_{03} = H(h_{01}, h_{23})$
$h_{01} = H(h_0, h_1)$
$h_{23} = H(h_2, h_3)$
$h_0 = H(d_0)$
$h_1 = H(d_1)$
$h_2 = H(d_2)$
$h_3 = H(d_3)$
d_0
d_1
d_2
d_3

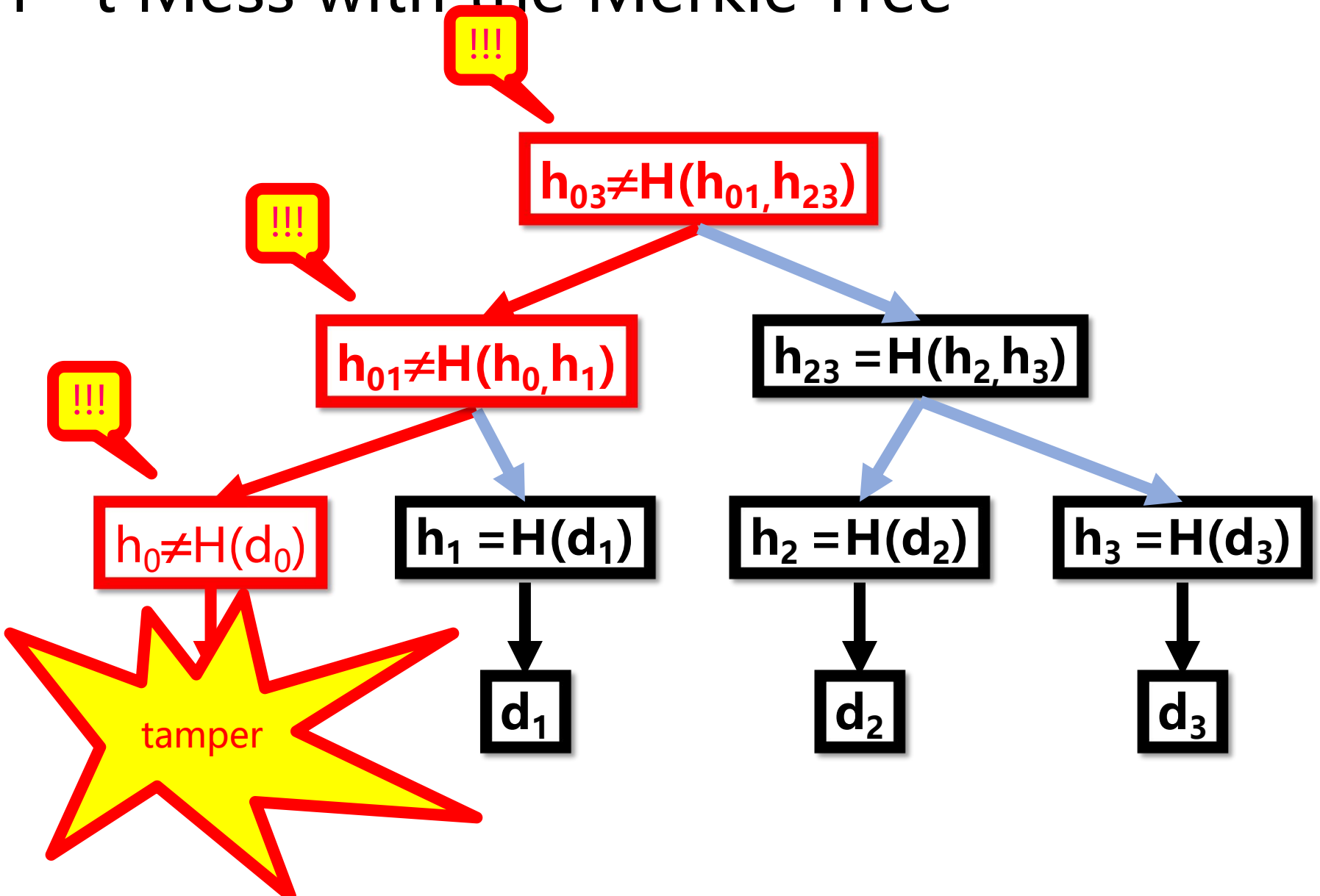
Merkle Tree


Nodes have hashes of children
$h_{03} = H(h_{01}, h_{23})$
$h_{01} = H(h_0, h_1)$
$h_{23} = H(h_2, h_3)$
$h_0 = H(d_0)$
$h_1 = H(d_1)$
$h_2 = H(d_2)$
$h_3 = H(d_3)$
d_0
d_1
d_2
d_3

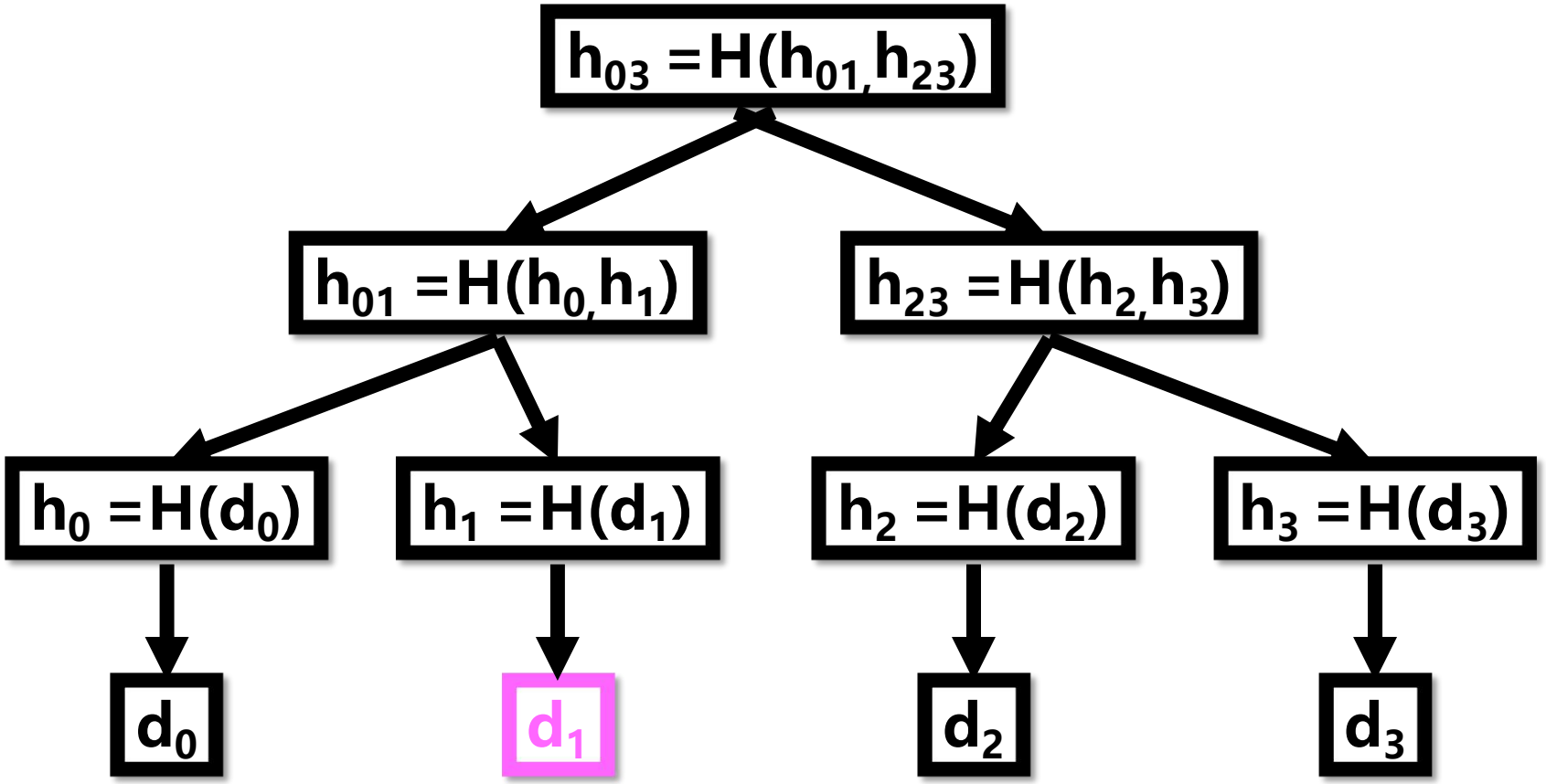
Don’ t Mess with the Merkle Tree



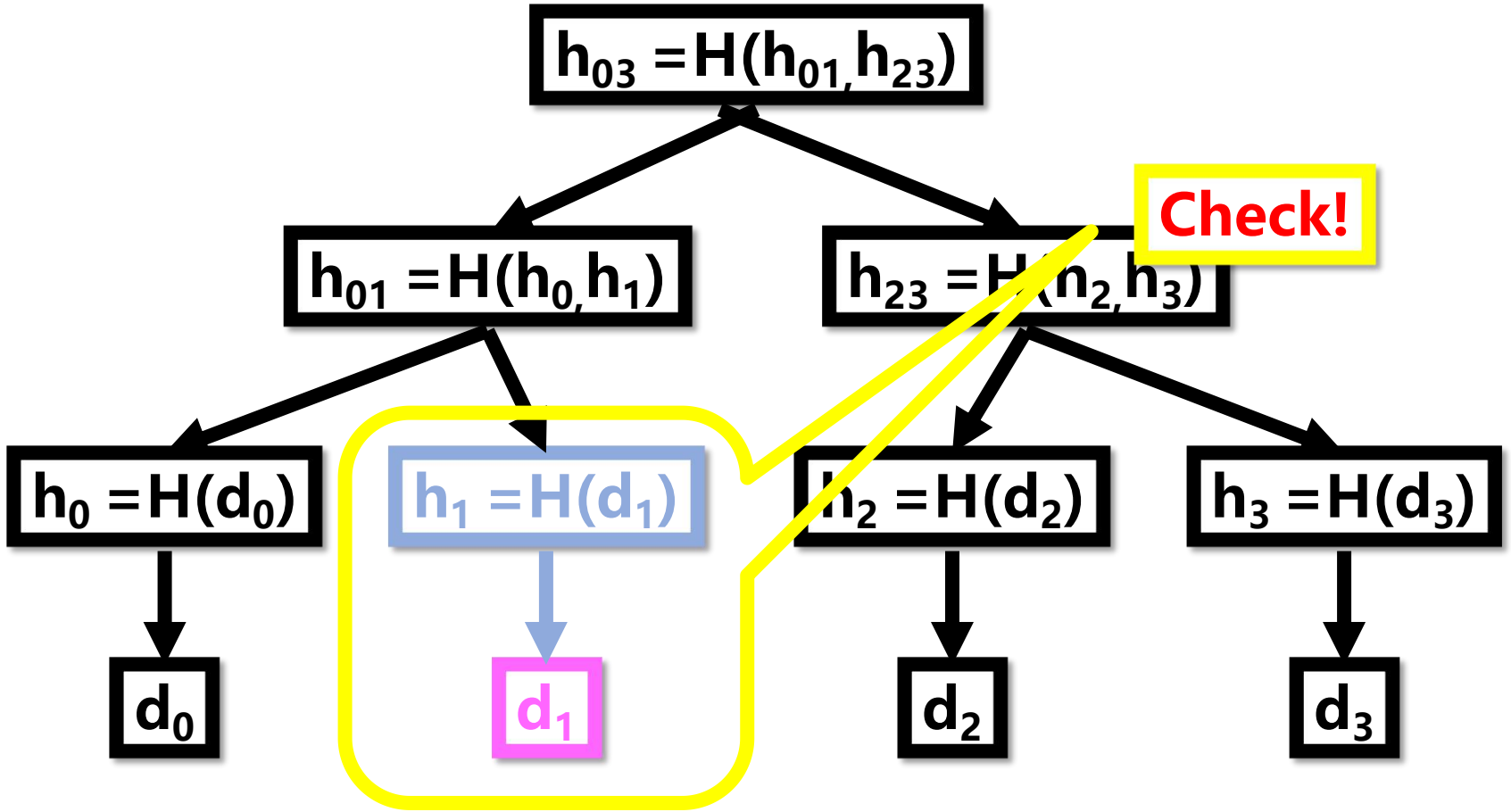
Don’ t Mess with the Merkle Tree



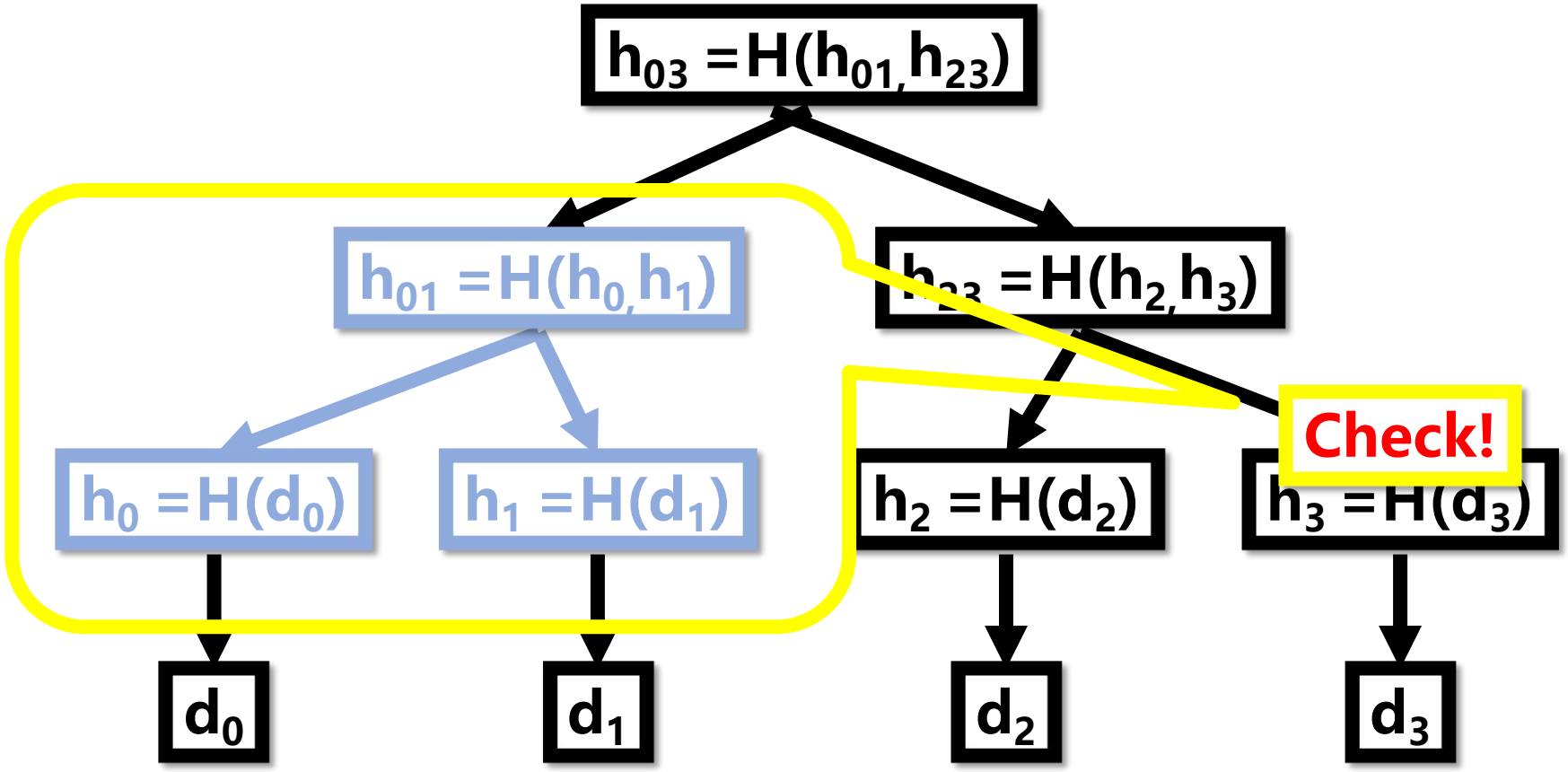
Proof d_1 is in the Tree



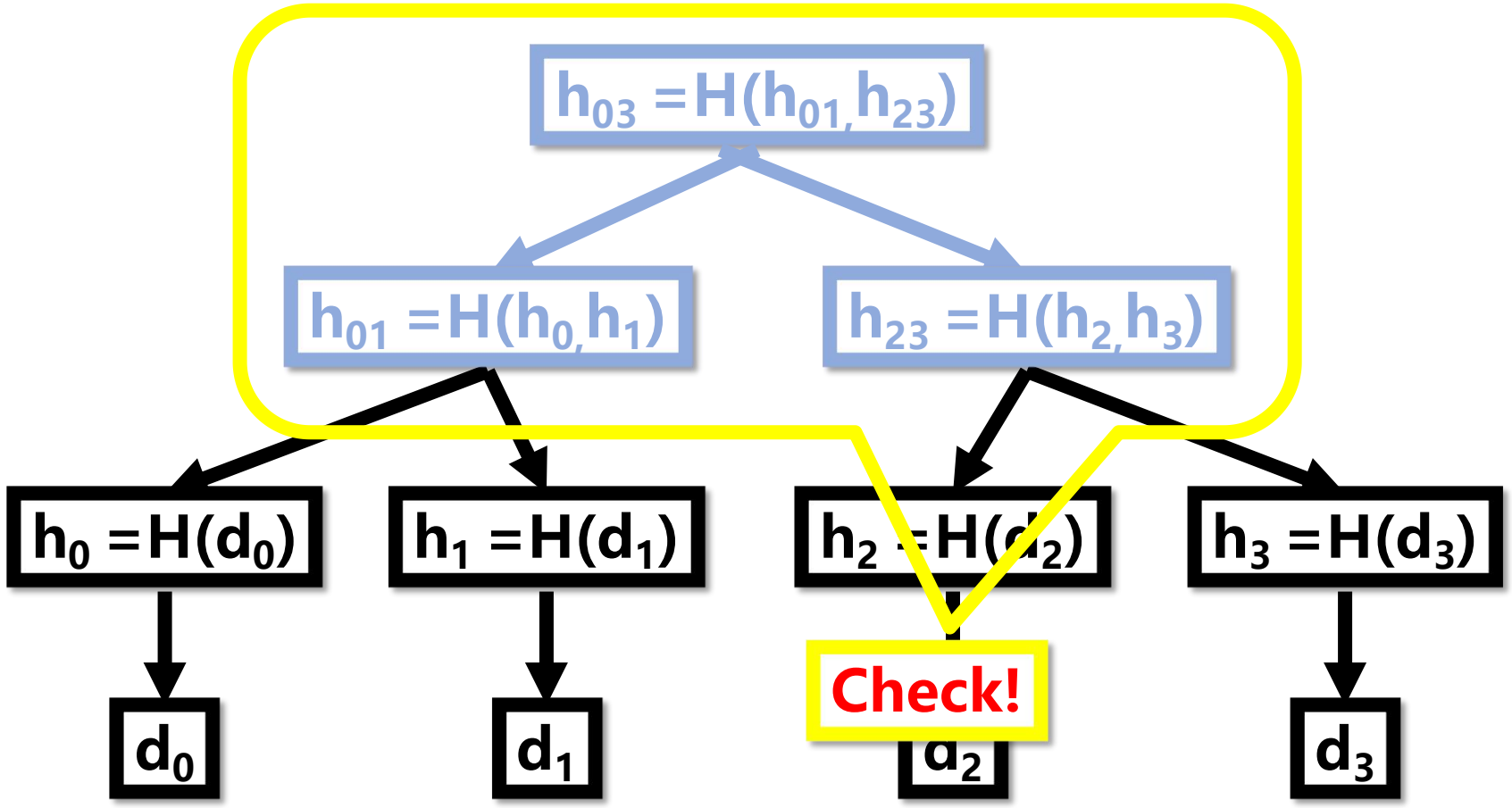
Proof d_1 is in the Tree



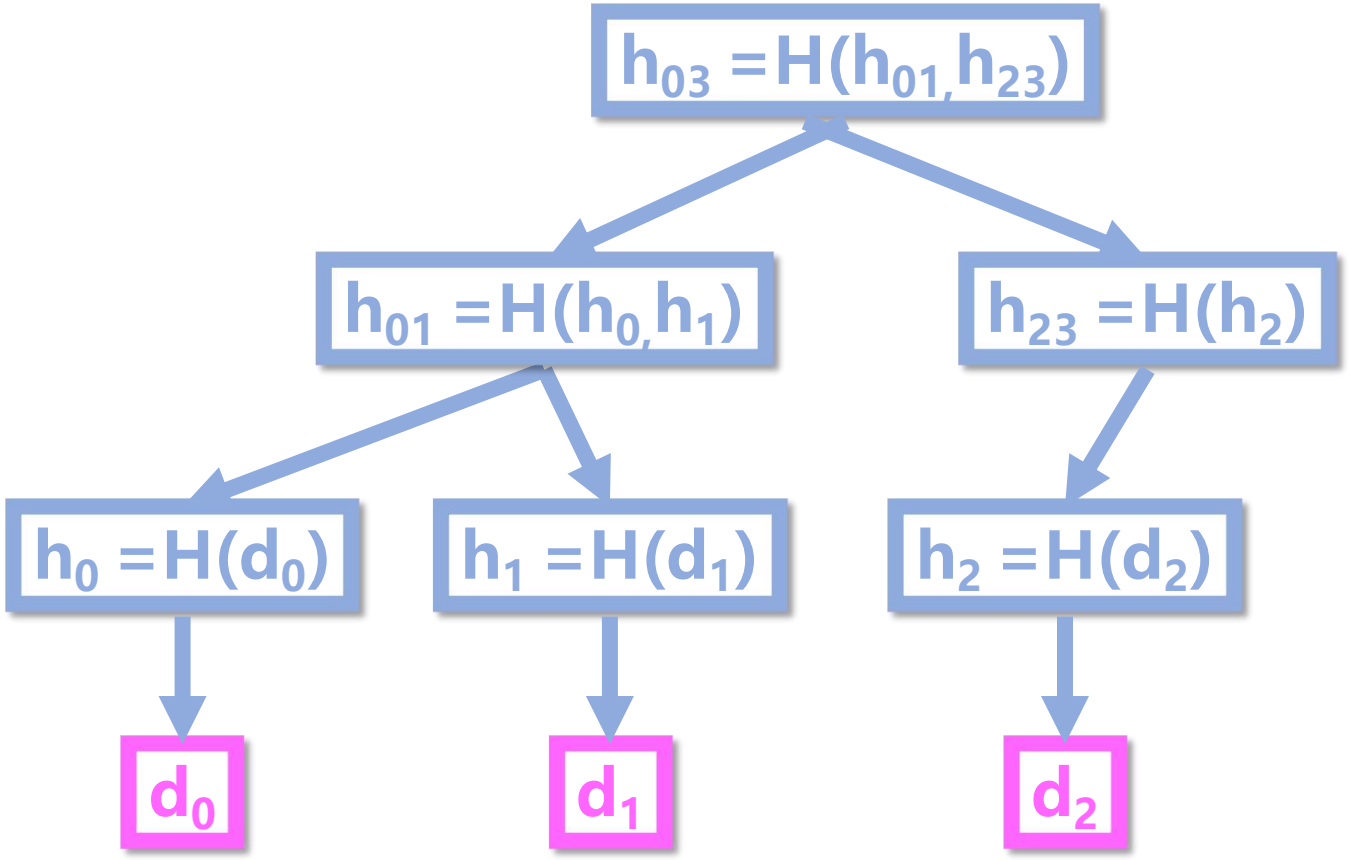
Proof d_1 is in the Tree



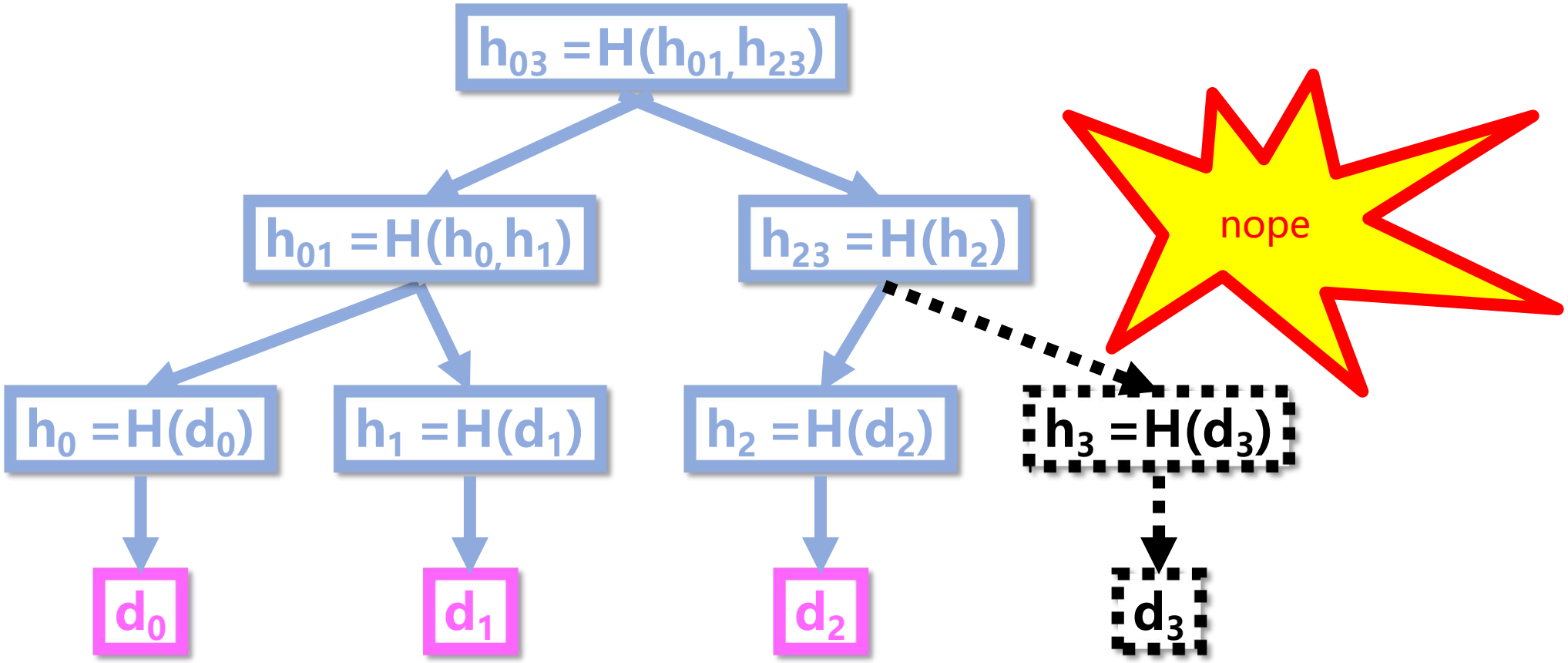
Proof d_1 is in the Tree



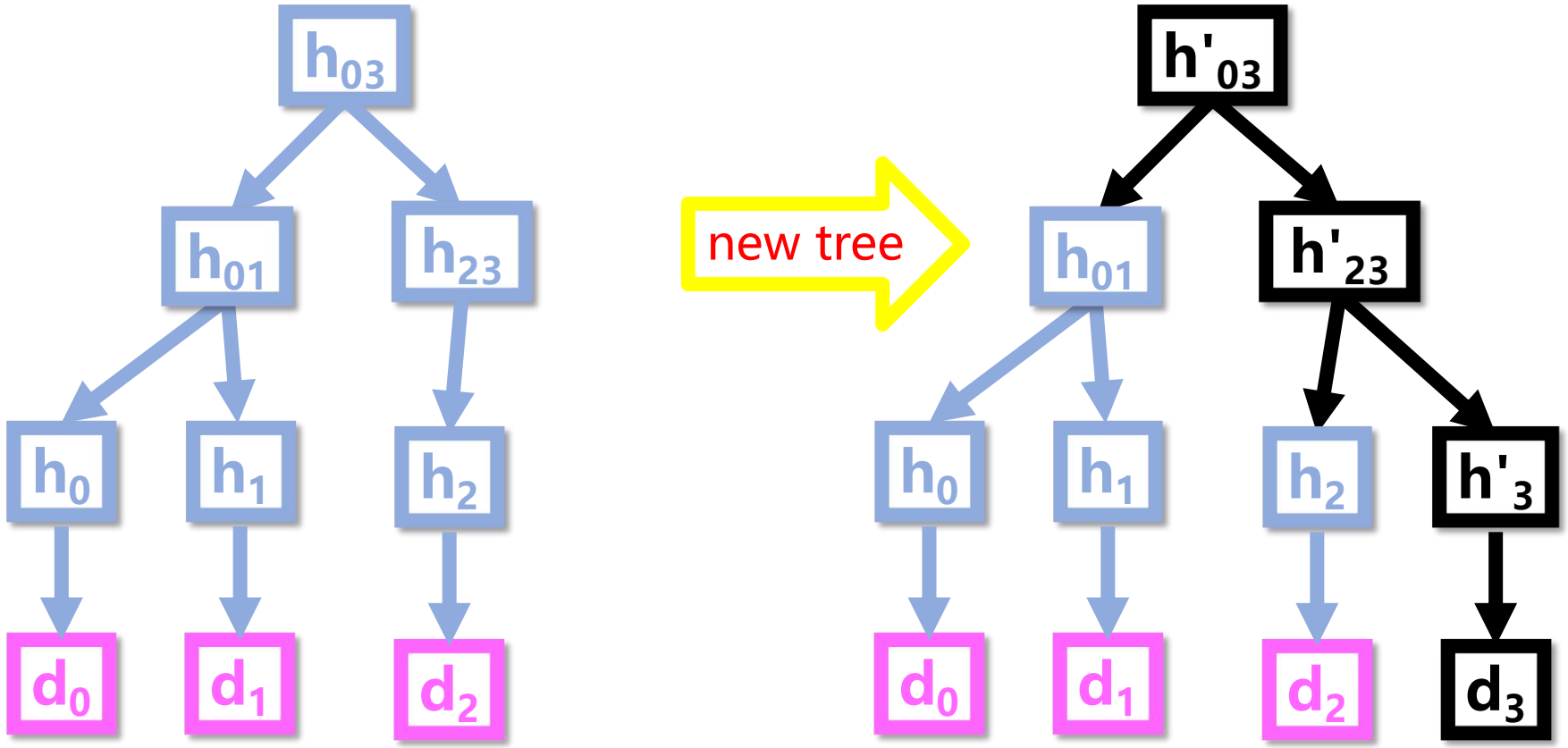
Updating a Merkle Tree



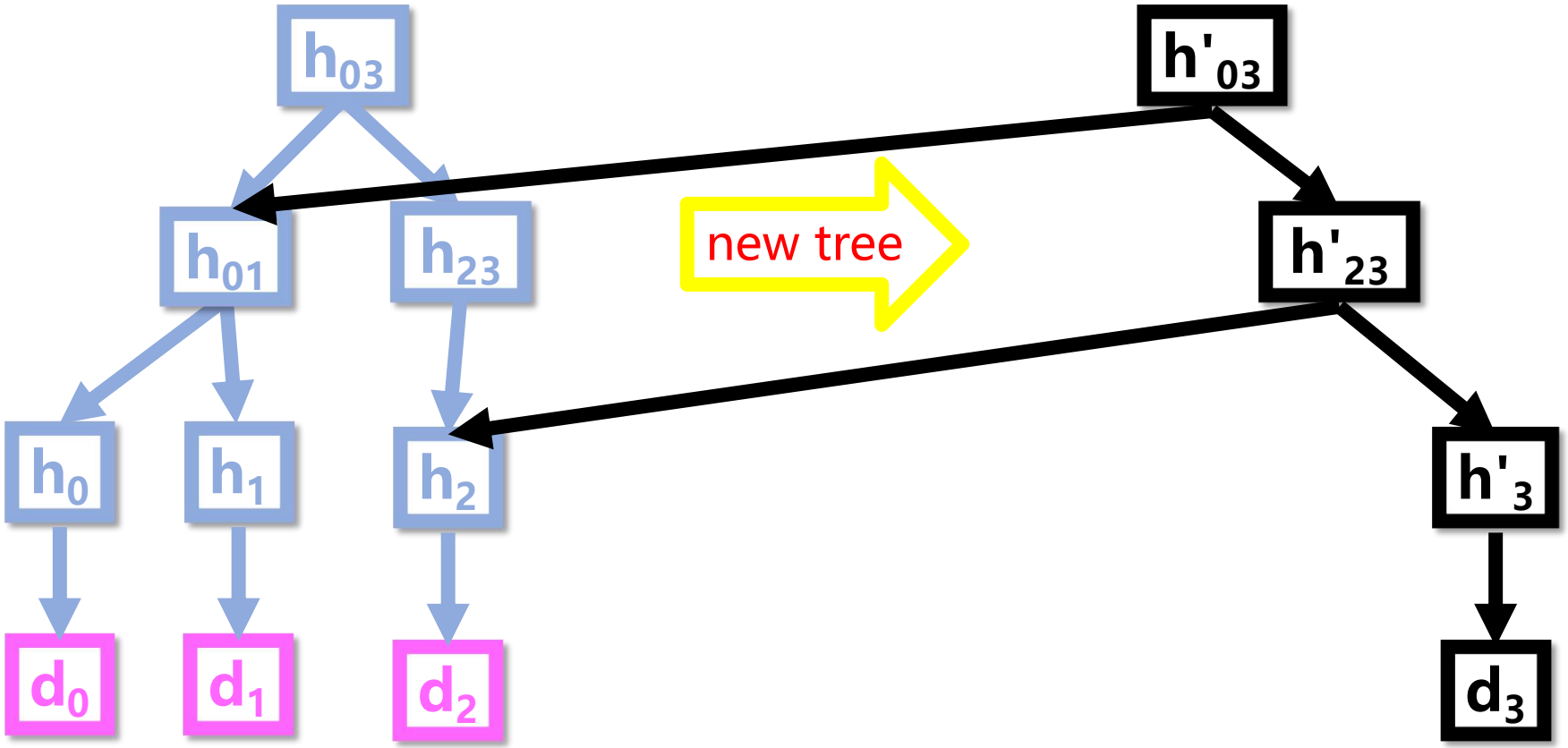
Updating a Merkle Tree



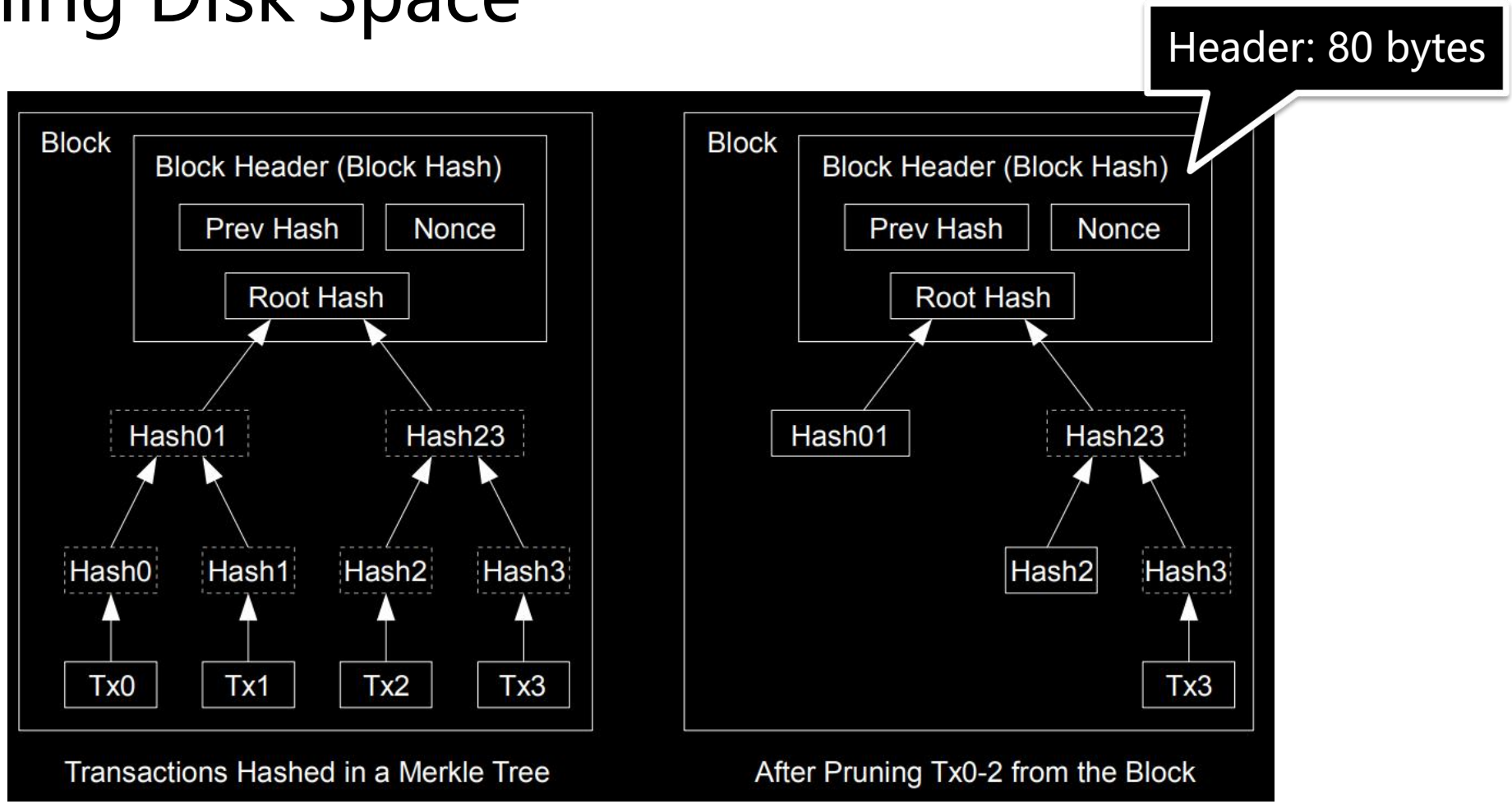
Updating a Merkle Tree



Updating a Merkle Tree

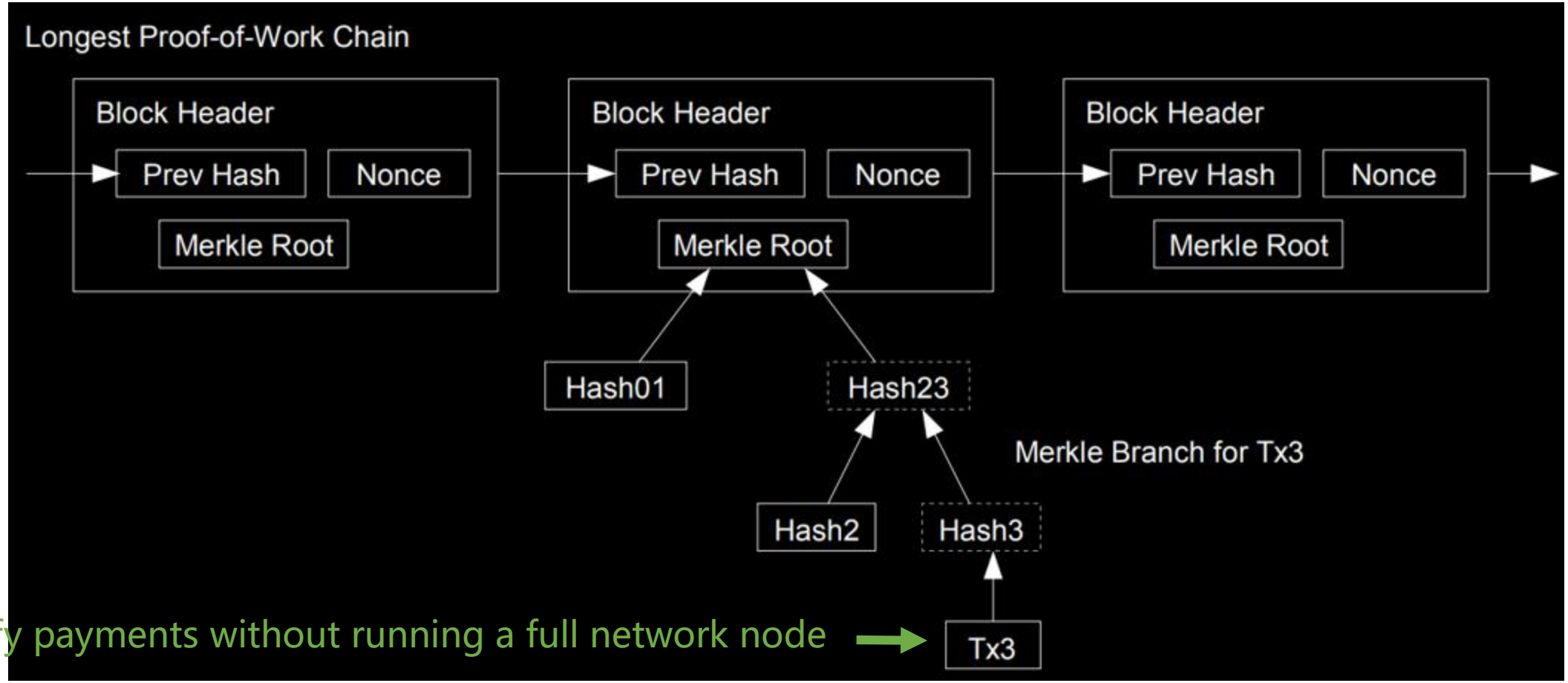


Reclaiming Disk Space



Blocks are generated every 10 minutes, 80 bytes * 6 * 24 * 365 = 4.2MB per year

Simplified Payment Verification



Verify payments without running a full network node

Bitcoin System

Incentive Mechanism

As calculating the valid block hash have to use many computational resources, bitcoin system provides an Incentive Mechanism to encourage miners to contribute the system.

A miner receives 2 types of rewards when it broadcasts a valid block successfully:

1. **Incentive Reward**
2. **Transaction Fee**

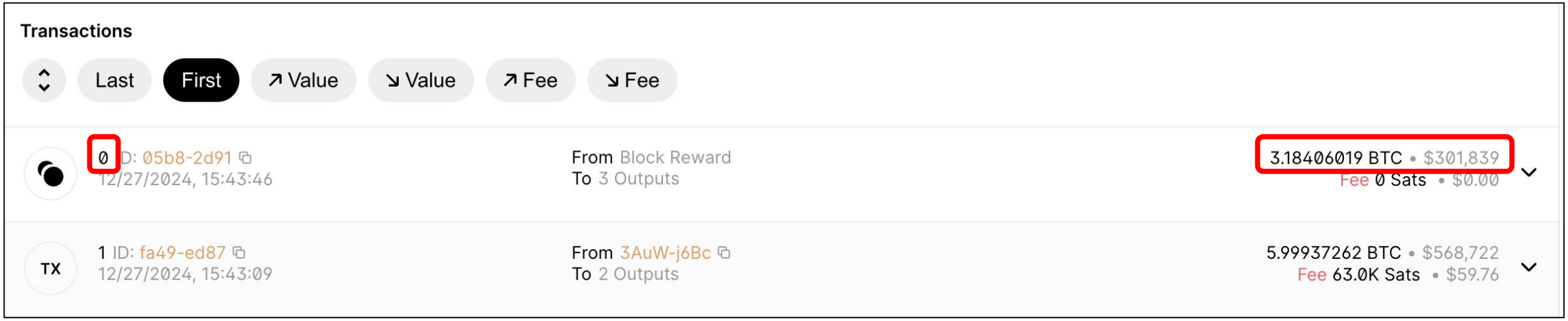
Bitcoin System

Incentive Mechanism

1. Incentive Reward

- Before a miner calculates the block hash, the Bitcoin protocol allows it to add a transaction that creates a specific value of Bitcoin to its address. This transaction usually appears as the first transaction (index = 0) in the block.

- When a miner successfully finds a valid block hash, this reward transaction is recorded in the global ledger, then the miner can claim the reward.



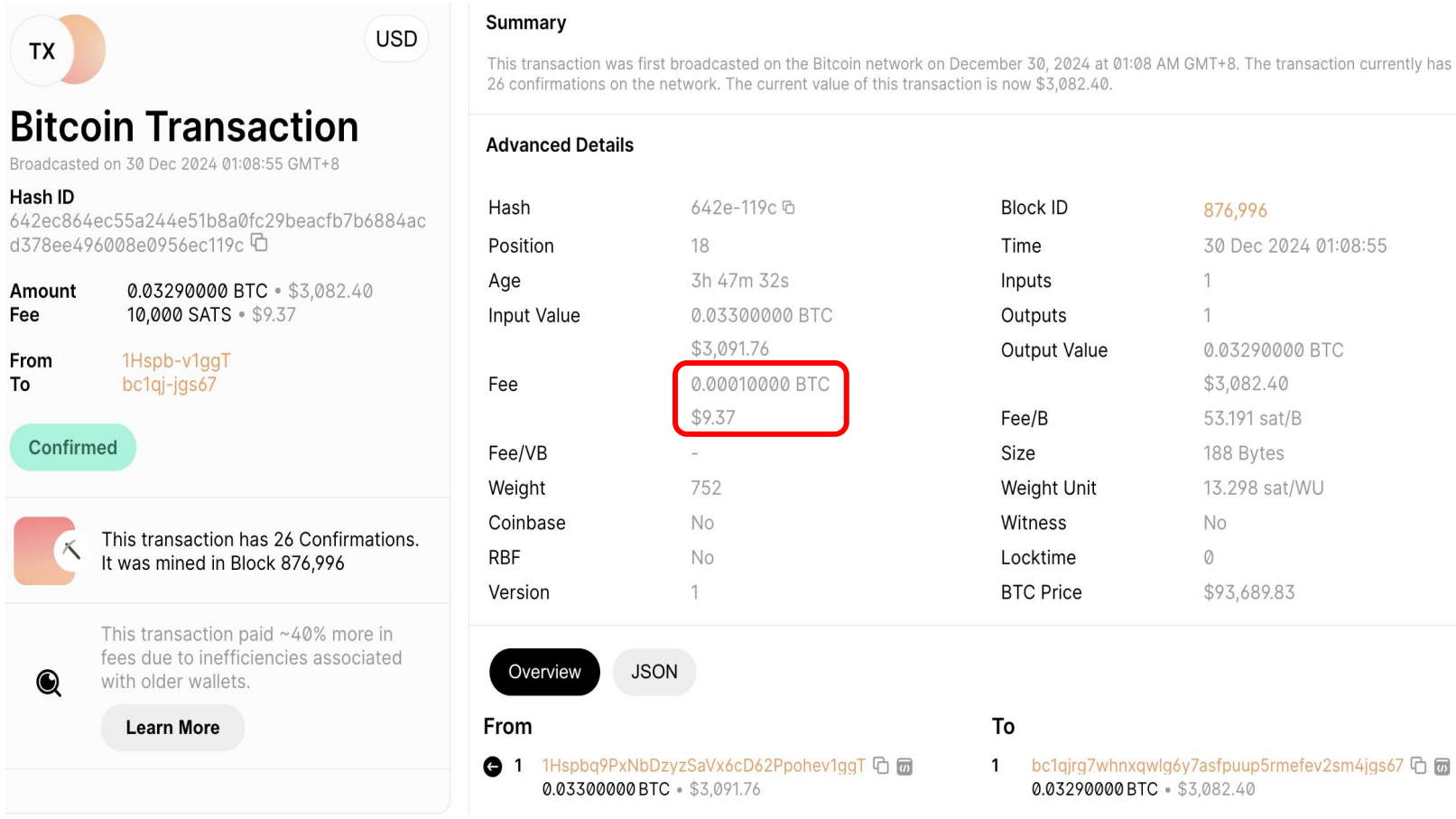
Bitcoin System

Incentive Mechanism

2. Transaction Fee

- The solution implemented in the Bitcoin system is to create an **auction system**.

- Each transaction sender is required to provide a transaction fee amount. When a miner includes a transaction, they can earn the transaction fee associated with it.



Fee = Input Value – Output Value

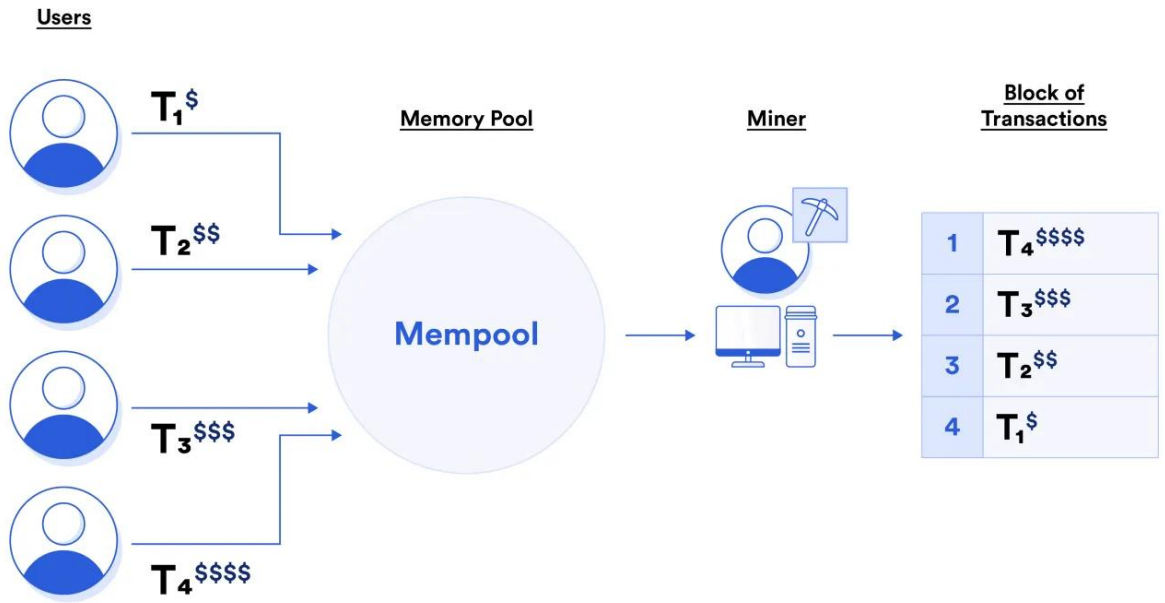
https://www.blockchain.com/explorer/transactions/btc/642ec864ec55a244e51b8a0fc29beacfb7b6884acd378ee496008e0956ec119c

Bitcoin System

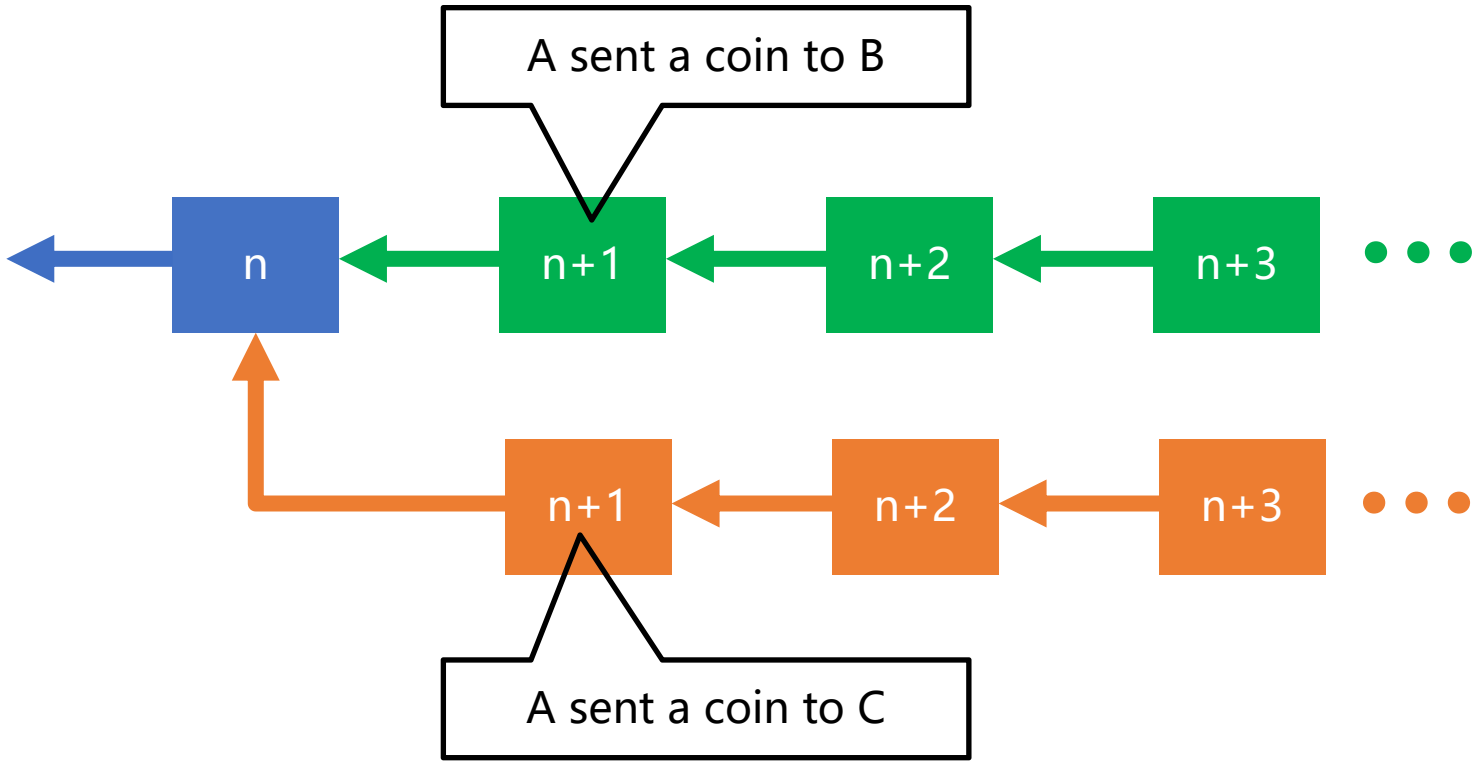
Incentive Mechanism

2. Transaction Fee

- In this auction system, miners are motivated to include transactions with the highest transaction fees in order to maximize their rewards. This encourages transactions with higher fees can be processed and written into the global ledger faster.



The Essence of Double Spending



Which chain is the correct one?

Review: Longest Chain Rule

in it. If a majority of CPU power is controlled by honest nodes, the honest chain will grow the fastest and outpace any competing chains. To modify a past block, an attacker would have to

Honest miners build on longest chain …



… longest chain reflects will of honest miners

Dishonest miners would have to out-compute all honest miners

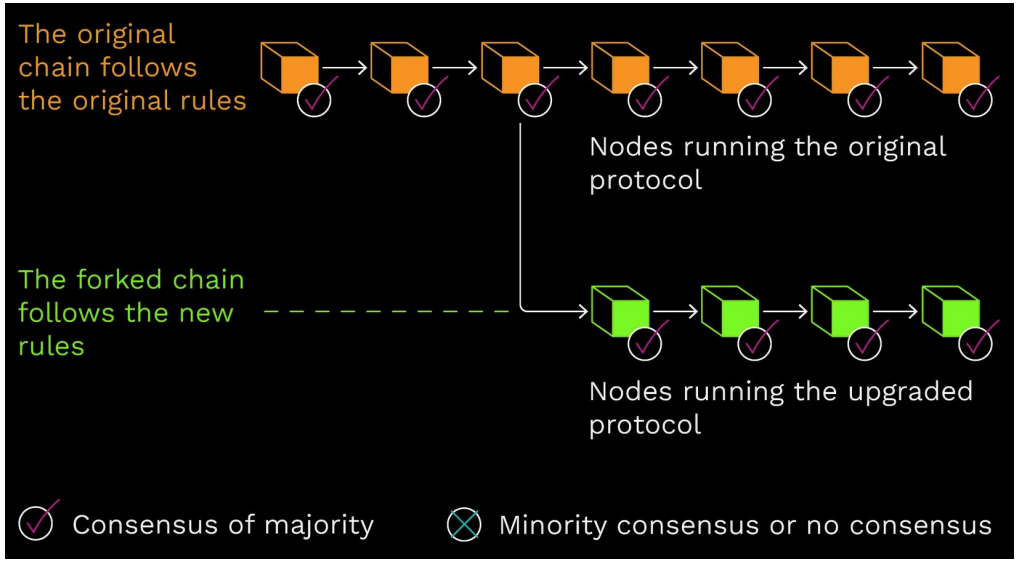
Chain Property

Control x% of miners, control x% of blocks

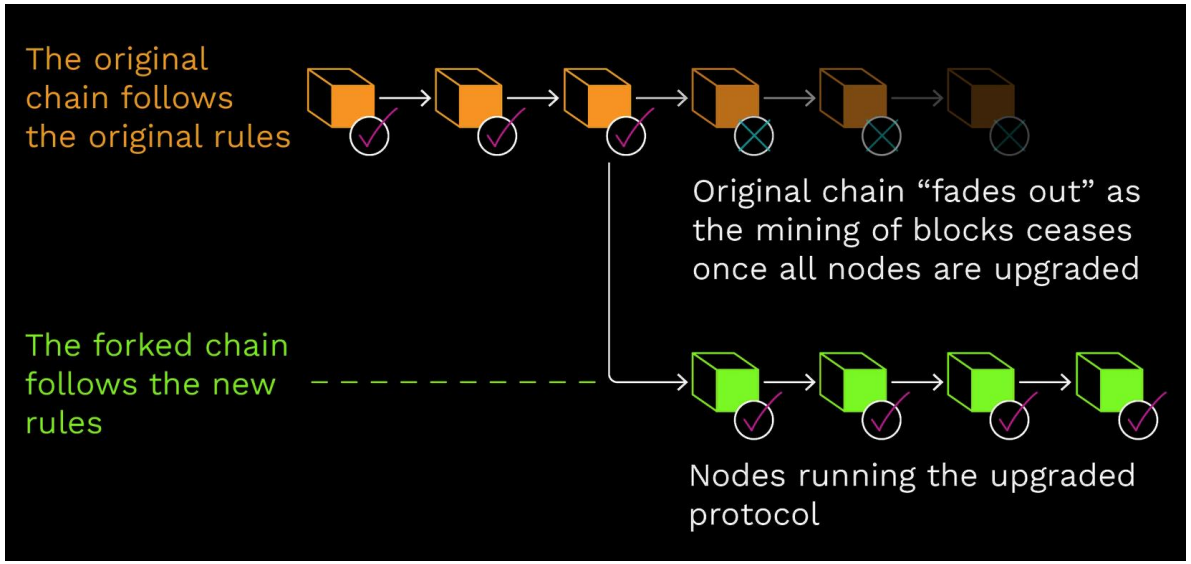
Not always true but close enough for now

Soft Fork and Hard Fork

Main Difference: the change in protocol is gradual or not

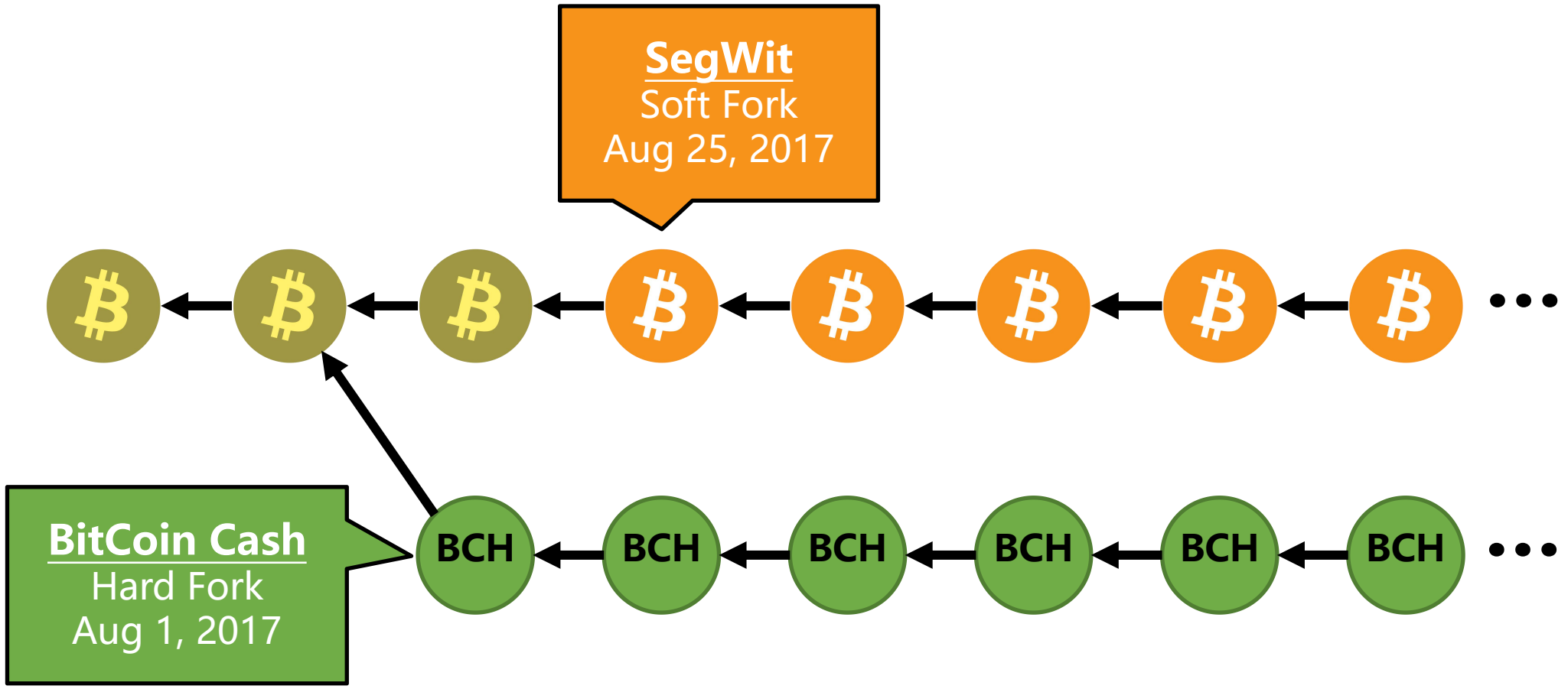


Hard Fork: Backwards Incompatible



Soft Fork: Backwards Compatible

Two Major Forks on Block Size Limit

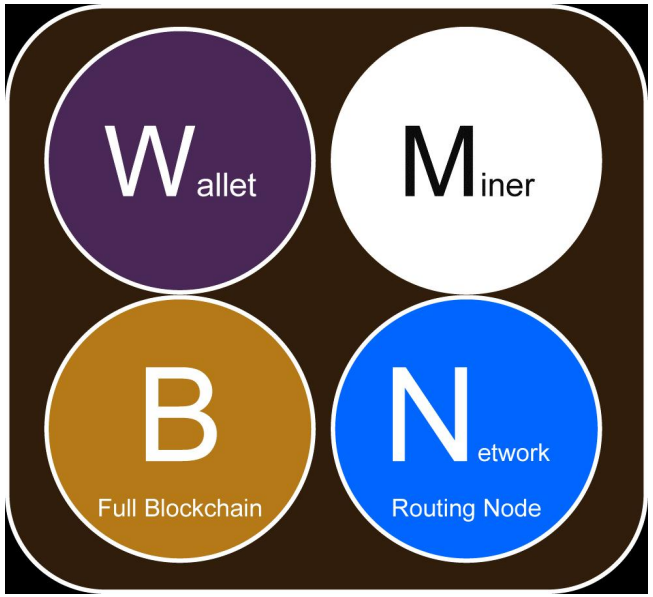


Nodes in Bitcoin Network

All nodes include the routing function to participate in the network

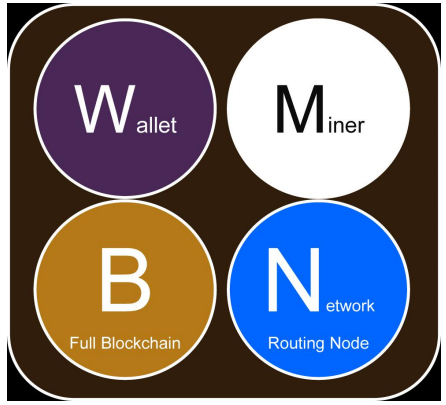
Discover &maintain connections to peers

Validate & Propagate transactions & blocks

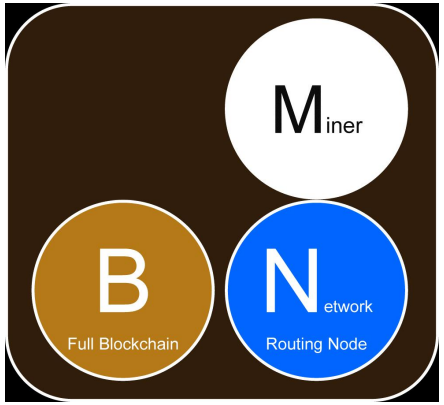


A Full Node

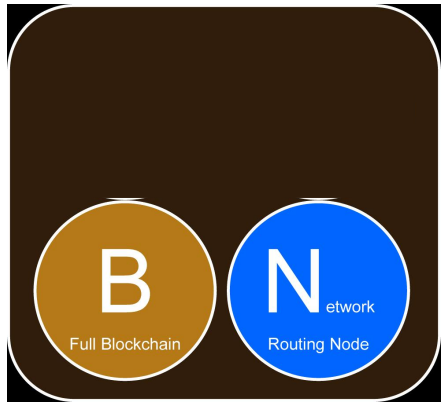
Nodes in BitCoin Network



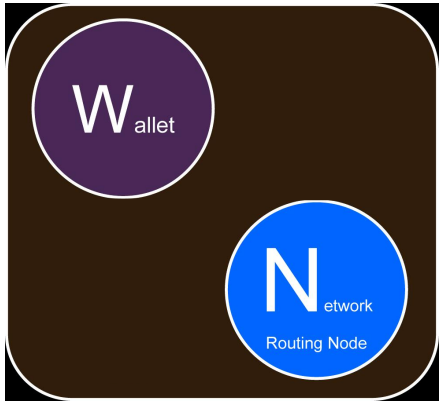
Reference Client (BitCoin Core)



Solo Miner



Full Blockchain Node

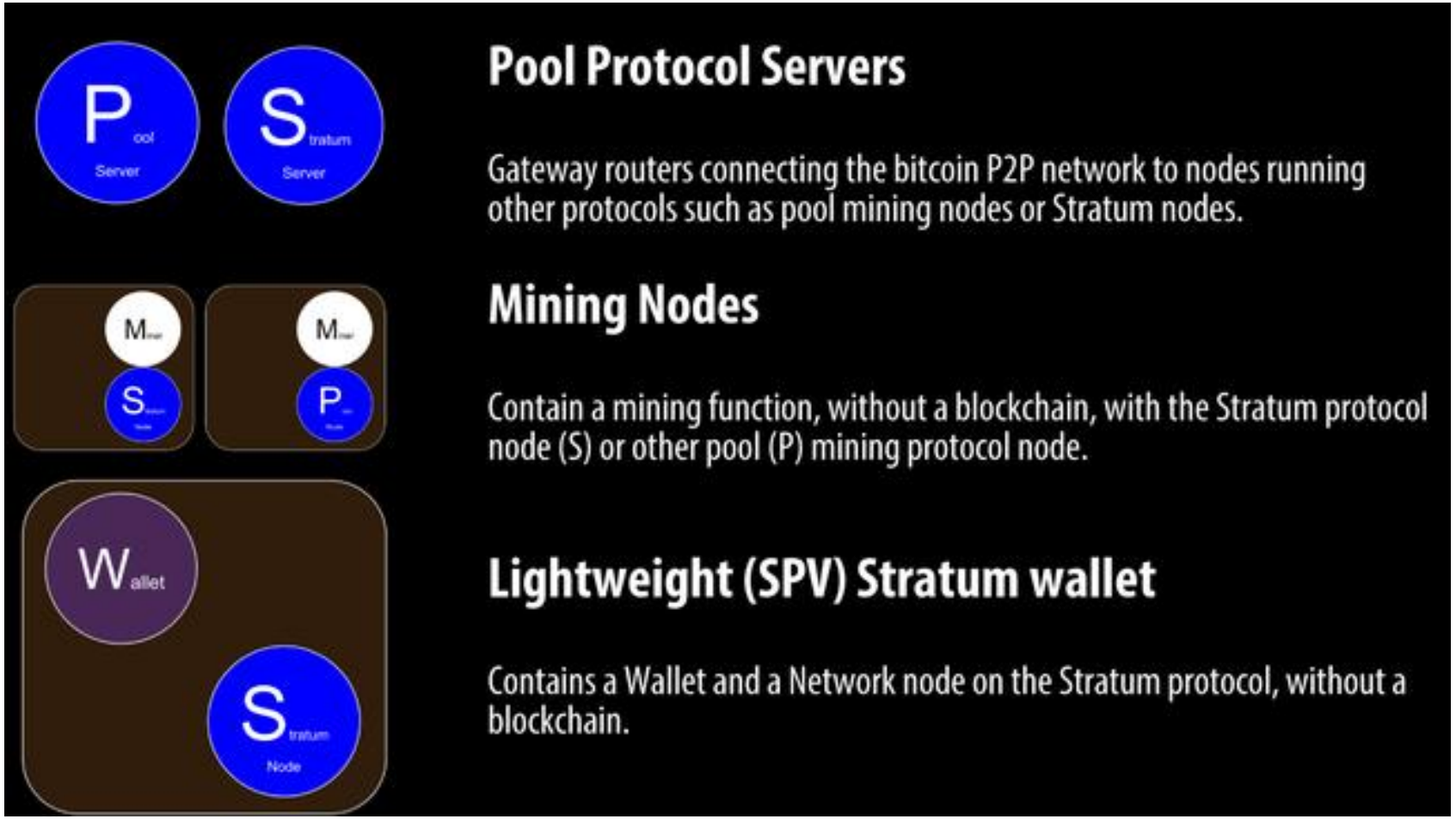


Simplified Payment Verification (SPV) Nodes
Lightweight Wallet

Simplified Payment Verification (SPV) Nodes

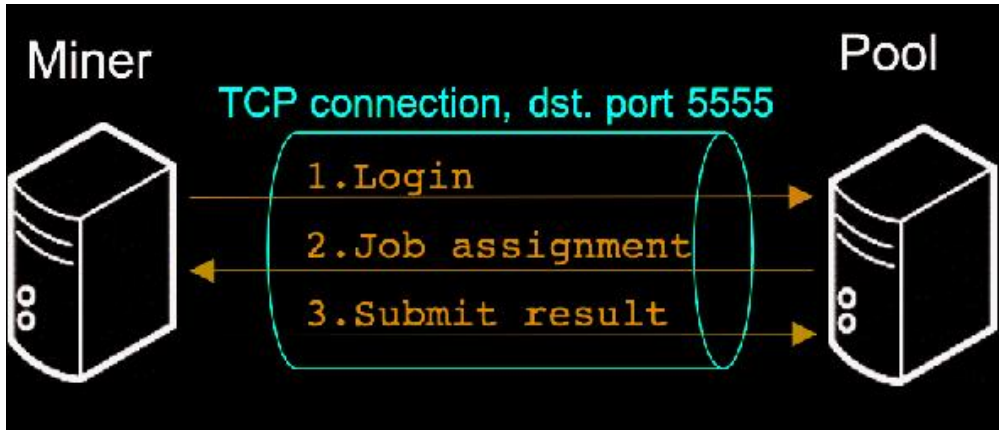
- The most common form of bitcoin node (bitcoin wallets)
  - for space- and power-constrains on nodes
- Download only the block headers
  - 1,000 times smaller than the full blockchain nodes
- Can verify transactions
  - request a merkle path proof
  - validate the PoW in the chain of blocks
- Cannot validate double spending
  - Cannot construct a full picture of all transactions and available UTXOs

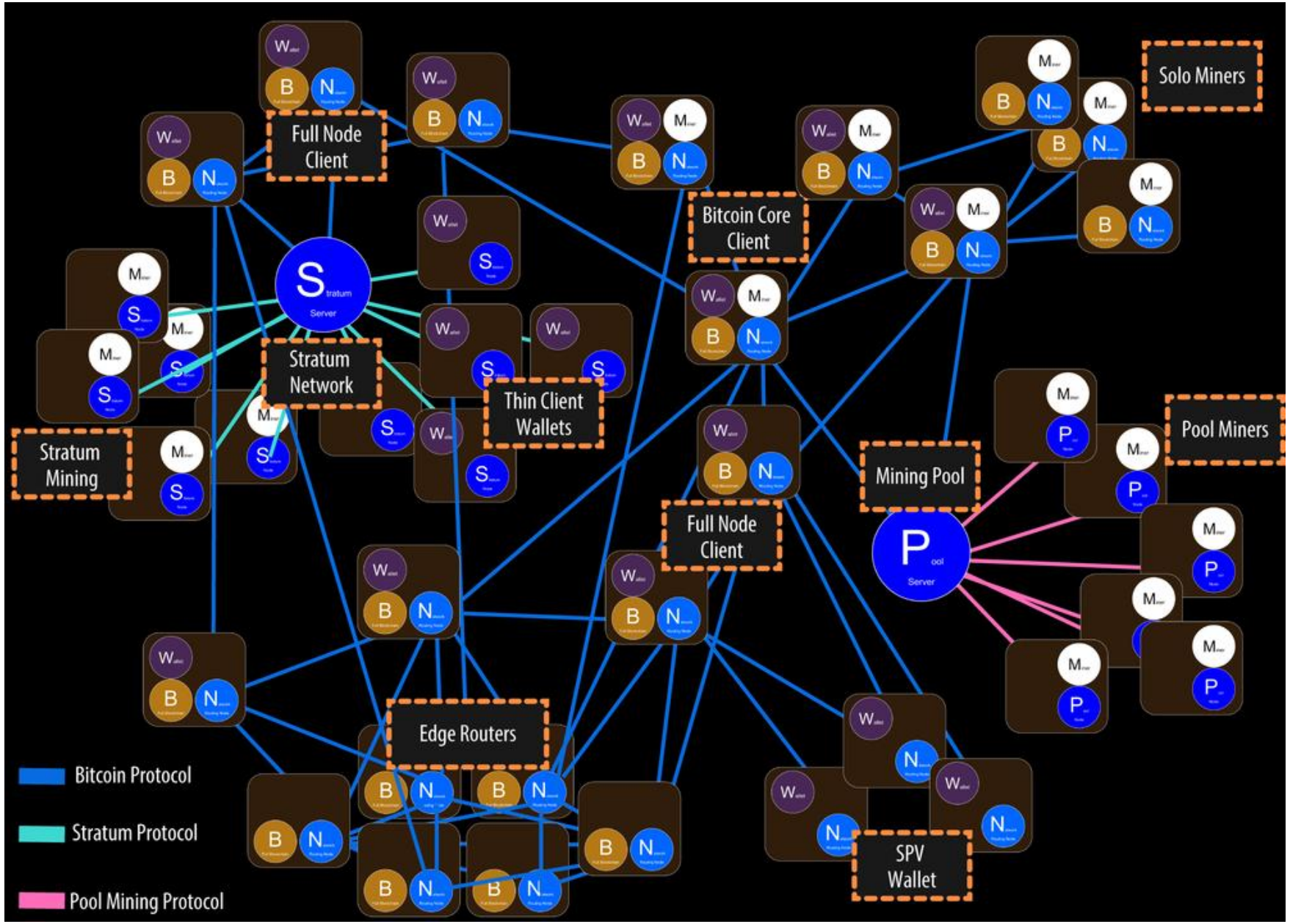
Nodes in BitCoin Network



Stratum Protocol

- Used by ASIC miners and mining pools
- Miners use this protocol to obtain a template for a new block





Extended Bitcoin Network

Privacy in Searching Addresses

Example: SPV nodes ask their peers for transactions for UTXOs

Solution: Bloom Filters

- Probabilistic Search Filter
- A way to describe a desired pattern without specifying it exactly
- An efficient way to express a search pattern while protecting privacy

Analog of Bloom Filters



Asking for directions to “23 Church St”

“Are there any streets whose name ends in R-C-H?”

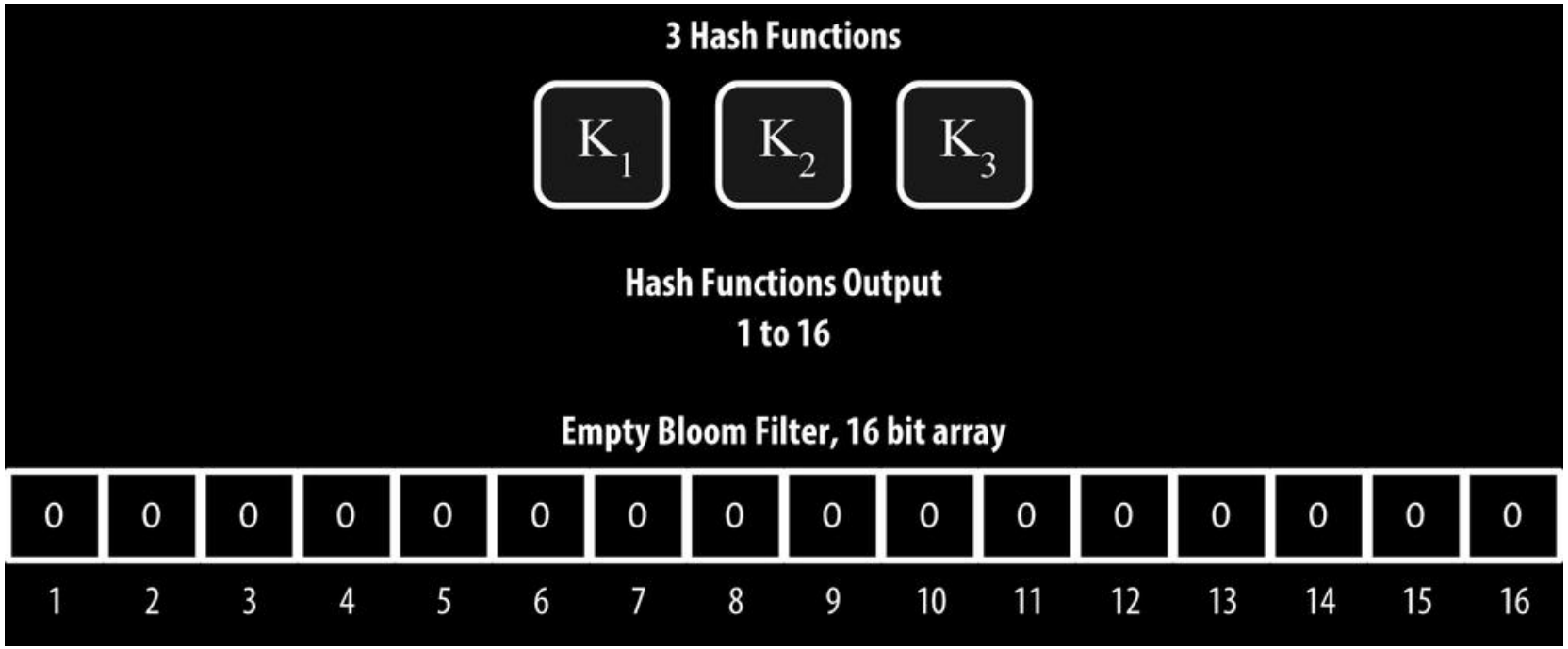
- more detail as “ending in U-R-C-H”
- less detail as “ending in H”

less specific -> more possible addresses & better privacy

more specific -> fewer results & loses privacy

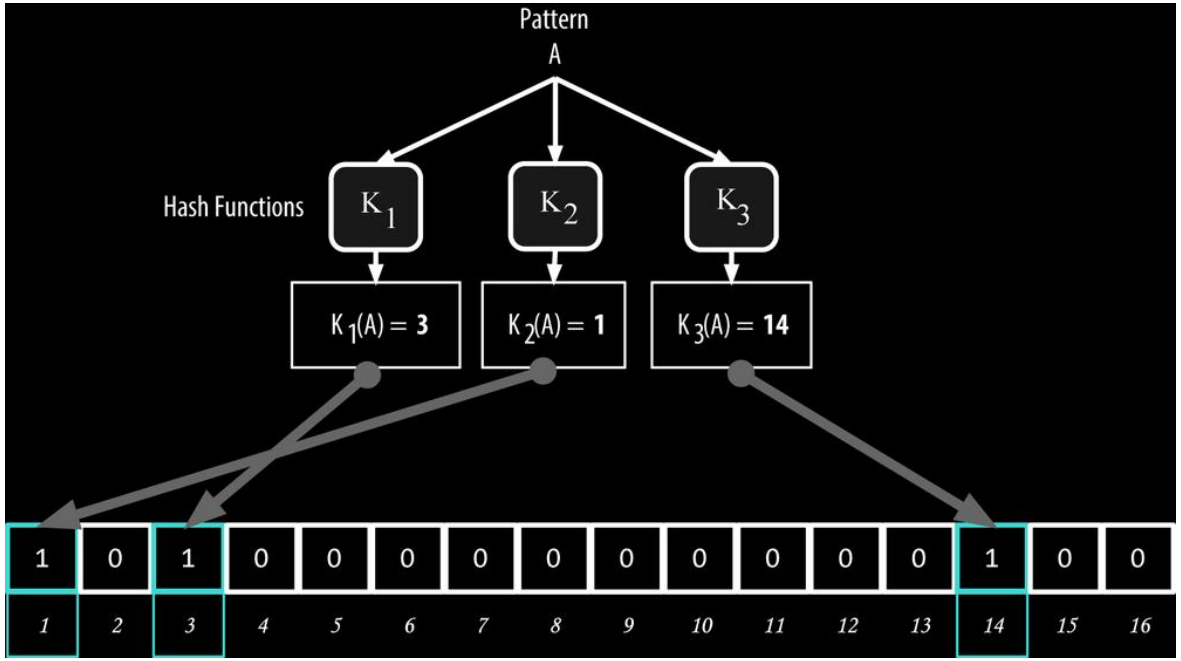
Bloom Filters

- Bloom filters are implemented as
  - a variable-size array of N binary digits (a bit field)
  - a variable number of M hash functions with output between 1 and N

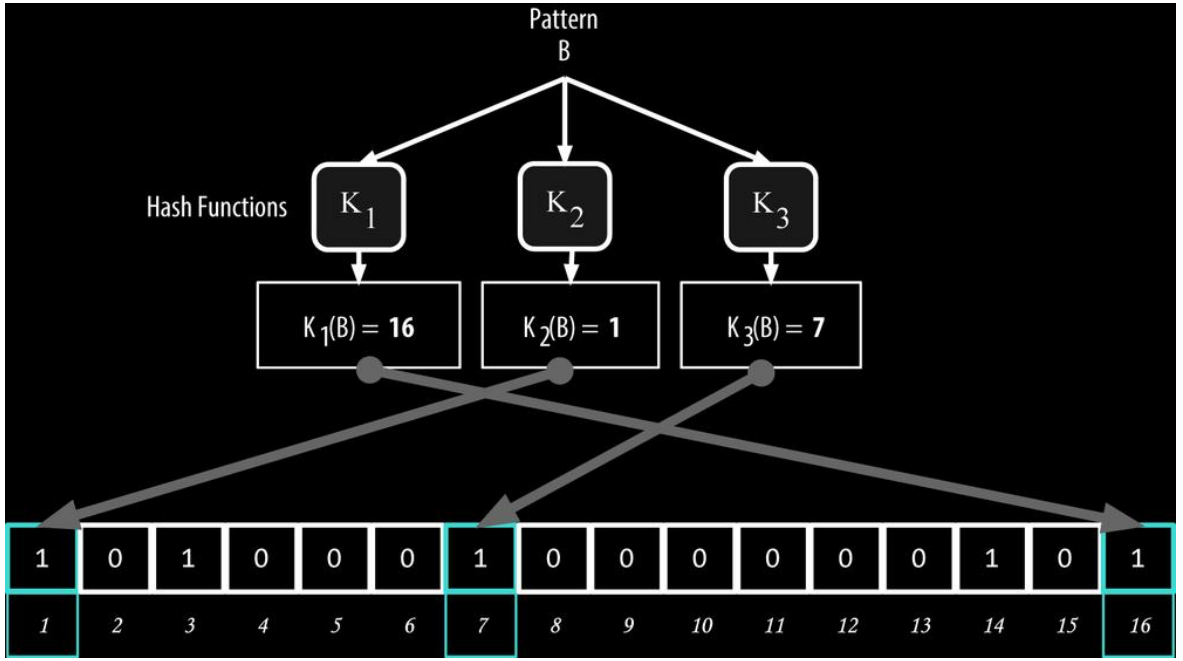


Bloom Filters: Adding Data

Adding Pattern A

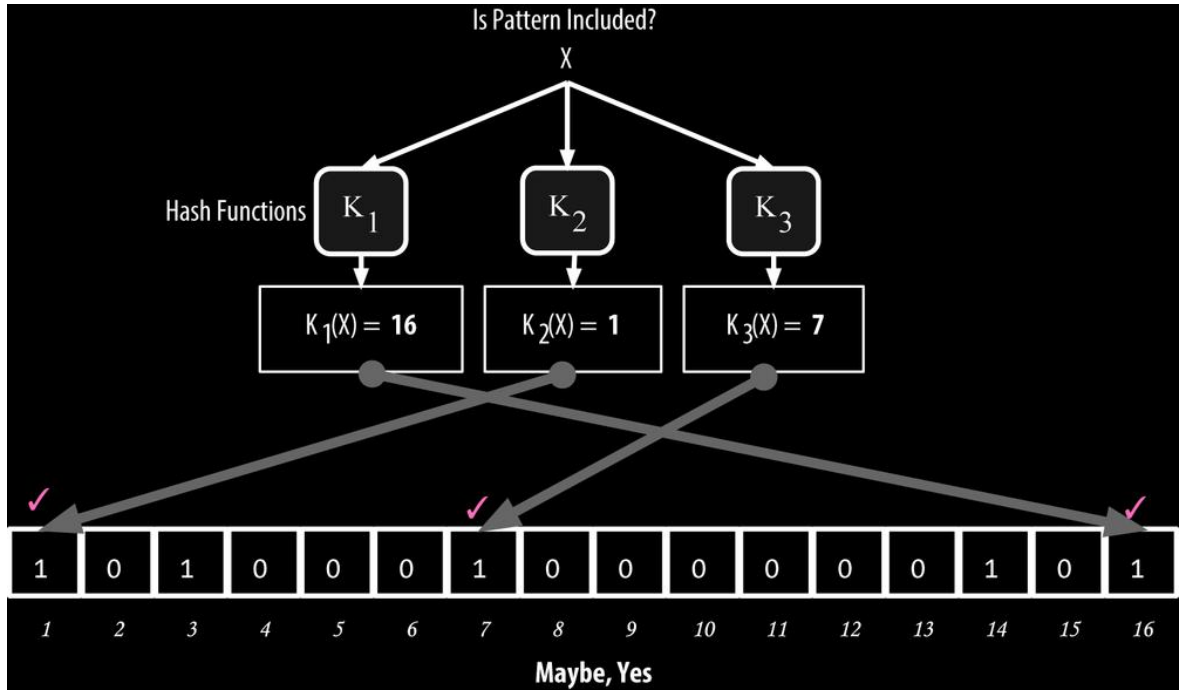


Adding Pattern B

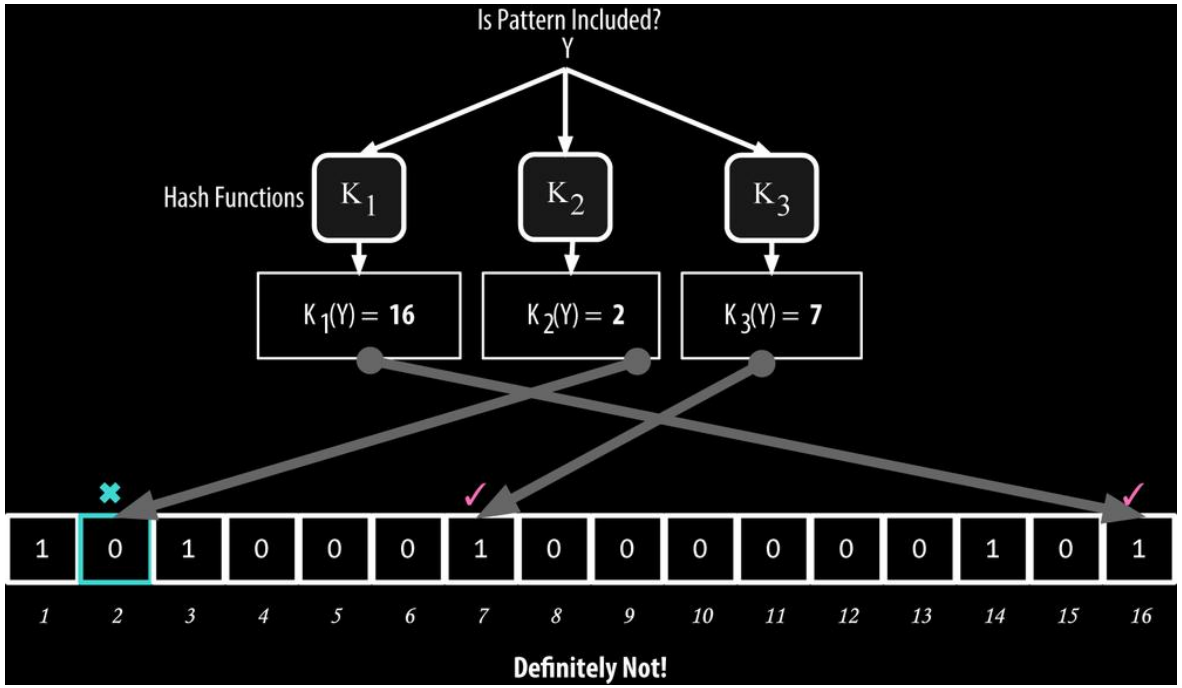


Bloom Filters: Testing Existence of Pattens

Checking Pattern X



Checking Pattern Y



Bloom Filters and Requesting Data

① SPV nodes create bloom filters
- Example: matches only the addresses held in the SPV node’ s wallet

② SPV nodes send a *filterload* message to the peer
- containing the bloom filter to use on the connection

③ The peer only send transactions matches the bloom filter

Transaction Pools

Transaction Pools (mempool)

- Temporary List of Unconfirmed Transactions
- tracking broadcasted transactions not yet included in blocks

Orphaned Transaction Pool

- store transactions with inputs from unknown transactions
- always checking for matching parents

UTXO Pool

- List of All Confirmed Unspent Outputs
- represents the emergent consensus of the network

Find PoW Nonce

```
def proof_of_work(header, difficulty_bits):

target = 2 ** (256-difficulty_bits)          # calculate the difficulty target

    for nonce in xrange(max_nonce):
        hash_result = hashlib.sha256(str(header)+str(nonce)).hexdigest()

        if long(hash_result, 16) < target:          # check if this is a valid result, below the target
            print "Success with nonce %d" % nonce
            print "Hash is %s" % hash_result
            return (hash_result,nonce)

    print "Failed after %d (max_nonce) tries" % nonce
    return nonce
```

Reward

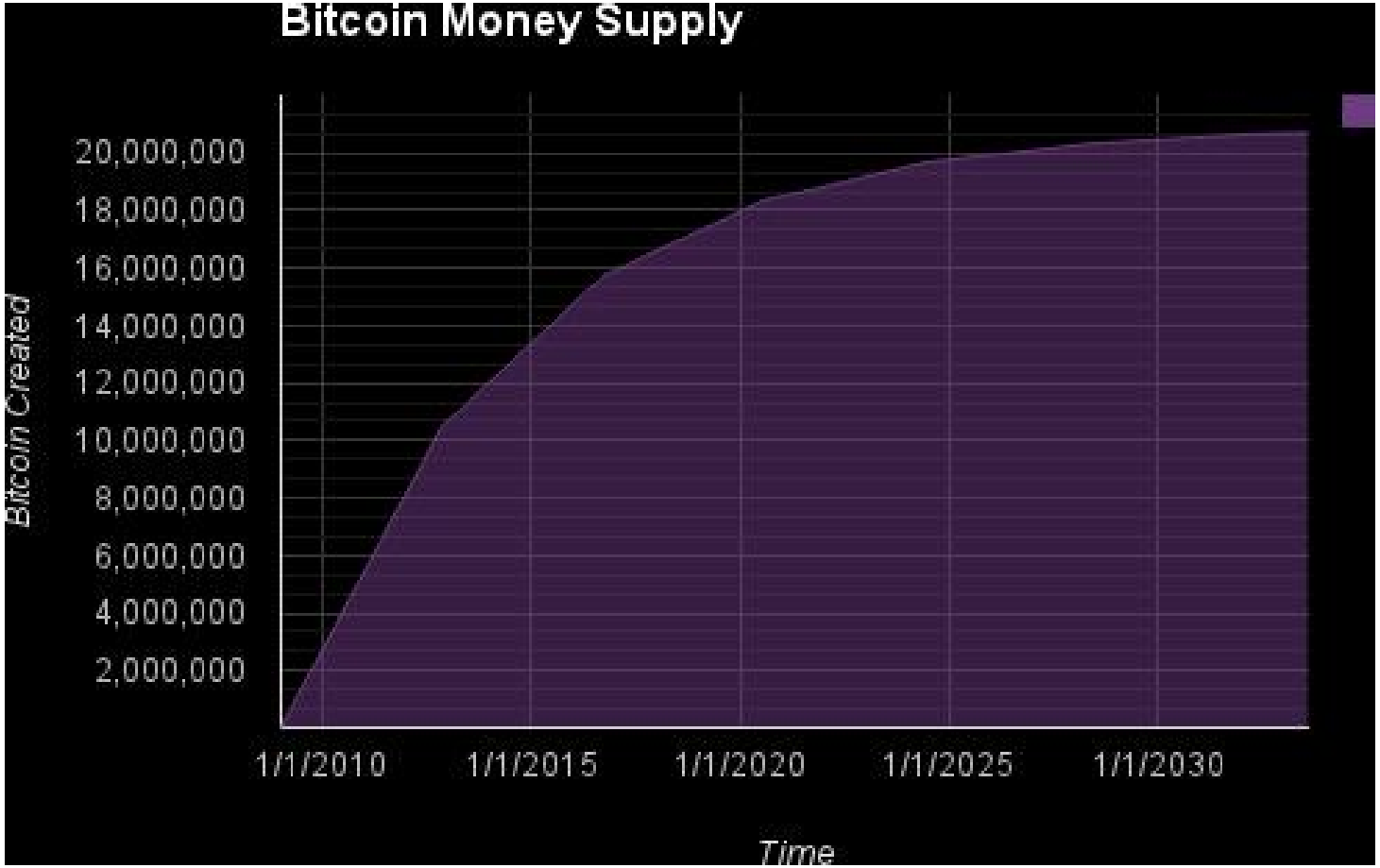
Bitcoins are “minted” during the creation of each block

- Block Generation Rate: 10 minutes

Reward Issuance at a Fixed and Diminishing Rate

- Each block contained 50 $BTC for the first 4 years of operation,
- Currency Issuance Rate is decreased by 50% every 210,000 blocks

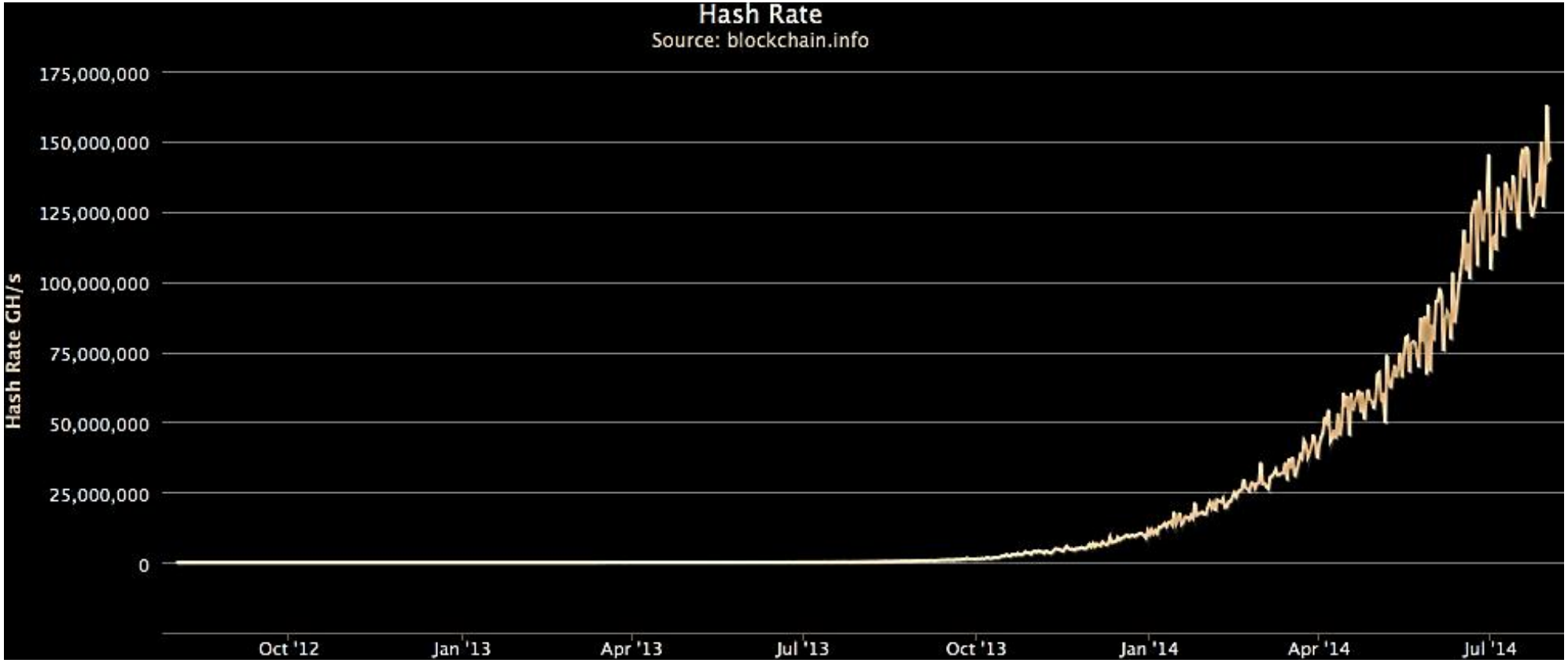
BitCoin Supply



Hash Rate in PoW Mining

- 2009
  - 0.5 MH/sec–8 MH/sec (16× growth)
- 2010
  - 8 MH/sec–116 GH/sec (14,500× growth)
- 2011
  - 16 GH/sec–9 TH/sec (562× growth)
- 2012
  - 9 TH/sec–23 TH/sec (2.5× growth)
- 2013
  - 23 TH/sec–10 PH/sec (450× growth)
- 2014
  - 10 PH/sec–150 PH/sec in August (15× growth)

Increasing Hash Rate in Mining



Increasing Hash Rate in Mining



Difficulty Target and Retargeting

Bitcoin blocks design to be generated every 10 minutes

- Bitcoin’ s Heartbeat
- Underpins the frequency of currency issuance
- Underpins the speed of transaction settlement

Increasing/Decreasing Hash Power May Change the Rate

- Computer Power Increase at a Rapid Pace
- Number of Mining Participants Change

Mining Difficulty Must be adjusted to these changes

Decentralized Retargeting

All nodes retarget the PoW difficulty every 2,016 blocks

$$\text{New Difficulty} = \text{Old Difficulty} * \frac{\text{Actual Time of Last 2016 Blocks}}{20160 \text{ minutes}}$$

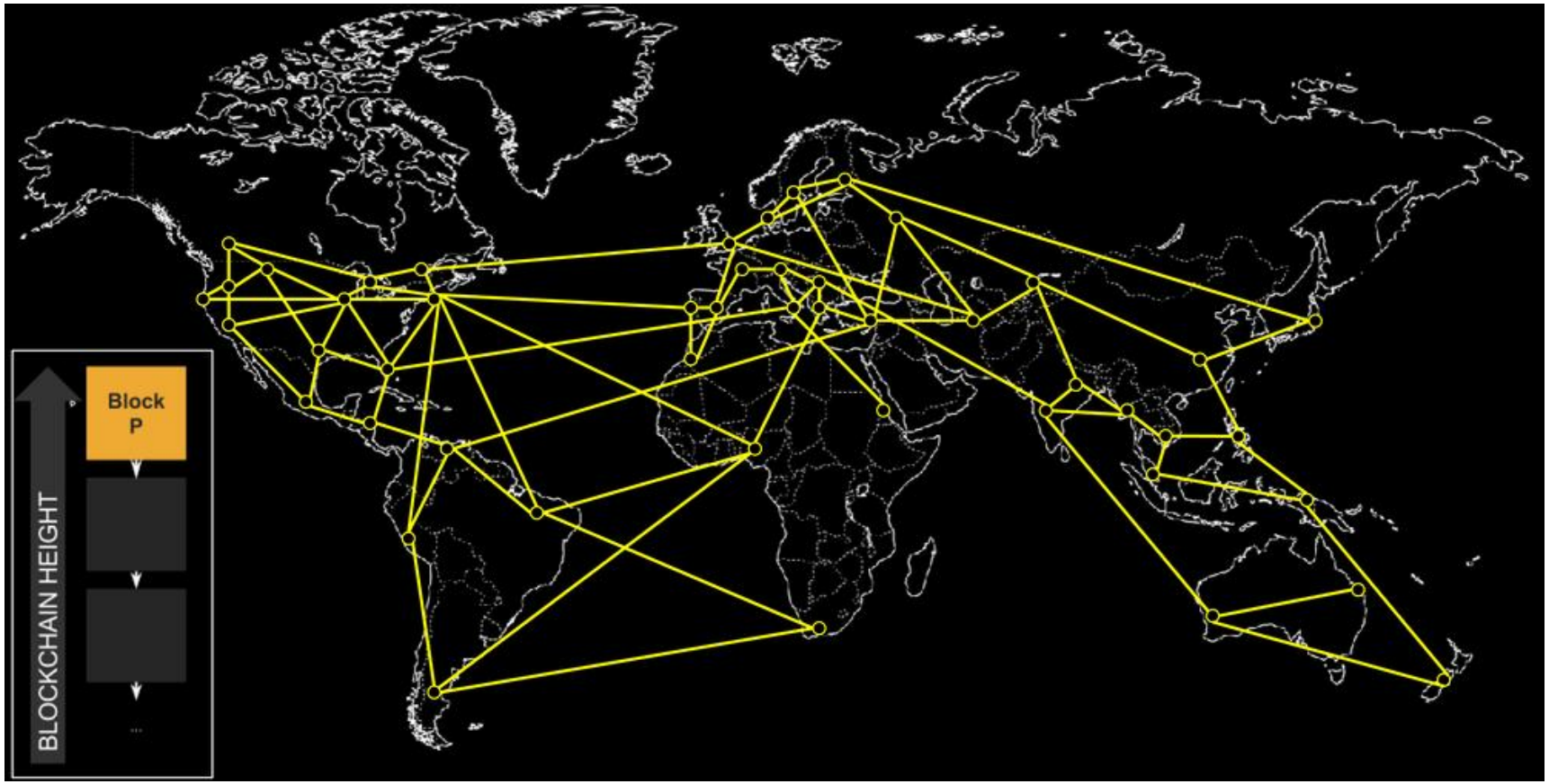
Maximum Retargeting Adjustment must be less than 4x per cycle

- to avoid extreme volatility in the difficulty

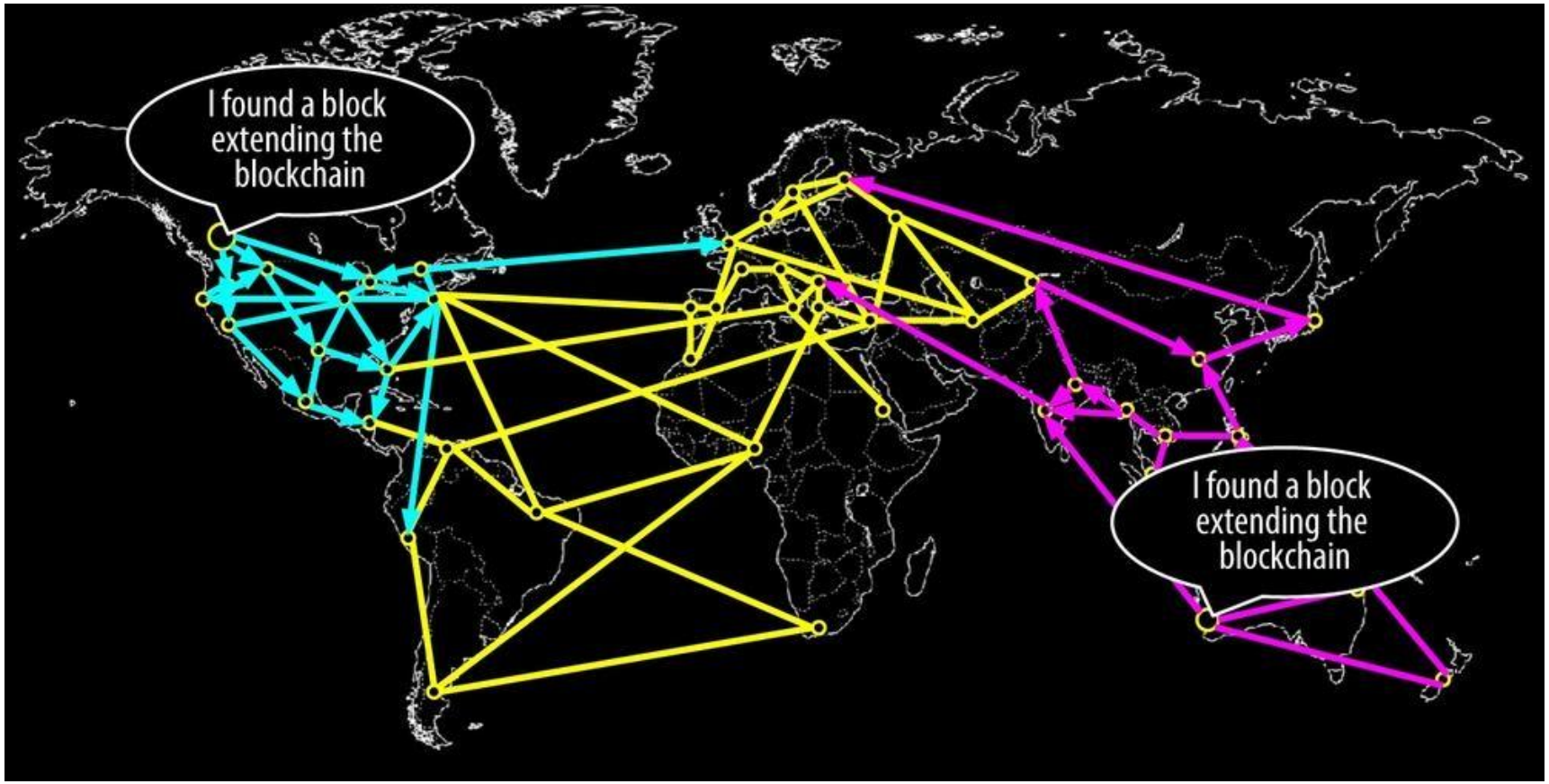
Difficulty in PoW Mining



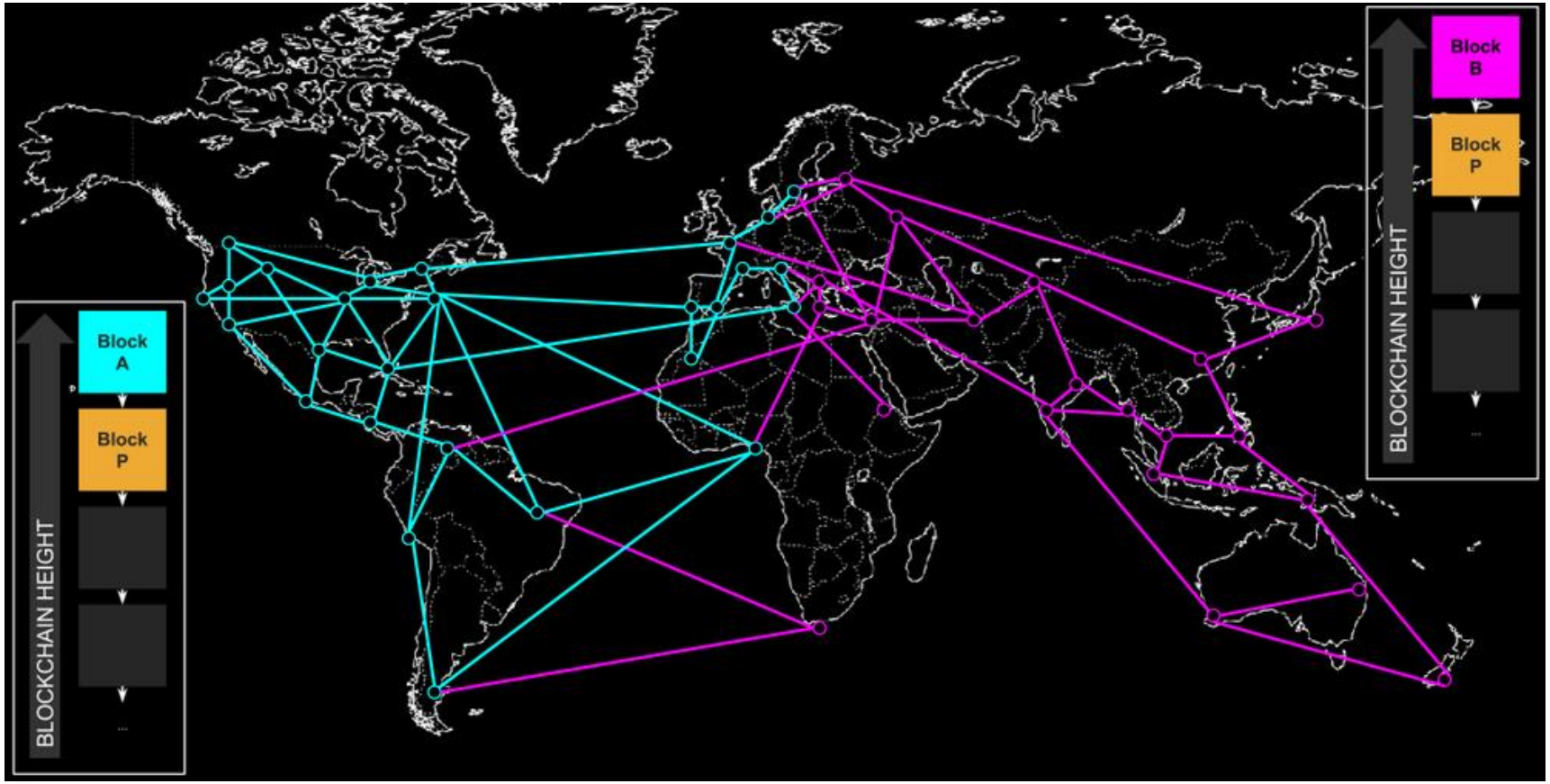
Mining Concurrency and Fork



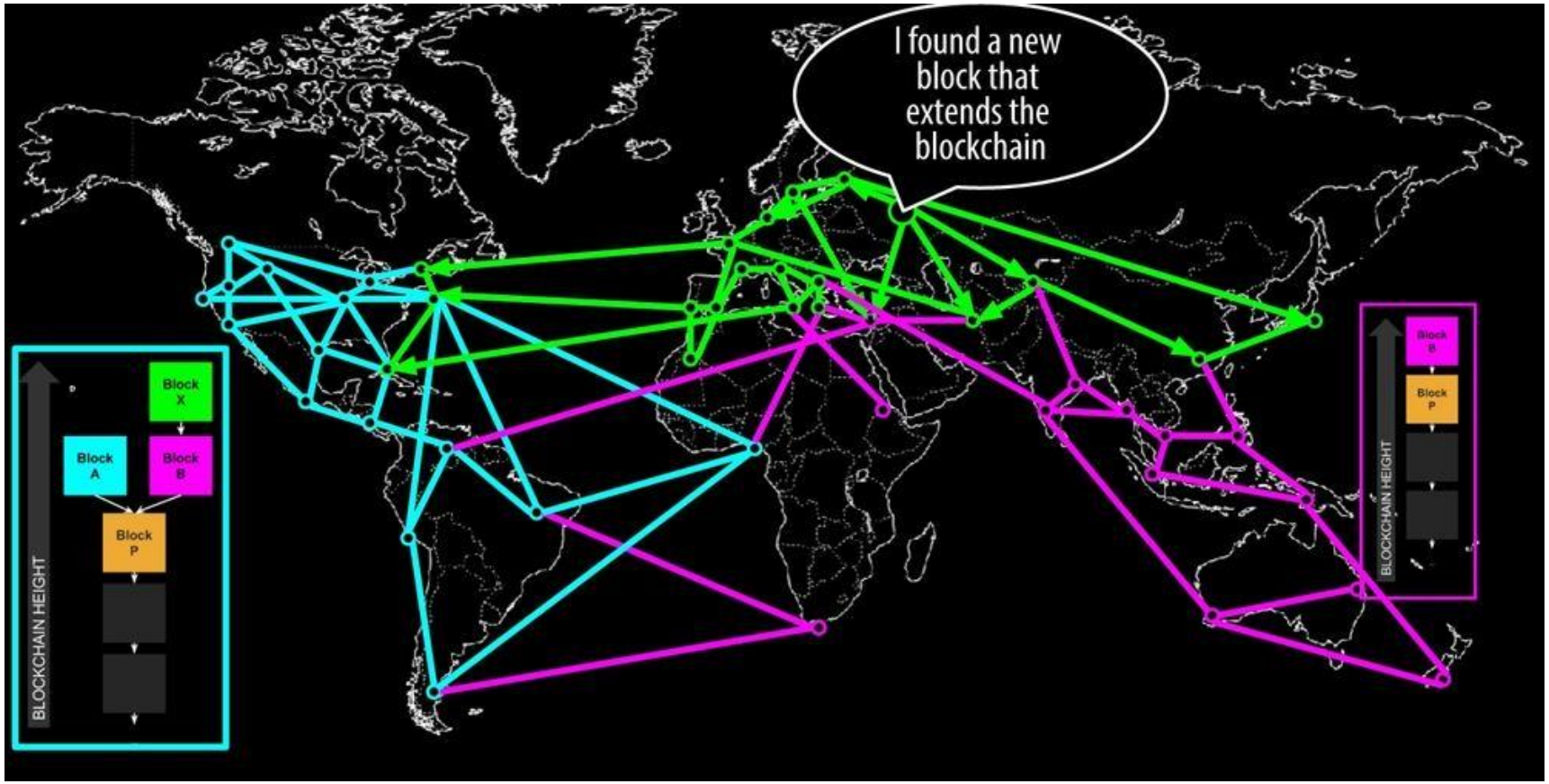
Mining Concurrency and Fork



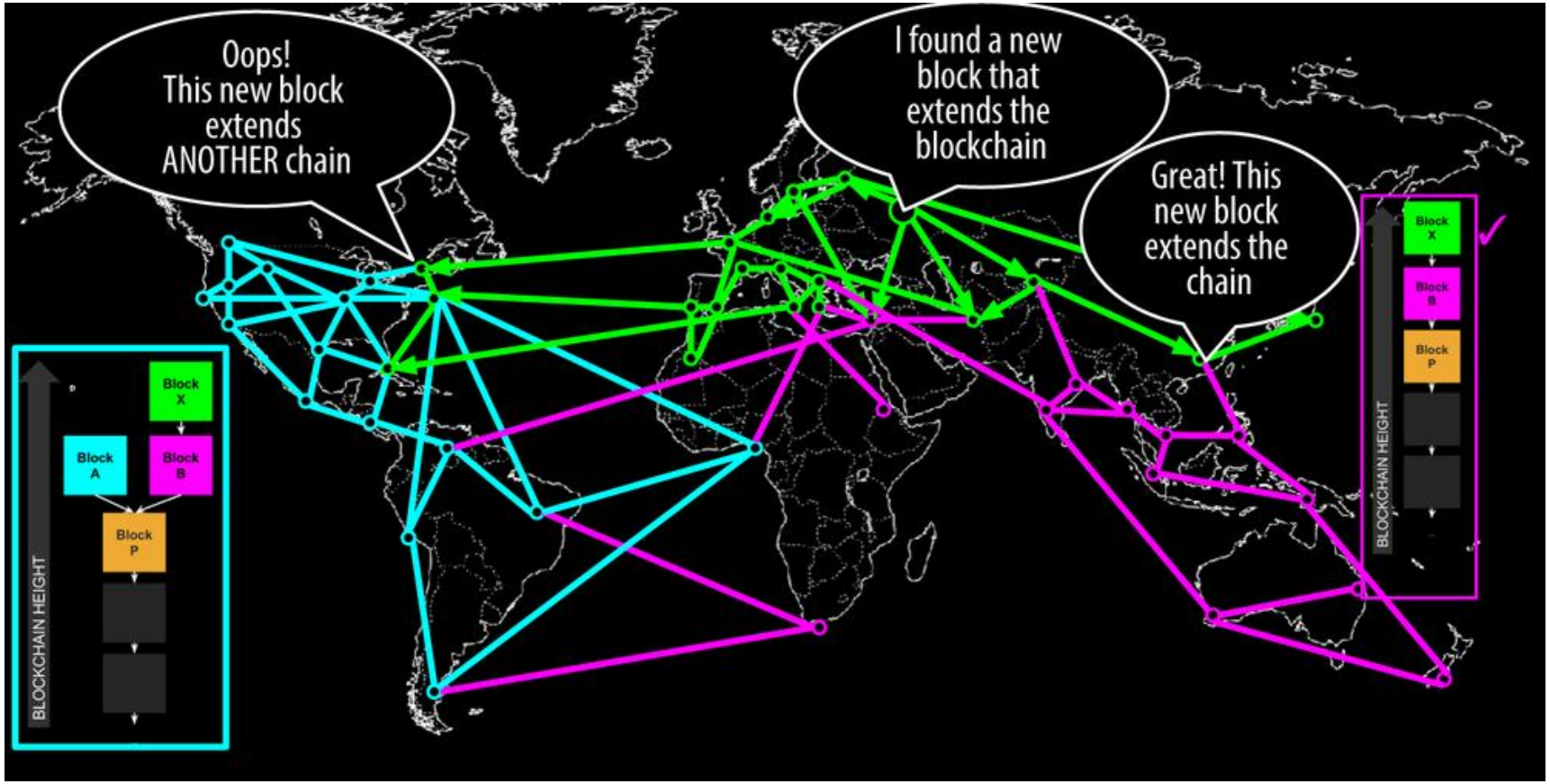
Mining Concurrency and Fork



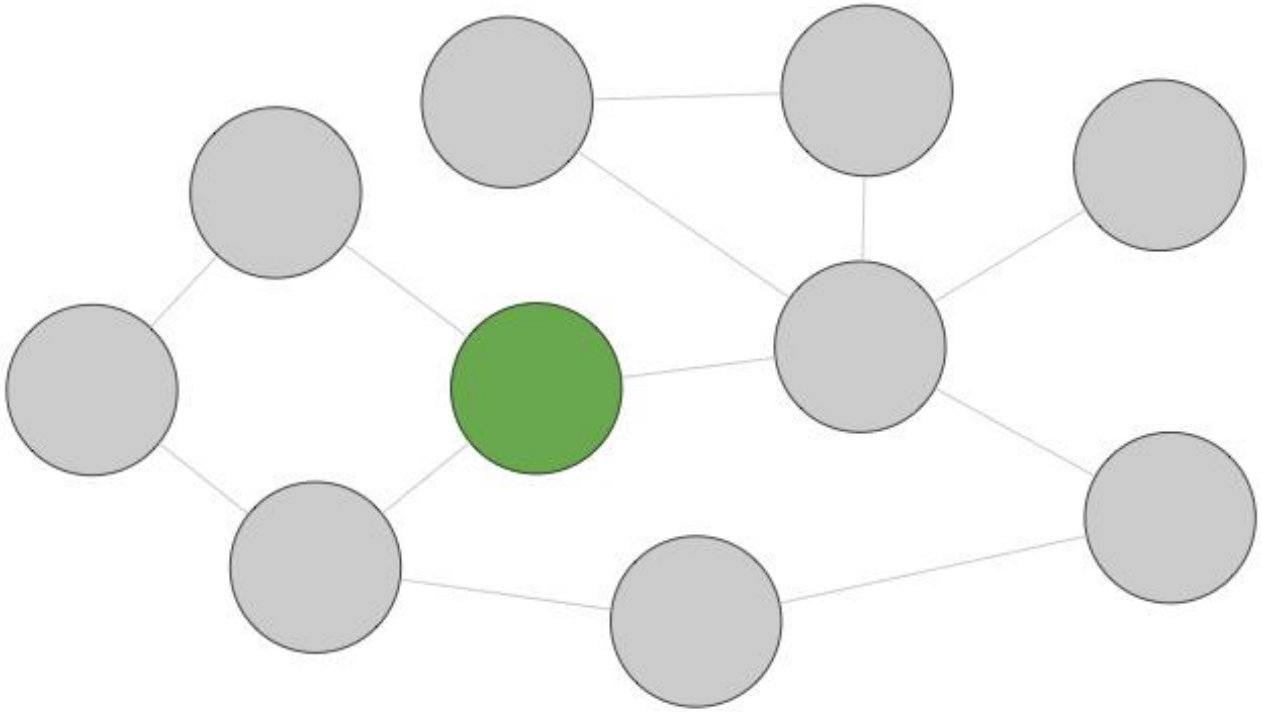
Mining Concurrency and Fork



Mining Concurrency and Fork



BitCoin Transaction Propagation



Empty Blocks Due to Delays in Block Propogation

- Block header and body are broadcasted separately
- header (~80 bytes) v.s. body (~ 1MB)
- 10 seconds gap between receiving header and body

- Miners' Deicisons:
  - Wait for the body and then mine the block
  - Mine an empty block to avoid transaction
- 0.75% empty blocks in 210,000 blocks when the memory pool is not empty since second halving

The Resulting Empty Blocks

788711	BTC.com	5,425	2023/05/08 08:06:50	1,883,704	6.25 + 4.55094627	0.00083889
788710	F2Pool	4,506	2023/05/08 08:03:44	1,776,160	6.25 + 5.3678926	0.00119128
788709	AntPool	5,503	2023/05/08 07:56:02	1,940,625	6.25 + 5.34502354	0.0009713
788708	AntPool	4,440	2023/05/08 07:53:43	1,705,264	6.25 + 5.87448005	0.00132309
788707	AntPool	4,952	2023/05/08 07:43:10	1,811,636	6.25 + 5.50692618	0.00111207
788706	ViaBTC	5,455	2023/05/08 07:31:32	1,906,934	6.25 + 5.06209725	0.00092798
788705	Foundry USA	5,149	2023/05/08 07:28:23	1,875,066	6.25 + 6.08940768	0.00118264
788704	F2Pool	5,016	2023/05/08 07:18:39	1,804,696	6.25 + 5.90042441	0.00117633
788703	ViaBTC	1	2023/05/08 07:16:40	260	6.25 + 0	0
788702	3LwoXDTo...p2LM	4,233	2023/05/08 07:16:26	1,685,311	6.25 + 6.52943977	0.00154251
788701	ViaBTC	4,906	2023/05/08 06:38:44	1,836,149	6.25 + 5.0725438	0.00103395
788700	Foundry USA	5,417	2023/05/08 06:35:32	1,900,540	6.25 + 6.33947083	0.0011703
788699	Foundry USA	4,835	2023/05/08 06:24:58	1,799,647	6.25 + 6.17870654	0.00127792
788698	AntPool	5,377	2023/05/08 06:19:17	1,883,537	6.25 + 6.08708589	0.00113206
788697	Foundry USA	4,682	2023/05/08 06:10:36	1,742,750	6.25 + 5.22415107	0.0011158
788696	Foundry USA	4,568	2023/05/08 06:09:36	1,754,221	6.25 + 5.78441609	0.0012663

14 seconds

Infamous BitCoin

- Sex, Drugs, etc.

6 Major Porn Sites Now Accept Bitcoin



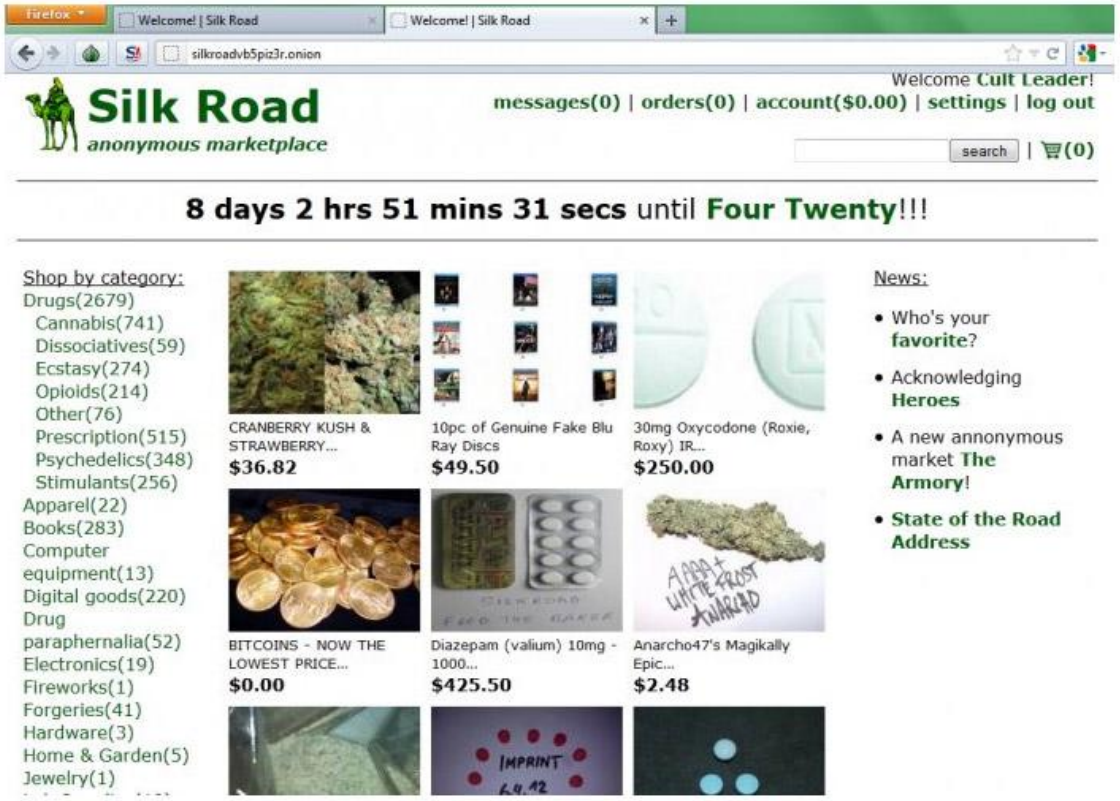
Bitcoin offers the cannabis industry an alternative to banks

Infamous BitCoin

- Scammers, Frauds, Money Laundering, etc.



Infamous BitCoin



A "dark web" site used to sell drugs, arms, and even murder-for-hire.

The FBI initially seized 26,000 bitcoins from accounts on Silk Road, worth approximately $3.6 million at the time

Infamous BitCoin

- Mysterious Identities

Craig Steven Wright claims to be Satoshi Nakamoto. Is he?

All latest update

Evaluating his claim will involve a multi-step paternity test

May 2nd 2016 | Online extra

Timekeeper

Like 2.2K Tweet



Craig Wright is not Satoshi Nakamoto

Craig Wright is not Satoshi Nakamoto. He wasn't Satoshi Nakamoto before or after Wired and Gizmodo suspected him to be last year, and he still isn't Satoshi Nakamoto after trying to reveal himself to be on his own blog and to The BBC, The Economist, GQ, Jon Matonis and Gavin Andresen.

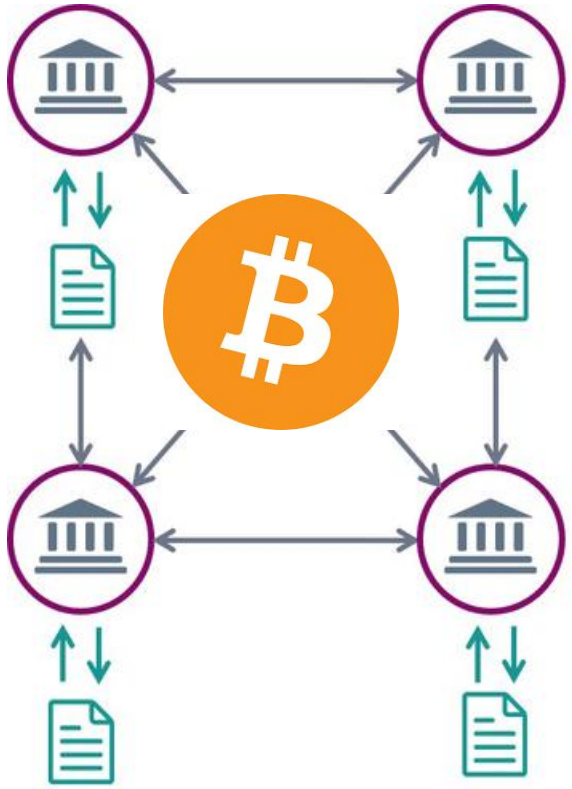
There is a long and fraught history in Bitcoin of claims and counterclaims about who Satoshi is, and one would think that lessons had been learned and a high standard would be set for subsequent claims regarding Satoshi Nakamoto. The



🤐 *A Homeland Security agent, at a conference attended by the SEC, said the US government knows who "Satoshi Nakamoto" is and that it is a group of 4 people!* 🤹

— CoinHub (@CoinHubCC) July 16, 2022

Bitcoin



Distributed Ledger



Consensus

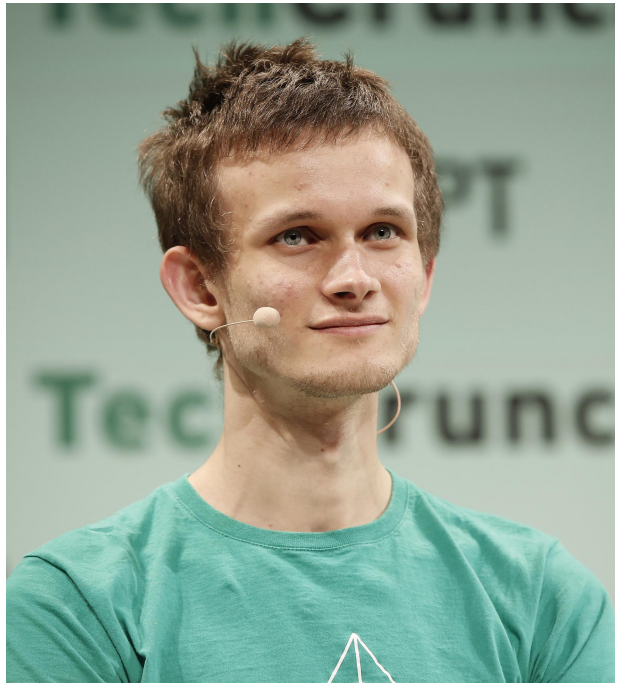
Not Utilized

Ethereum: *Blockchain 2.0*

- Not a Payment Alternative Like BitCoin
- A Platform for Decentralized Smart Contracts

Smart Contract			
Immutable On-Chain Code	Invoked by Contract Users, Executed by Miners	Driven by Ether ($ETH)	Solidity: Turing-Complete Language

- Initial Release Date: July 30, 2015
- PoW Consensus Model (Jul. 2015 - Sep. 2022)



@VitalikButerin
Russian-Canadian

Ethereum White Paper



Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform.
By Vitalik Buterin (2014).

When Satoshi Nakamoto first set the Bitcoin blockchain into motion in January 2009, he was simultaneously introducing two radical and untested concepts. The first is the "bitcoin", a decentralized peer-to-peer online currency that maintains a value without any backing, intrinsic value or central issuer. So far, the "bitcoin" as a currency unit has taken up the bulk of the public attention, both in terms of the political aspects of a currency without a central bank and its extreme upward and downward volatility in price. However, there is also another, equally important, part to Satoshi's grand experiment: the concept of a proof of work-based blockchain to allow for public agreement on the order of transactions. Bitcoin as an application can be described as a first-to-file system: if one entity has 50 BTC, and simultaneously sends the same 50 BTC to A and to B, only the transaction that gets confirmed first will process. There is no intrinsic way of determining from two transactions which came earlier, and for decades this stymied the development of decentralized digital currency. Satoshi's blockchain was the first credible decentralized solution. And now, attention is rapidly starting to shift toward this second part of Bitcoin's technology, and how the blockchain concept can be used for more than just money.

Ethereum Yellow Paper

ETHEREUM: A SECURE DECENTRALISED GENERALISED TRANSACTION LEDGER

BYZANTIUM VERSION 69351d5 - 2018-12-10

DR. GAVIN WOOD
FOUNDER, ETHEREUM & PARITY
GAVIN@PARITY.IO

Abstract. The blockchain paradigm when coupled with cryptographically-secured transactions has demonstrated its utility through a number of projects, with Bitcoin being one of the most notable ones. Each such project can be seen as a simple application on a decentralised, but singleton, compute resource. We can call this paradigm a transactional singleton machine with shared-state.

Ethereum implements this paradigm in a generalised manner. Furthermore it provides a plurality of such resources, each with a distinct state and operating code but able to interact through a message-passing framework with others. We discuss its design, implementation issues, the opportunities it provides and the future hurdles we envisage.

1. Introduction

With ubiquitous internet connections in most places of the world, global information transmission has become incredibly cheap. Technology-rooted movements like Bitcoin have demonstrated through the power of the default, consensus mechanisms, and voluntary respect of the social contract, that it is possible to use the internet to make a decentralised value-transfer system that can be shared across the world and virtually free to use. This system can be said to be a very specialised version of a cryptographically secure, transaction-based state machine. Follow-up systems such as Namecoin adapted this original "currency application" of the technology into other applications albeit rather simplistic ones.

Ethereum is a project which attempts to build the generalised technology; technology on which all transaction-based state machine concepts may be built. Moreover it is often lacking, and plain old prejudices are difficult to shake.

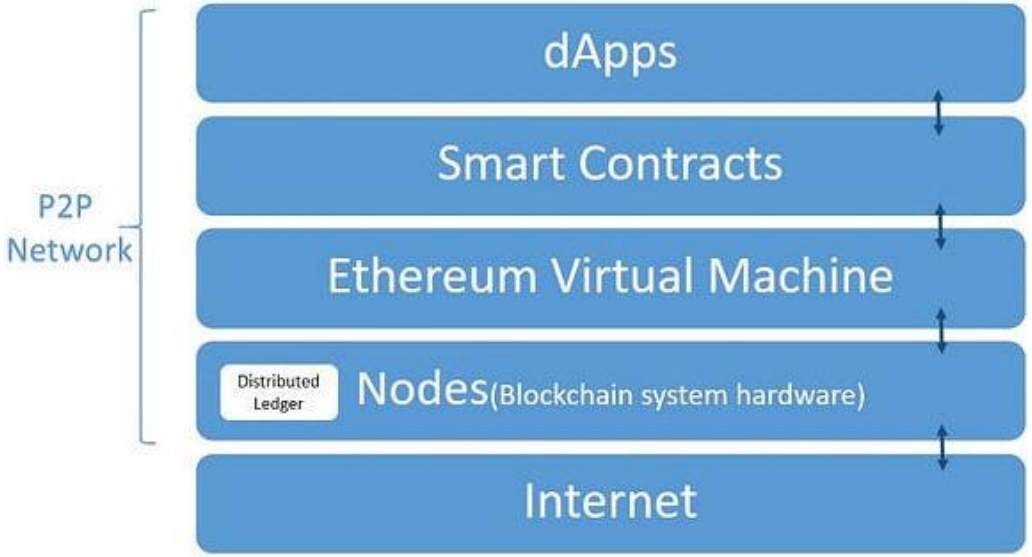
Overall, we wish to provide a system such that users can be guaranteed that no matter with which other individuals, systems or organisations they interact, they can do so with absolute confidence in the possible outcomes and how those outcomes might come about.

1.2. **Previous Work.** Buterin [2013a] first proposed the kernel of this work in late November, 2013. Though now evolved in many ways, the key functionality of a blockchain with a Turing-complete language and an effectively unlimited inter-transaction storage capability remains unchanged.

Dwork and Naor [1992] provided the first work into the usage of a cryptographic proof of computational expenditure ("proof-of-work") as a means of transmitting a value signal over the Internet. The value-signal was utilised here

Ethereum: World Computer

Ethereum is powered by the **Ethereum Virtual Machine** which allows smart contracts to run on a decentralized blockchain.





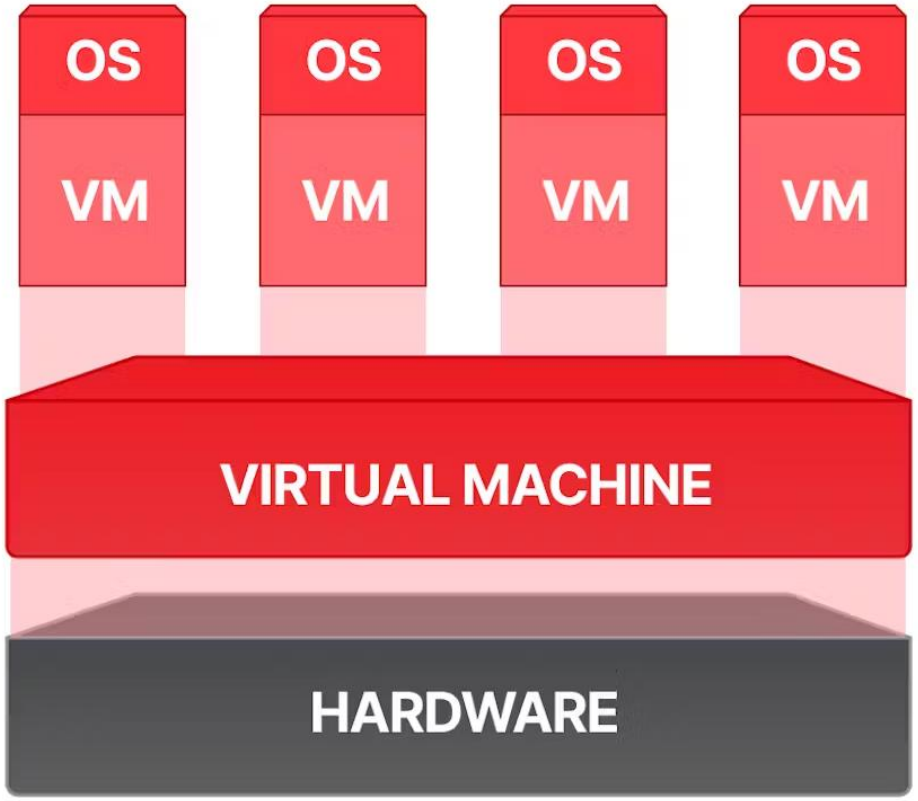
Ethereum Virtual Machine (EVM): Definition

The Ethereum Virtual Machine is the **runtime environment** for smart contracts in Ethereum.

- EVM is a Virtual Machine …
- EVM is a State Machine …
- EVM is a Quasi-Turing complete machine…

Ethereum Virtual Machine (EVM): Virtual Machine

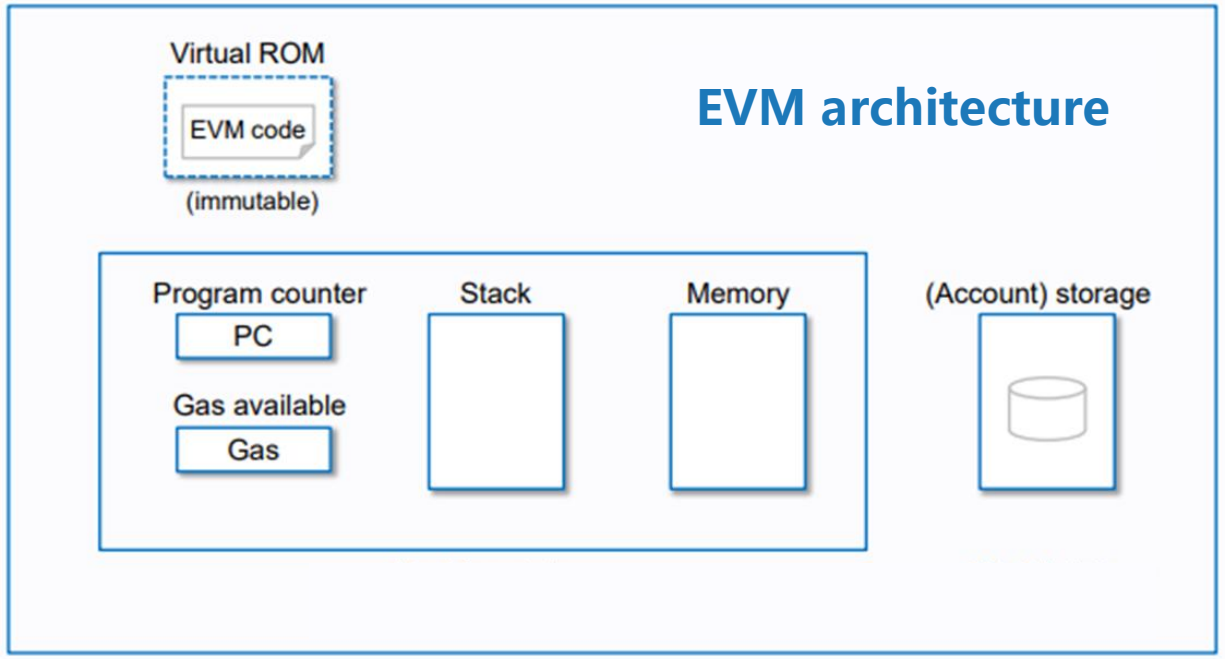
A **Virtual Machine** is a program that simulates the operation of a computer by isolating its real components and operating system.



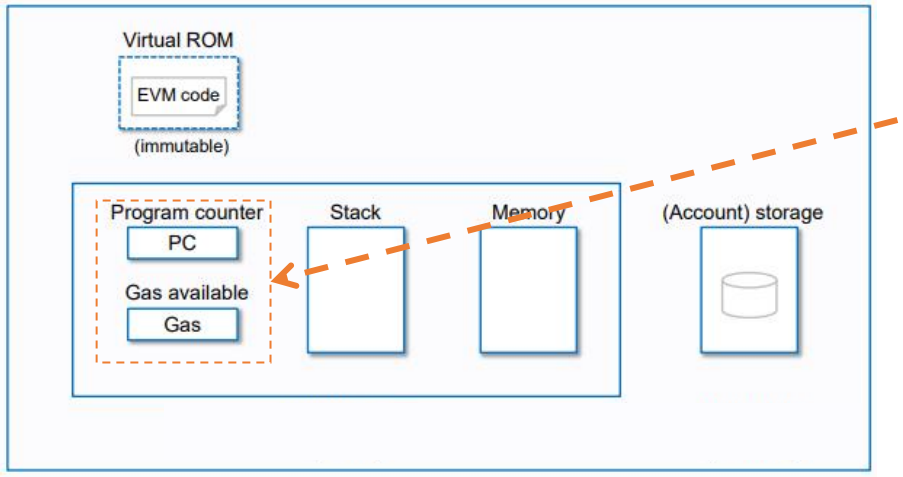


Ethereum Virtual Machine (EVM): Architecture

EVM is a Virtual Machine which means it can run on multiple hardware irrespective of the underlying platform/OS with a simple stack-based architecture.



Ethereum Virtual Machine (EVM): Architecture



The EVM uses the program counter to keep track of the execution flow of the smart contract, and gas to manage the computational resources each contract is allowed to use.

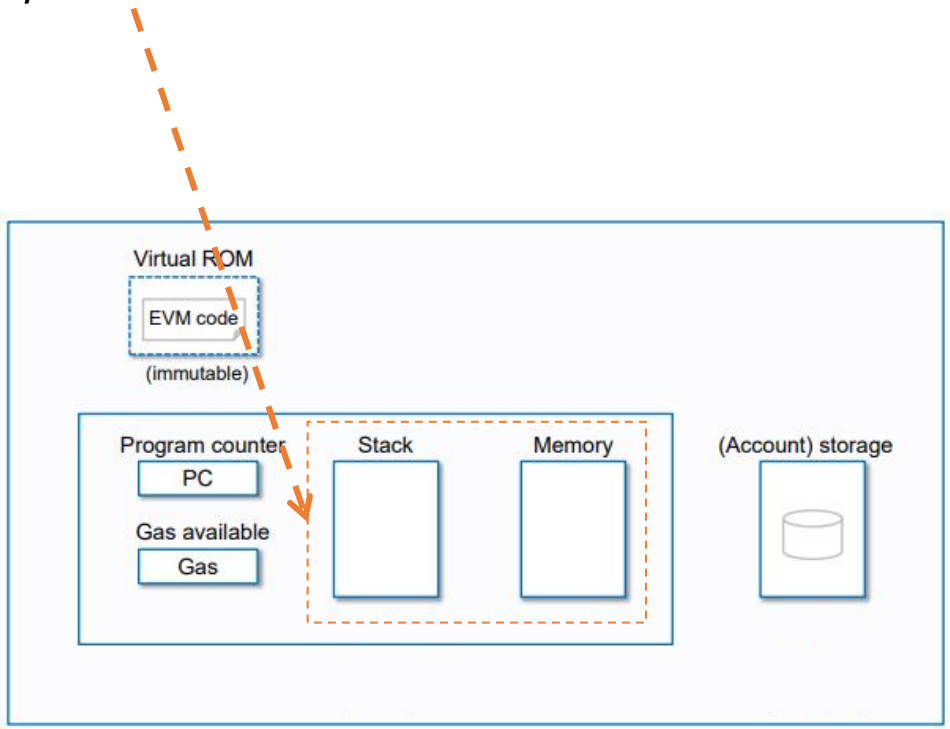
Appendix G. Fee Schedule

The fee schedule G is a tuple of 31 scalar values corresponding to the relative costs, in gas, of a number of abstract operations that a transaction may effect.

Name	Value	Description*
G_{zero}	0	Nothing paid for operations of the set W_{zero}.
G_{base}	2	Amount of gas to pay for operations of the set W_{base}.
$G_{verylow}$	3	Amount of gas to pay for operations of the set $W_{verylow}$.
G_{low}	5	Amount of gas to pay for operations of the set W_{low}.
G_{mid}	8	Amount of gas to pay for operations of the set W_{mid}.
G_{high}	10	Amount of gas to pay for operations of the set W_{high}.
$G_{extcode}$	700	Amount of gas to pay for operations of the set $W_{extcode}$.
$G_{balance}$	400	Amount of gas to pay for a BALANCE operation.

Ethereum Virtual Machine (EVM): Architecture

In the EVM, the stack is used for immediate computations, while memory is a more flexible space for storing data during execution.

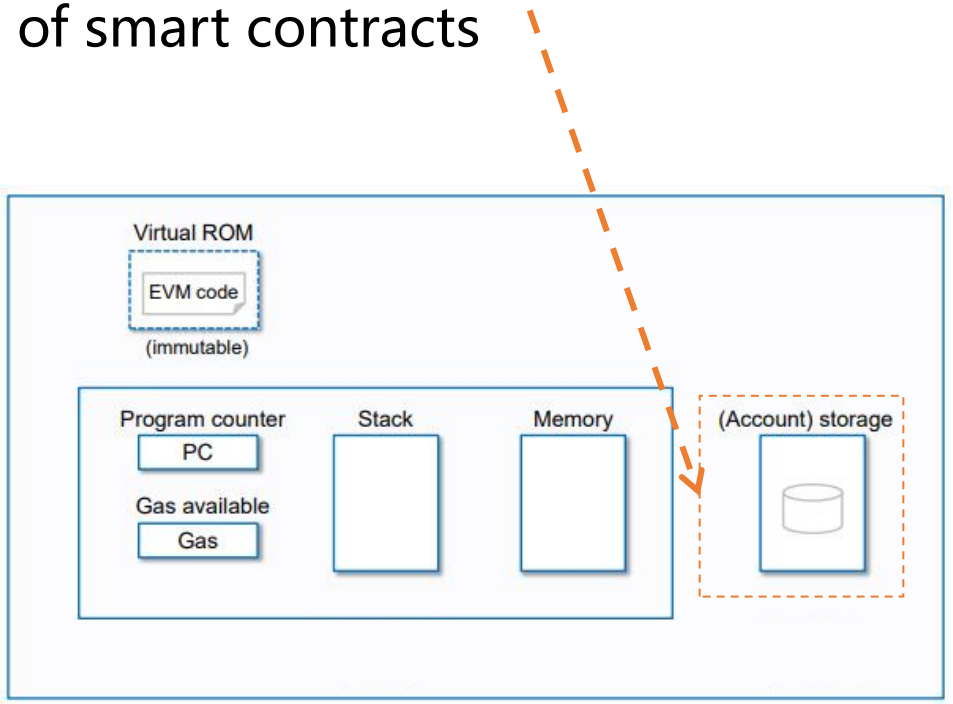


- **Stack:** Temporary storage for executing operations, similar to a CPU's stack.
  - All operation are performed on the stack.
- **Memory:** Short-term storage, like RAM, holding data that is needed by the stack or for temporary storage during execution.
  - Memory is linear and can be addressed at byte level.
  - All locations in memory are well-defined initially as zero

Ethereum Virtual Machine (EVM): Architecture

In the EVM, account storage is where the persistent data of smart contracts is kept

- **Storage:** The **long-term** storage for each account on the Ethereum network, like a hard drive in a traditional computer.
  - **Ether Balance:** The amount of Ether held by the account.
  - **Contract Code:** The compiled smart contract code if the account is a contract.
  - **Contract State:** Data variables and their current values for the smart contract.
  - **Nonce:** A counter indicating the number of transactions sent from an account or, for contract accounts, the number of contracts created.

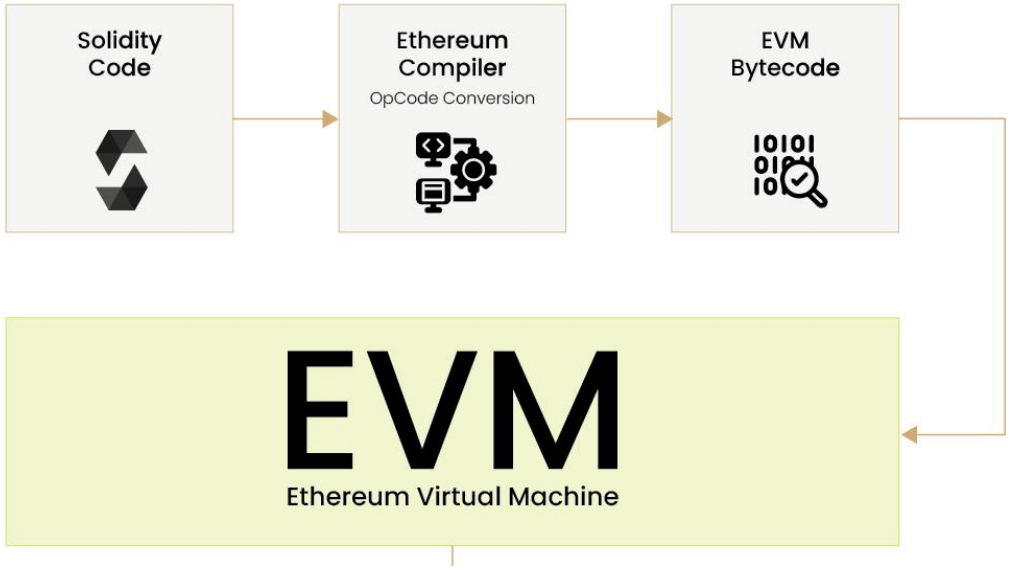


What Are Smart Contracts?

Definition:

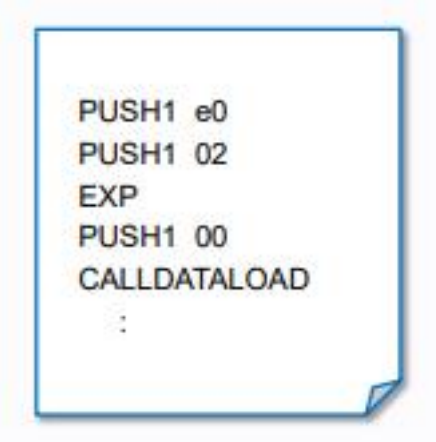
Smart contracts are programs deployed on the Ethereum blockchain that have associated storage and can execute arbitrarily complex logic.

- Smart Contracts are written in higher level programming languages like Solidity, Vyper, etc. and are compiled down to EVM bytecode, which is a Turing complete low level programming language.



EVM Architecture: Basic Knowledge Supplement

Assembly View



- This is a more human-readable form of code, showing the opcodes (operations) that the EVM will execute.
- The opcodes listed are assembly language instructions for the EVM.
  - PUSH1 is an instruction to push a 1-byte value onto the stack.
  - EXP is an instruction to perform exponential calculation.
  - CALLDATALOAD is an instruction to load data from the call payload.

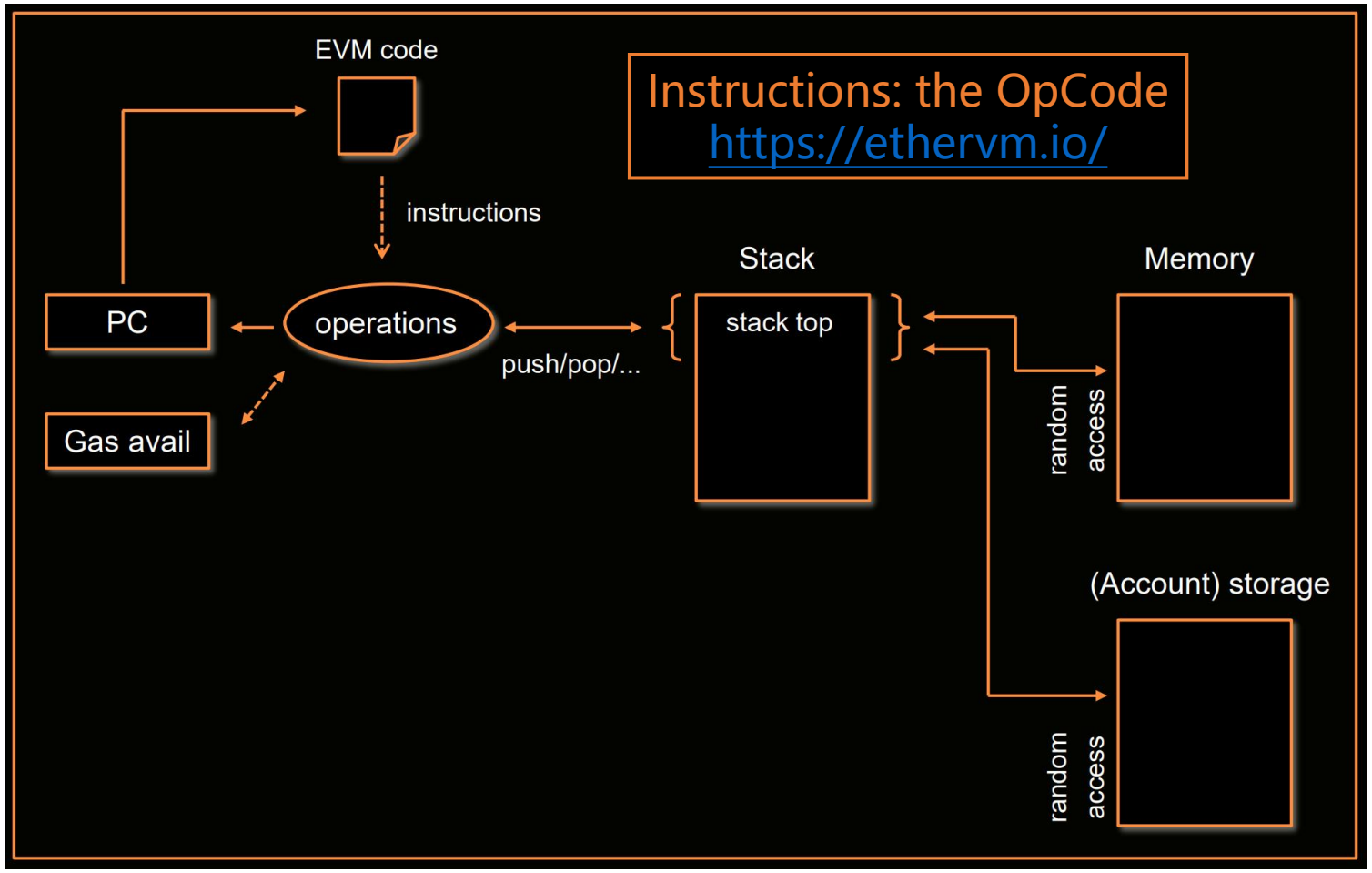
Bytecode View:



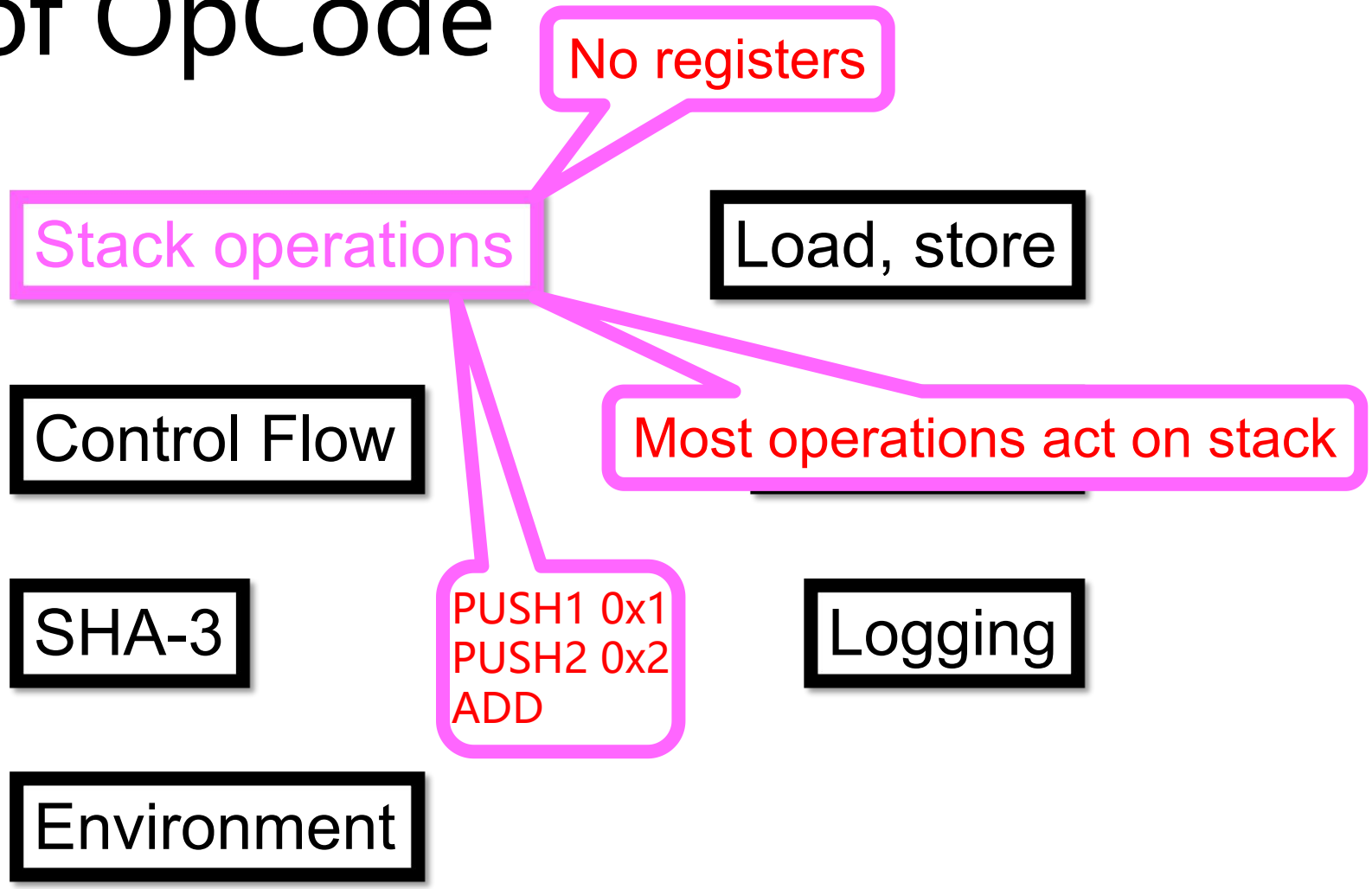
- The compiled form of the smart contract code that the EVM directly executes.
- It's a hexadecimal representation of the contract's assembly code.
- This code is what gets deployed to the Ethereum blockchain.

■The hexadecimal sequence (0x60e060020a600035...) corresponds to the raw bytes that make up the contract.

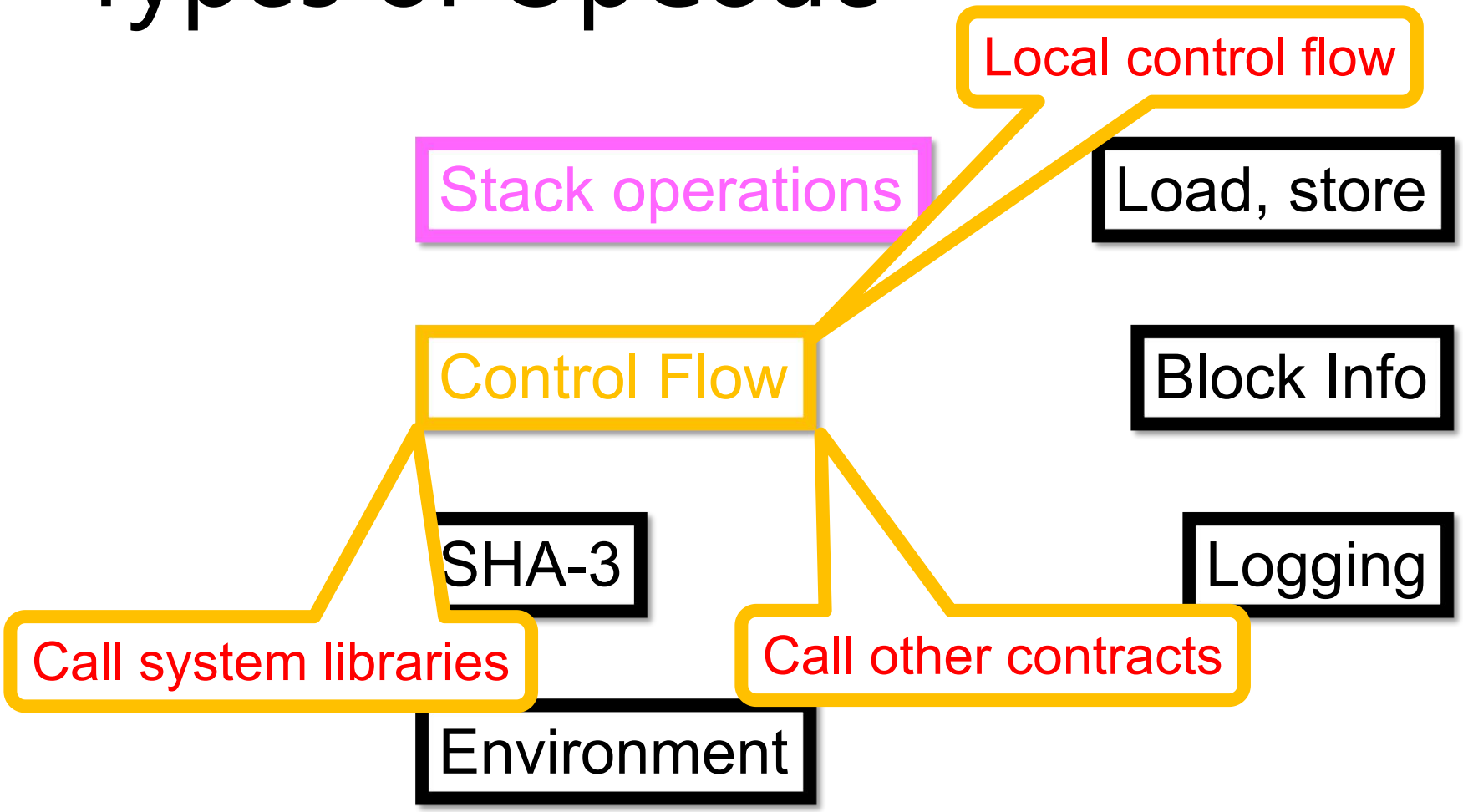
EVM Execution Model



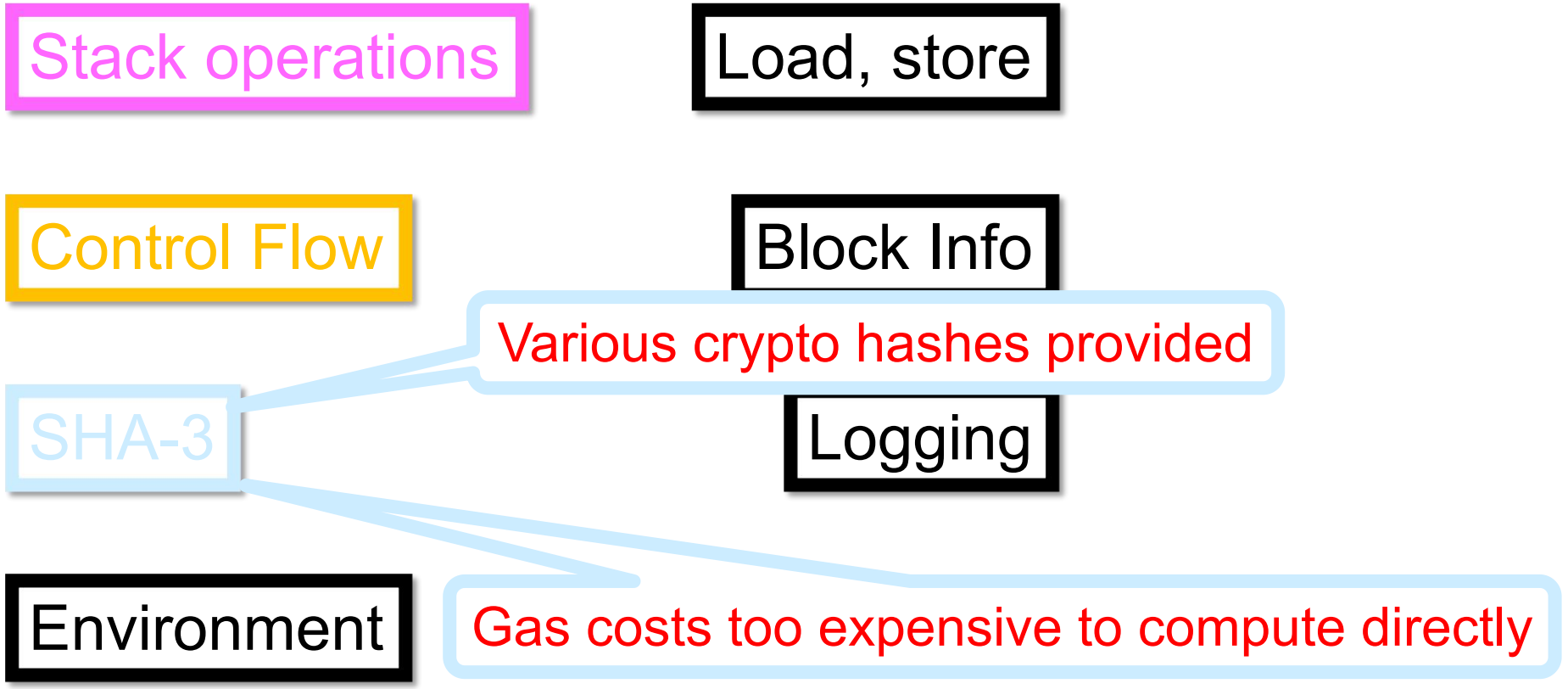
Types of OpCode



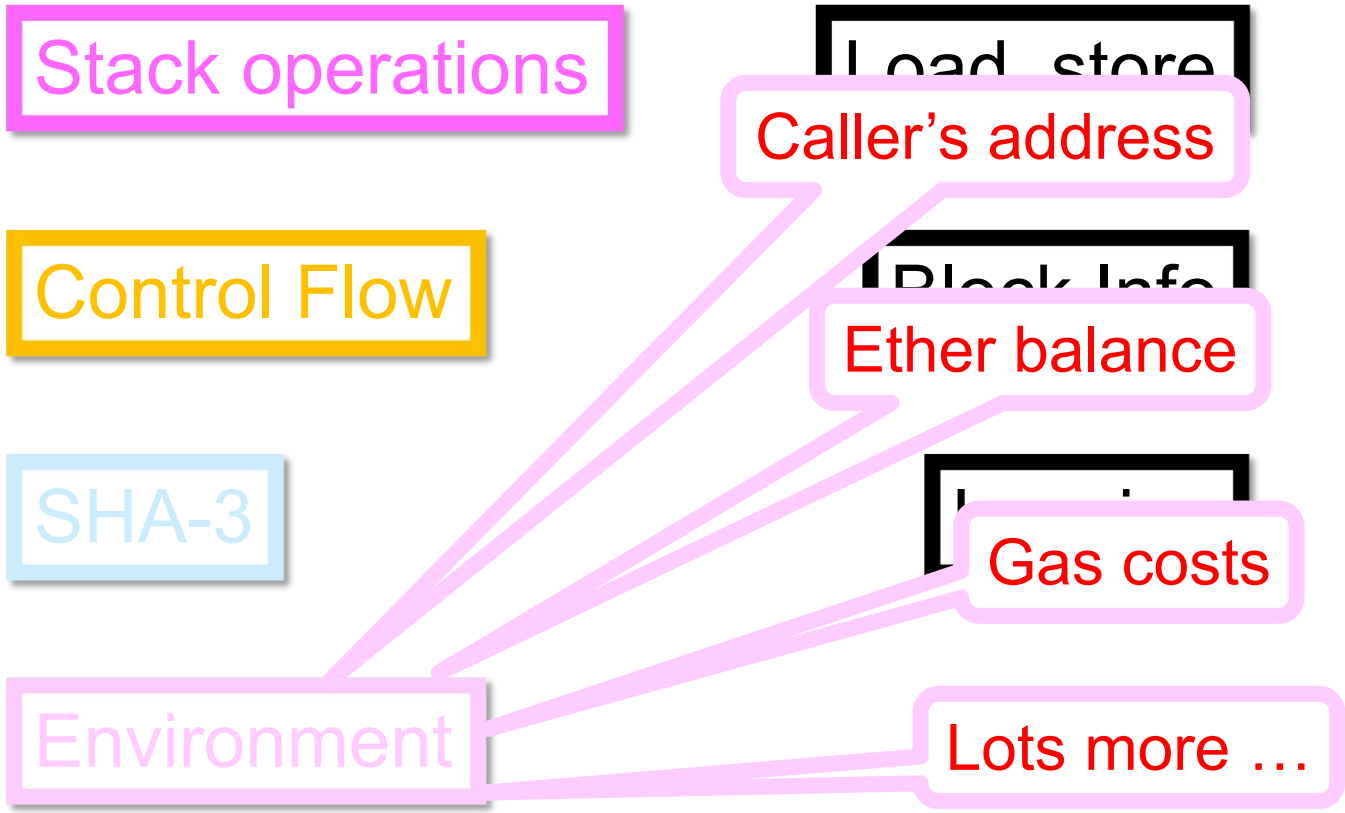
Types of OpCode



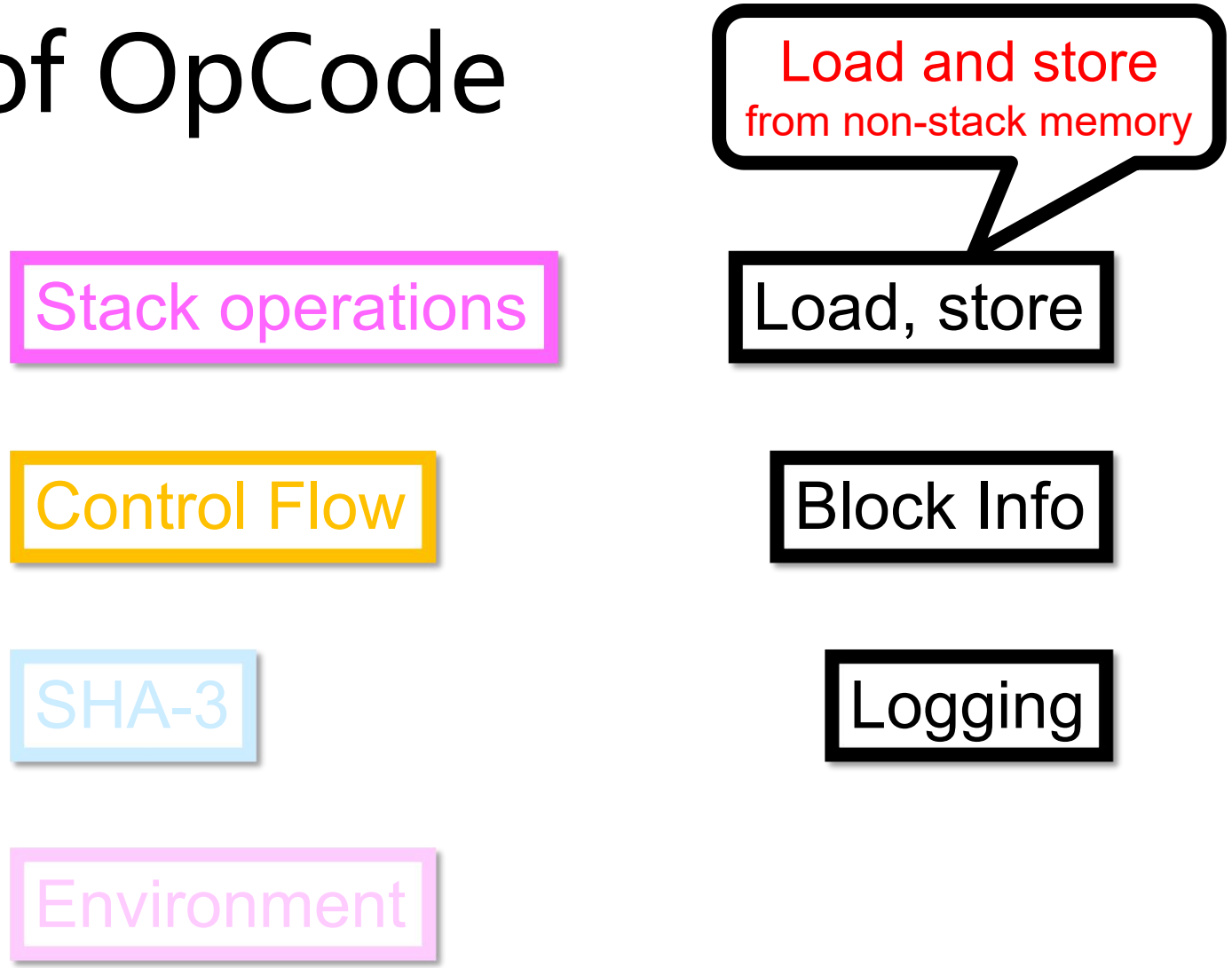
Types of OpCode



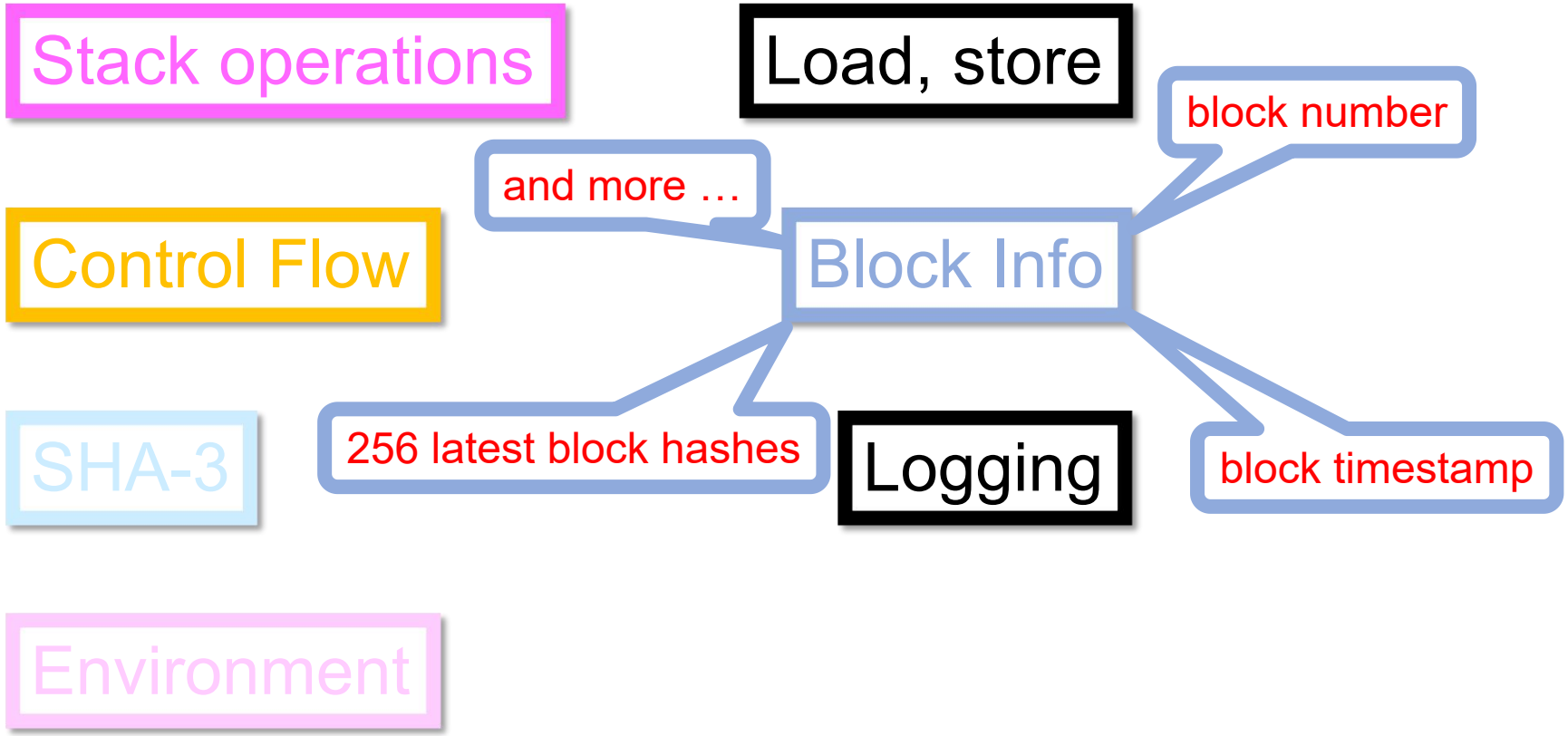
Types of OpCode



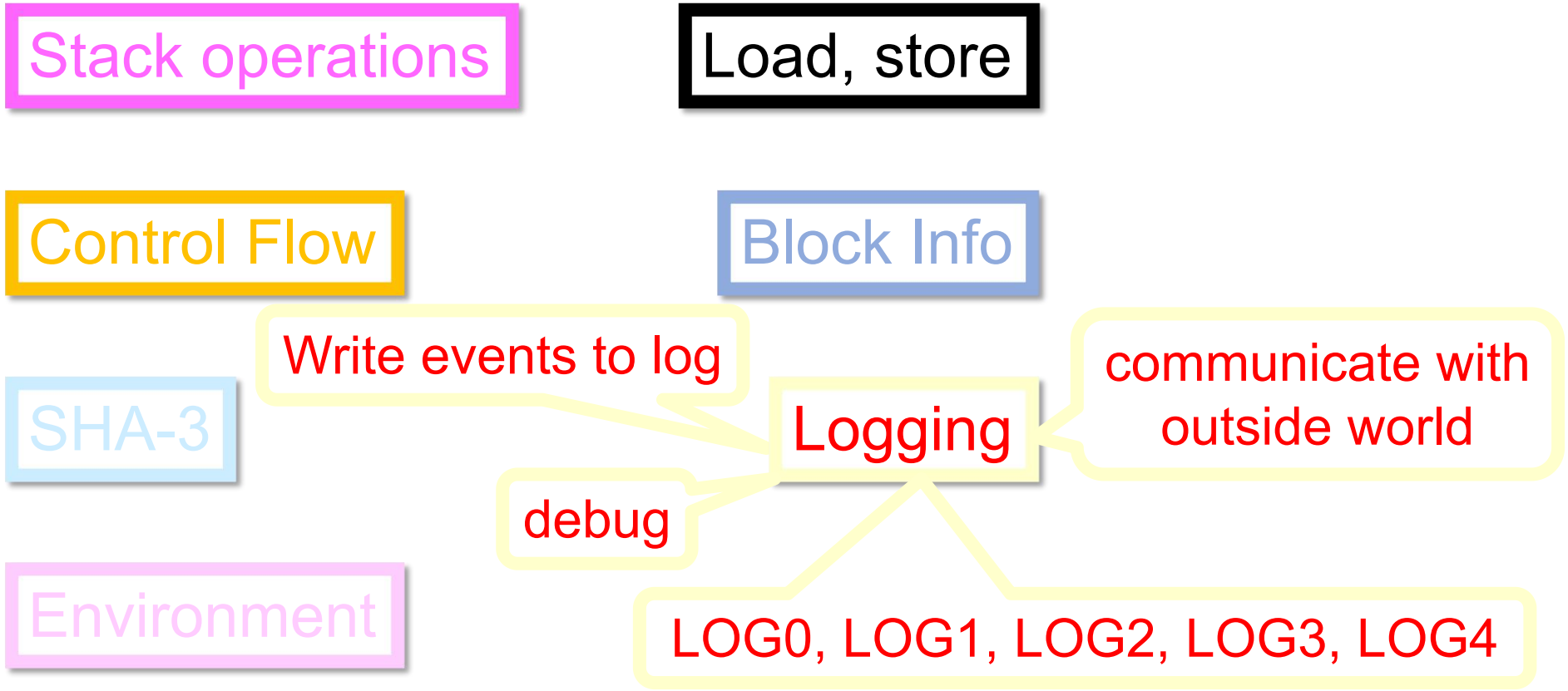
Types of OpCode



Types of OpCode



Types of OpCode



Extended Readings

- Ethereum Yellow Paper
  - https://github.com/ethereum/yellowpaper
- EVM OpCode
  - https://ethervm.io/

Smart Contracts Function

```
destination: public(address)

@external
@payable
def __default__():
    assert self.balance >= 10 * 10**18,
"Insufficient balance to transfer"
    send(self.destination, 10 * 10**18)

@external
def withdraw():
    send(msg.sender, self.balance)
```

Programming Language in Vyper

```
[2] PUSH2 0x00ec
[5] PUSH2 0x000f
[7] PUSH1 0x00
[8] CODECOPY
[11] PUSH2 0x00ec
[13] PUSH1 0x00
[14] RETURN
[15] PUSH0 0x
[16] CALLDATALOAD
[18] PUSH1 0xe0
[19.....
[256] STOP
[257] LOG1
[264] PUSH6 0x767970657283
[265] STOP
[266] SUB
[267] EXP
[268] STOP
[269] SGT
```

Assembly View (Opcode)

```
0x6100ec61000f6000396100ec6000f35f3560e01c6002600382
0660011b6100e601601e395f51565b63b269681d811861005f5
7346100e2575f5460405260206040f361005f565b633ccfd60b8
11861005f57346100e2575f5f5f5f47335ff1156100e257006100
5f5661005f565b5b678ac7230489e800004710156100ca57602
06040527f496e7375666696369656e742062616c616e636520
746f207472616e736657260605260405060405180606001601
f825f03163682375050630 8c379a05f526020602052601f19601
f60405101160440 1601cfd5b5f5f5f5f678ac7230489e800005f
545ff1156100e257005b5f80fd0037005e00188418ec810600a1
6576797065728300030a0013
```

Bytecode

Ethereum Virtual Machine (EVM): Definition

The Ethereum Virtual Machine is the **runtime environment** for smart contracts in Ethereum.

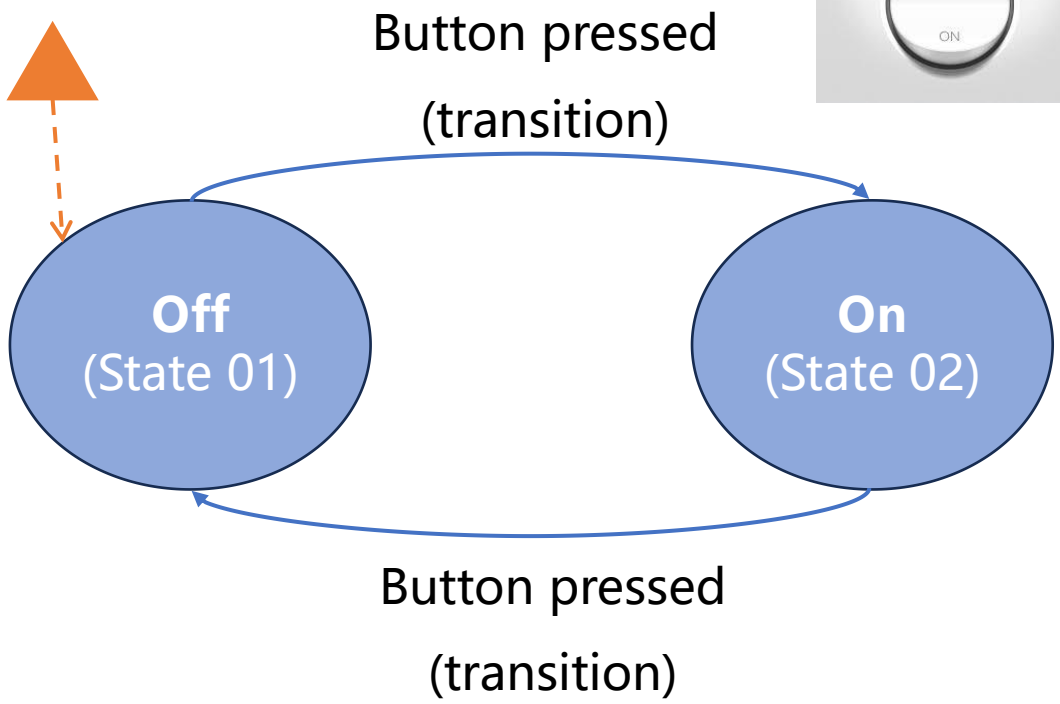
- EVM is a Virtual Machine …
- **EVM is a State Machine …**
- EVM is a Quasi-Turing complete machine…

Ethereum Virtual Machine (EVM): Definition

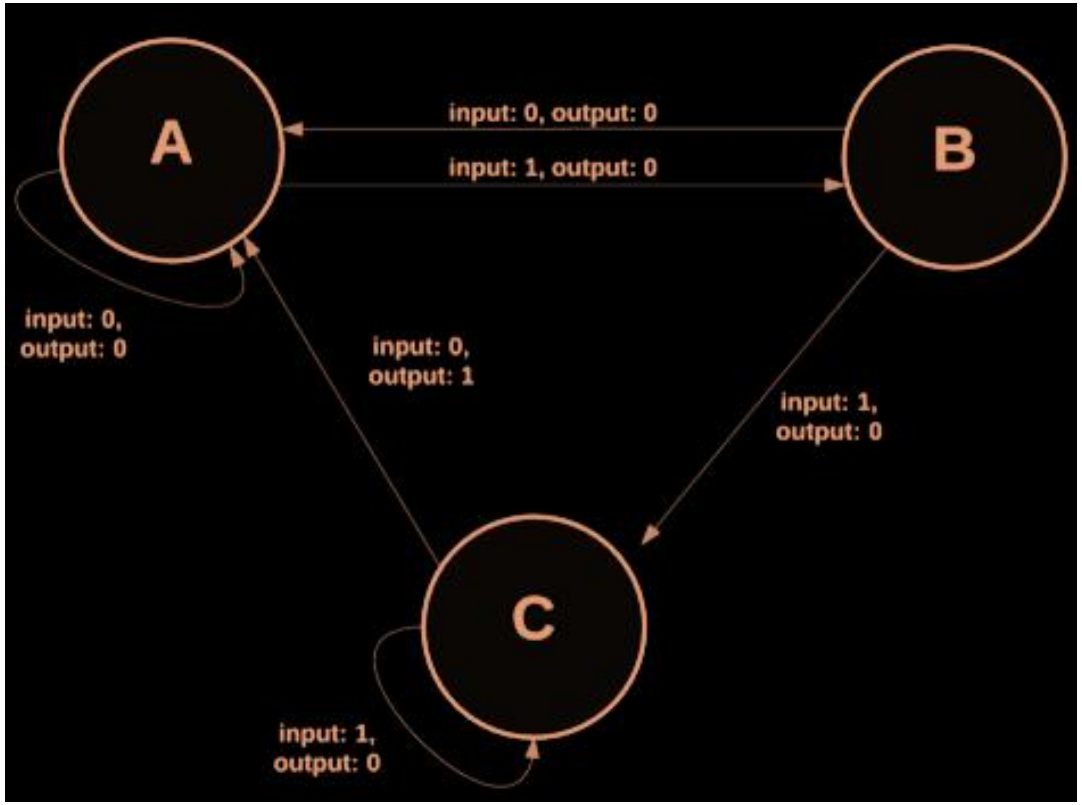
State Machine: A state machine is a mathematical abstraction used to design algorithms.

- A state machine reads a set of inputs and changes to a different state based on those inputs.
- A state is a description of the status of a system waiting to execute a transition





State Transition System

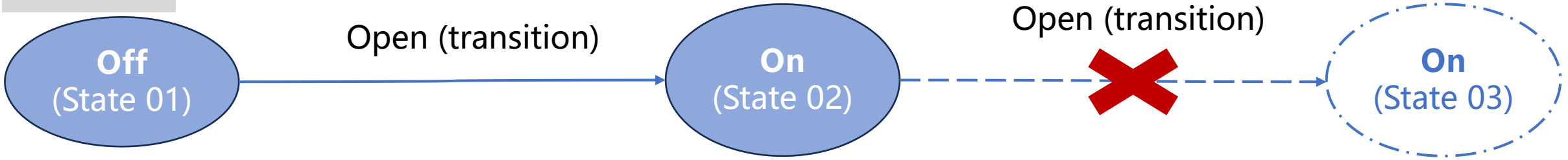


Ethereum Virtual Machine (EVM): Definition

- Finite Number of States: A state machine can only be in one of a finite set of states at any given time. It doesn't have an infinite number of conditions but a countable, manageable number of distinct states.
- Rollback Capability: The ability to revert to a previous state.

  This could be due to an undo operation, an error that needs to be corrected, or any other reason where the system must return to a prior configuration.





BitCoin As A State Transition System

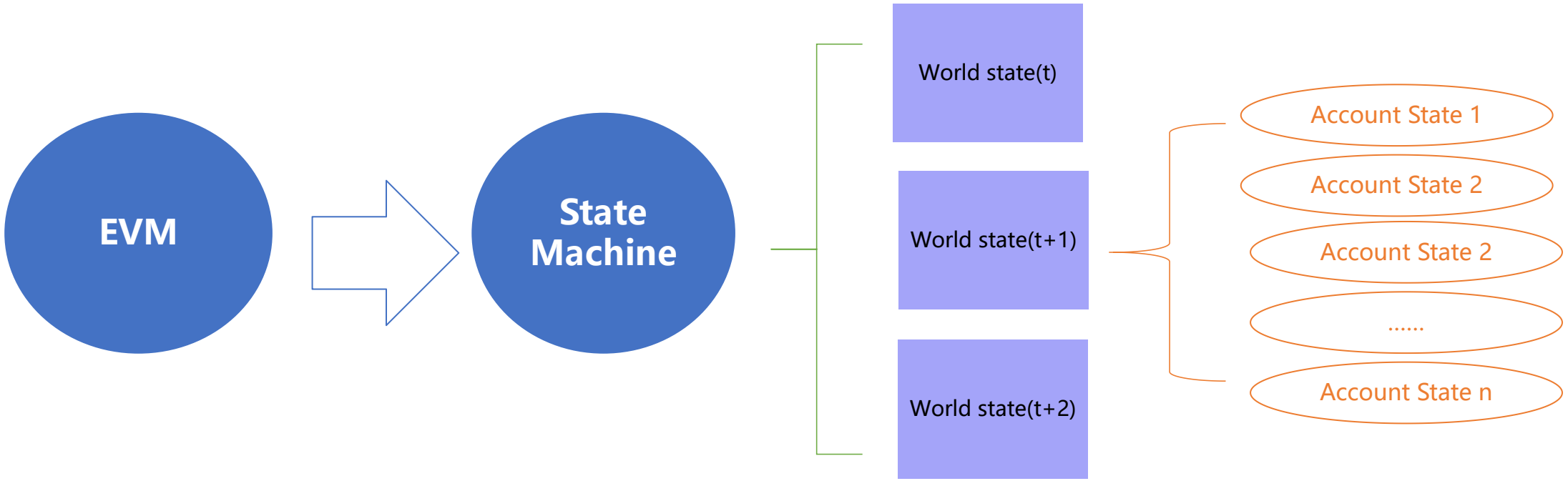


APPLY(S,TX) -> S' or ERROR

APPLY({ Alice: $50, Bob: $50 },"send $20 from Alice to Bob") = { Alice: $30, Bob: $70 }

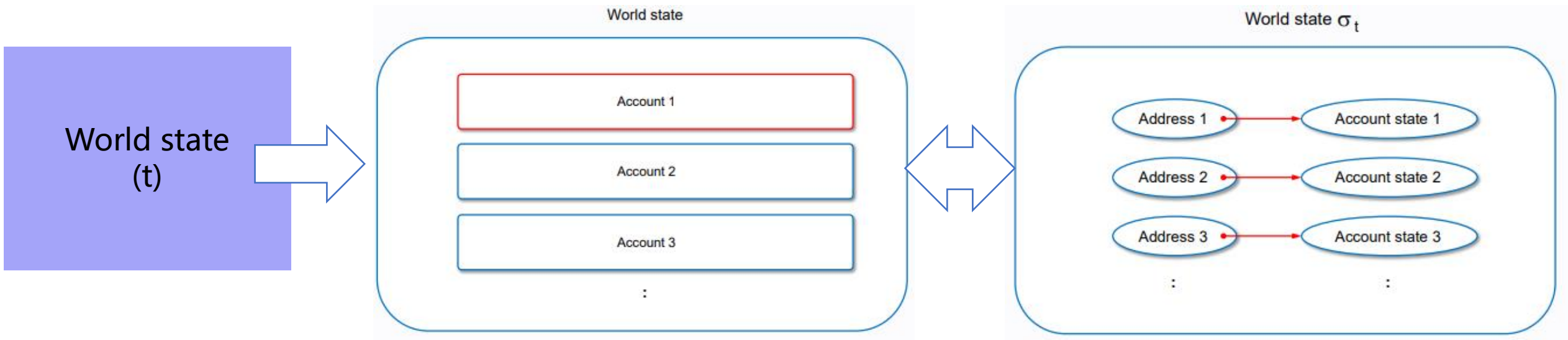
APPLY({ Alice: $50, Bob: $50 },"send $70 from Alice to Bob") = ERROR

EVM Related Concept: State



- EVM as can be viewed as a state machine which records the world state
- The world state consists of account states

EVM Related Concept: What is Account?



An account is an object in the world state.

An account is a mapping between address and account state

Account State

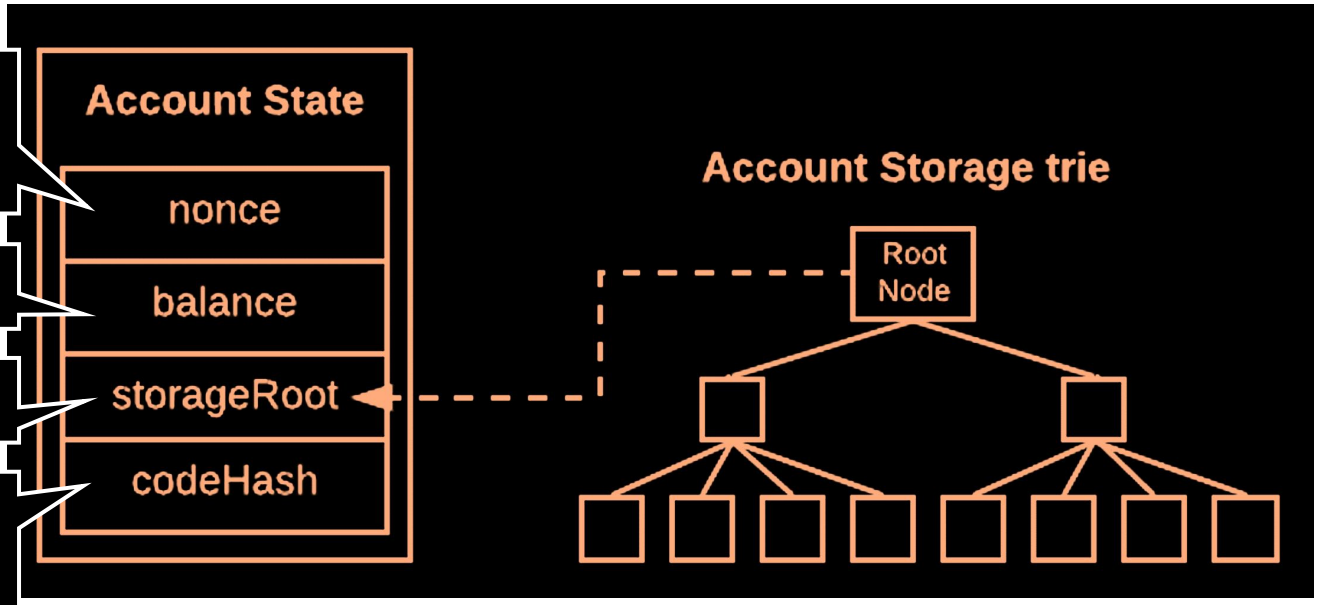
All fields (except the codeHash) are mutable

EOA: Number of transactions sent from this address
Contract: number of contracts created by this account

Total Ether (in Wei) owned by this account.

Hash of the root node of the account storage trie

EOA: Empty
Contract: hash of the EVM code of this account



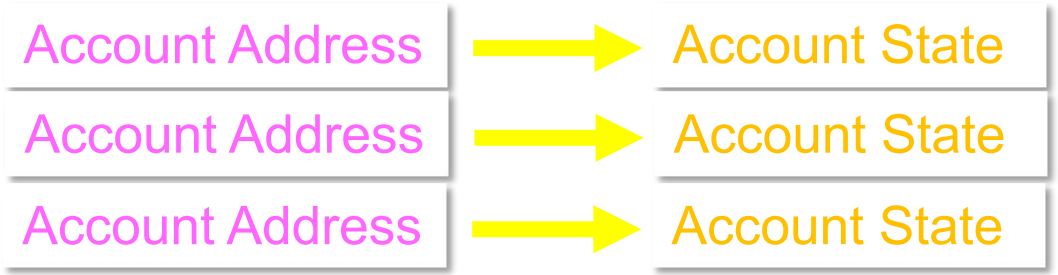
If you deploy a contract with a bug,

you need to deploy a new contract

World State (Ethereum State)

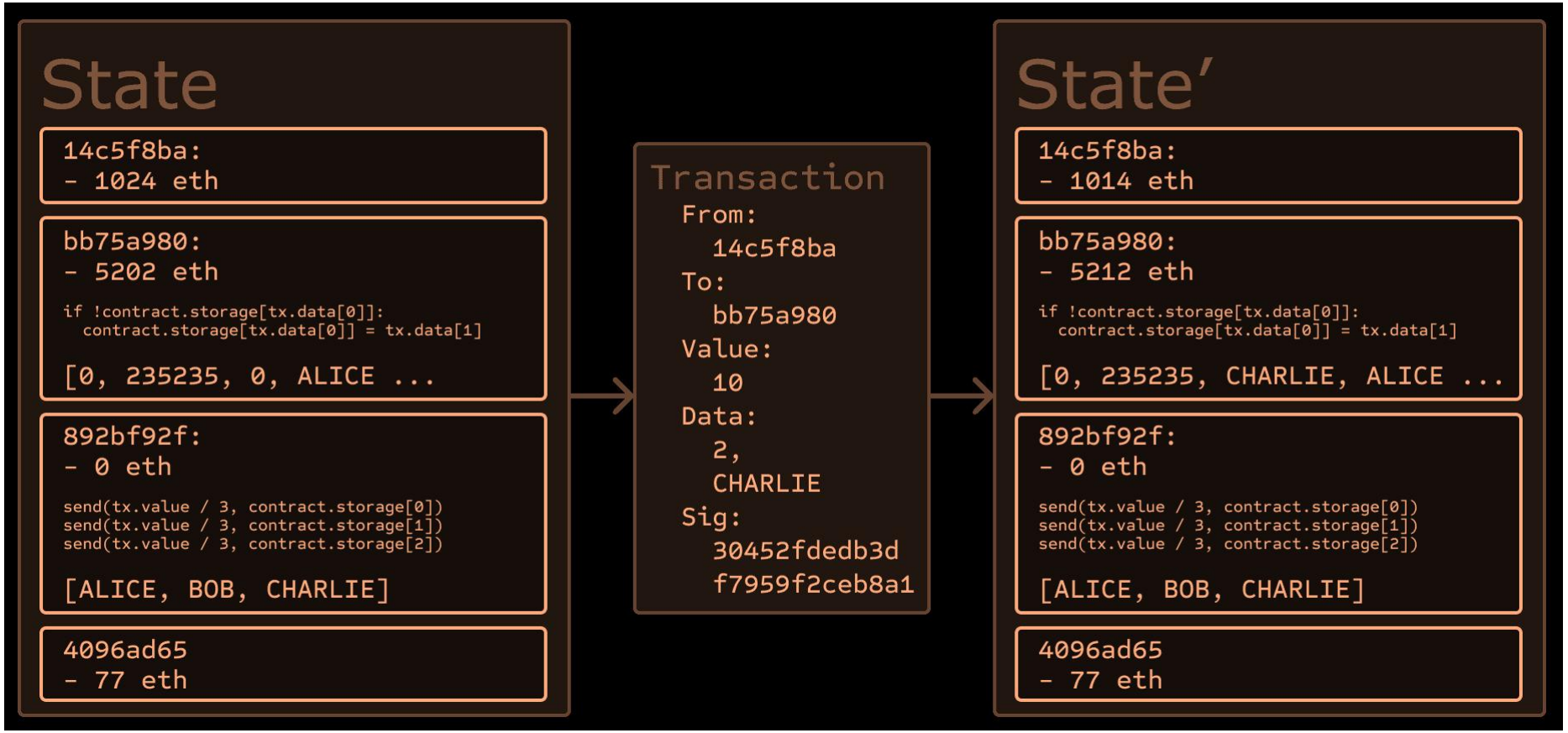
A mapping between accounts and account states

- Off-Chain Data in a trie format
- updated by transactions

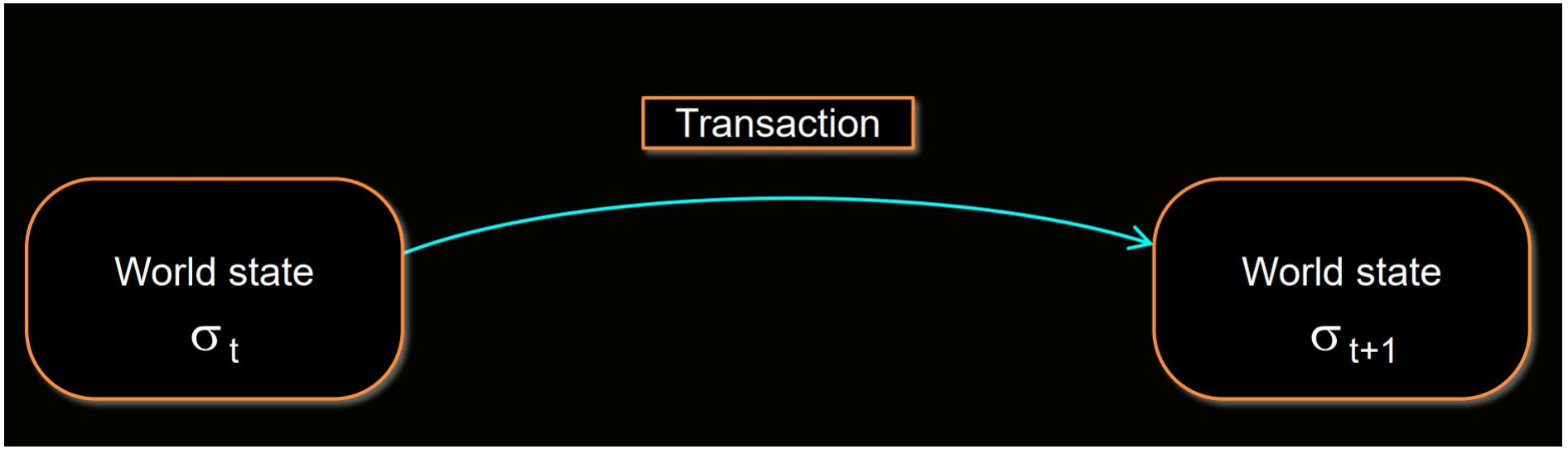


The Ethereum network is like a decentralized computer, the world state is this computer’ s hard drive.

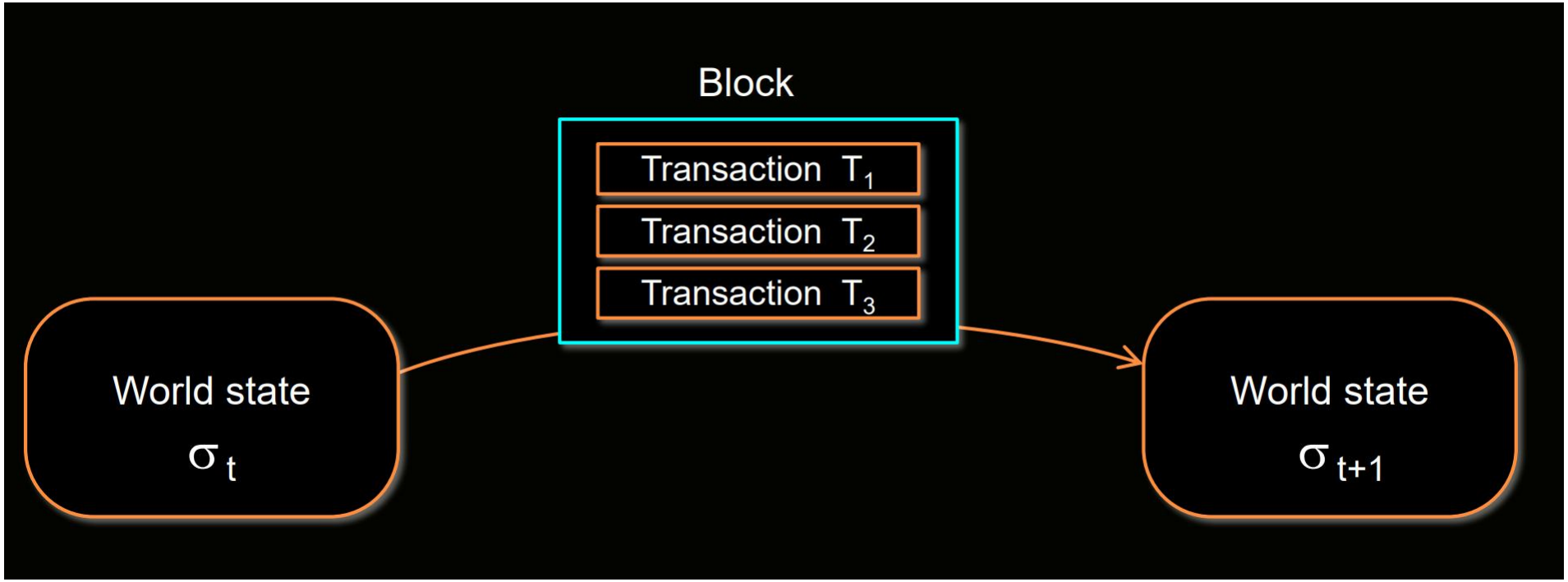
World State Transition



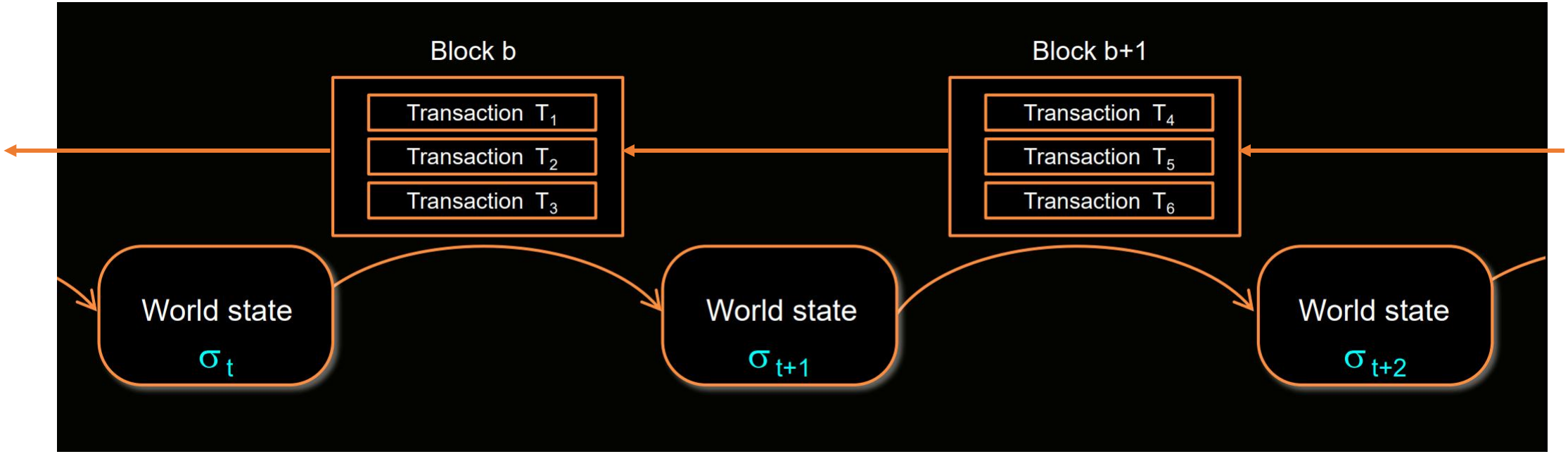
World State Transition



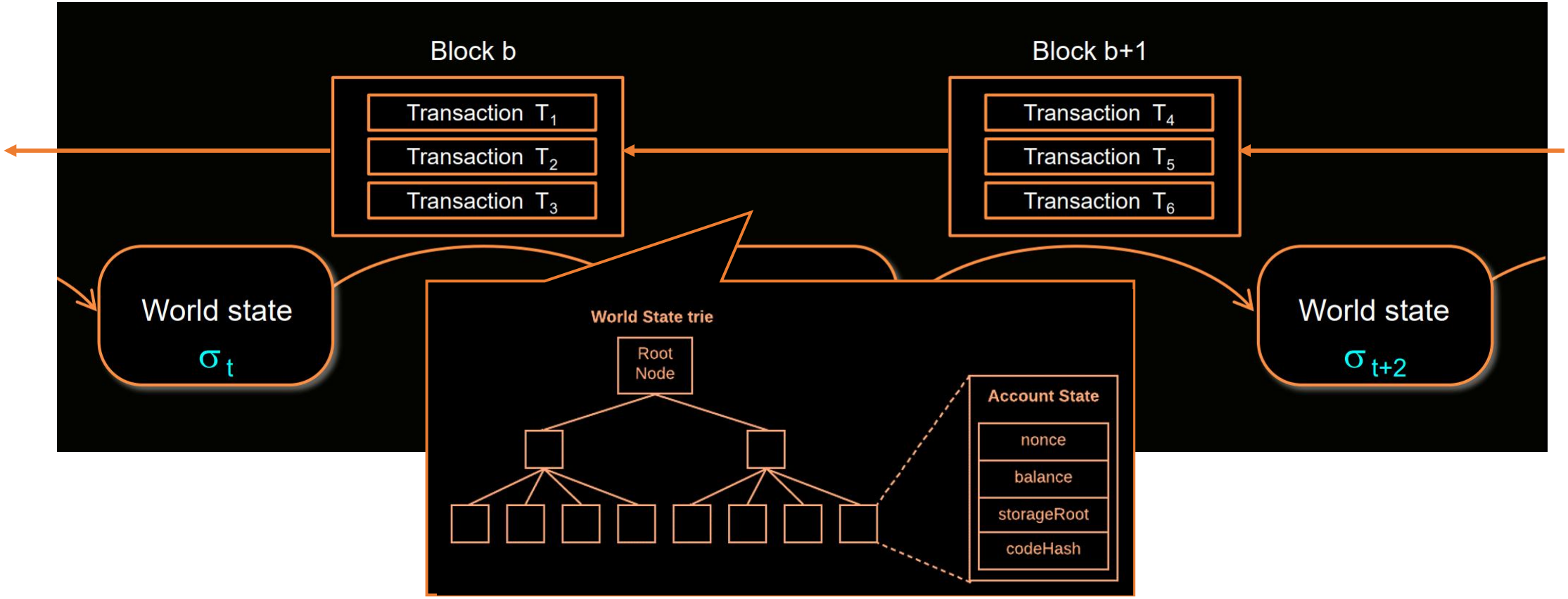
World State Transition



World State Transition

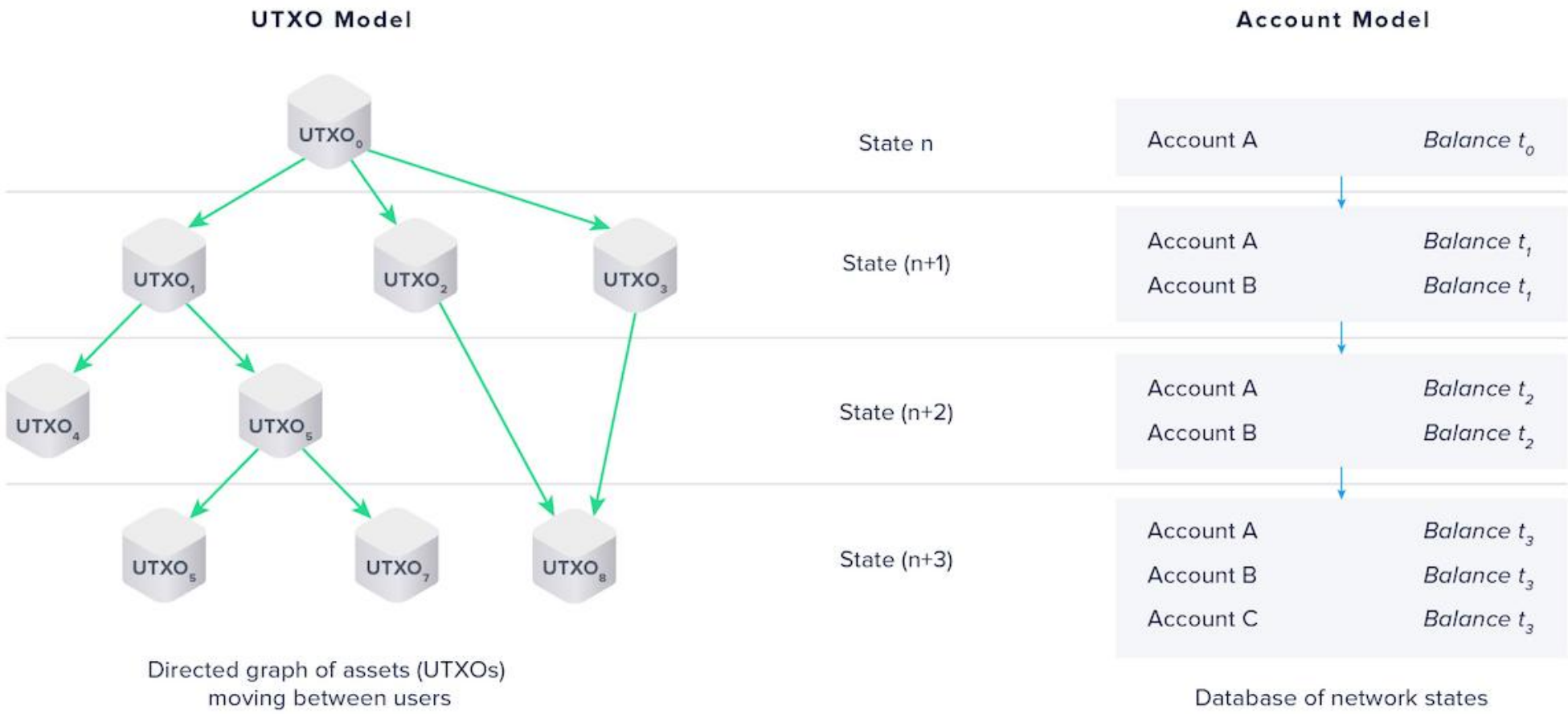


World State Transition



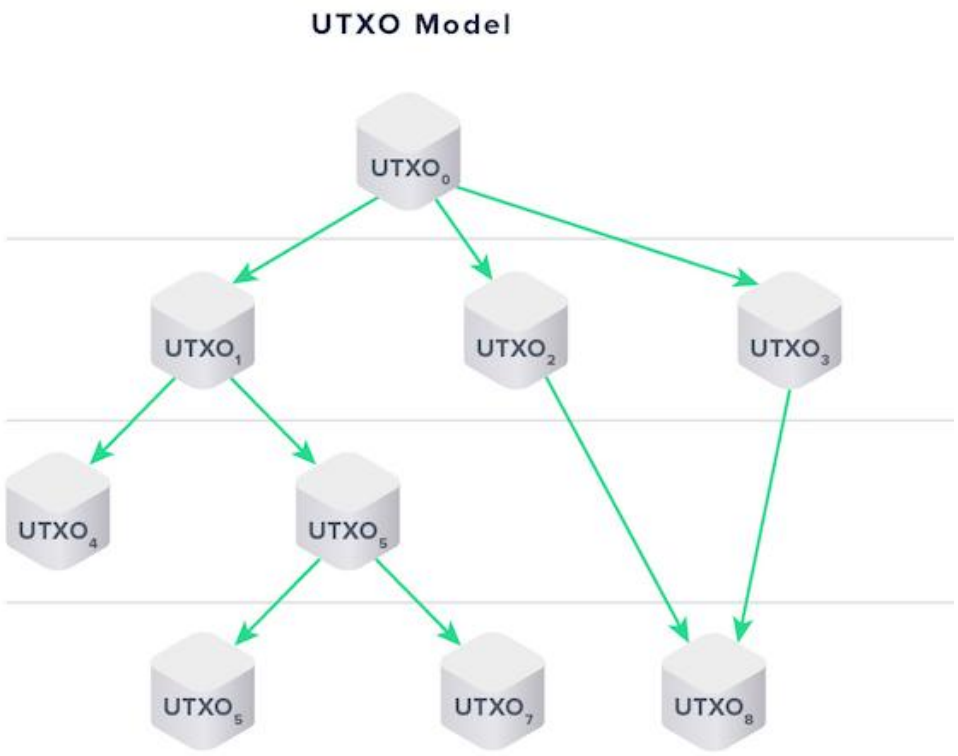
Transaction Models: The UTXO Vs Account-Based Model

UTXO (Unspent Transaction Output) and **Account-Based** are two different blockchain transaction models, each with distinct ways of managing transactions and account balances:



Transaction Models: The UTXO Vs Account Model

Unspent Transaction Output (UTXO) Model——Cash Transaction System



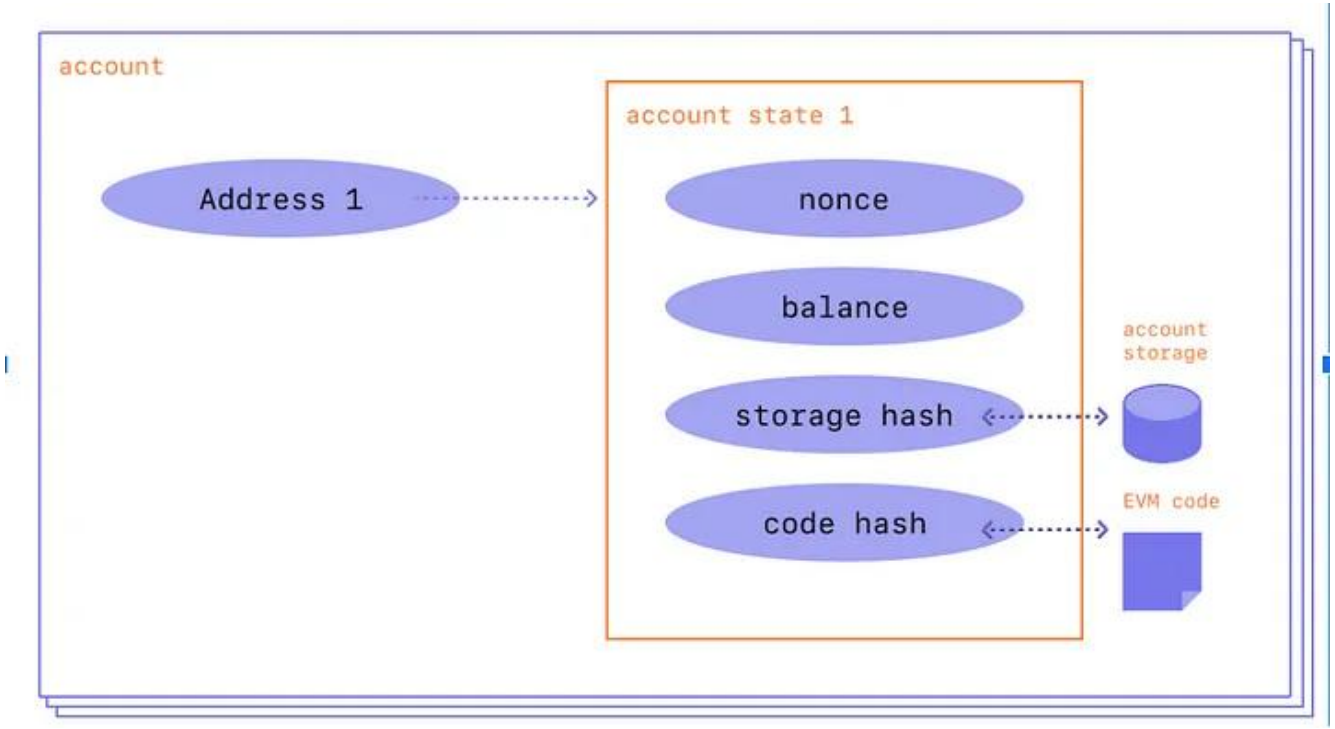
Directed graph of assets (UTXOs) moving between users

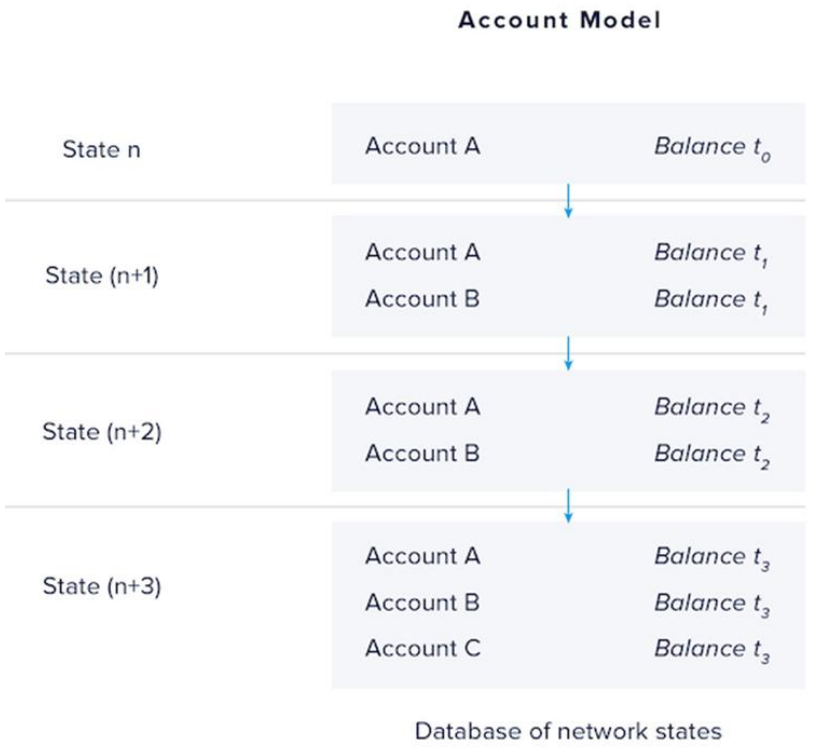
- A UTXO, or Unspent Transaction Output, represents the amount of digital currency remaining after a transaction has been executed.
- In this system, each person carry a wallet containing several cash notes of various denominations, which have been obtained from previous transactions.
- Every time a transaction is executed, the input UTXOs are deleted and the output UTXOs are generated. Any output of the transaction as a new UTXO that can be later spent in a new transaction.

Bookkeeping Models: The UTXO Vs Account Model

Account-based Model —— Bank Accounts

- **Definition:** The account model maintains the state of accounts directly on the blockchain.
- This is enabled by global state tracking - a database of all accounts, keys, contract code, and balances across the network.

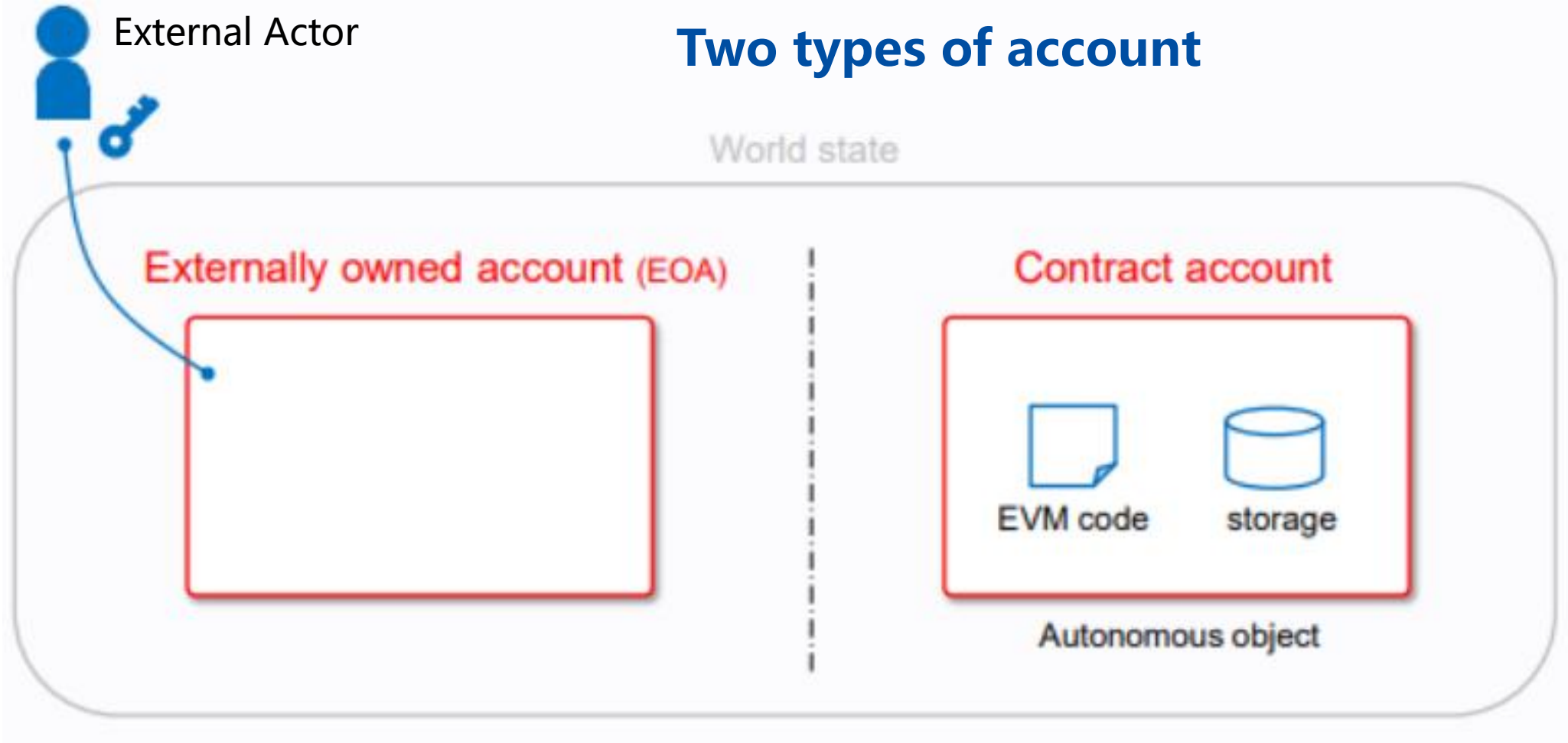




UTXO Model v.s. Account/Balance Model

UTXO	Account/Balance
Scalability Parallel UTXOs Processing	**Simplicity** A More Intuitive Model
Privacy Pseudonymous with new addresses	**Efficiency** Efficient Balance Check in Payment
Paper Bills (banknotes)	ATM/Debit Card

Account



- **Externally Owned Accounts (EOAs)** are controlled by individuals, with a secret key.
- **Contract Accounts (CAs)** are for smart contracts. CAs are not controlled by a user.

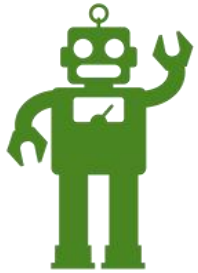
Account in Ethereum



External Owned Account (EOA)

Address → { balance }

- Owned by person or organization
- Controlled by private keys
- Holds Ether balance
- Active Agent: transfers Ether, calls contract code

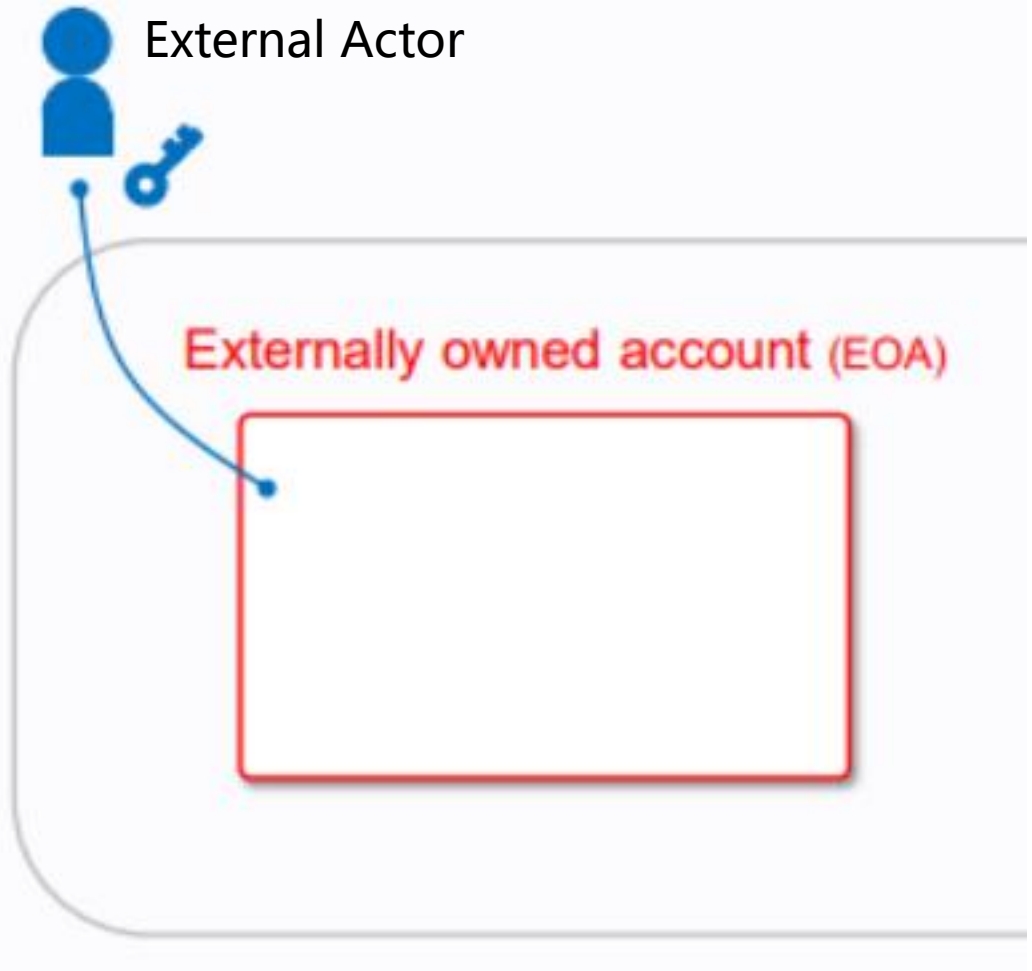


Contract Account

Address → { balance, code, storage }

- Holds Ether balance
- Associated with code
- Passive Agent: Code called by other accounts
- Code can transfer Ether, call other contracts

Account



Externally Owned Accounts (EOAs)

- **Controlled by Individuals:** EOAs are controlled by private keys, essentially cryptographic proof of ownership.
- **Direct Transactions:** They can send Ether and initiate transactions directly with other EOAs or Contract Accounts.

Account

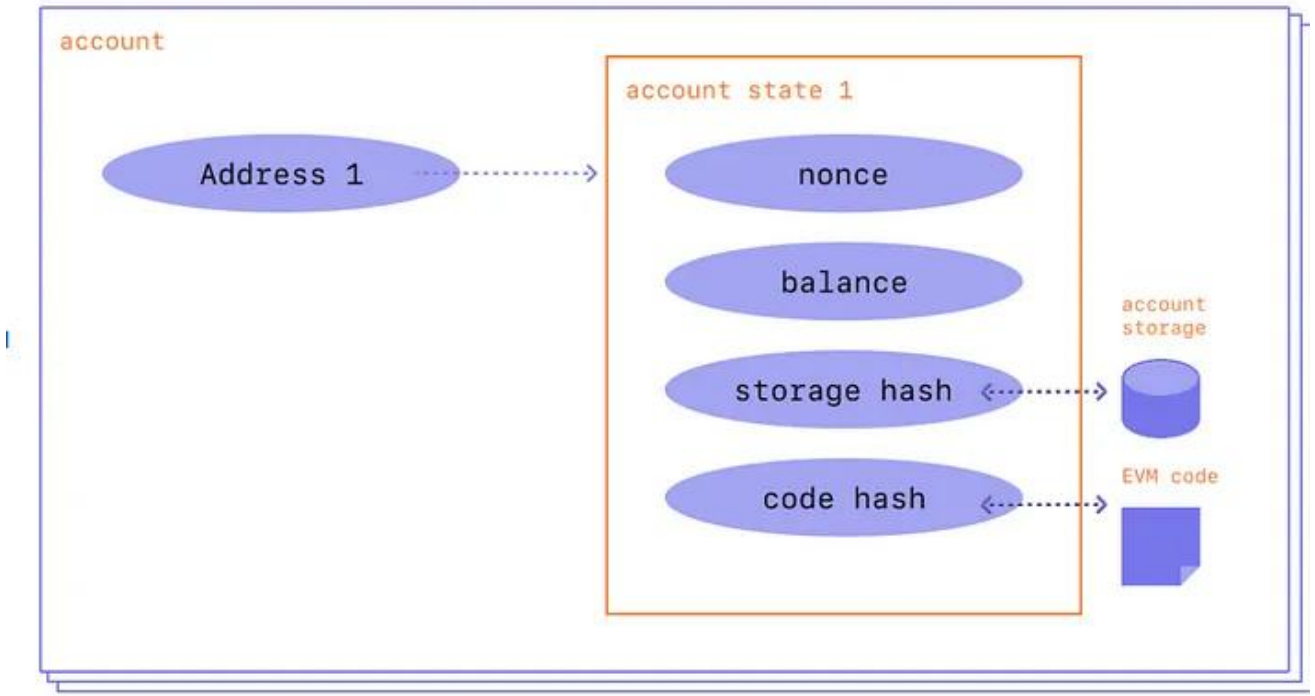


Contract Accounts (CAs)

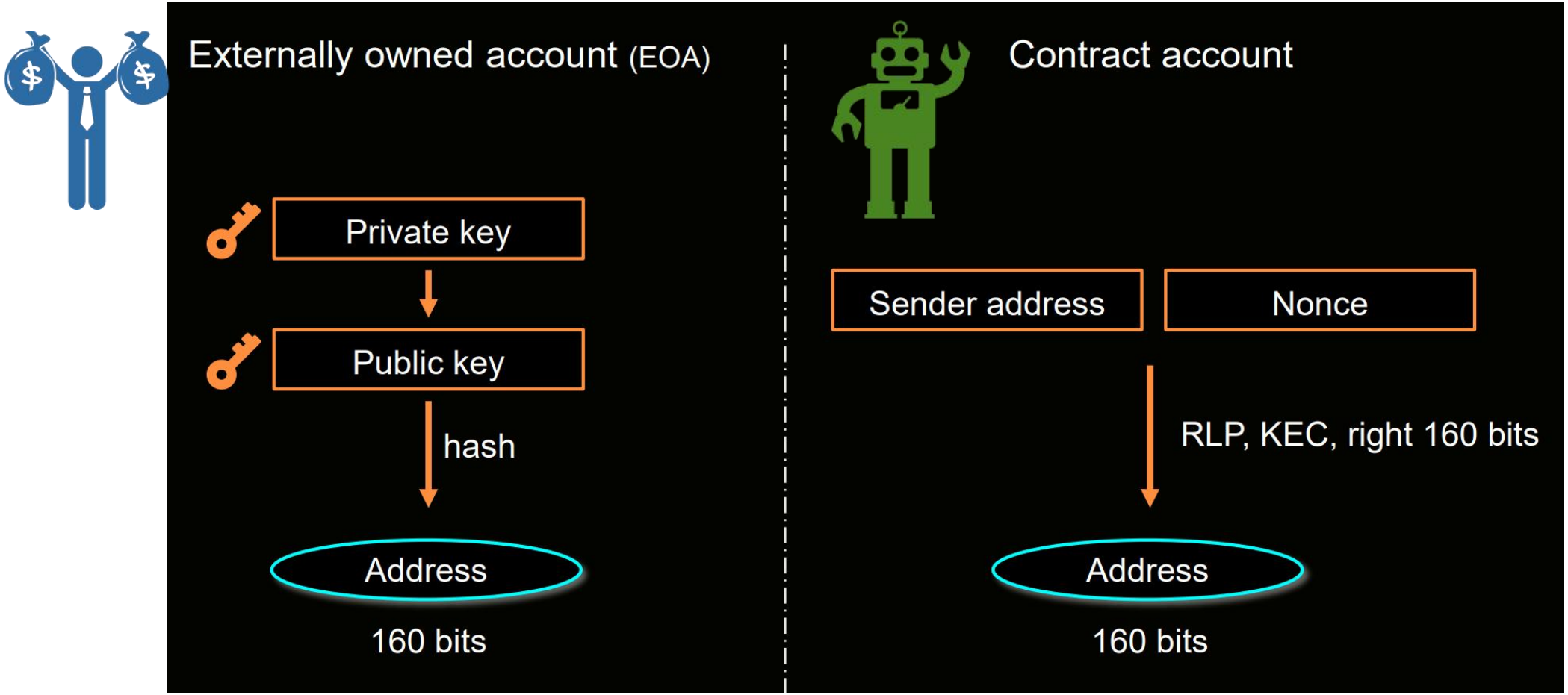
- **Powered by Code:** Contract account contains EVM code, and act is controlled by EVM code autonomously when triggered by transactions.
- **Internal Storage:** These accounts have their own storage, holding the contract's state, variables, and other relevant data.
- **Autonomous Functionality:** CAs are governed by their contract code, enabling complex functions like automated token distribution, decentralized finance operations, and more.
- **Interaction with the EVM:** The contract code within these accounts is executed by the EVM, consuming gas in the process.

Account State

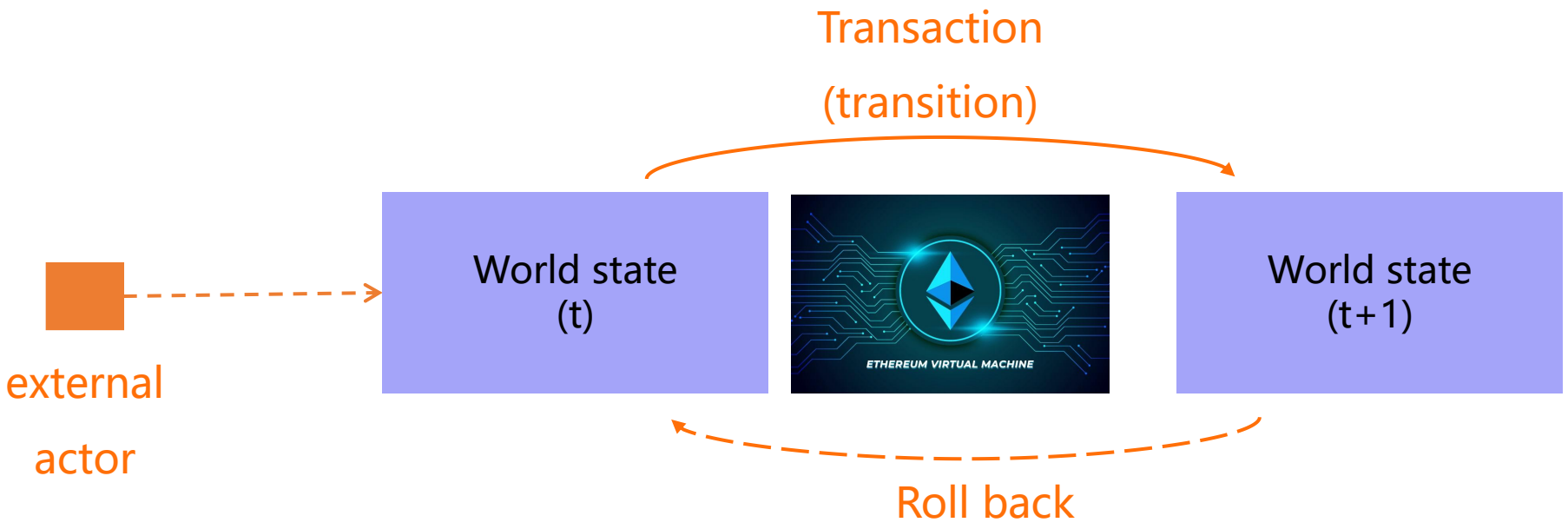
- **Nonce:** counter that records the number of transactions made by the account that increments by 1 every time a transaction is made.
- **Balance:** Total Ether (in Wei) owned by this account
- **Storage hash:** A 256-bit hash that encodes the storage contents of the account. It is empty by default
- **Code Hash:** The EVM code hash that gets executed when a contract is called. For EOAs, it is the hash of an empty string



Two Types Address Generation



Transactions



Transactions are cryptographically signed instructions from accounts which is submitted by external actor.

Ethereum Virtual Machine (EVM): Transactions

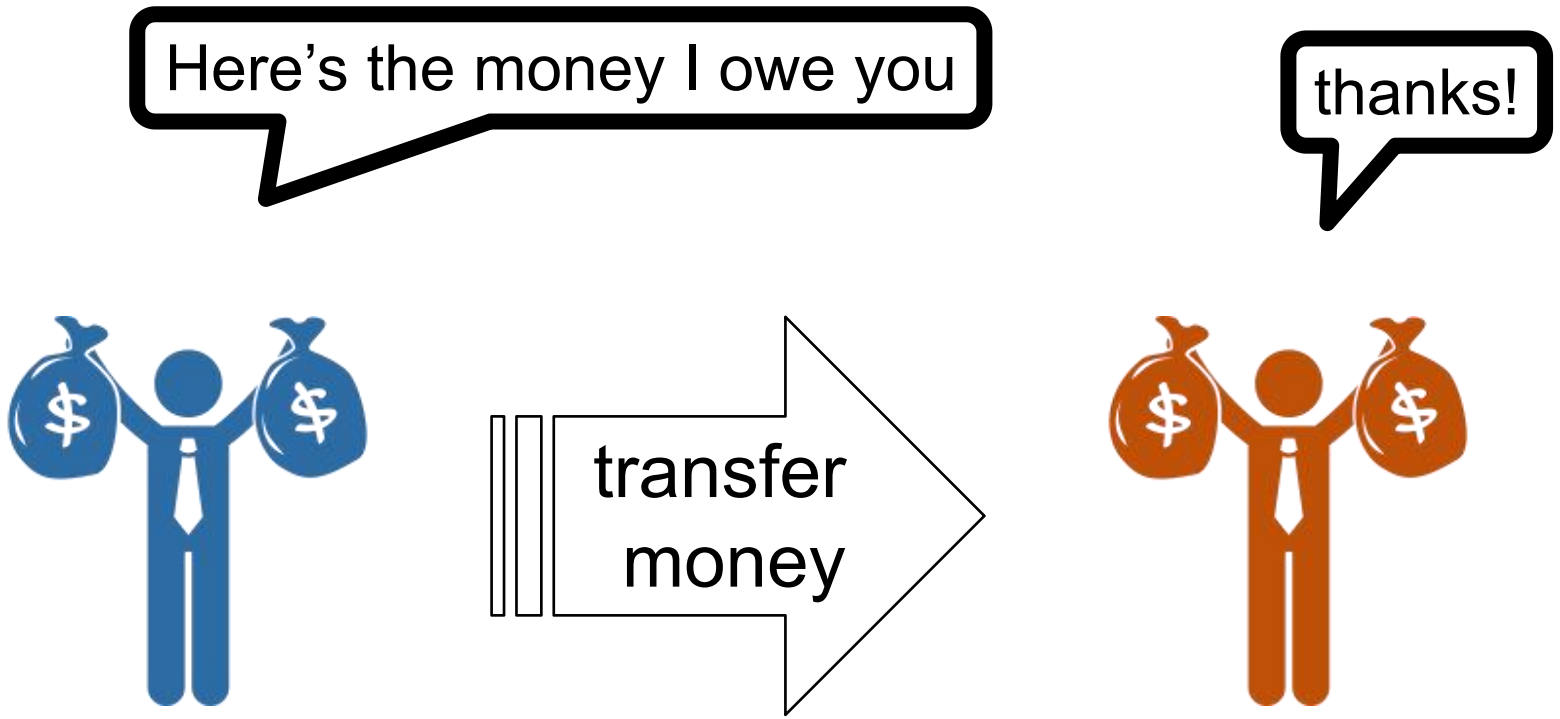
Type of Transactions

Funds Transfer

Contract Deployment

Function Execution

EOA to EOA Message



Ethereum Virtual Machine (EVM): Transactions

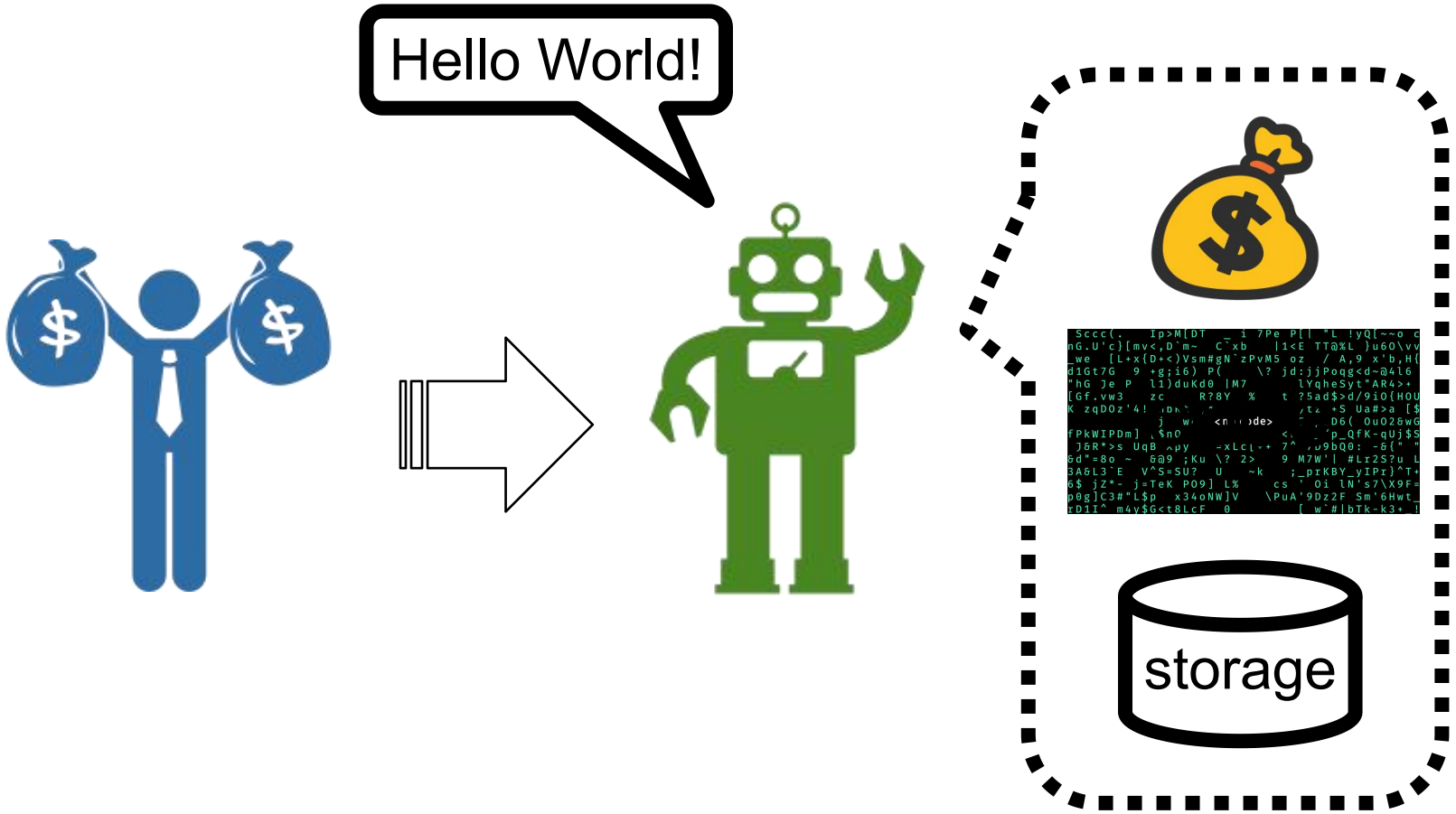
Type of Transactions

Funds Transfer

- This type of transaction takes place whenever an account sends some funds (in form of Ether) to another account.
- This transaction has a value associated with it but does not have any data.
  - For example, when account A sends some money to account B for buying something. The gas price is set by the sender.



Contract Creation Transaction

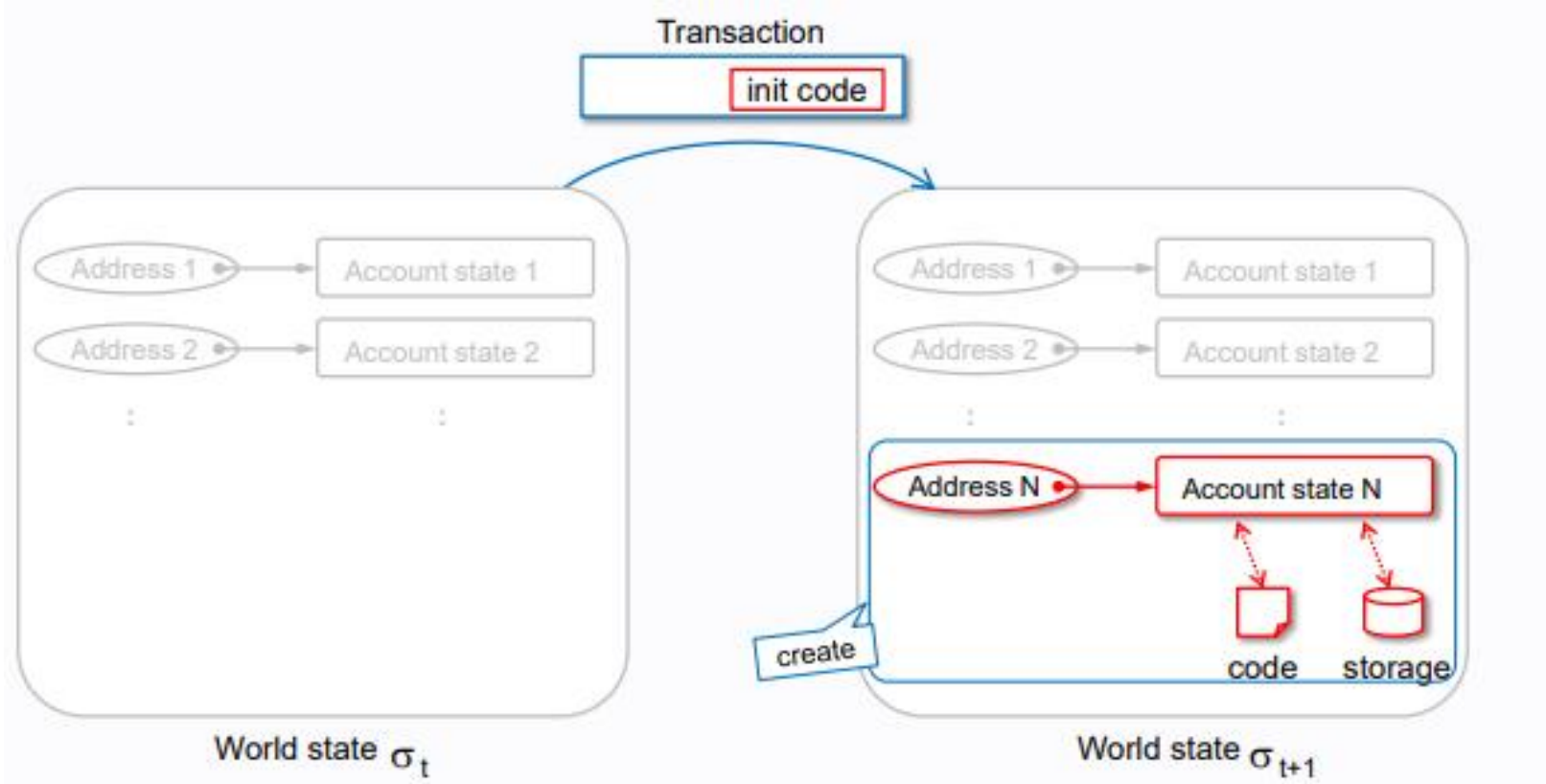


Ethereum Virtual Machine (EVM): Transactions

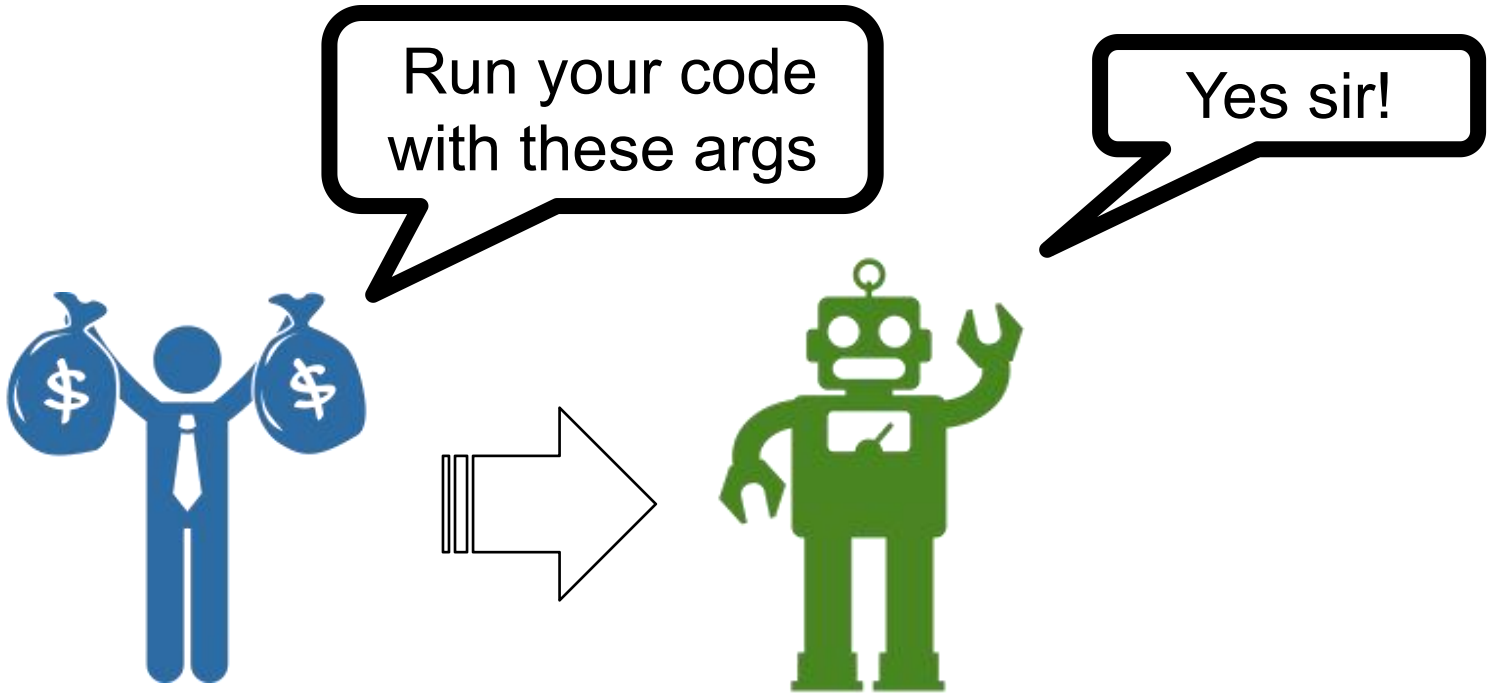
Type of Transactions

Contract Deployment

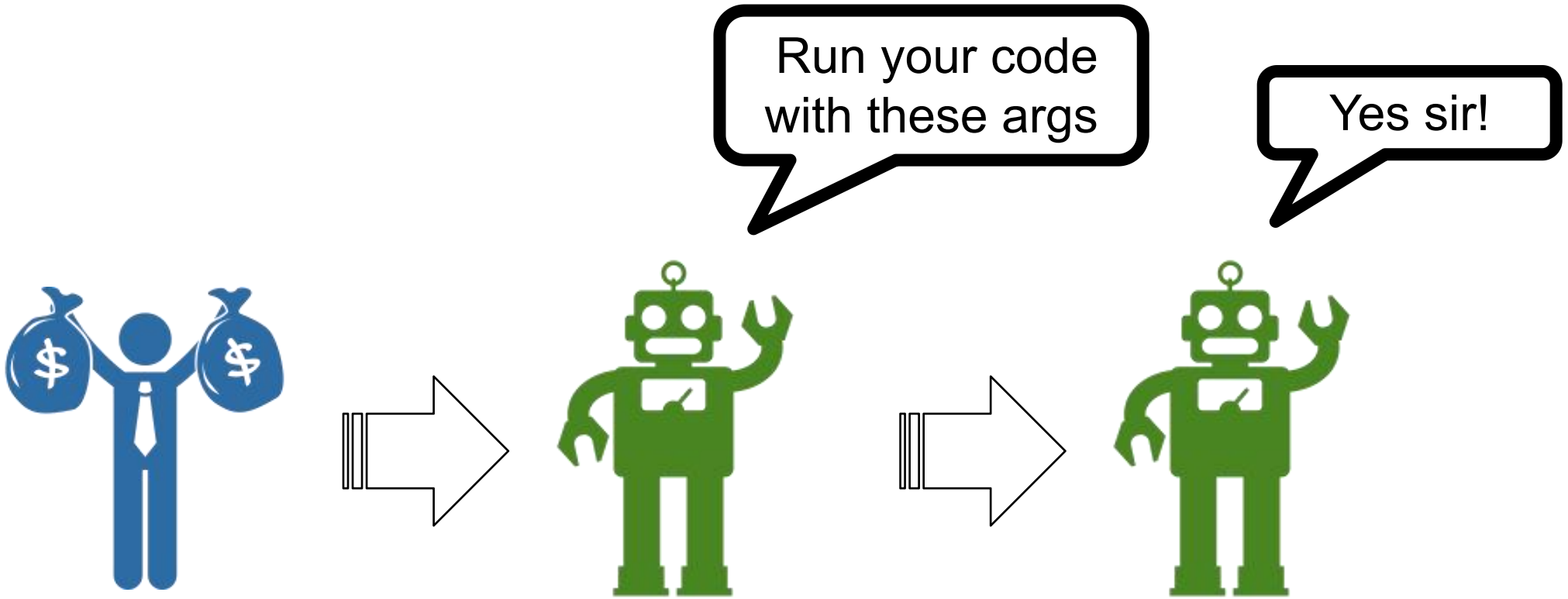
The process by which a new smart contract is deployed to the Ethereum blockchain.



Message Call Transaction



Message Call Transaction

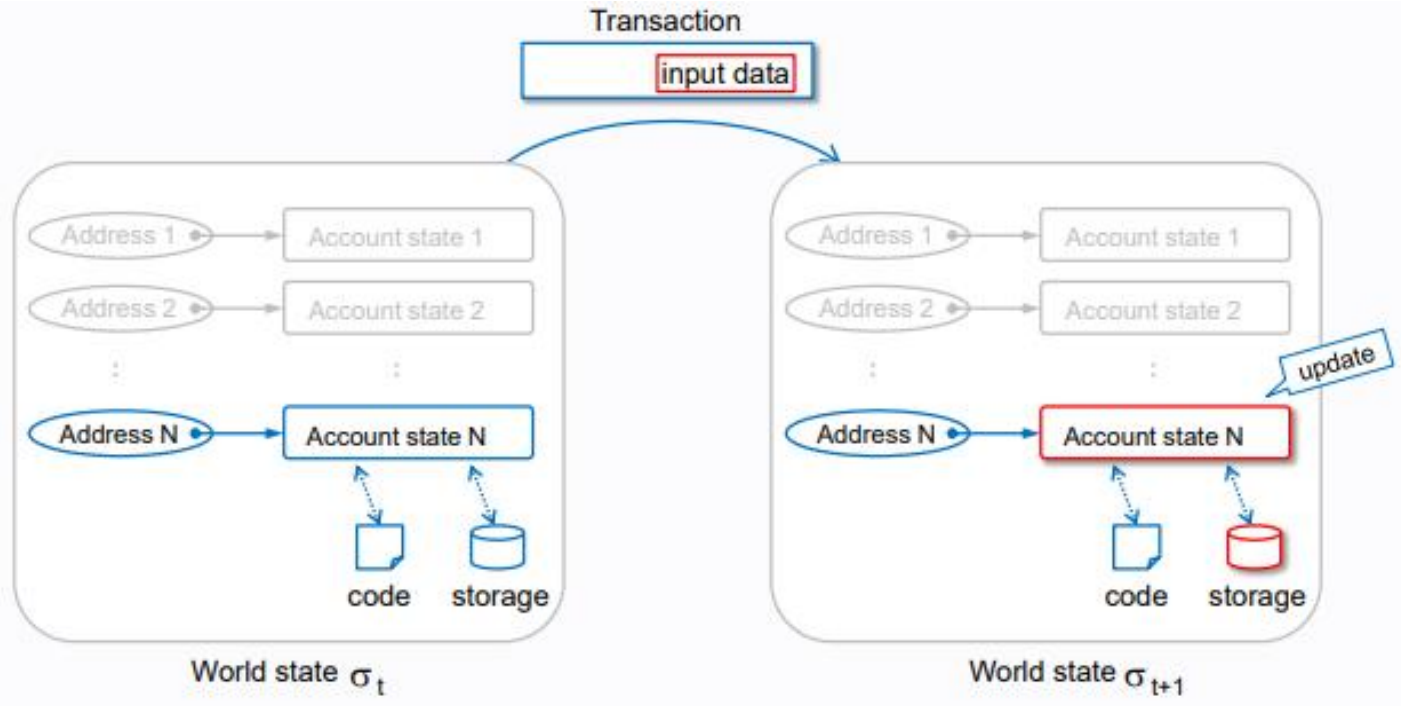


Ethereum Virtual Machine (EVM): Transactions

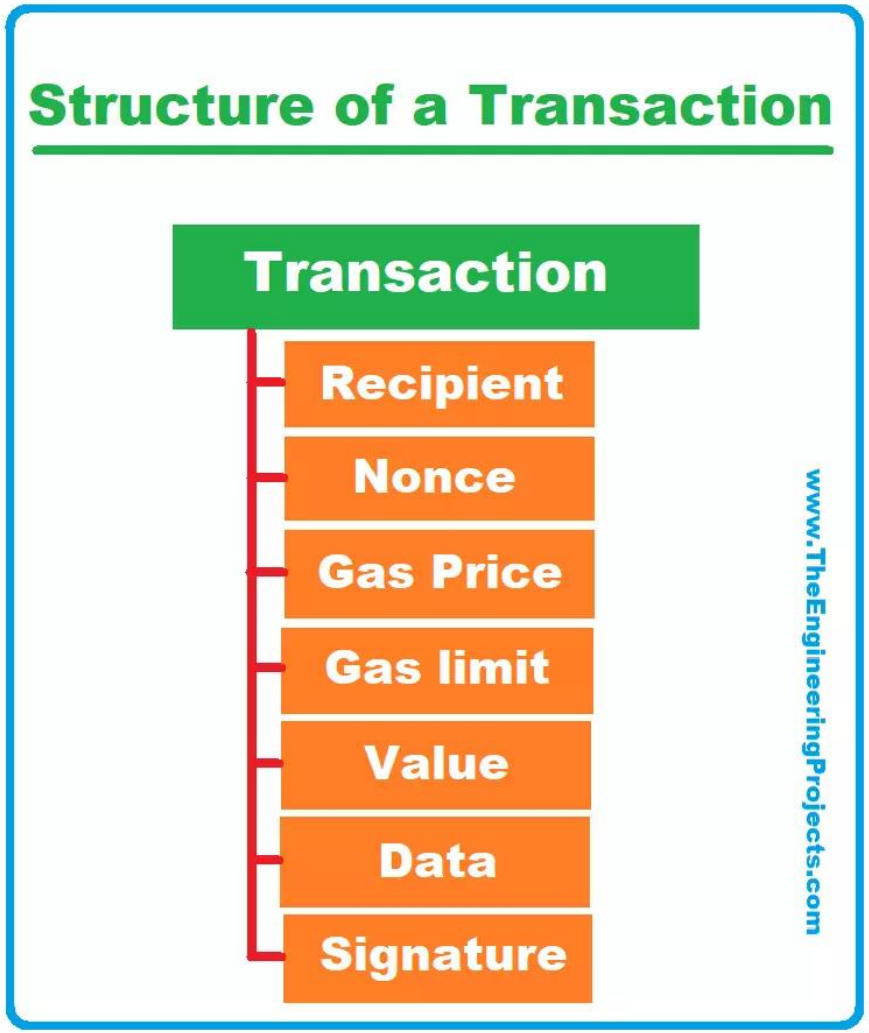
Type of Transactions

Function Execution

The process in which a transaction is sent to an existing smart contract on the Ethereum blockchain to execute its code



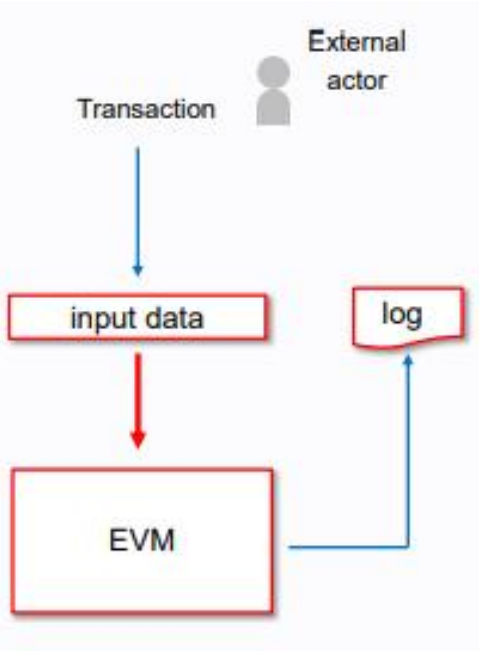
EVM: What Are Contained in Transactions



- **Recipient:** 160-bit address of the recipient. .
- **Nonce:** This "number only used once" is simply a counter that counts how many transactions the account of the sender of the transaction has already sent.
- **GasPrice:** The amount of Wei the sender is willing to pay per unit of gas.
- **Gas limit:** The maximum amount of computational effort the transaction is allowed to use, expressed in units of gas.
- **Value:** The amount of Wei (the native currency of Ethereum) to transfer from the sender to the recipient.
- **Data:** Optional data field, which can be accessed by smart contracts.
- **Signature:** The transaction is signed by the user controlling the EOA.

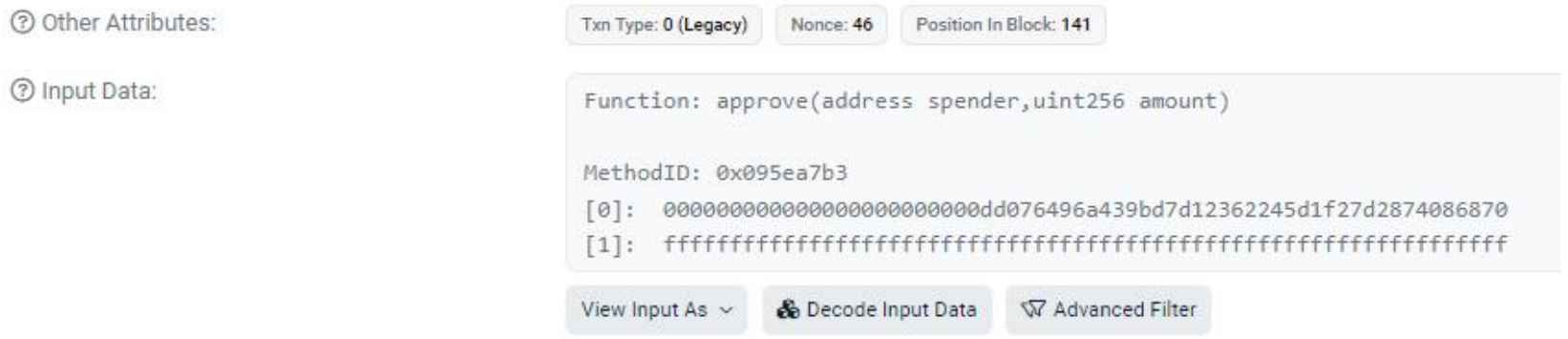
Transaction: Input and Output

- **Input:** External data from a transaction.
- **Output:** EVM can output log.



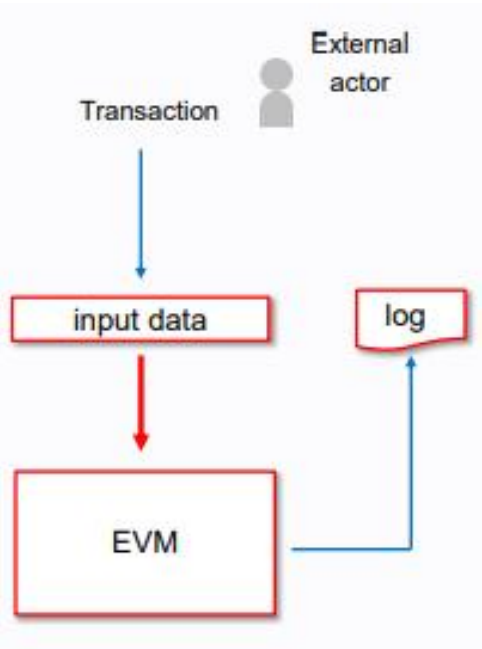
Two main types of input data:

- **Function call data**
  - The most common type of input data
  - Used to call a specific function in a smart contract
  - Typically includes the function signature (or methodid) and any parameters that the function requires.



Transaction: Input and Output

- **Input:** External data from a transaction.
- **Output:** EVM can output log.



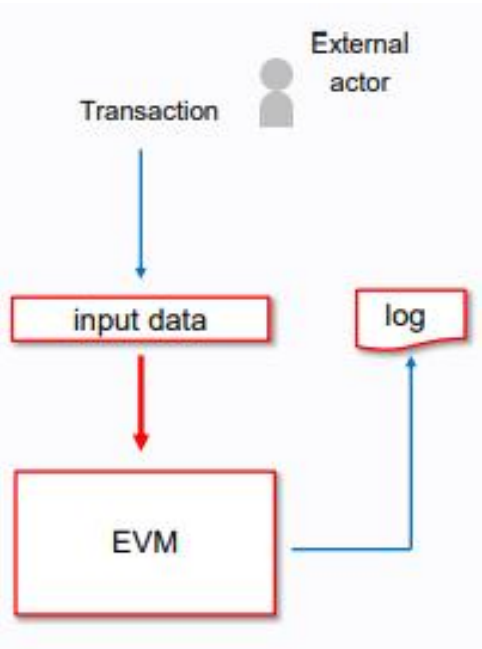
Two main types of input data:

- **Arbitrary data**
  - Any data that is not related to a specific function call in a smart contract
  - Doesn't change any state on ethereum
  - Can be useful for attaching a message to a transaction and providing metadata about the transaction



Transaction: Input and Output

- **Input:** External data from a transaction.
- **Output:** EVM can output log.



Log

- Data that smart contracts can emit during transactions and are recorded into the ethereum blockchain.
- Logs are used to track events and record the execution of contract functions.

Transaction Receipt Event Logs

340

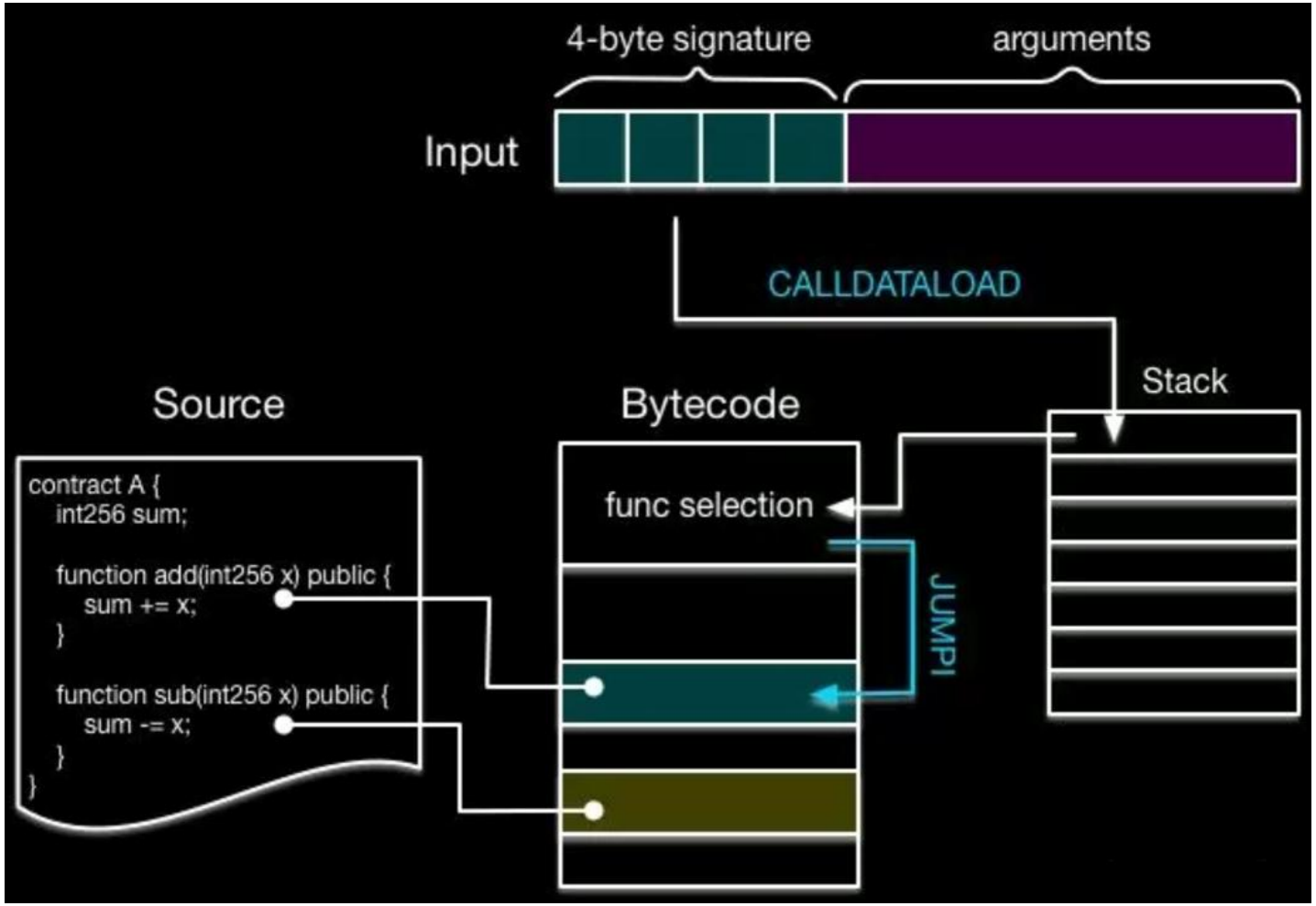
Address Memecoin: MEME Token

Name Approval (index_topic_1 address owner, index_topic_2 address spender, uint256 value) View Source

Topics
0 0x8c5be1e5ebec7d5bd14f71427d1e84f3dd0314c0f7b2291e5b200ac8c7c3b925
1: owner Dec → 0x6fD1DD3888cD5ce26daBB727F76348cceDFd3Dd4
2: spender Dec → 0xDd076496a439bd7d12362245D1F27d2874086870

Data value : 115792089237316195423570985008687907853269984665640564039457584007913129639935 Dec Hex

EVM: Input


4-byte signature
arguments
Input
CALLDATALOAD
Stack
Source
Bytecode
contract A {
int256 sum;
function add(int256 x) public {
sum += x;
}
function sub(int256 x) public {
sum -= x;
}
}
func selection
JUMPI

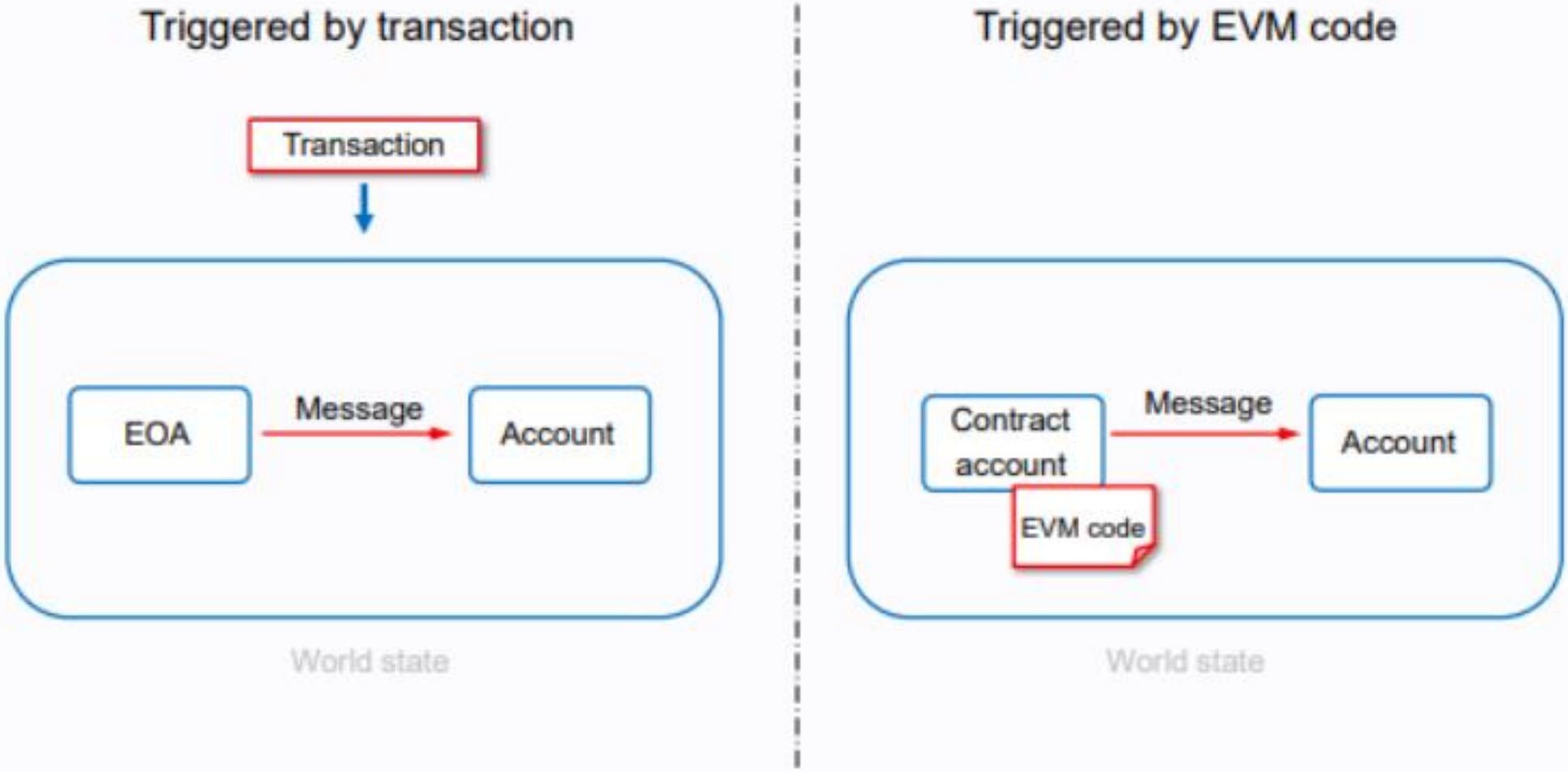
Communication between Accounts: Message

Message is passed between two Accounts.

Message is Data (as a set of bytes) and Value (specified as Ether) .



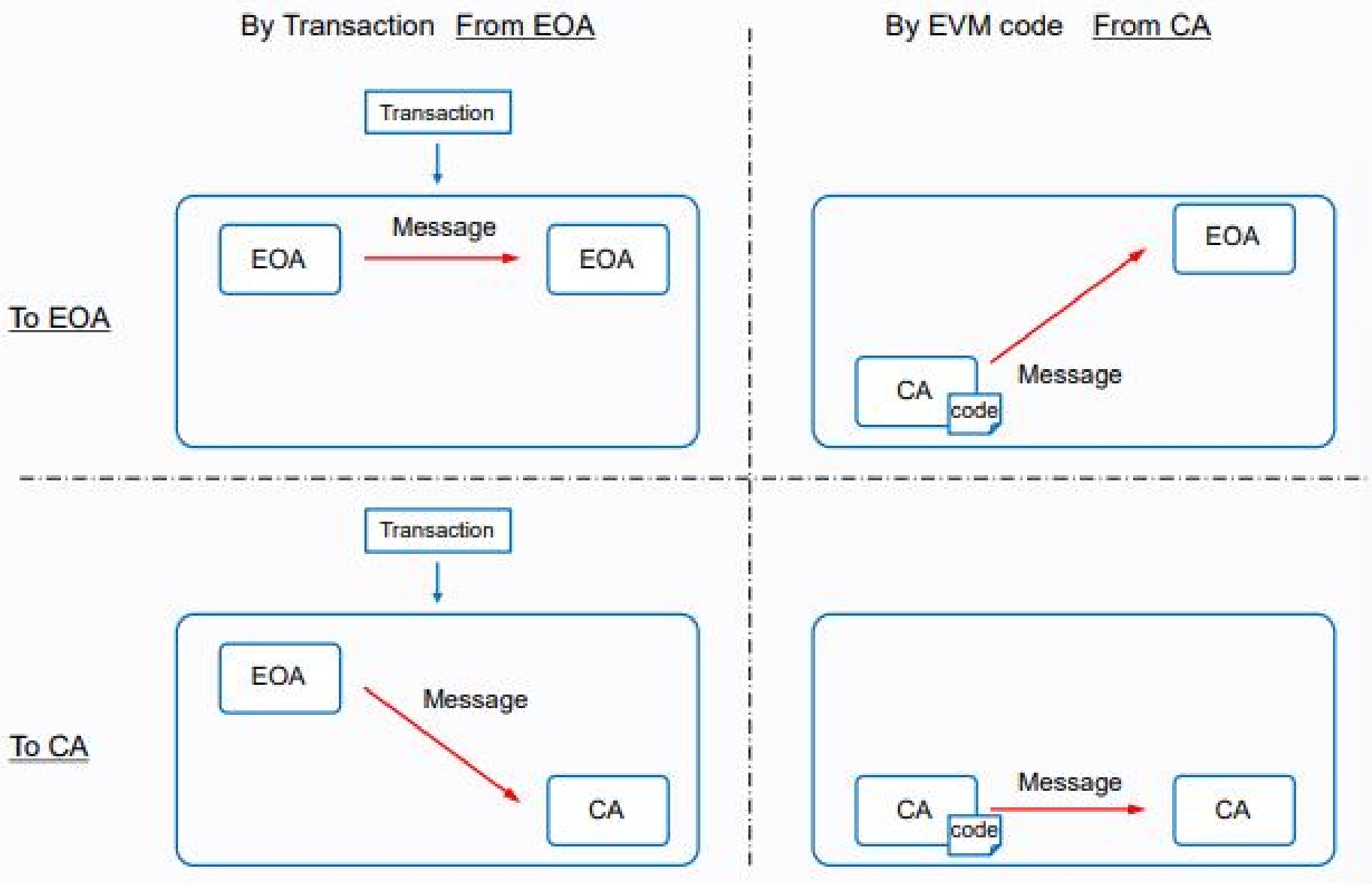
Message



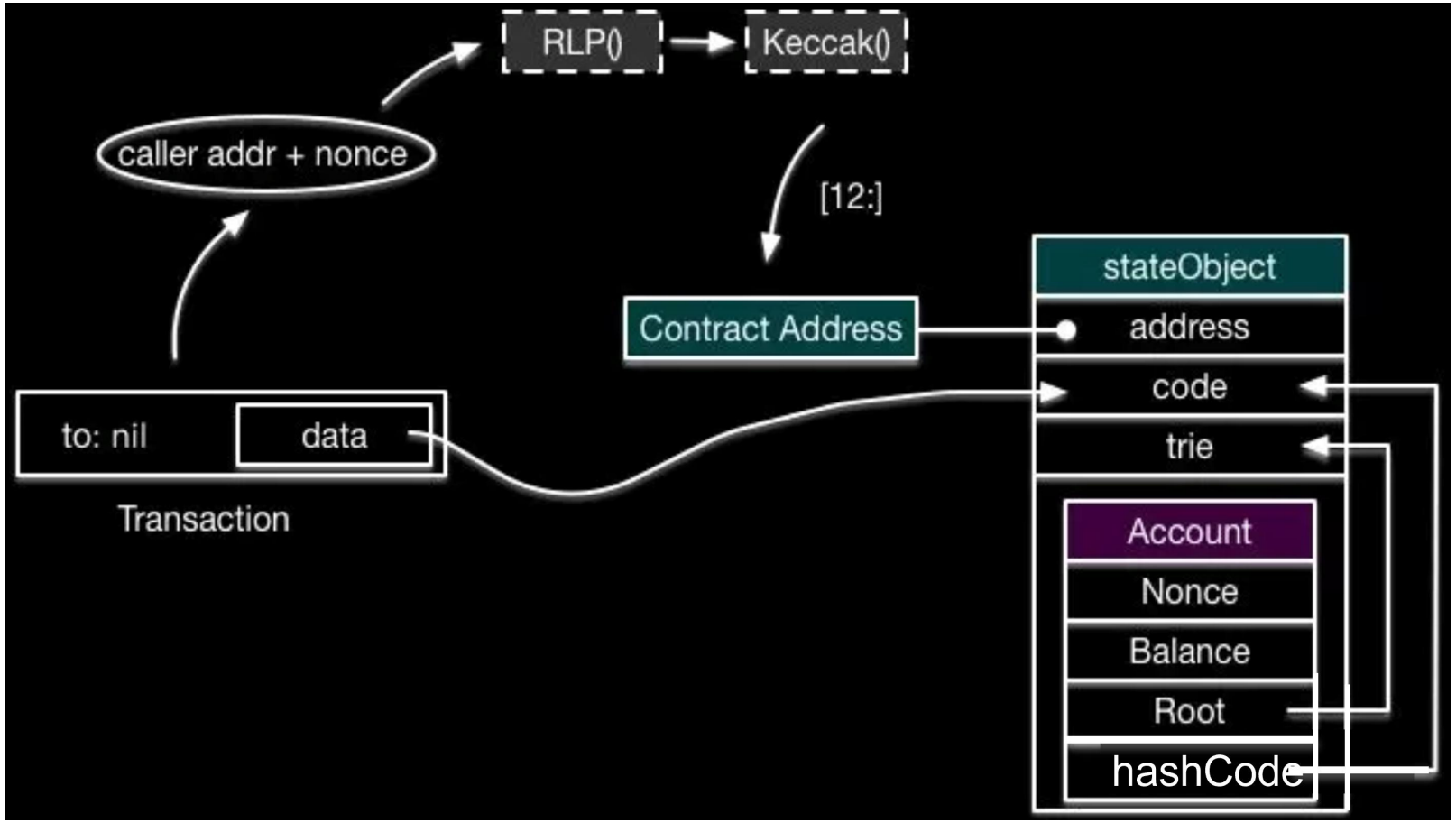
Transaction triggers an associated message

EVM can also send a message

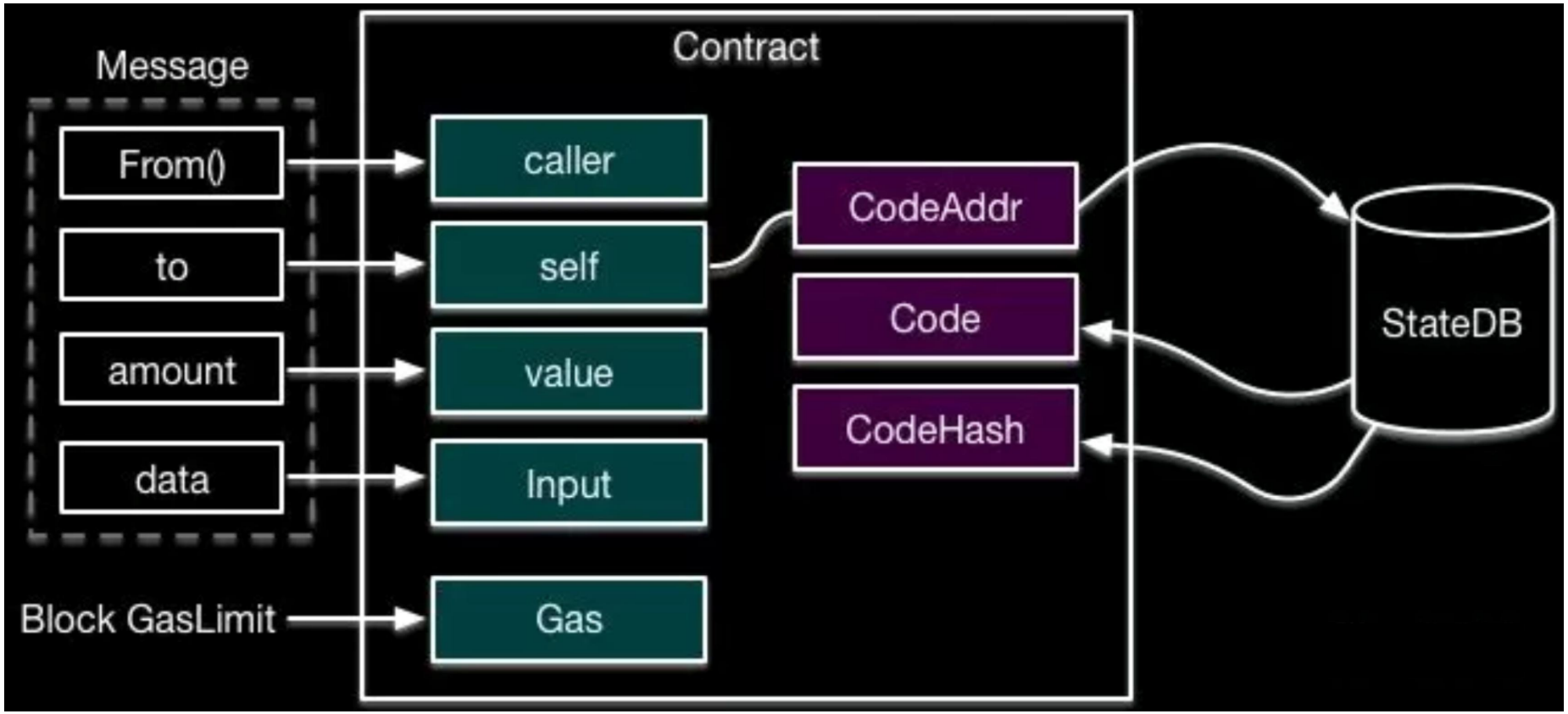
Message


By Transaction From EOA
By EVM code From CA
To EOA
To CA
Transaction
EOA
Message
EOA
CA
code
Message
EOA
Transaction
EOA
Message
CA
CA
code
Message
CA

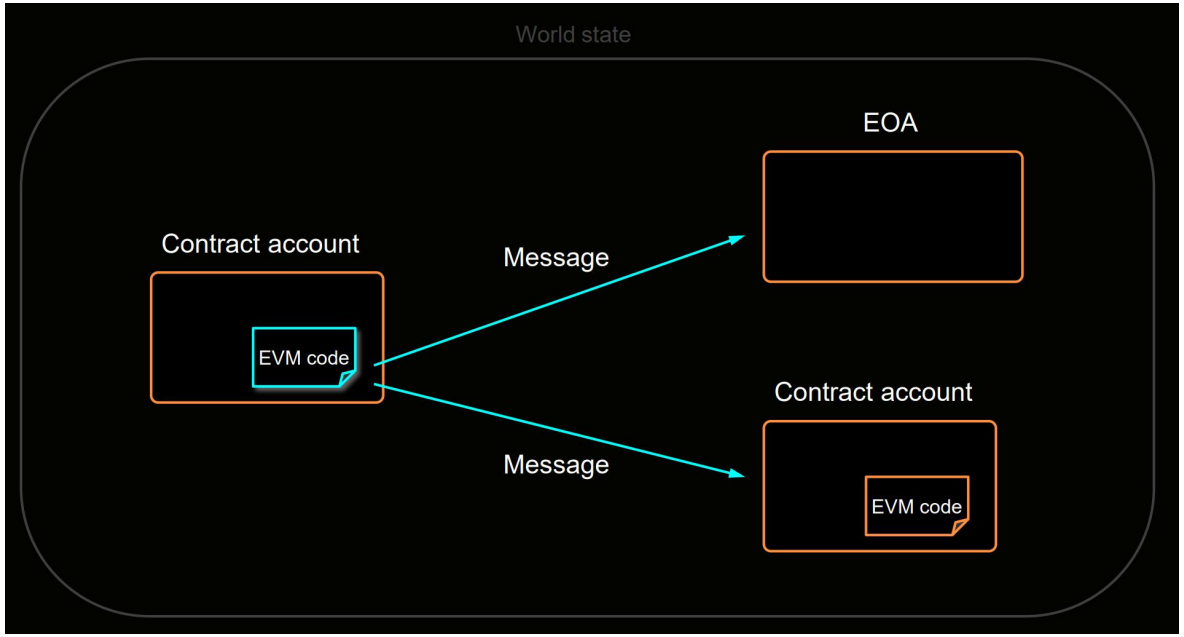
EVM: Create Contract



EVM: Calling A Contract

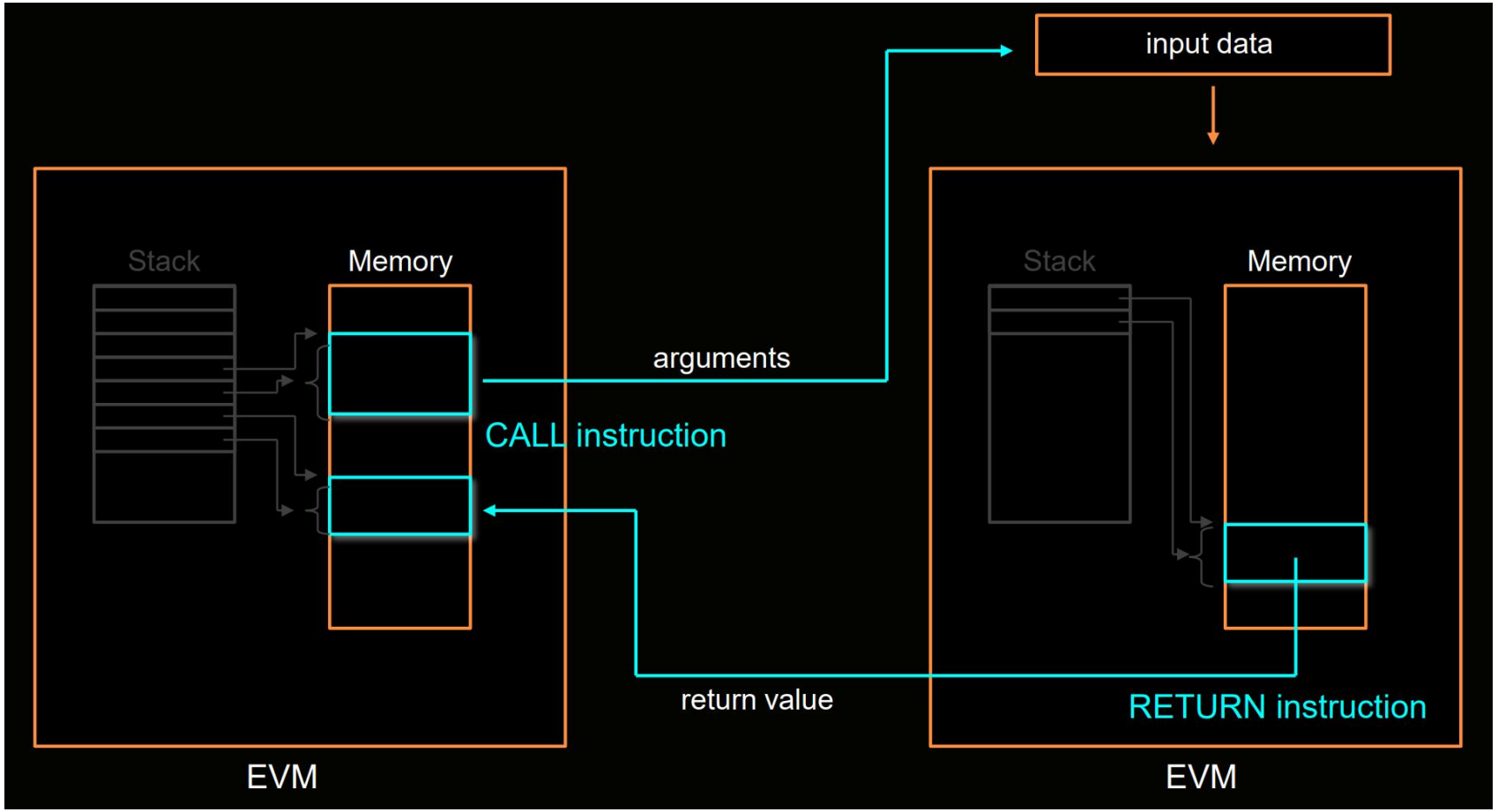


EVM Message Call



EVM can send a message to other account.
The depth of message call is limited to less than 1024 levels.

Instructions for Message Call

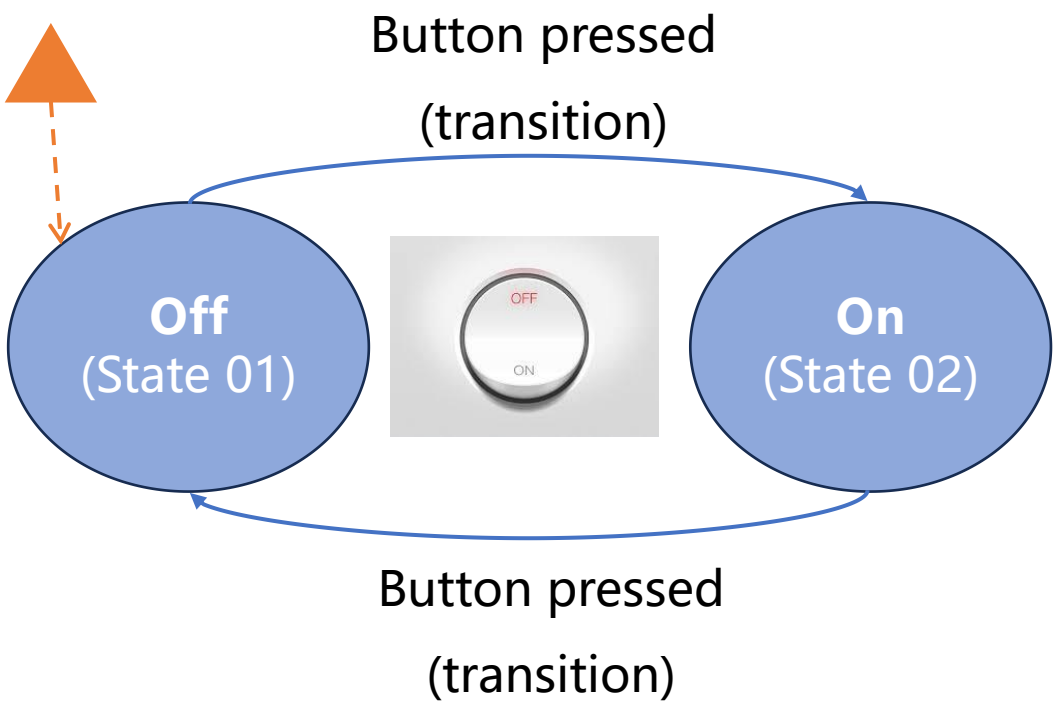


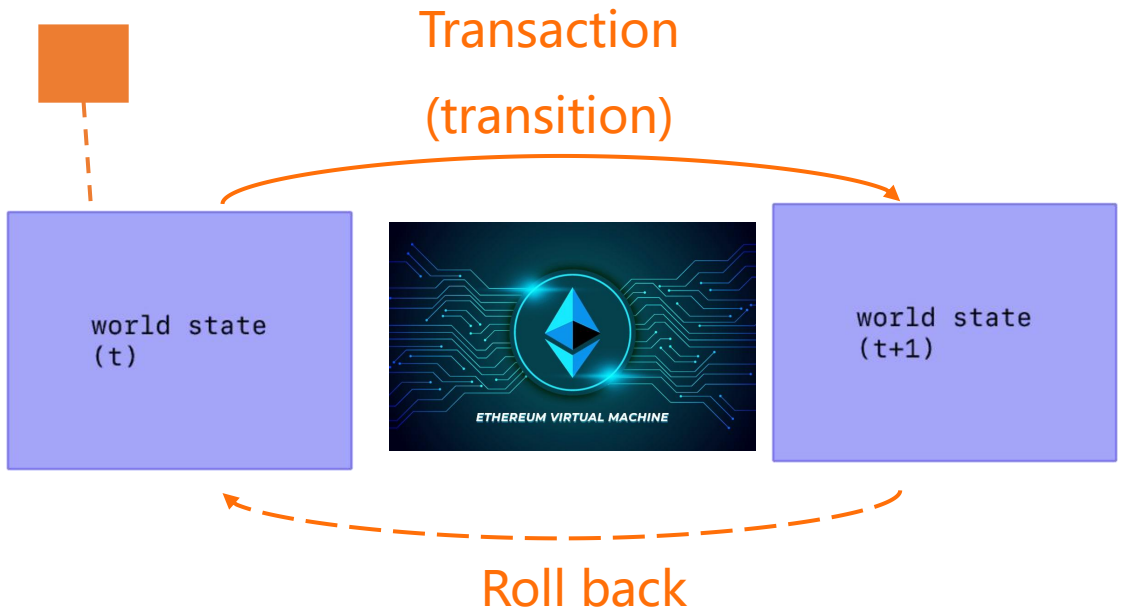
State Machine

State Machine is a mathematical abstraction used to design algorithms.

- A state machine reads a set of inputs and changes to a different state based on those inputs.

Ethereum Virtual Machine (EVM) can be viewed as a **transaction-based state machine**

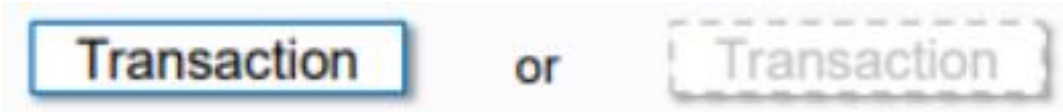




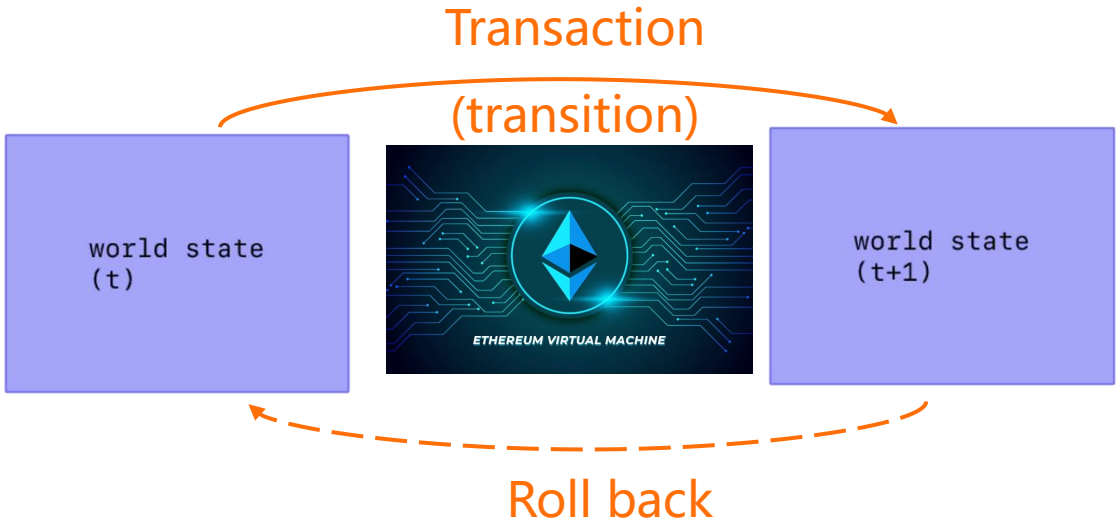
Atomicity of Transactions

■ Atomicity of transaction

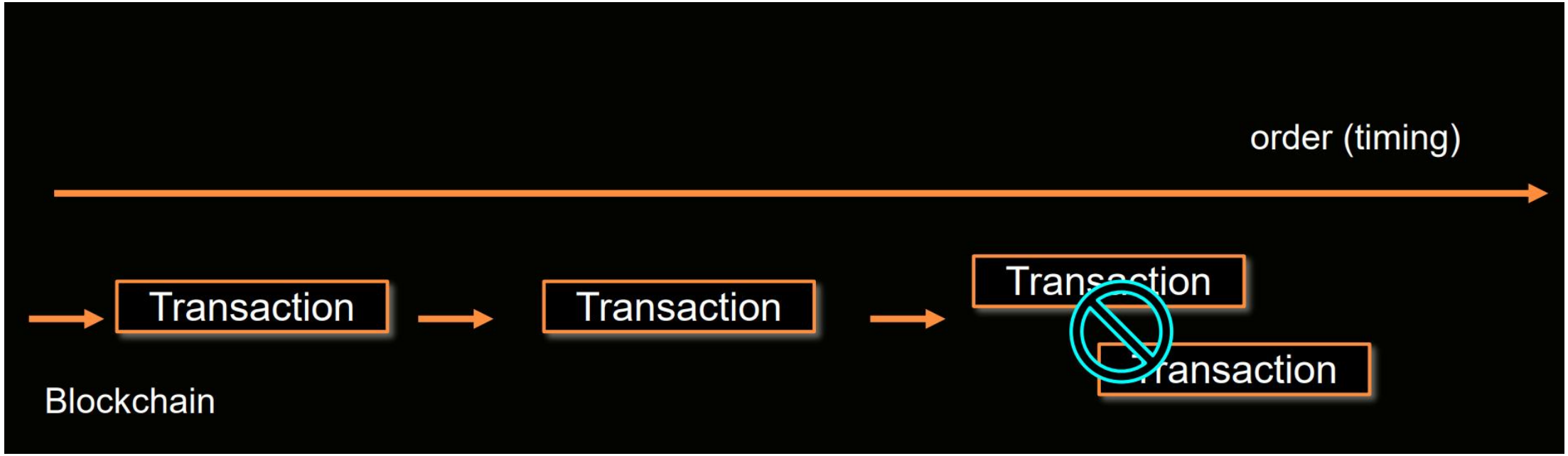
A transaction is an atomic operation. Can't divide or interrupt



That is, **All** (complete done) or **Nothing** (zero effect)

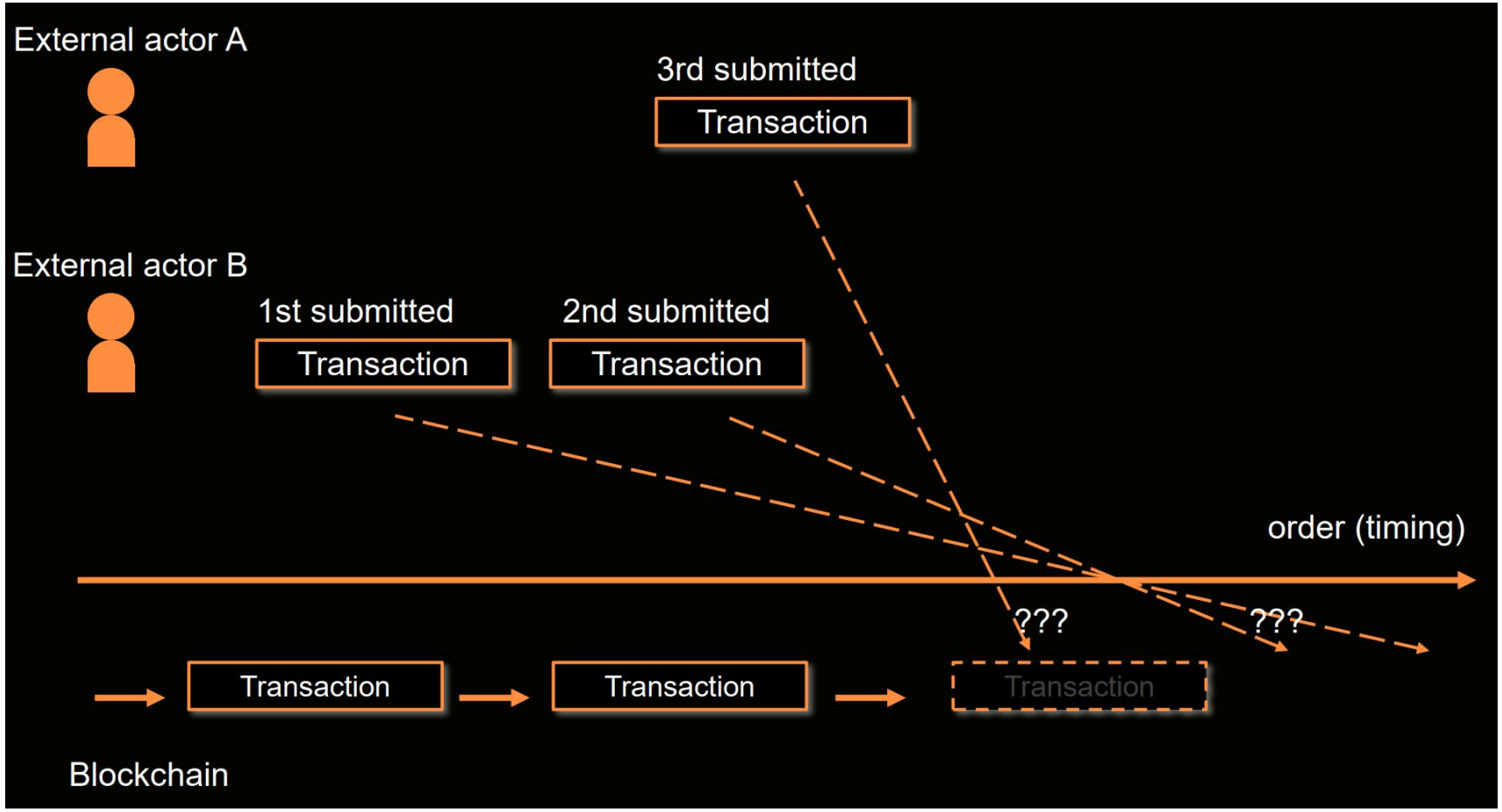


Order of Transactions

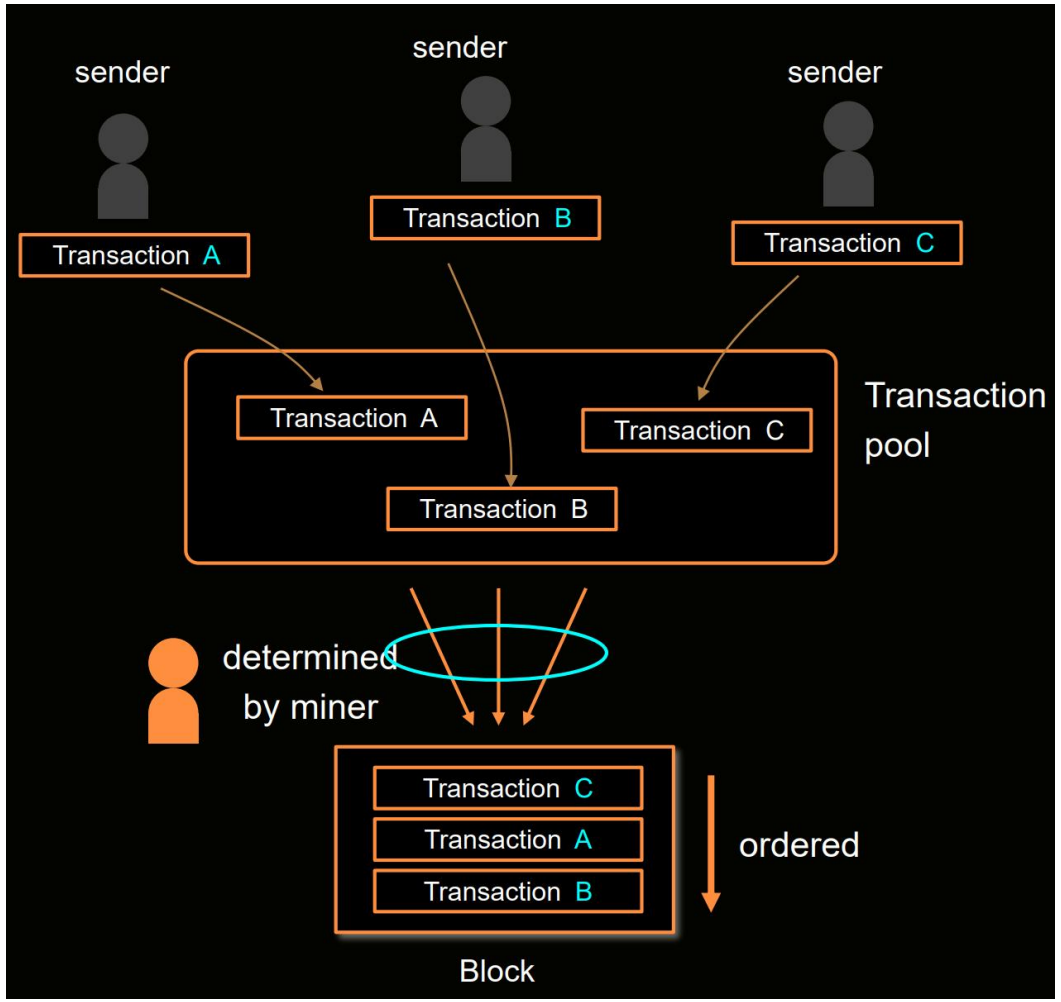


Transactions cannot be overlapped.
Transactions must be executed sequentially.

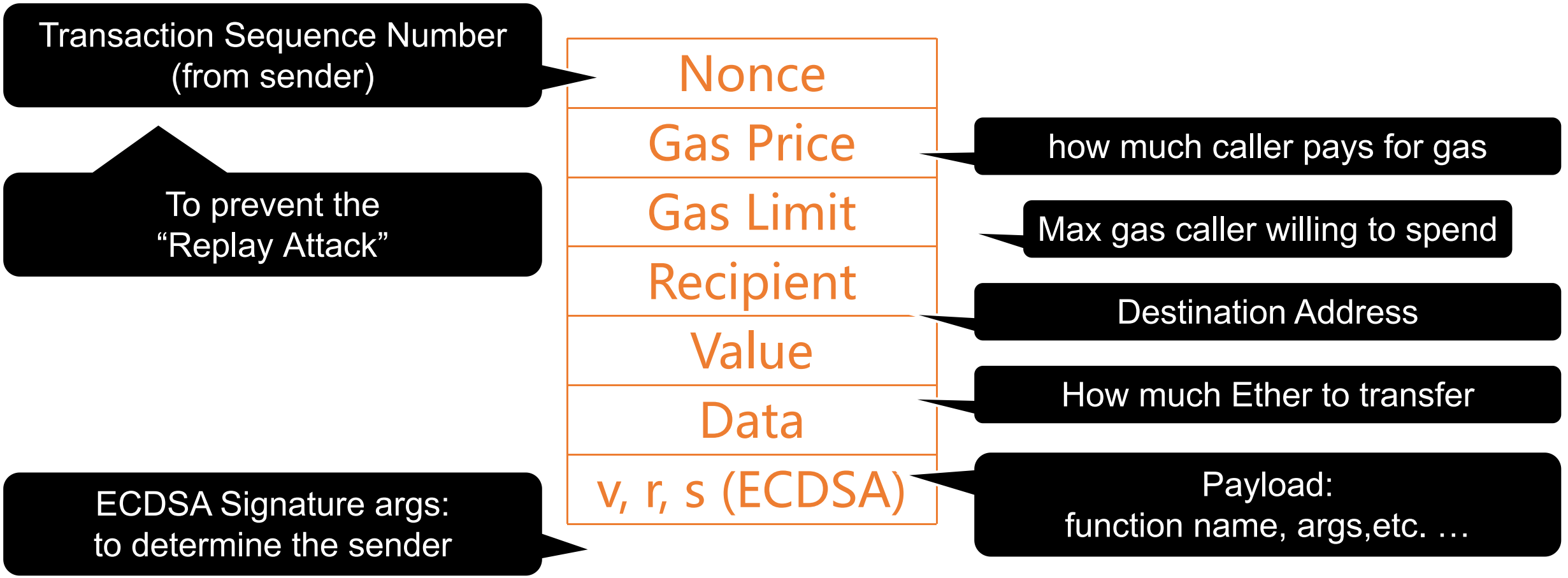
Transaction Order is Not Guaranteed



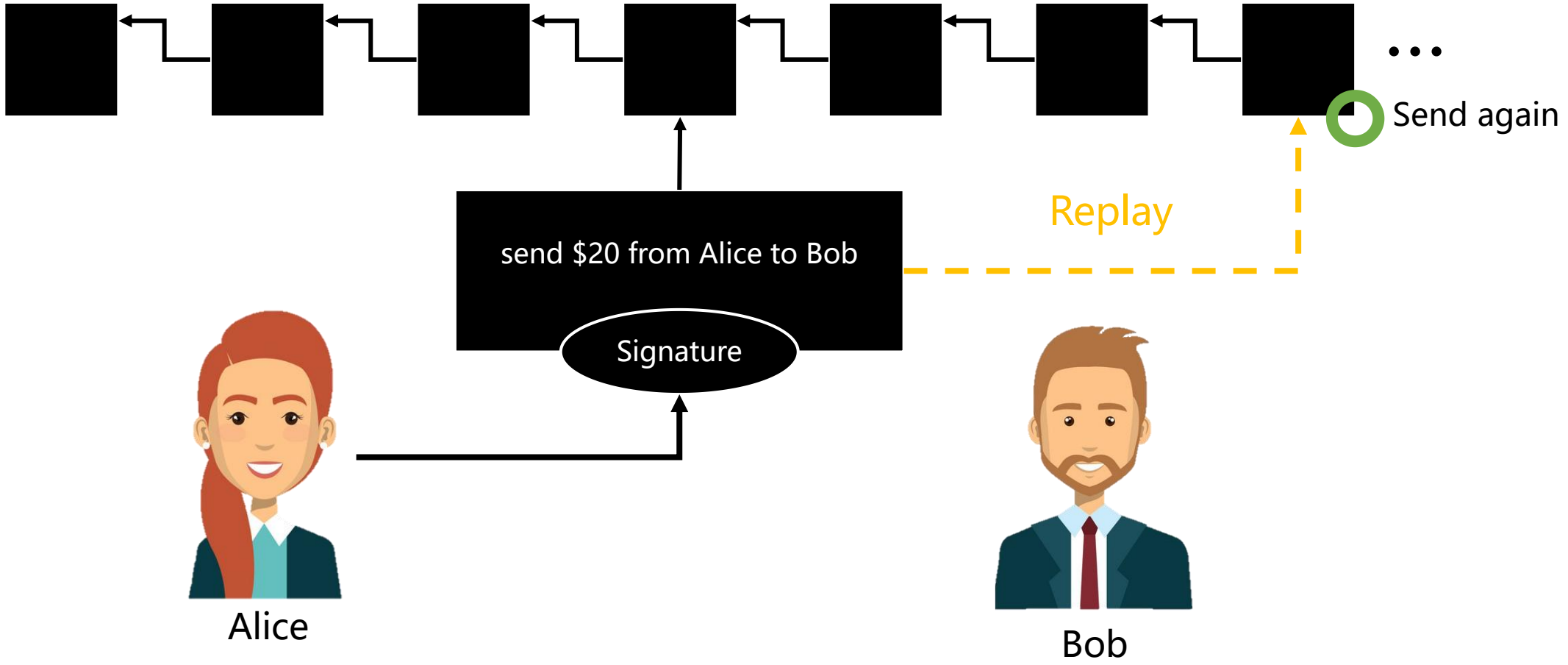
The Order of Transactions in a Block



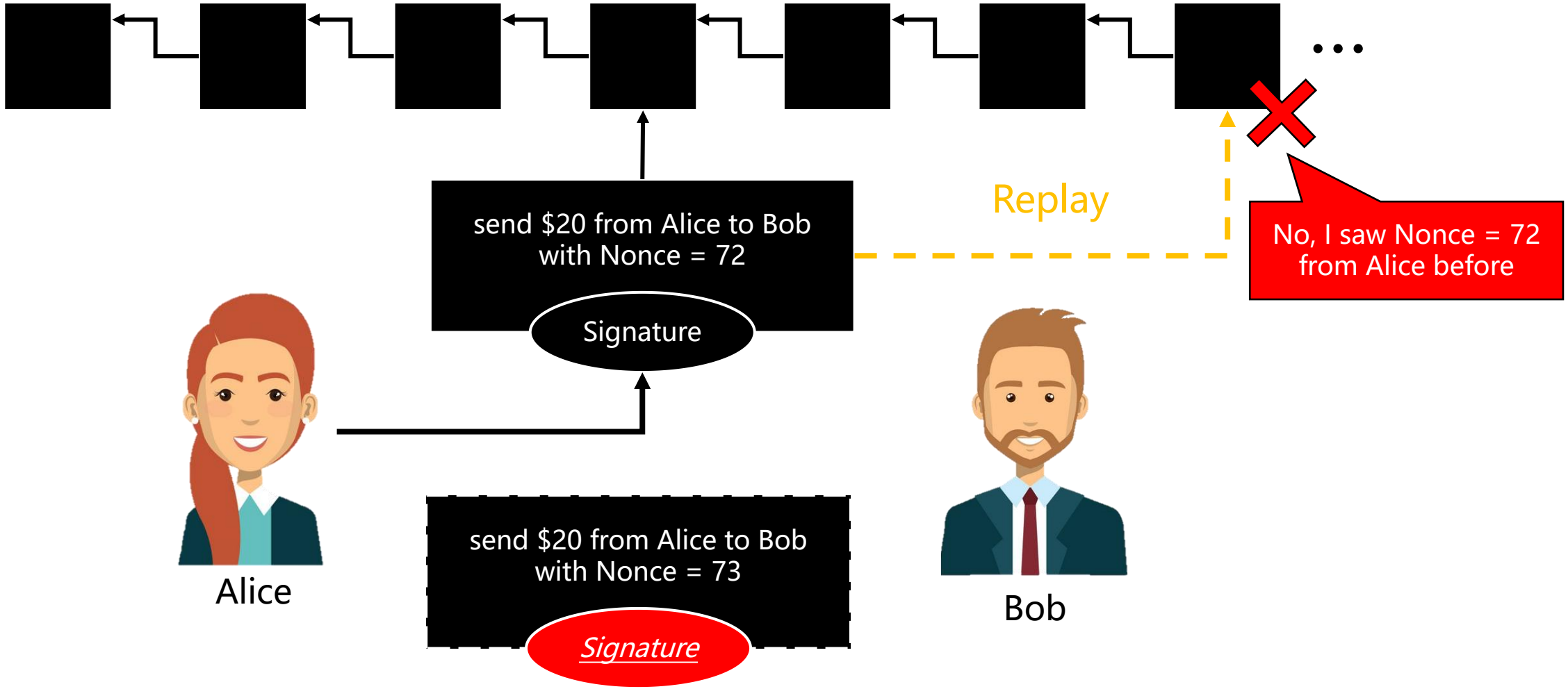
Ethereum Transaction



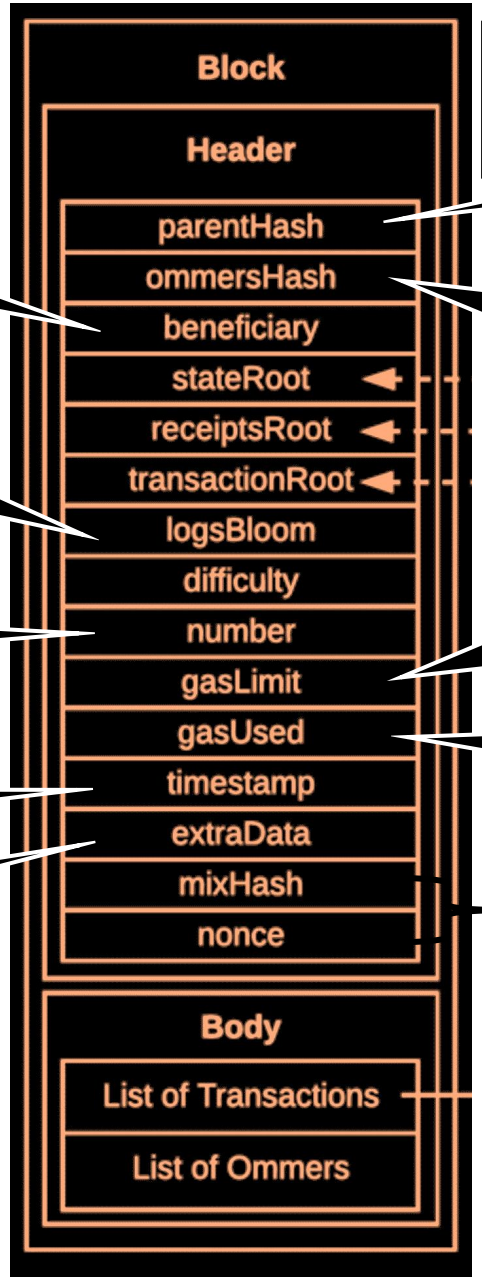
Replay Attack: A Problem in Account Model



Replay Attack: Nonce Solution



Ethereum Block



Hash of the block header from the Previous Block

Mining & Fee Reward Recipient

Hash of the Uncle Blocks headers; Receive 1/32 of the block reward

Bloom filter for finding out if logs were generated on transactions in this block

The Maximum Gas can be used by the transactions in the block.

The Height. Genesis : 0.

Total Gas Cost of Transactions in the block

Block Creation Time

Ethash (Dagger-Hashimoto) PoW Proof

Arbitrary byte array that can contain anything.

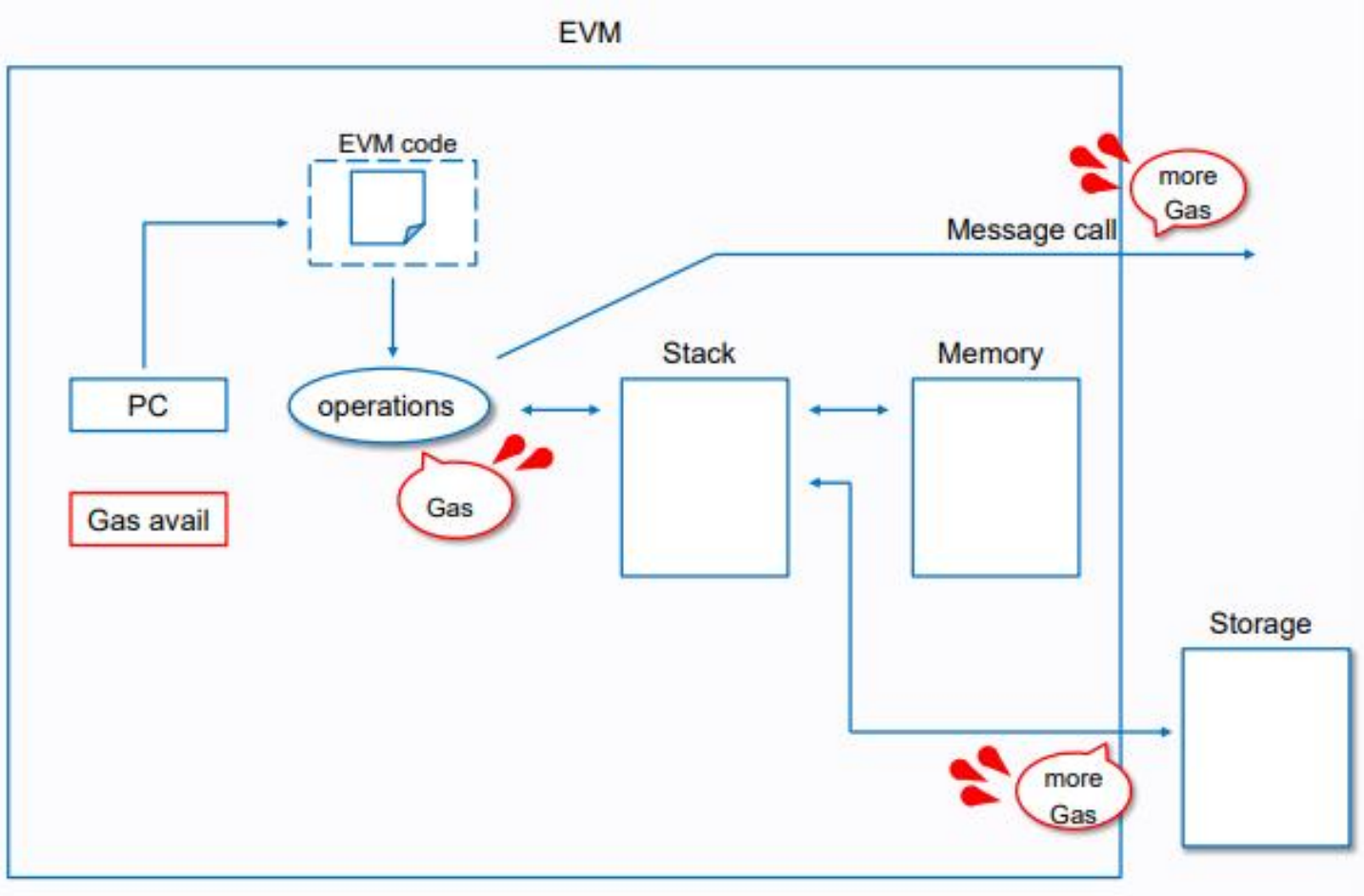
Ethereum Virtual Machine (EVM): Definition

The Ethereum Virtual Machine is the **runtime environment** for smart contracts in Ethereum.

- EVM is a Virtual Machine …
- EVM is a State Machine …
- **EVM is a Quasi-Turing complete machine…**

Turing complete machine: The kind of machine that enables running a program that can answer/solve any computational problem, irrespective of the time or memory it takes to do so.

EVM Execution Model



Turing Complete Framework:

- Can theoretically execute any algorithm.
- Comprises EVM code, stack, memory, and persistent storage.

Quasi-Turing Complete:

- Practical execution limited by gas.
- Prevents infinite loops, ensuring network security.

What is Gas?

Definition: Gas refers to the unit that measures the amount of computational effort required to execute specific operations on the Ethereum network.



Gas

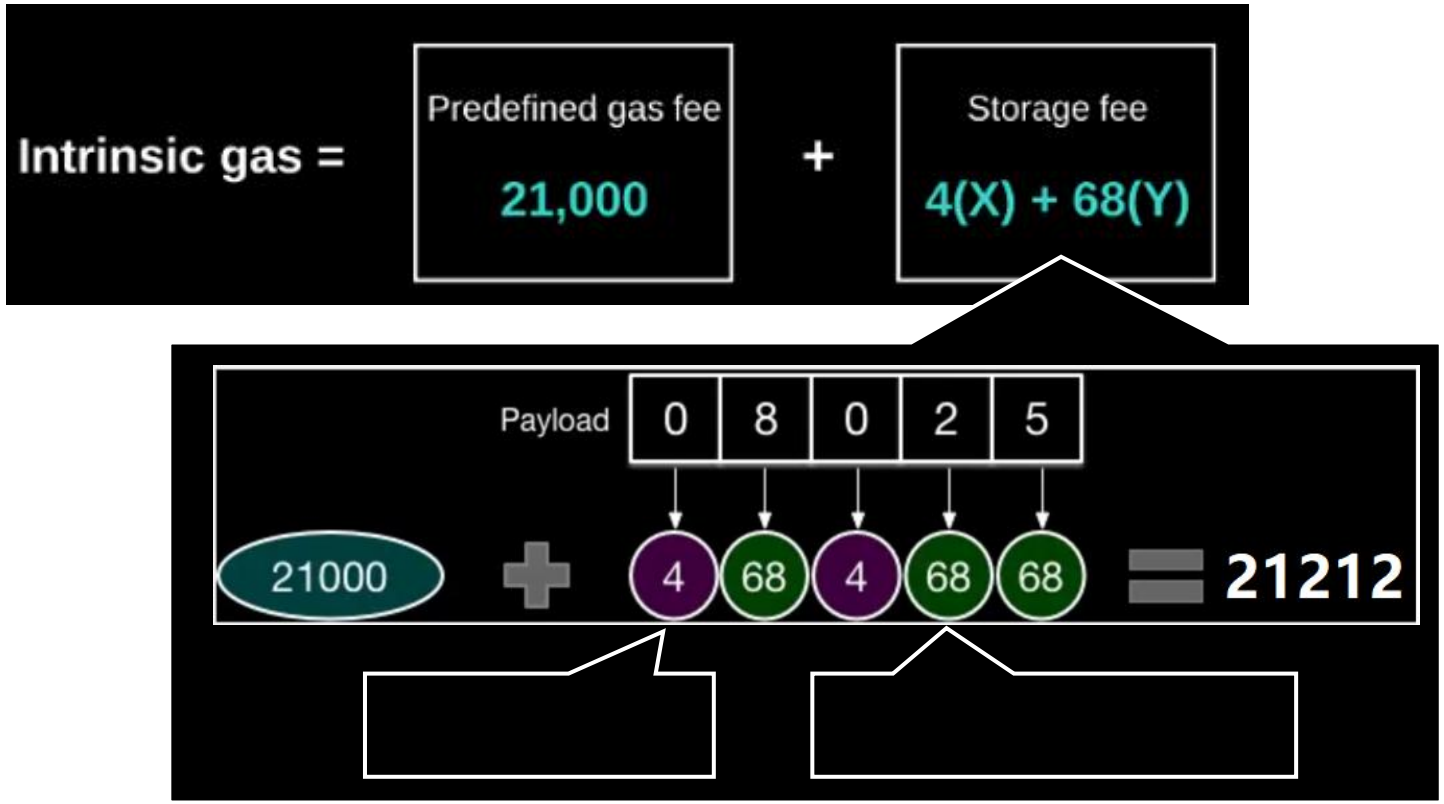
Caller pays fee for each transaction step

Total Gas = Intrinsic Gas + Operation Gas



Intrinsic Gas

Each step has Intrinsic (fixed) “gas” fee





Operation Gas





Appendix G. Fee Schedule

The fee schedule G is a tuple of 31 scalar values corresponding to the relative costs, in gas, of a number of abstract operations that a transaction may effect.

Name	Value	Description*
G_{zero}	0	Nothing paid for operations of the set W_{zero}.
G_{base}	2	Amount of gas to pay for operations of the set W_{base}.
$G_{verylow}$	3	Amount of gas to pay for operations of the set $W_{verylow}$.
G_{low}	5	Amount of gas to pay for operations of the set W_{low}.
G_{mid}	8	Amount of gas to pay for operations of the set W_{mid}.
G_{high}	10	Amount of gas to pay for operations of the set W_{high}.
$G_{extcode}$	700	Amount of gas to pay for operations of the set $W_{extcode}$.
$G_{balance}$	400	Amount of gas to pay for a BALANCE operation.
G_{sload}	200	Paid for a SLOAD operation.
$G_{jumpdest}$	1	Paid for a JUMPDEST operation.
G_{sset}	20000	Paid for an SSTORE operation when the storage value is set to non-zero from zero.
G_{sreset}	5000	Paid for an SSTORE operation when the storage value's zeroness remains unchanged or is set to zero.
R_{sclear}	15000	Refund given (added into refund counter) when the storage value is set to zero from non-zero.
$R_{selfdestruct}$	24000	Refund given (added into refund counter) for self-destructing an account.
$G_{selfdestruct}$	5000	Amount of gas to pay for a SELFDESTRUCT operation.
G_{create}	32000	Paid for a CREATE operation.
$G_{codedeposit}$	200	Paid per byte for a CREATE operation to succeed in placing code into state.
G_{call}	700	Paid for a CALL operation.
$G_{callvalue}$	9000	Paid for a non-zero value transfer as part of the CALL operation.
$G_{callstipend}$	2300	A stipend for the called contract subtracted from $G_{callvalue}$ for a non-zero value transfer.
$G_{newaccount}$	25000	Paid for a CALL or SELFDESTRUCT operation which creates an account.
G_{exp}	10	Partial payment for an EXP operation.
$G_{expbyte}$	50	Partial payment when multiplied by $\lceil \log_{256}(exponent) \rceil$ for the EXP operation.
G_{memory}	3	Paid for every additional word when expanding memory.
G_{txcreate}	32000	Paid by all contract-creating transactions after the *Homestead* transition.
$G_{txdatazero}$	4	Paid for every zero byte of data or code for a transaction.
$G_{txdatanonzero}$	68	Paid for every non-zero byte of data or code for a transaction.
$G_{transaction}$	21000	Paid for every transaction.
G_{log}	375	Partial payment for a LOG operation.
G_{logdata}	8	Paid for each byte in a LOG operation's data.
G_{logtopic}	375	Paid for each topic of a LOG operation.
G_{sha3}	30	Paid for each SHA3 operation.
$G_{sha3word}$	6	Paid for each word (rounded up) for input data to a SHA3 operation.
G_{copy}	3	Partial payment for *COPY operations, multiplied by words copied, rounded up.
$G_{blockhash}$	20	Payment for BLOCKHASH operation.
$G_{quaddivisor}$	20	The quadratic coefficient of the input sizes of the exponentiation-over-modulo precompiled contract.

CYBER AND CRYPTO LAB

Ethereum's Gas System

- **Gas per Instruction**: Each line of code consumes a specific amount of Gas when executed.
- **Defined Costs**: The Ethereum Yellow Paper details the Gas cost for each instruction.
- **Miner Incentives**: Gas fees incentivize miners to validate and execute transactions.

Appendix G. Fee Schedule

The fee schedule G is a tuple of 31 scalar values corresponding to the relative costs, in gas, of a number of abstract operations that a transaction may effect.

Name	Value	Description*
G_{zero}	0	Nothing paid for operations of the set W_{zero}.
G_{base}	2	Amount of gas to pay for operations of the set W_{base}.
$G_{verylow}$	3	Amount of gas to pay for operations of the set $W_{verylow}$.
G_{low}	5	Amount of gas to pay for operations of the set W_{low}.
G_{mid}	8	Amount of gas to pay for operations of the set W_{mid}.
G_{high}	10	Amount of gas to pay for operations of the set W_{high}.
$G_{extcode}$	700	Amount of gas to pay for operations of the set $W_{extcode}$.
$G_{balance}$	400	Amount of gas to pay for a BALANCE operation.
G_{sload}	200	Paid for a SLOAD operation.
$G_{jumpdest}$	1	Paid for a JUMPDEST operation.
G_{sset}	20000	Paid for an SSTORE operation when the storage value is set to non-zero from zero.
G_{sreset}	5000	Paid for an SSTORE operation when the storage value's zeroness remains unchanged or is set to zero.
R_{sclear}	15000	Refund given (added into refund counter) when the storage value is set to zero from non-zero.
$R_{suicide}$	24000	Refund given (added into refund counter) for suiciding an account.
$G_{suicide}$	5000	Amount of gas to pay for a SUICIDE operation.
G_{create}	32000	Paid for a CREATE operation.
$G_{codedeposit}$	200	Paid per byte for a CREATE operation to succeed in placing code into state.
G_{call}	700	Paid for a CALL operation.
$G_{callvalue}$	9000	Paid for a non-zero value transfer as part of the CALL operation.
$G_{callstipend}$	2300	A stipend for the called contract subtracted from $G_{callvalue}$ for a non-zero value transfer.
$G_{newaccount}$	25000	Paid for a CALL or SUICIDE operation which creates an account.
G_{exp}	10	Partial payment for an EXP operation.
$G_{expbyte}$	10	Partial payment when multiplied by $\lceil \log_{256}(exponent) \rceil$ for the EXP operation.
G_{memory}	3	Paid for every additional word when expanding memory.
$G_{txcreate}$	32000	Paid by all contract-creating transactions after the *Homestead transition*.
$G_{txdatazero}$	4	Paid for every zero byte of data or code for a transaction.
$G_{txdatanonzero}$	68	Paid for every non-zero byte of data or code for a transaction.
$G_{transaction}$	21000	Paid for every transaction.
G_{log}	375	Partial payment for a LOG operation.
$G_{logdata}$	8	Paid for each byte in a LOG operation's data.
$G_{logtopic}$	375	Paid for each topic of a LOG operation.
G_{sha3}	30	Paid for each SHA3 operation.
$G_{sha3word}$	6	Paid for each word (rounded up) for input data to a SHA3 operation.
G_{copy}	3	Partial payment for *COPY operations, multiplied by words copied, rounded up.
$G_{blockhash}$	20	Payment for BLOCKHASH operation.

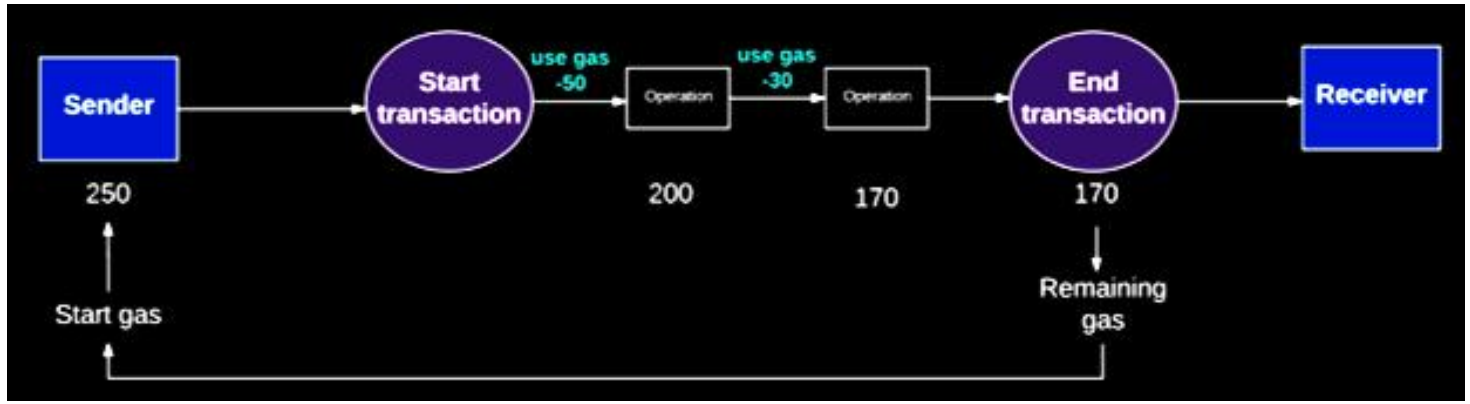
Gas Limit

Maximum amount of gas the users willing to pay

To prevent a outrageous bill from the system



Gas

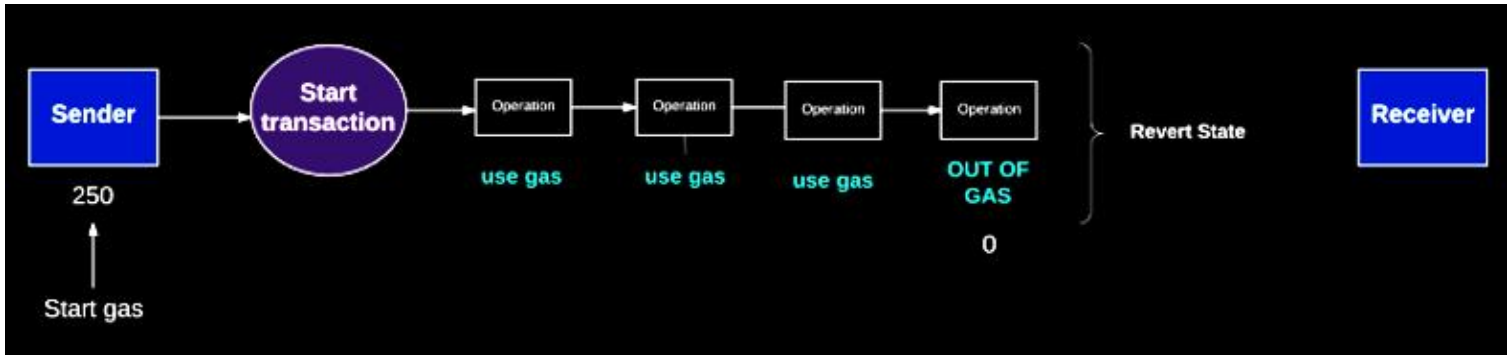


If a call has leftover gas …

Unused gas refunded



Gas



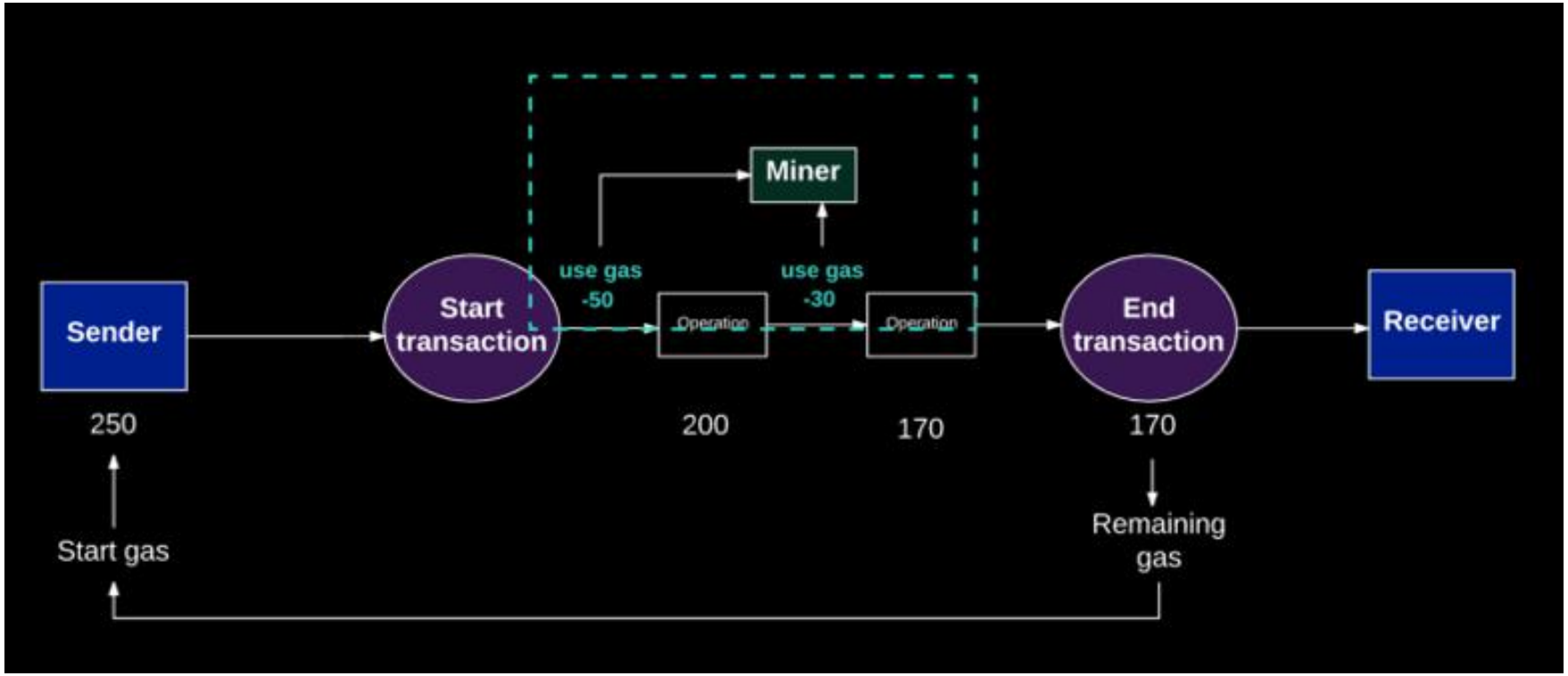
If a call runs out of gas …

Effects discarded

Gas not refunded



Where does the gas fee go?



Ethereum's Gas System

- **Gas Price: This is the amount of Ether you are willing to pay per unit of gas, measured in gwei,** with $10^{18} Wei = 10^{9}$ ***Gwei*** $= 10^{6}\ Szabo = 10^{3}\ Fonney = 1\ Ether$
- **Gas Used:** This is the unit that measures the amount of computational effort required to execute operations, like transactions or smart contract functions.
- **Gas Limit:** The maximum amount of gas you're willing to consume for the transaction.
- **Gas Fee:** This is the total cost of the transaction or computation, paid to miners or validators for processing and validating transactions on the blockchain.

Gas Fee = Gas Price × Gas Used

Gas Price

Gas price in Ether up to caller!



Low price means low priority …

and vice-versa

Common Gas Price Unit: Gwei



Gas War!

Block Gas Limit

Bitcoin has limit on block size

Ethereum has limit on block gas

Block full when transactions' gas costs reach limit



How Gas Is Involved In Ethereum Transactions

- **Setting Gas Limit and Gas Price:** When you initiate a transaction, you specify a Gas Limit and Gas Price:
  - **Gas Limit:** The maximum amount of gas you're willing to consume for the transaction.
  - **Gas Price:** The amount of Ether (in Gwei) you are willing to pay per unit of gas. This acts like a "bid" for network resources – higher gas prices usually mean faster transaction processing.





- **Preventing Spam and Network Abuse:**
  - Gas fees serve as a deterrent against network abuse by imposing costs on computations and storage, making it costly for malicious actors to spam the network or exploit smart contracts.



- **Compare to Bitcoin Transaction Fees:**
  - Bitcoin transaction fees are determined by the size of the transaction and tend to be more stable, primarily serving to incentivize.
  - Ethereum's gas fees depend on the complexity of actions being executed, and are designed to manage computational resources.

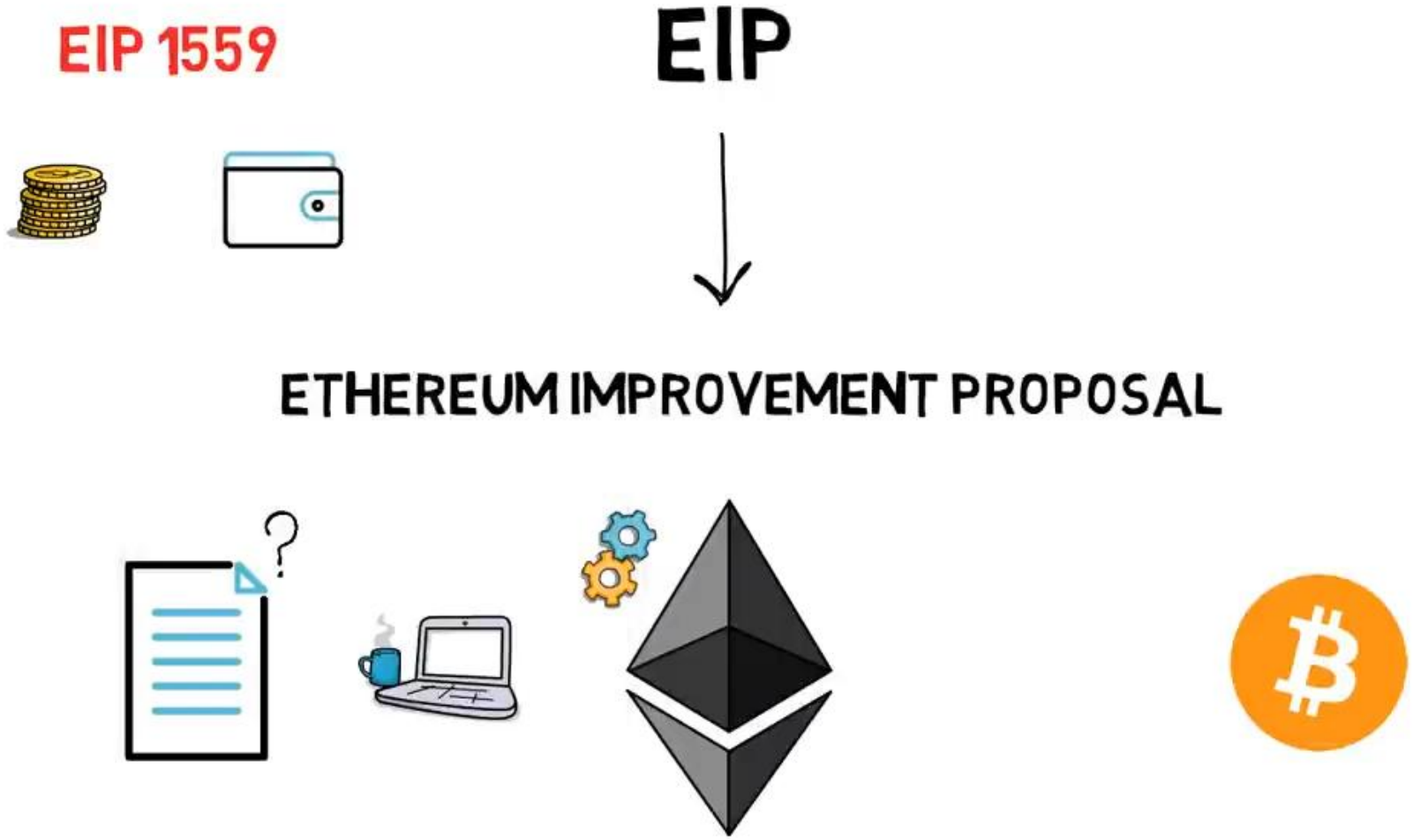
Why Gas Fee?

Reward for Mining Nodes

- Computation Cost
- Storage Cost

Protection for the Ethereum

- Prevent Infinite Loop Problem
- Make Denial of Service Attacks Expensive



London Upgrade (EIP-1559)

London Upgrade (EIP-1559)

Hard Fork on August 2021

Base Fee

- Burned: will be removed from circulation
- Predictable: maximal increase 12.5% per block when busy

Priority Fee (Tips)

- to incentivize miners to include a transaction in the block
- a higher tip for faster transactions

Max Fee (Gas Limit)

- a.k.a. maxFeePerGas
- set by users (21000 by default)

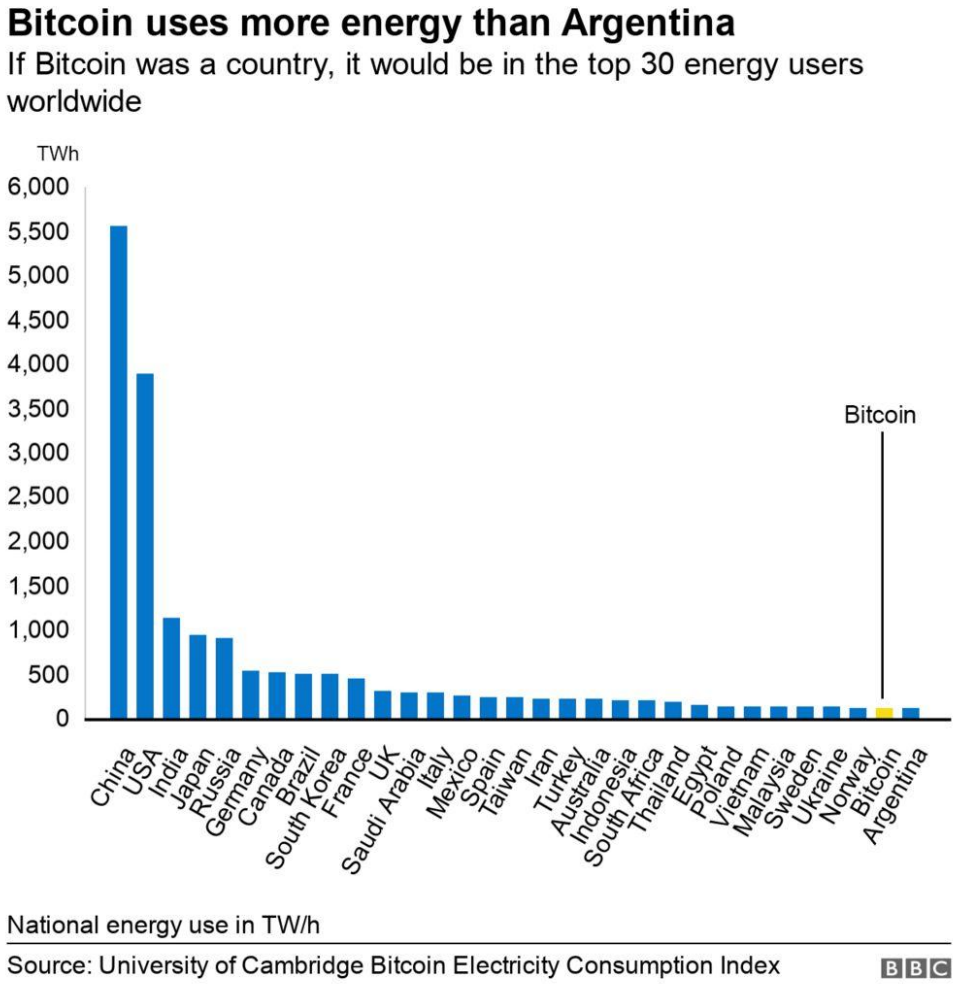
Max Base Fee

Some wallets (e.g. metamask) does not have Max Fee settings
Instead, you can set Max Base Fee



If your Max Base Fee < Base Fee, your transaction can not be included

Why Need Proof-of-stake (PoS)?



Bitcoin's Surging Energy Consumption

- Electricity used in Bitcoin mining surpasses power consumption of 159 countries
- Over the course of a year that’s equal to around 64 TWh or terawatt hours of energy consumption. That’s more than the country of Switzerland uses over the same time period (58 TWh per year)

Proof-of-stake (PoS)

What is Proof-of-stake (PoS)?

- Proof-of-Stake (PoS) is a consensus mechanism in blockchain technology where the creation of new blocks and the validation of transactions are determined by the stake (i.e., the amount of cryptocurrency) held by a network participant.
- In PoS systems, the probability of validating a new block typically depends on the participant's proportion of the total currency staked, rather than on computational work, as in Proof-of-Work (PoW) systems.
- This mechanism aims to achieve distributed consensus while reducing the energy consumption and hardware requirements associated with PoW.



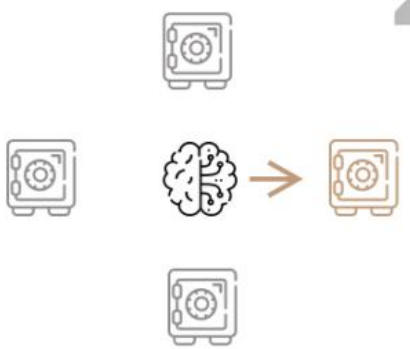
CYBER AND CRYPTO LAB

How PoS Works?

Owner stakes their native tokens in staking pools 1



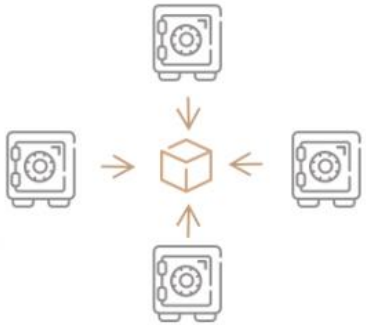
Algo pseudo-randomly selects next validator 2



The selected validator proposes a block of transactions 3



Other validators verify and approve the transaction 4



The block is added to the existing blockchain 5

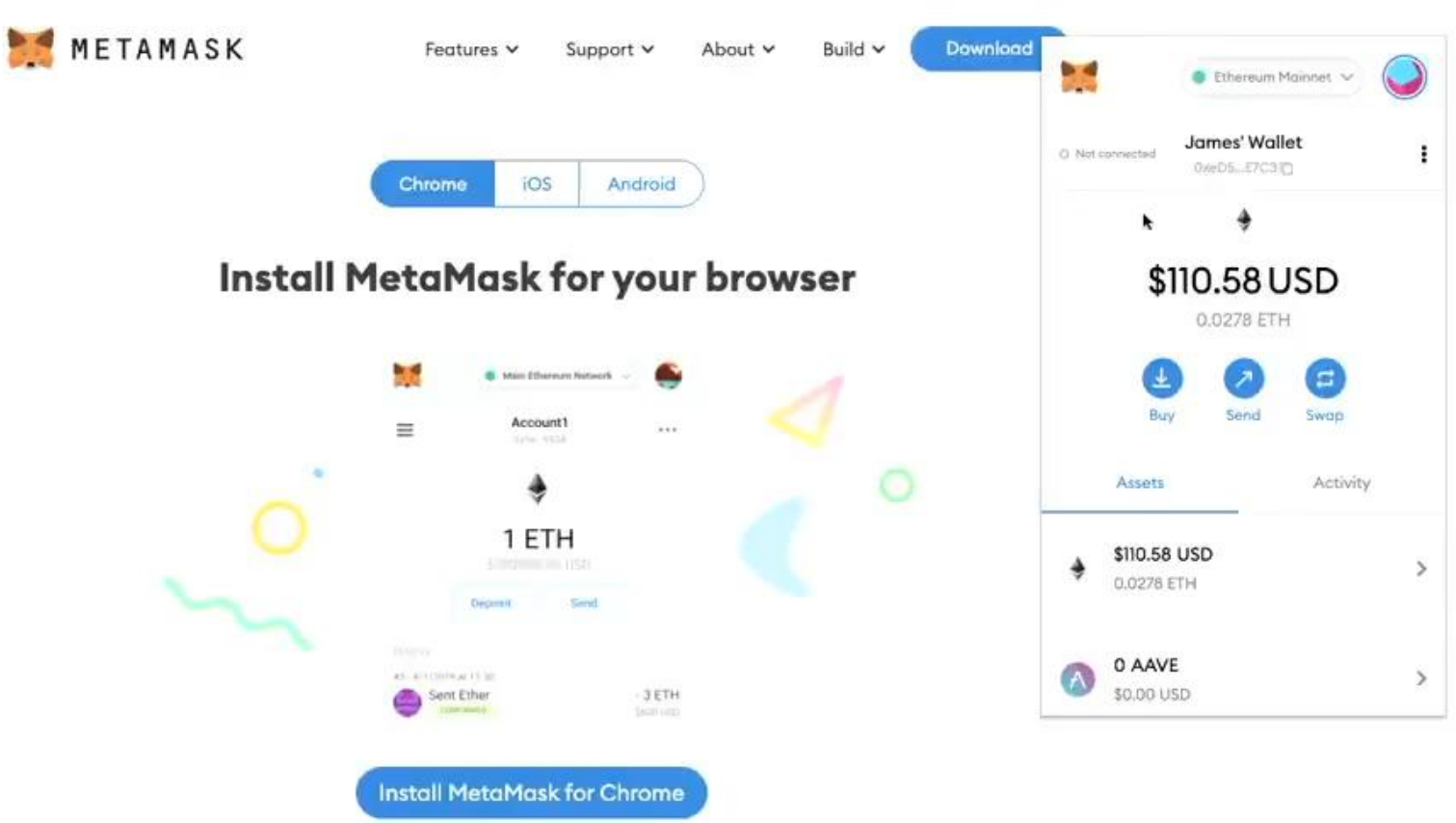


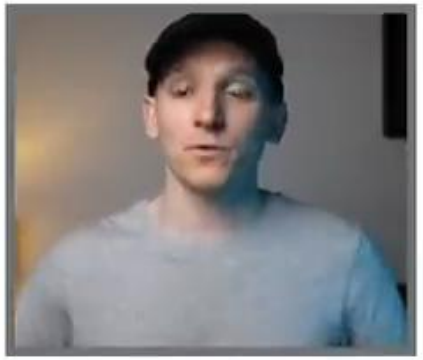
The validator earns a transaction fee 6



Summary

- EVM
- Smart Contract
- Transaction
- Account
- Gas
- Proof of Stake





Supported Browsers

Chrome Firefox Brave Edge

MetaMask Tutorial

CYBER AND CRYPTO LAB

Workshop:

Get Your On-chain Assets

澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

CYBER AND CRYPTO LAB

Content

1. Create Your Wallet  METAMASK
2. Buy Cryptocurrencies in a CEX (Centralized Exchange) BINANCE
3. Withdraw Cryptocurrencies to Your Wallet
4. Swap Assets in a DEX (Decentralized Exchange) UNISWAP

CYBER AND CRYPTO LAB

1. Create Your Wallet

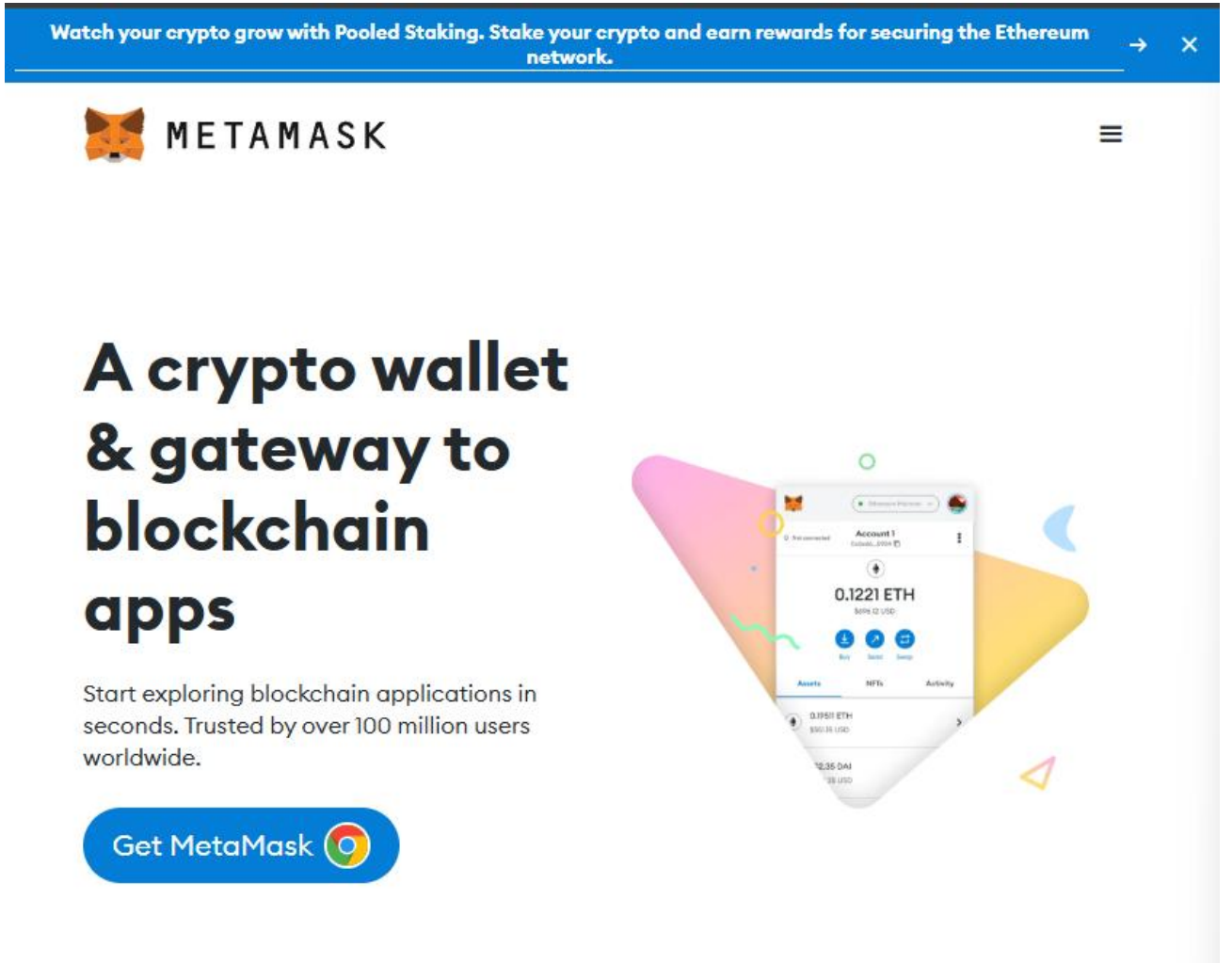
METAMASK

How to Access Metamask

- **Search on Search Engines**

1. Type " **Metamask** " in the search bar.
2. Be cautious to select the official website:

https://metamask.io/



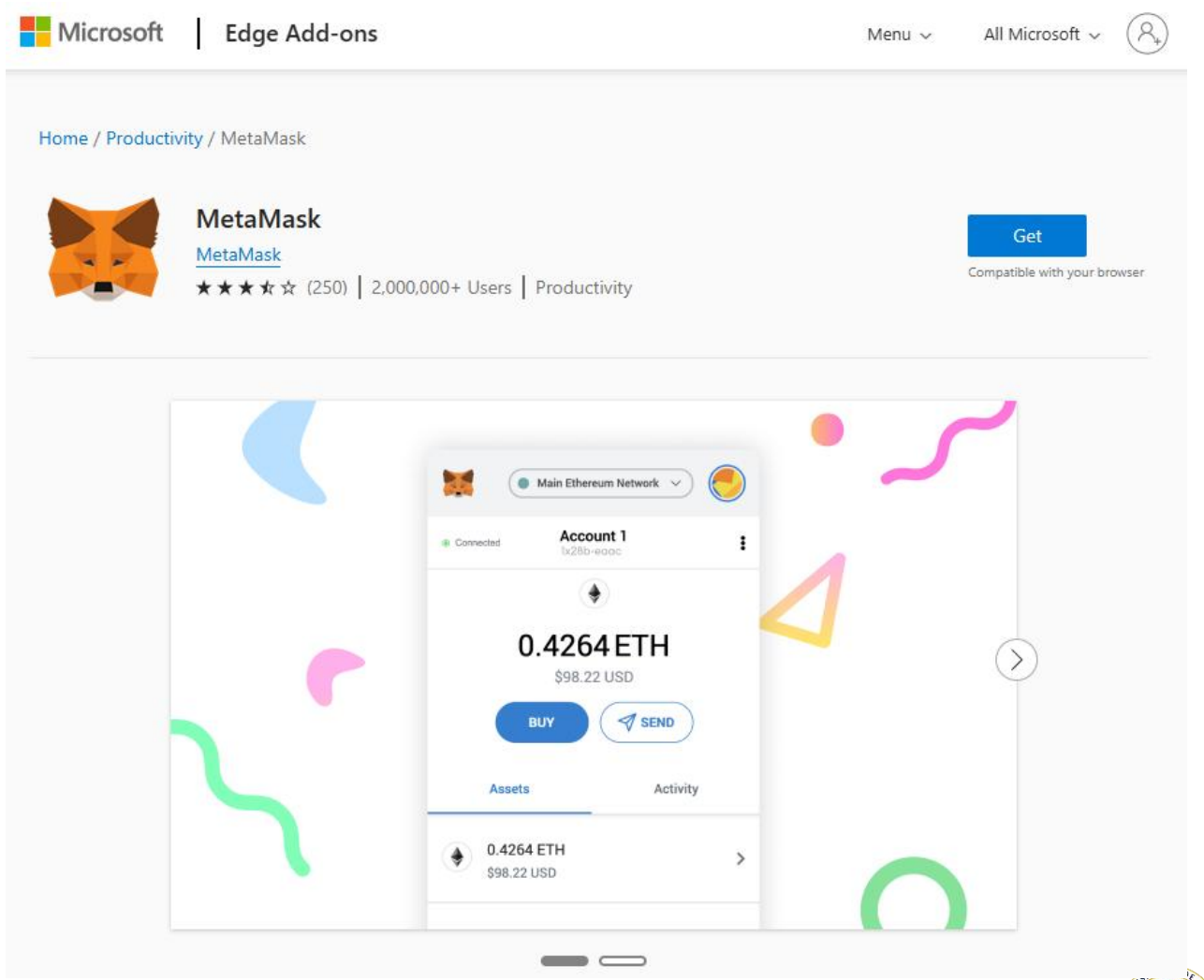
CYBER AND CRYPTO LAB

1. Create Your Wallet

METAMASK

How to Access Metamask

- **Install the browser plugin**



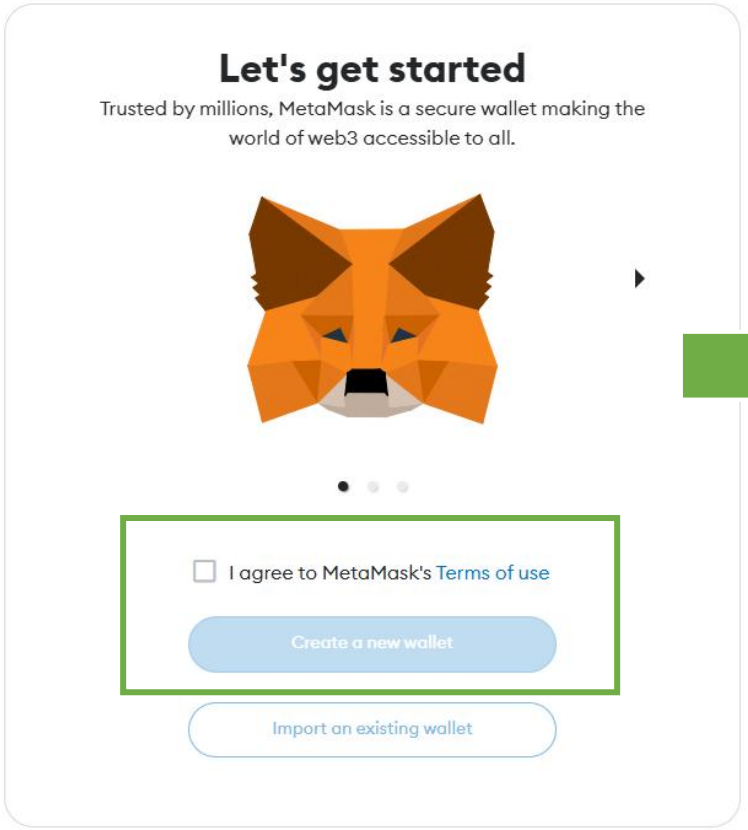
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

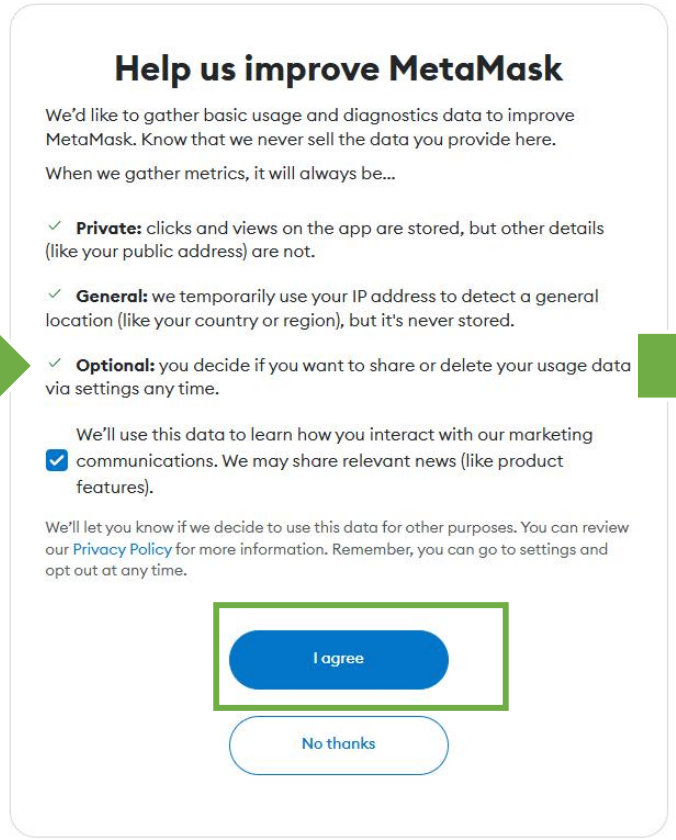
1. Create Your Wallet

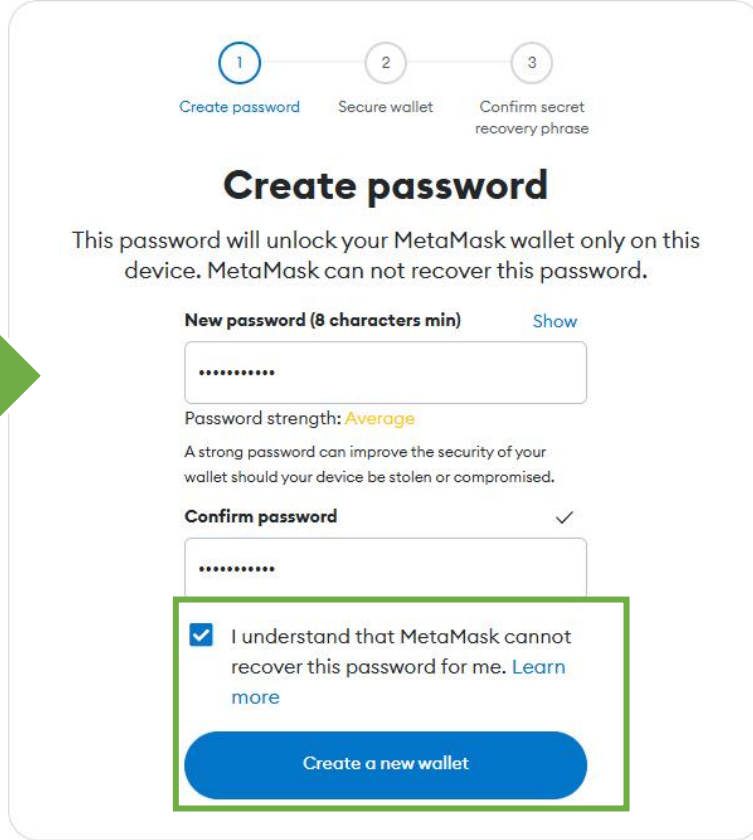
METAMASK

How to Access Metamask

- **Create your wallet**







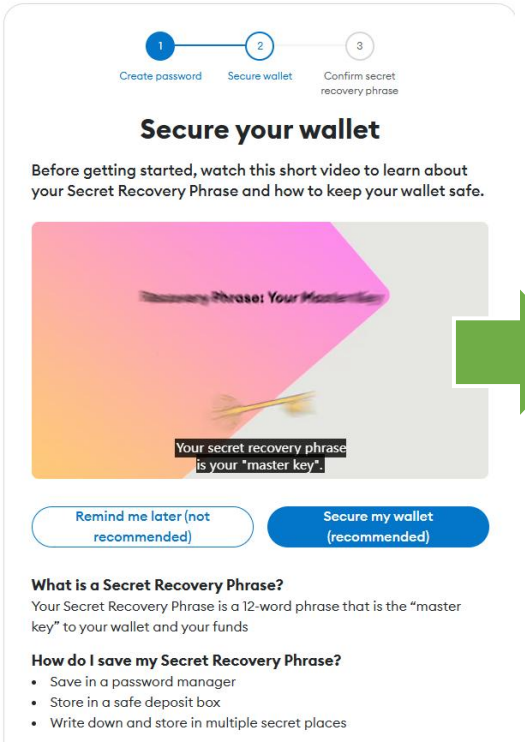
CYBER AND CRYPTO LAB

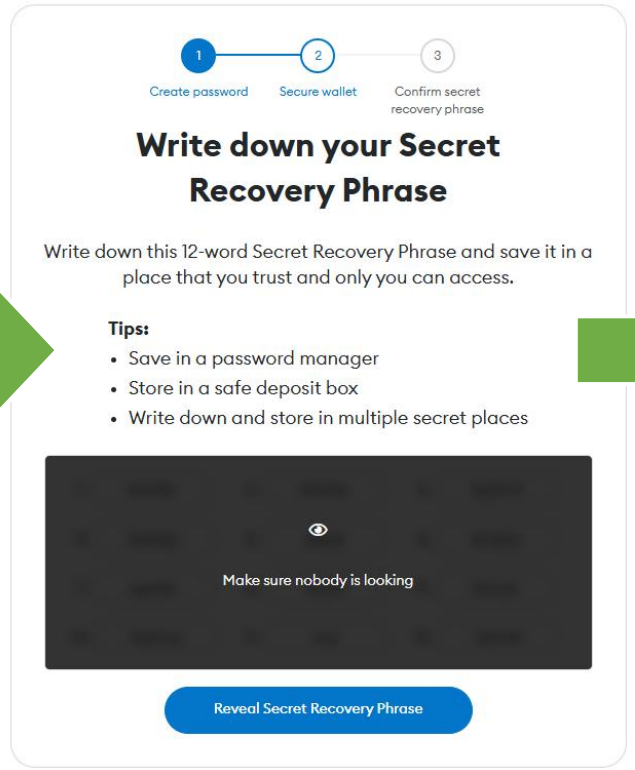
1. Create Your Wallet

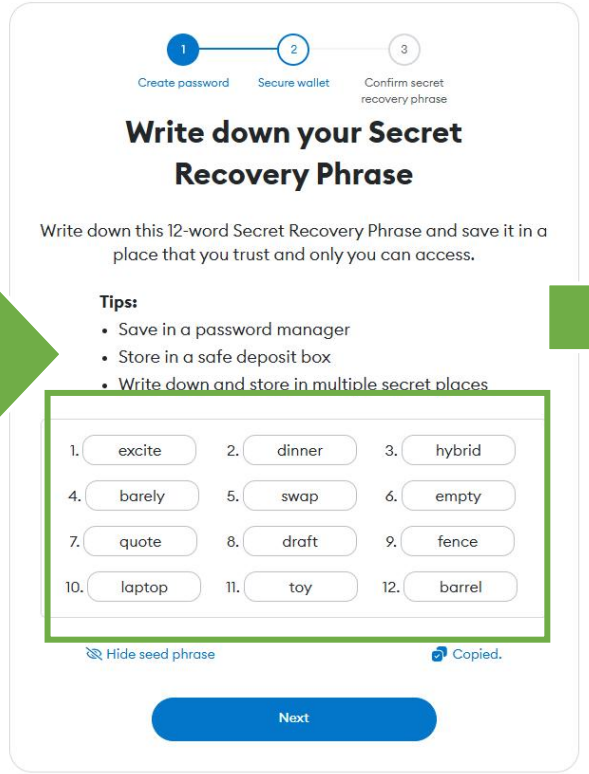
METAMASK

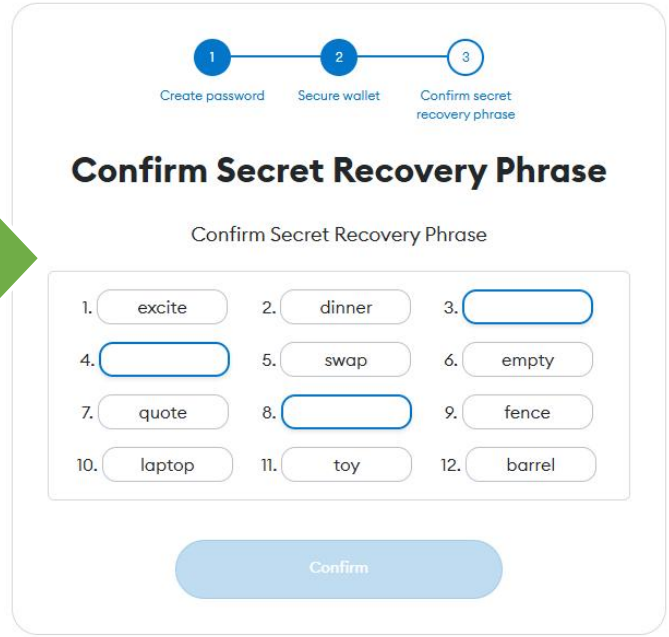
How to Access Metamask

■ Create your wallet







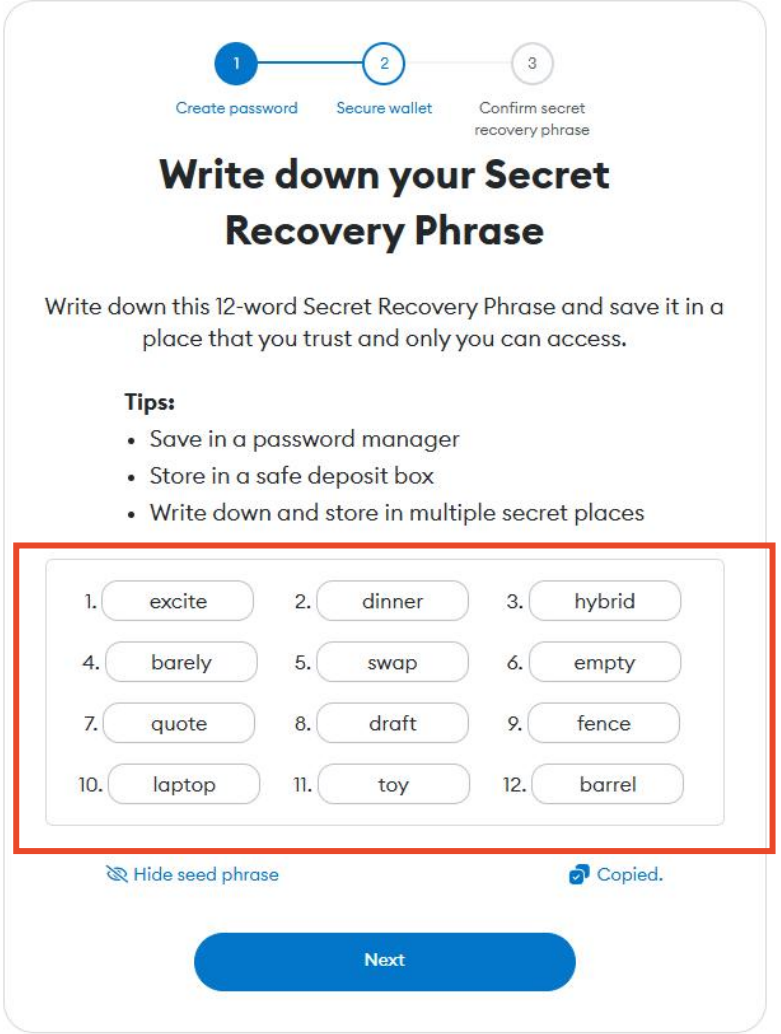


1. Create Your Wallet METAMASK

How to Access Metamask

Protect Your Secret Recovery Phrase!!

- **Why Protect Your Secret Recovery Phrase?**
  - It is the key to your wallet
  - Once lost, it cannot be recovered
  - Anyone with access to it can control your wallet
- **How to Secure Your Secret Recovery Phrase?**
  - Write it down on paper
  - Store it offline
  - **NEVER SHARE IT**



澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

CYBER AND CRYPTO LAB

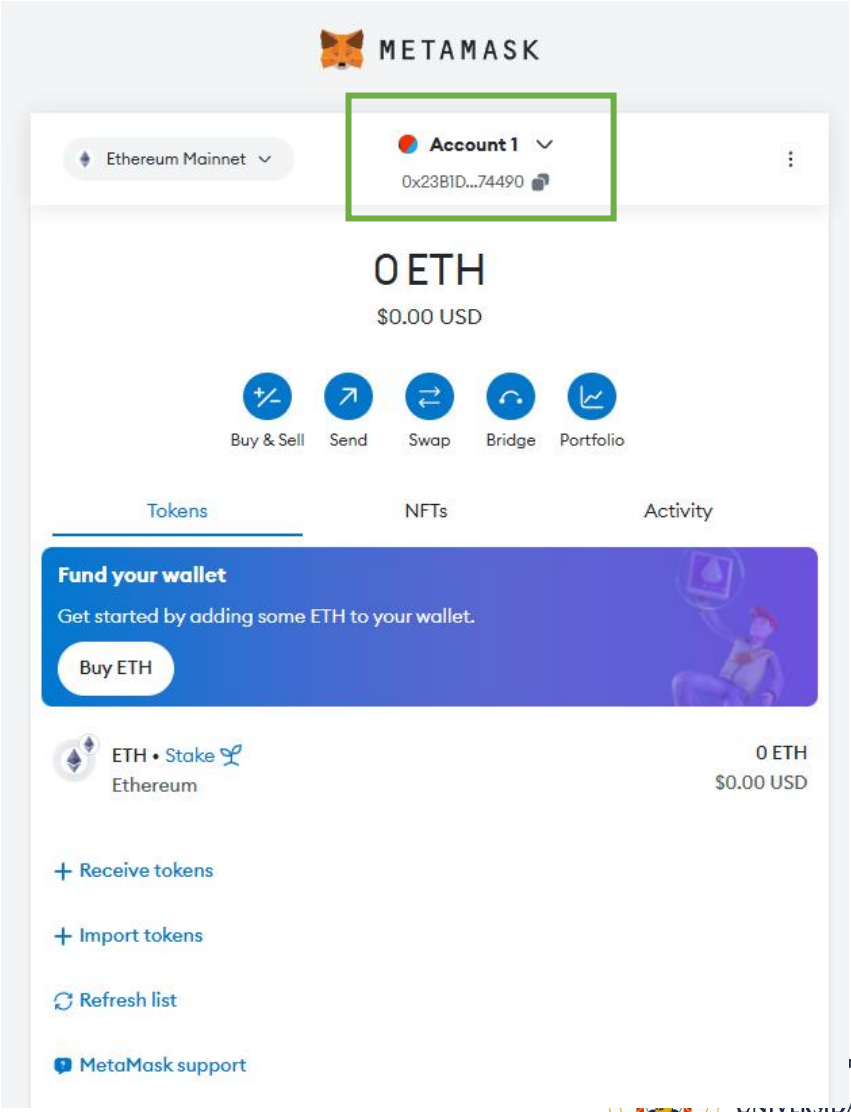
1. Create Your Wallet

METAMASK

Overview of the MetaMask Wallet Interface

Account Name & Address

- Shown at the top (e.g., Account 1 with address starting "0x23BID...").
- Use this to copy your wallet address for receiving tokens.



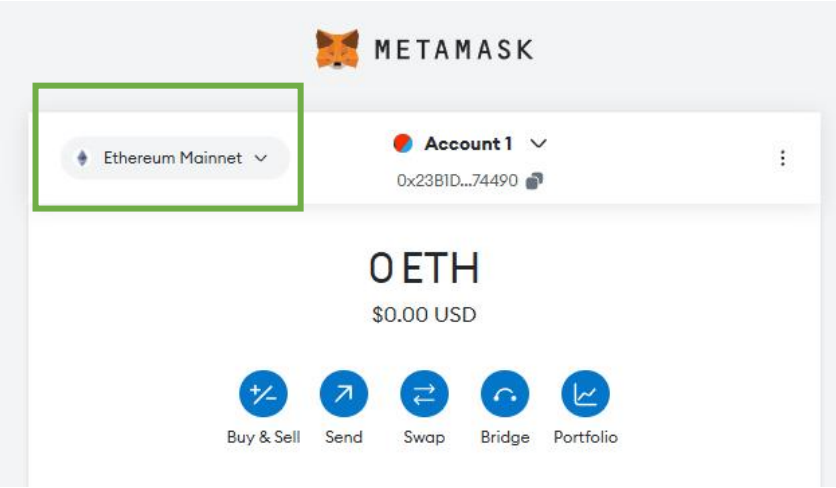
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

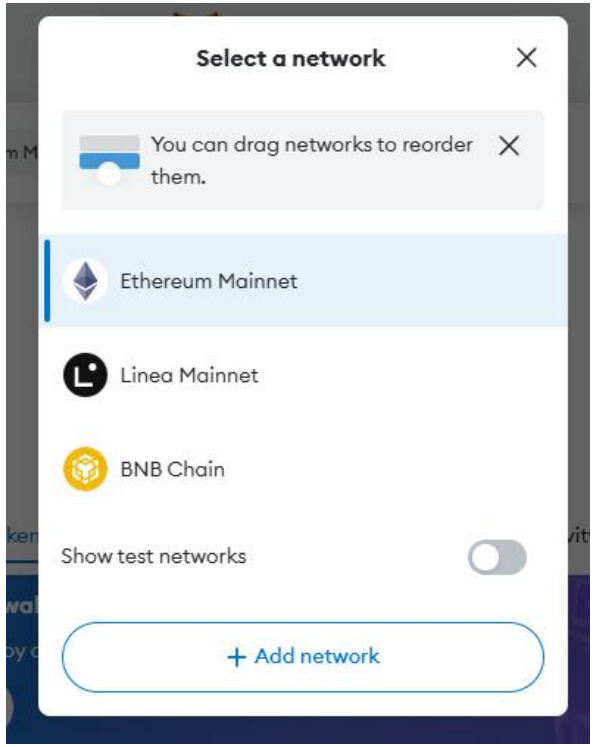
1. Create Your Wallet

Overview of the MetaMask Wallet Interface

Network Selection

- Located in the top-left (Ethereum Mainnet by default).
- Switch between different blockchain networks (e.g., Polygon, Binance Smart Chain).





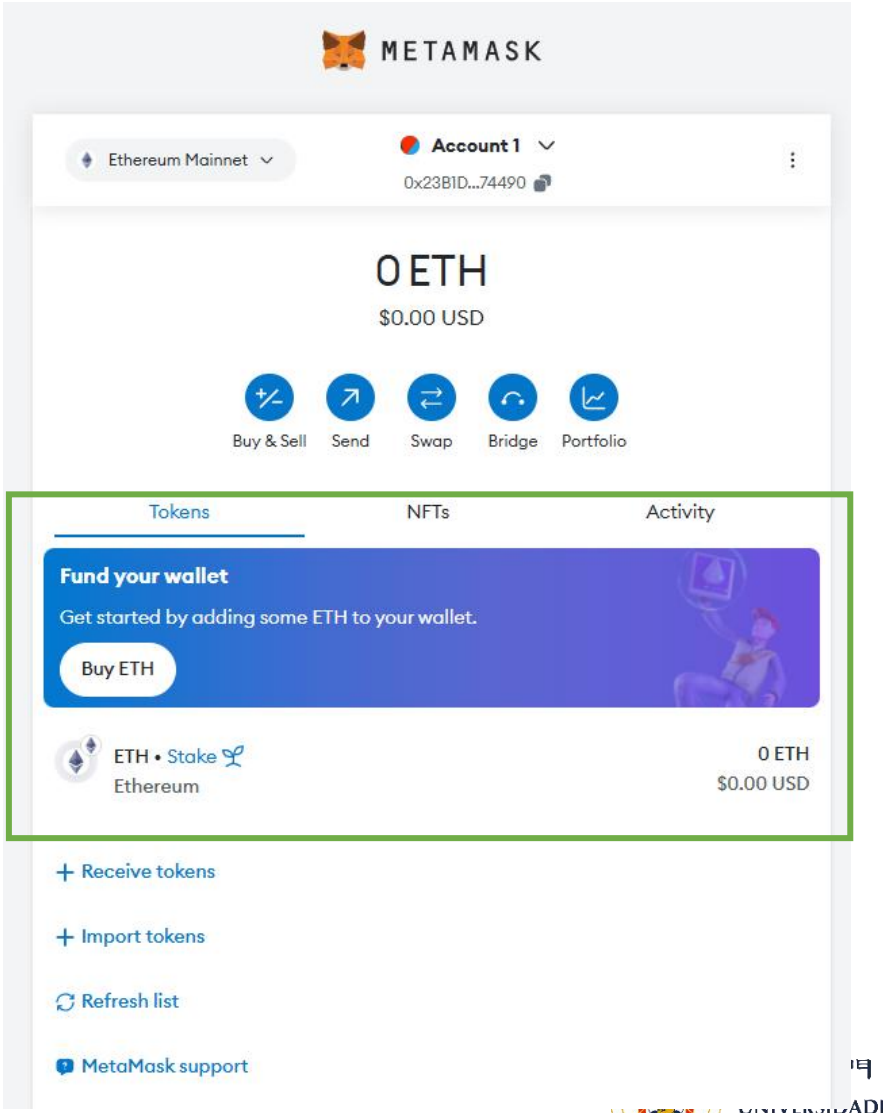
1. Create Your Wallet

METAMASK

Overview of the MetaMask Wallet Interface

Token Overview

- Displays the balance of your current tokens (e.g., ETH: 0 ETH, $0.00 USD).
- Toggle between Tokens, NFTs, and Activity tabs.



CYBER AND CRYPTO LAB

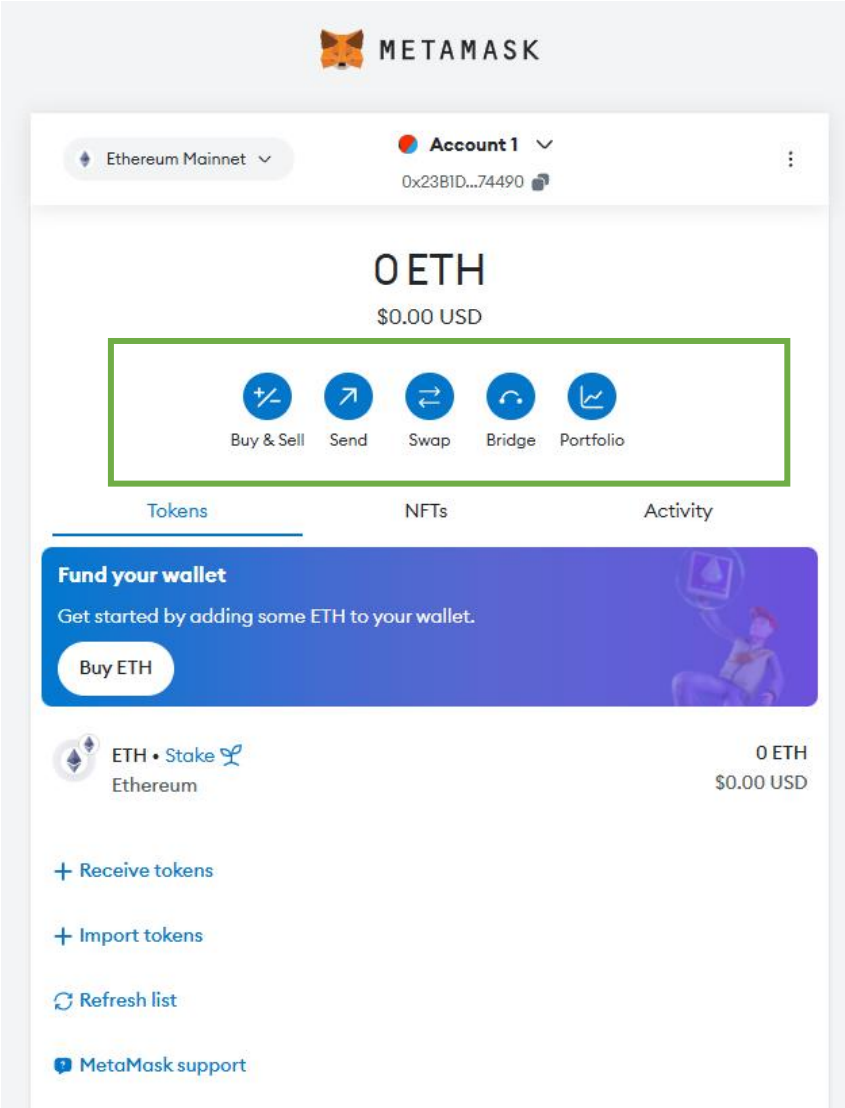
1. Create Your Wallet

METAMASK

Overview of the MetaMask Wallet Interface

Main Functions

- Buy & Sell: Purchase or sell cryptocurrencies.
- Send: Transfer assets to another wallet.
- Swap: Trade one cryptocurrency for another.
- Bridge: Move assets between blockchains.
- Portfolio: View your holdings in detail.

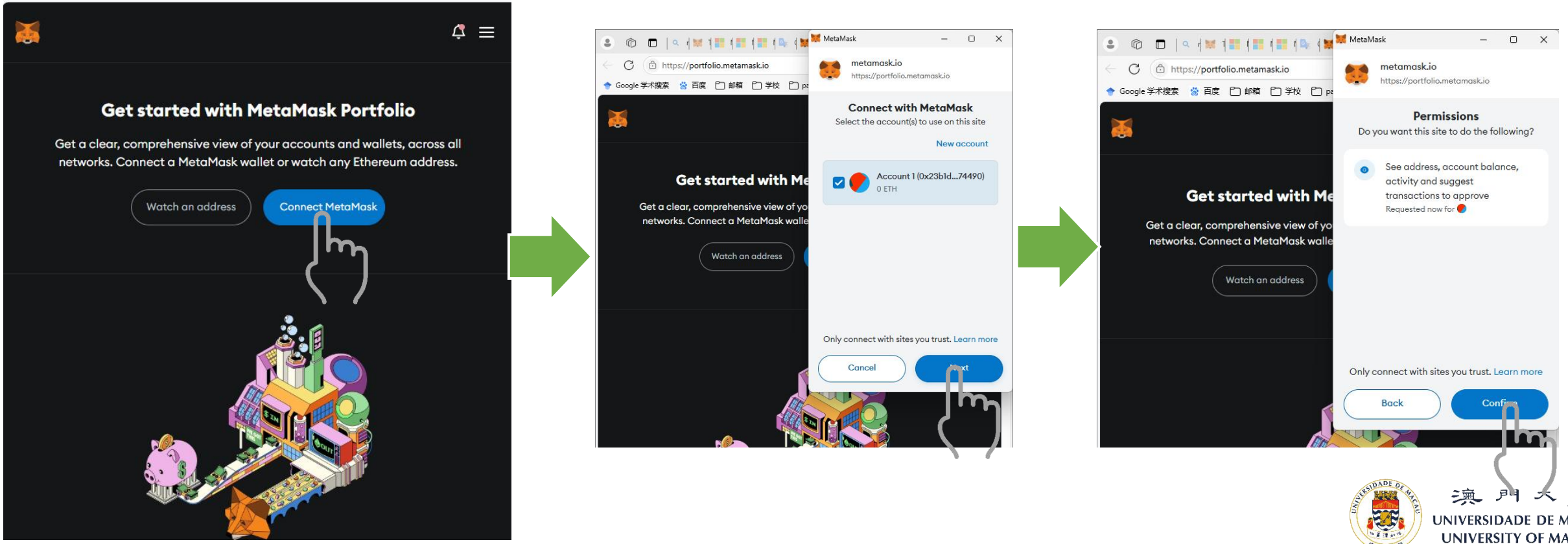


UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

1. Create Your Wallet

Overview of the MetaMask Portfolio https://portfolio.metamask.io/

Get a clear, comprehensive view of your accounts and wallets, across all networks. Connect a MetaMask wallet or watch any Ethereum address.



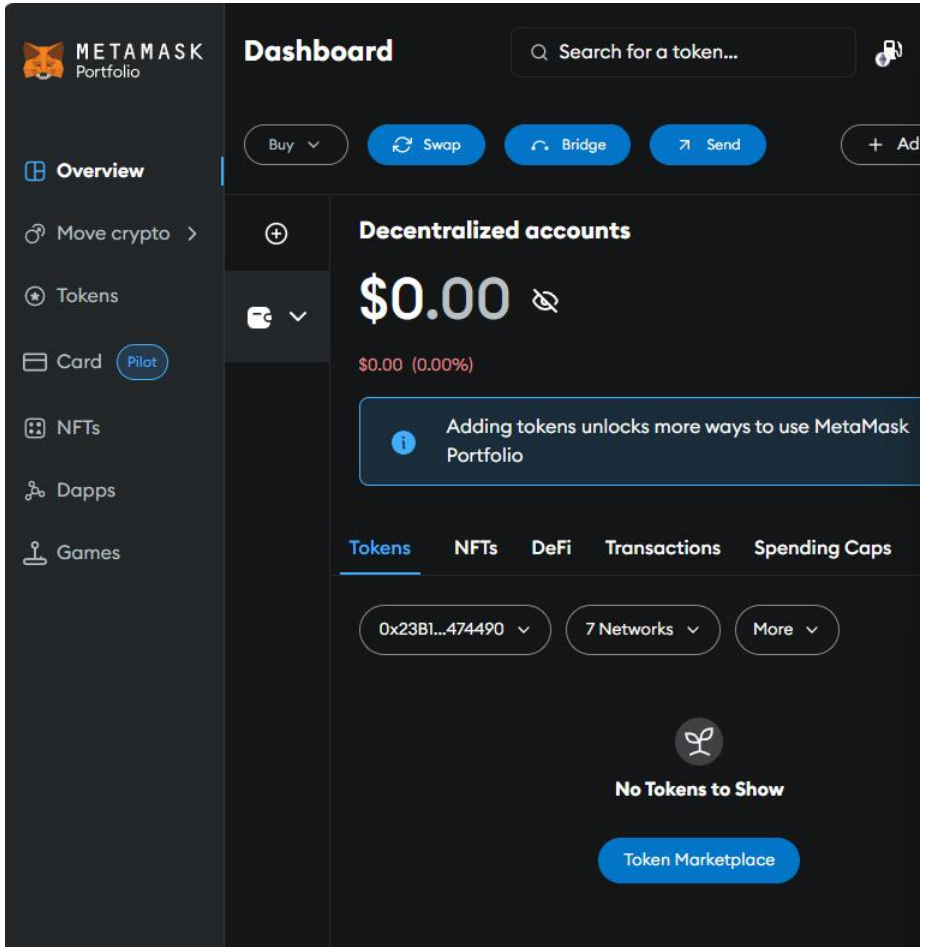
CYBER AND CRYPTO LAB

1. Create Your Wallet METAMASK

Overview of the MetaMask Portfolio

https://portfolio.metamask.io/

Get a clear, comprehensive view of your accounts and wallets, across all networks. Connect a MetaMask wallet or watch any Ethereum address.



2. Buy Cryptocurrencies in a CEX

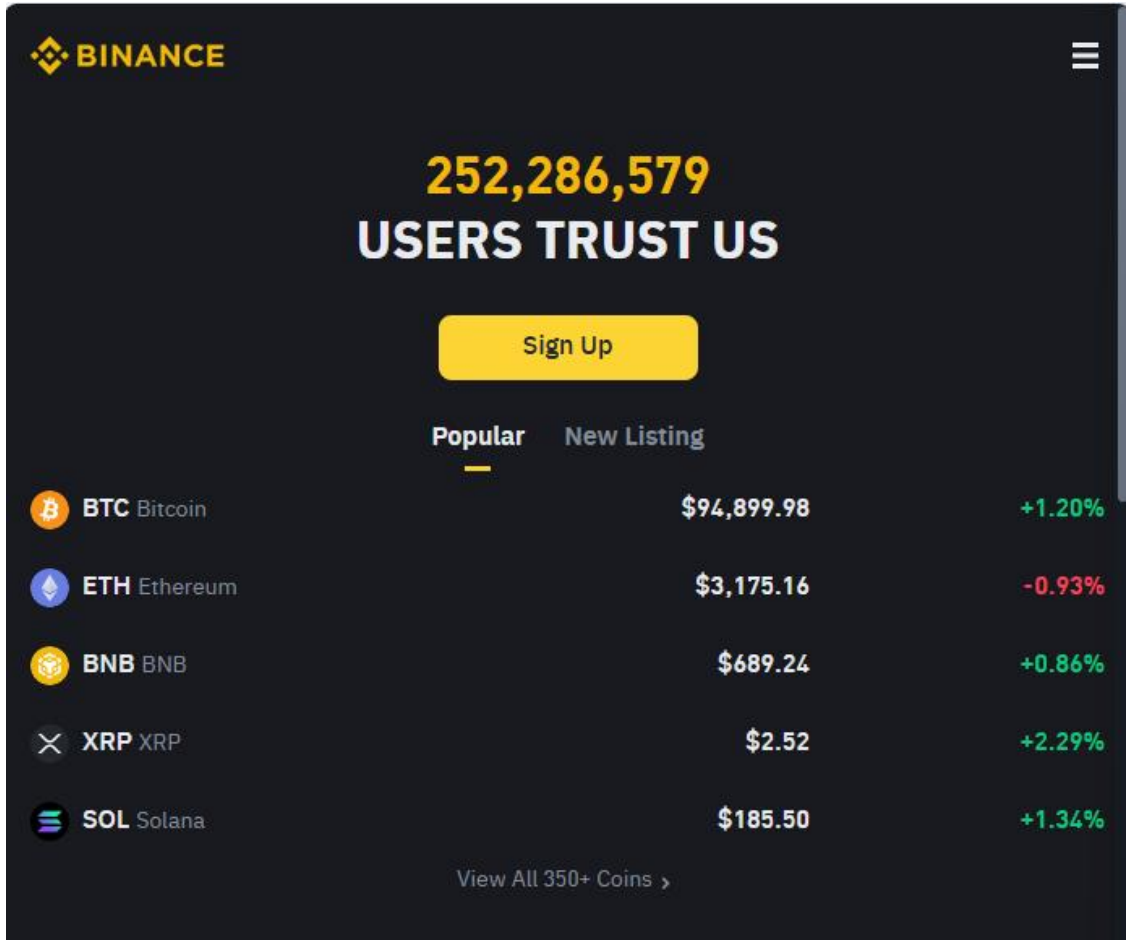
How to Access Binance

- **Search on Search Engines**

1. Type " **Binance** " in the search bar.
2. Be cautious to select the official website:

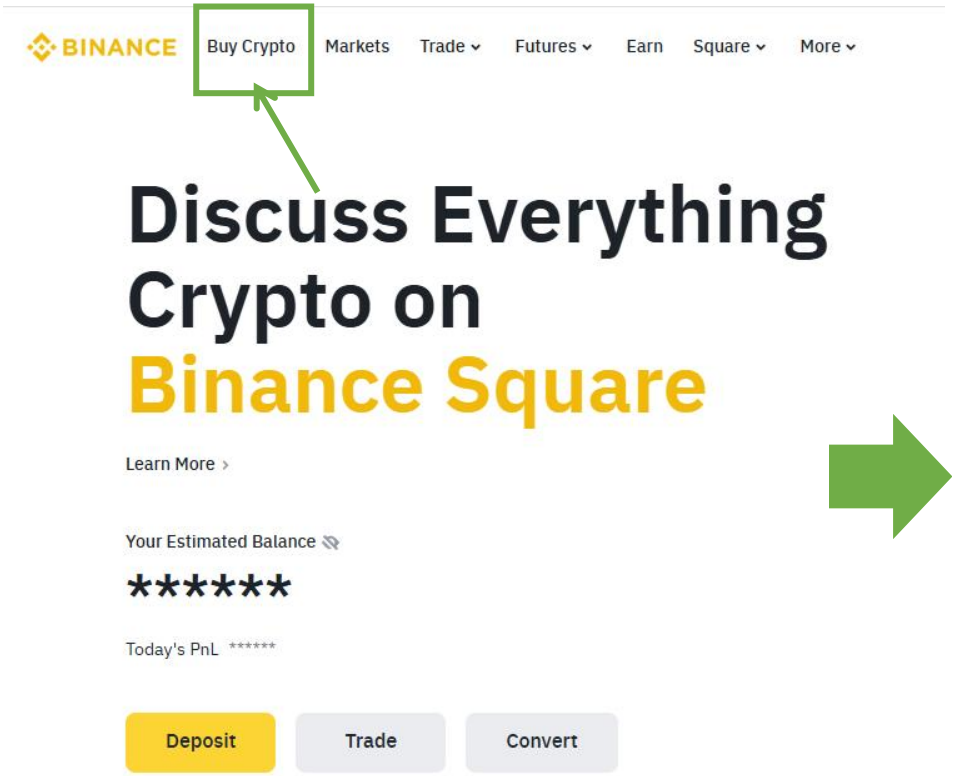
   https://www.binance.com/en

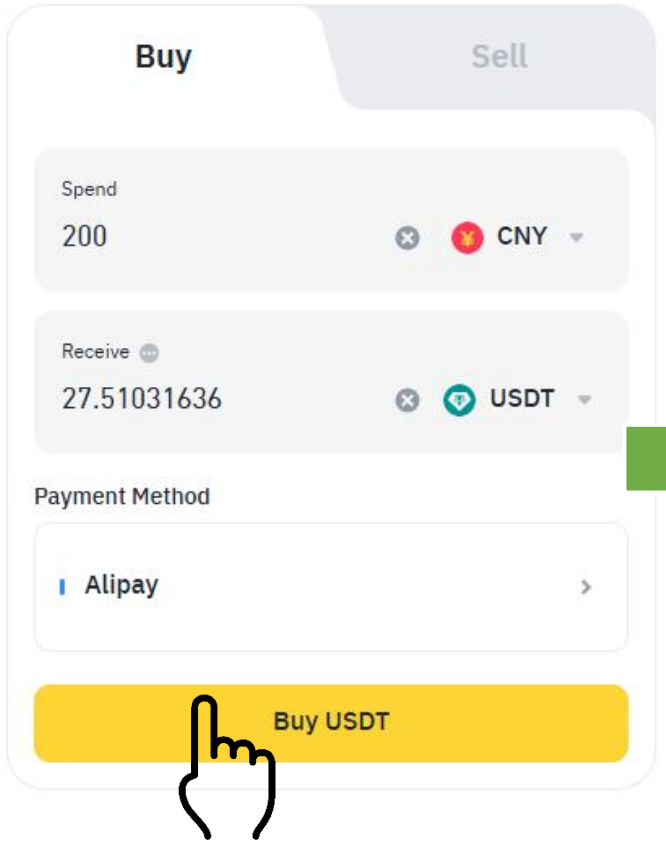
3. Complete registration and KYC according to binance's instructions

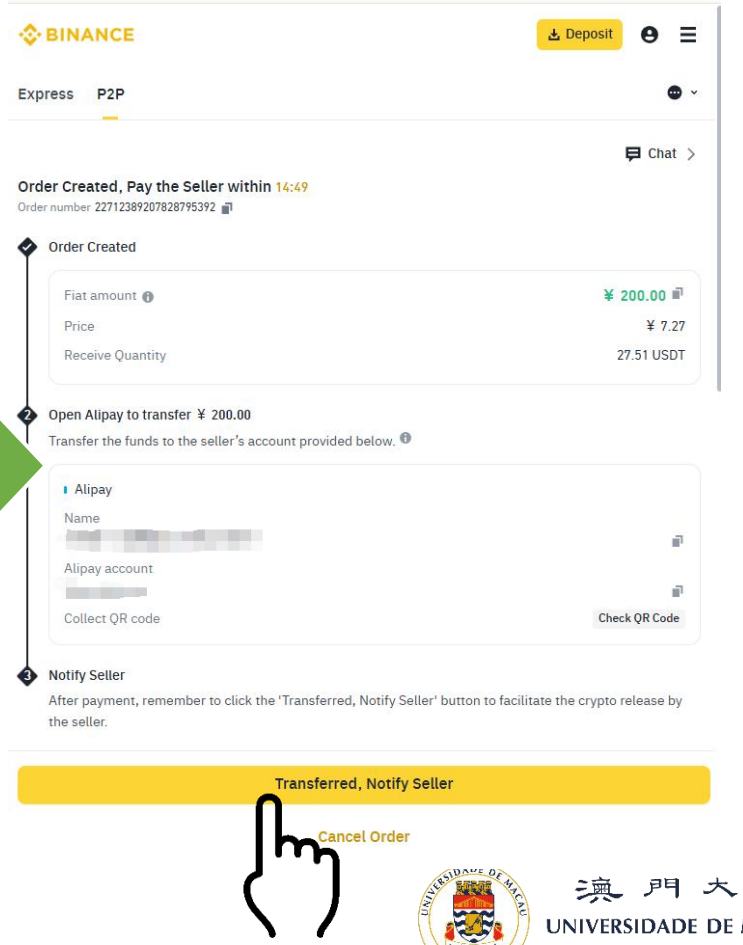


2. Buy Cryptocurrencies in a CEX

Buy Cryptocurrencies in Binance



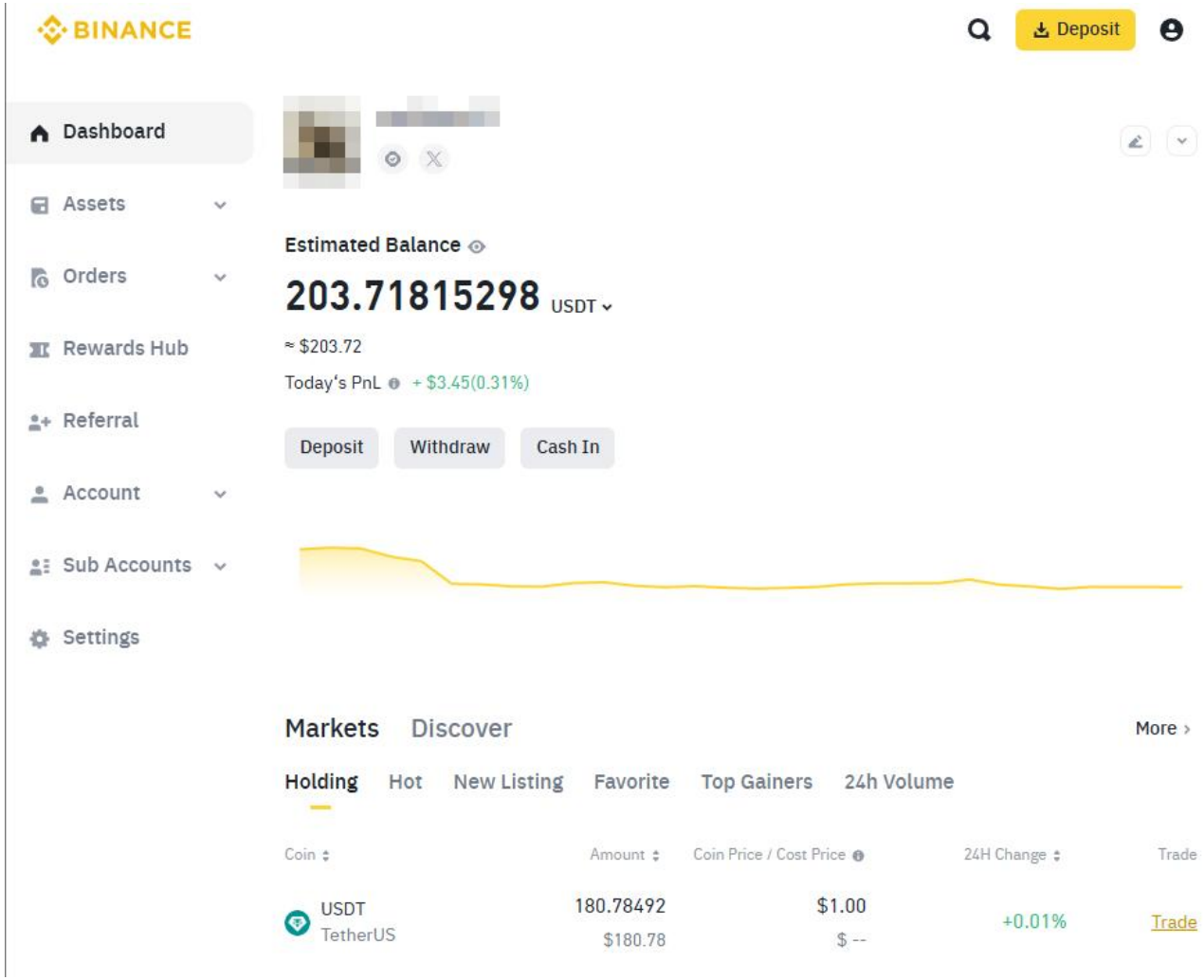




UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

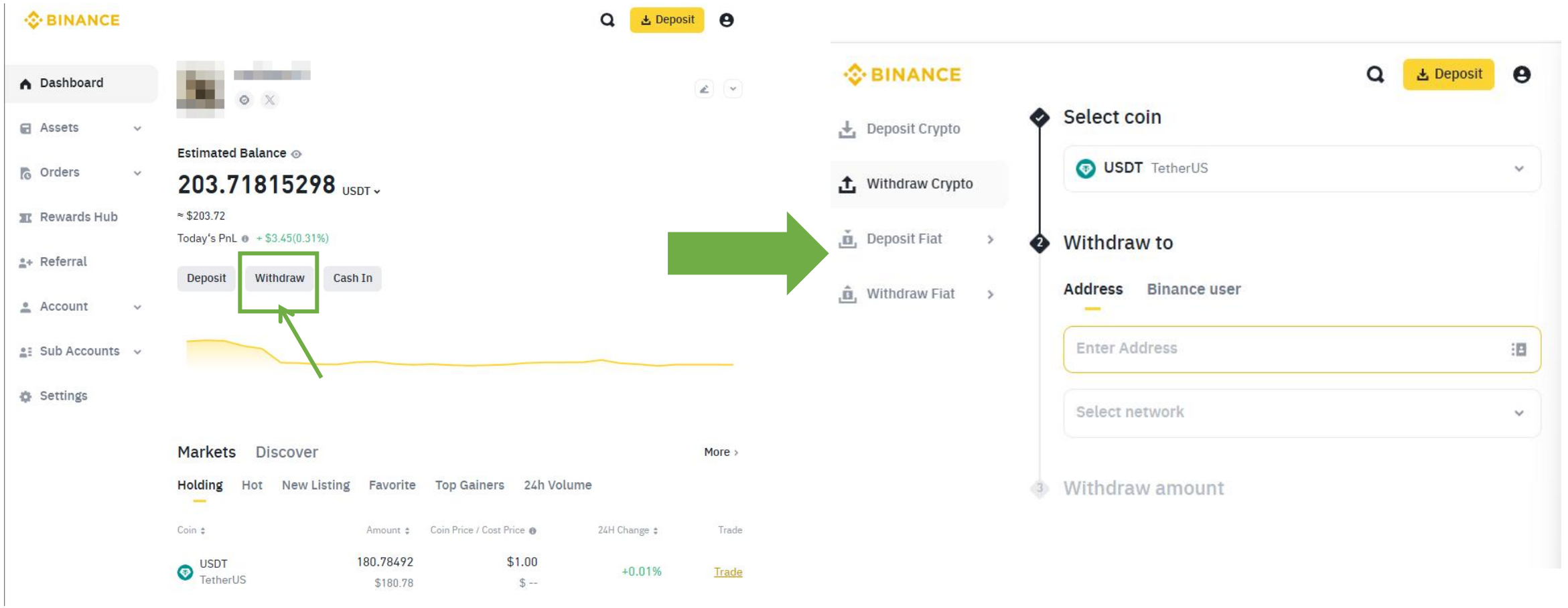
2. Buy Cryptocurrencies in a CEX

Check Your Asset in Binance



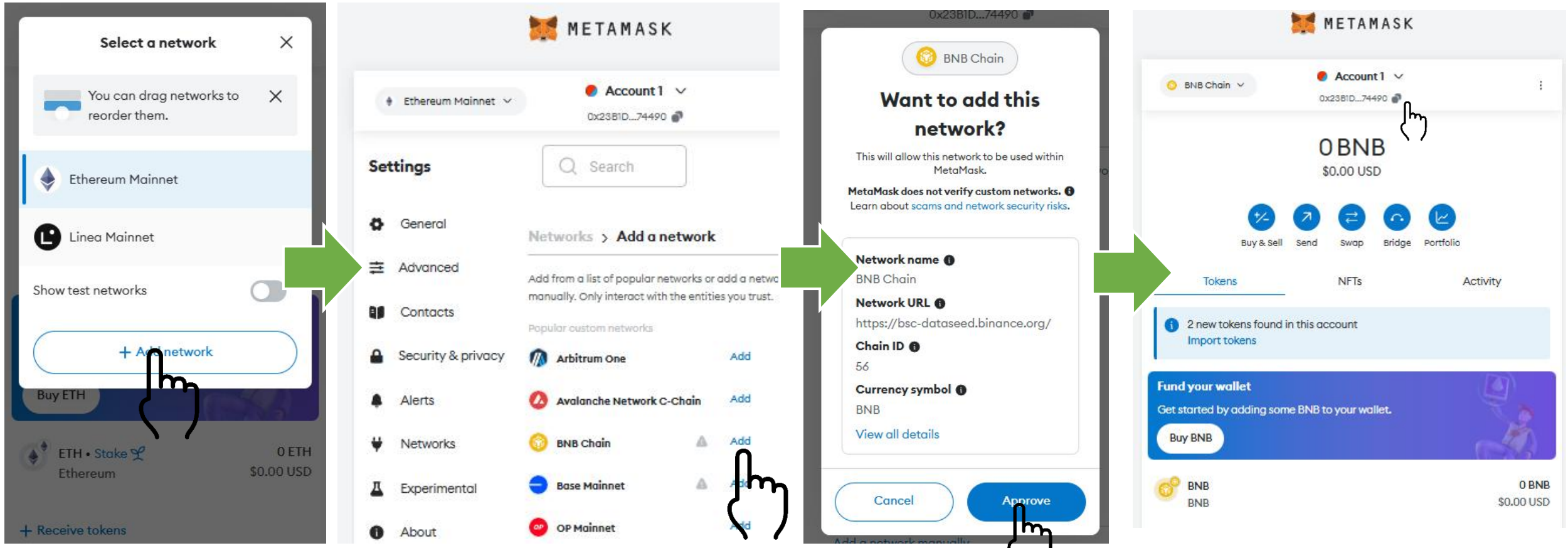
3. Withdraw Cryptocurrencies from CEX

Withdraw from CEX



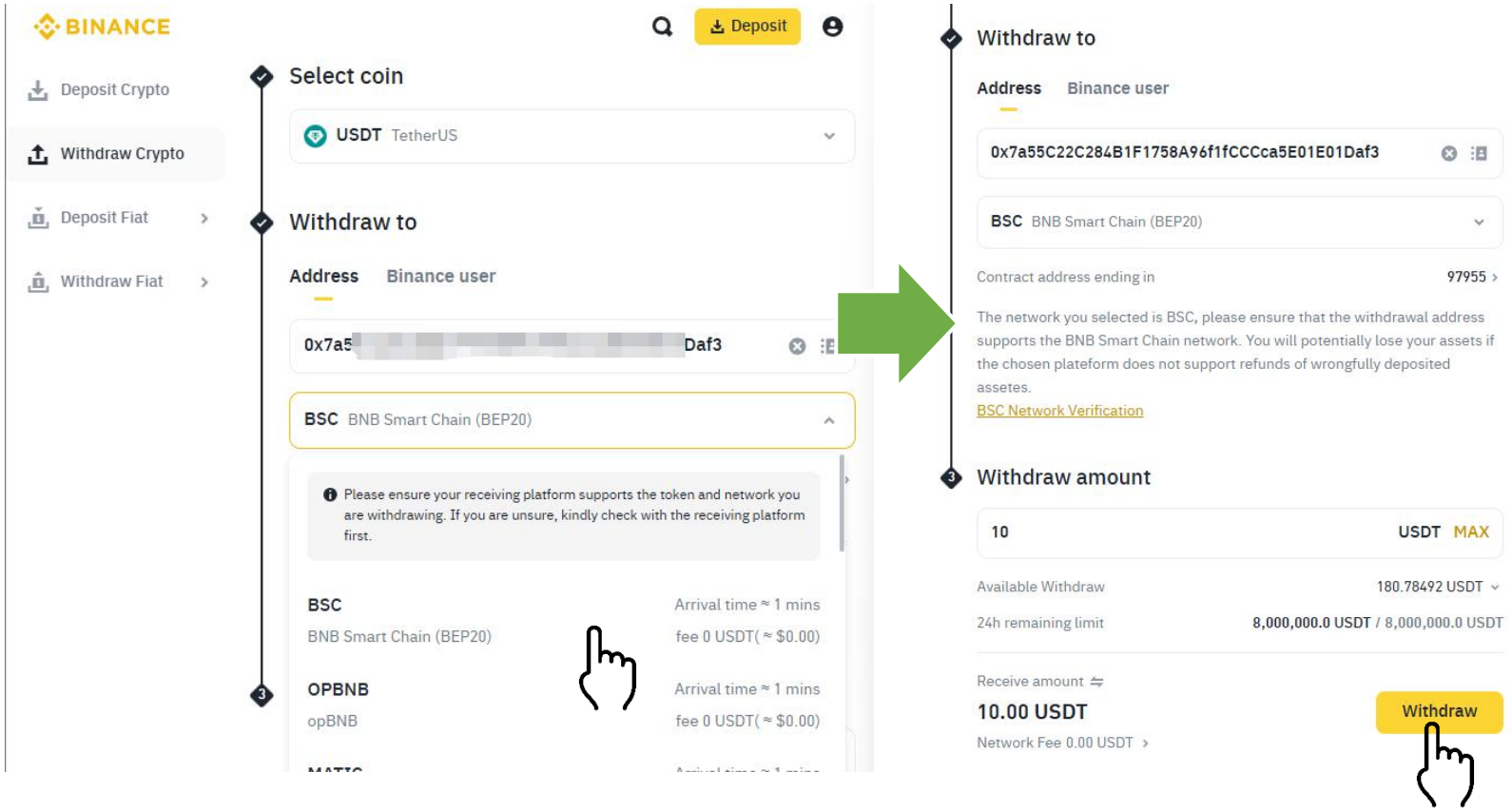
3. Withdraw Cryptocurrencies from CEX

Find Suitable Address in Metamask



3. Withdraw Cryptocurrencies from CEX

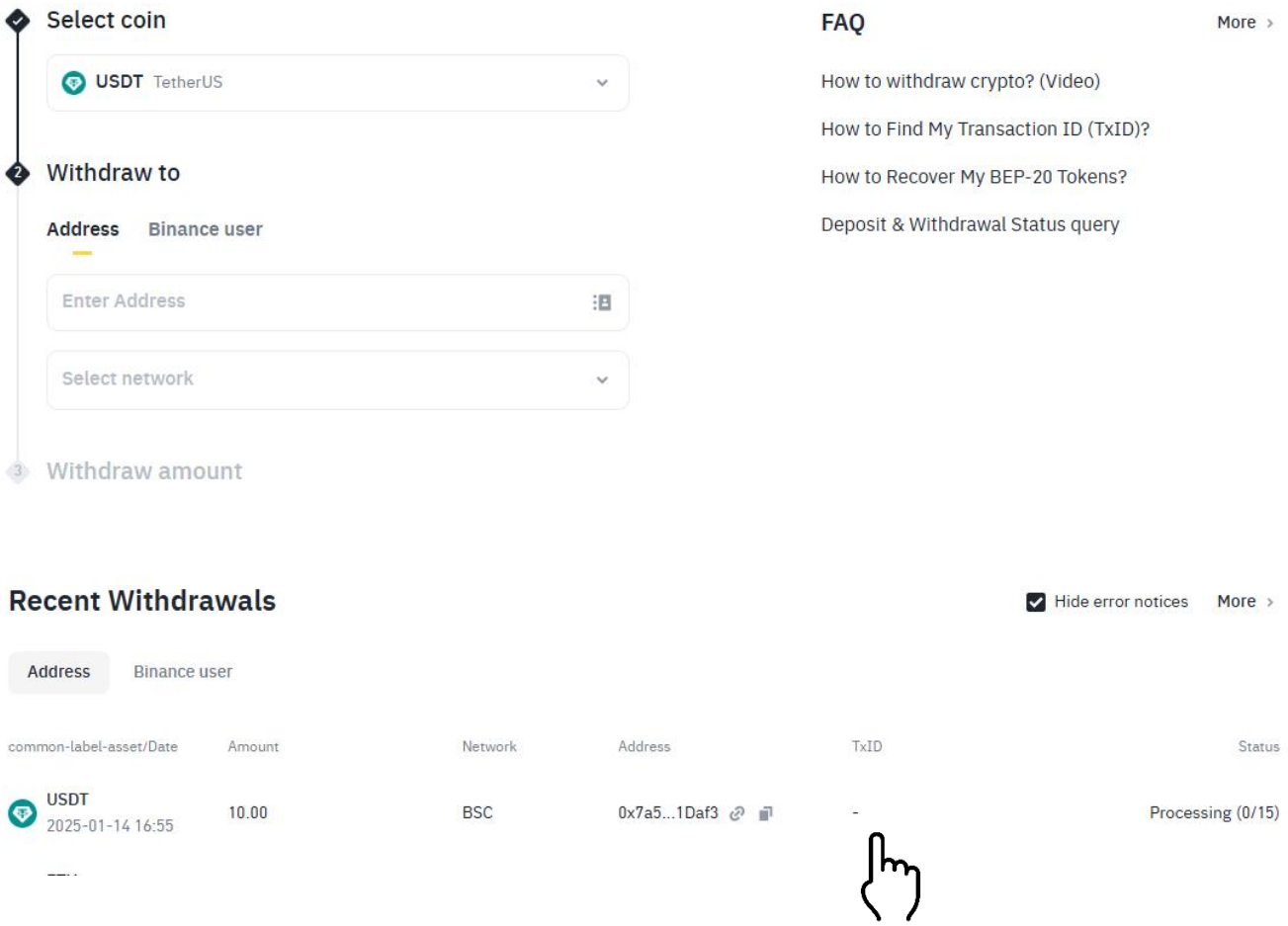
Fill in Address in Binance and Choose the Right Chain

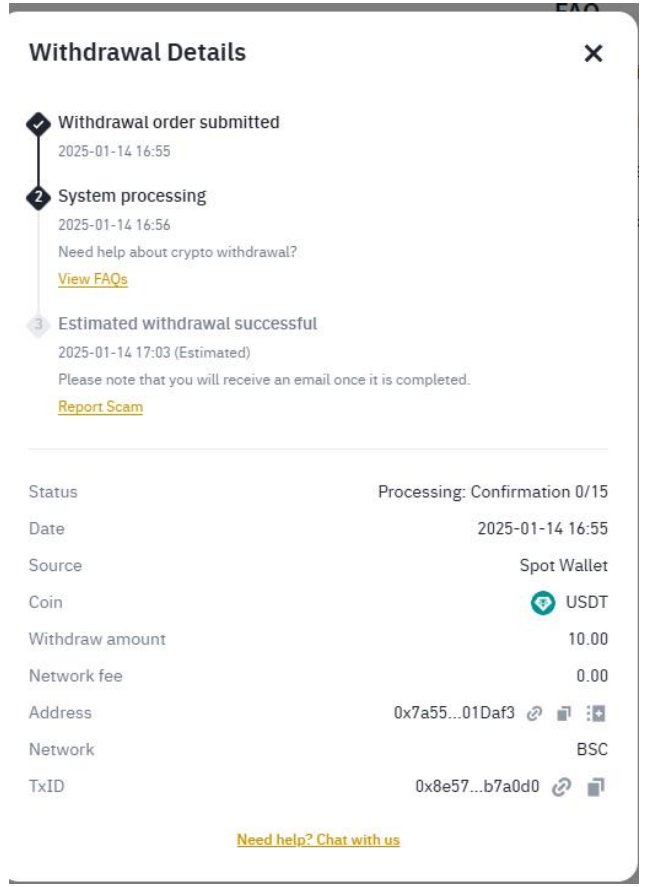


澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

3. Withdraw Cryptocurrencies from CEX

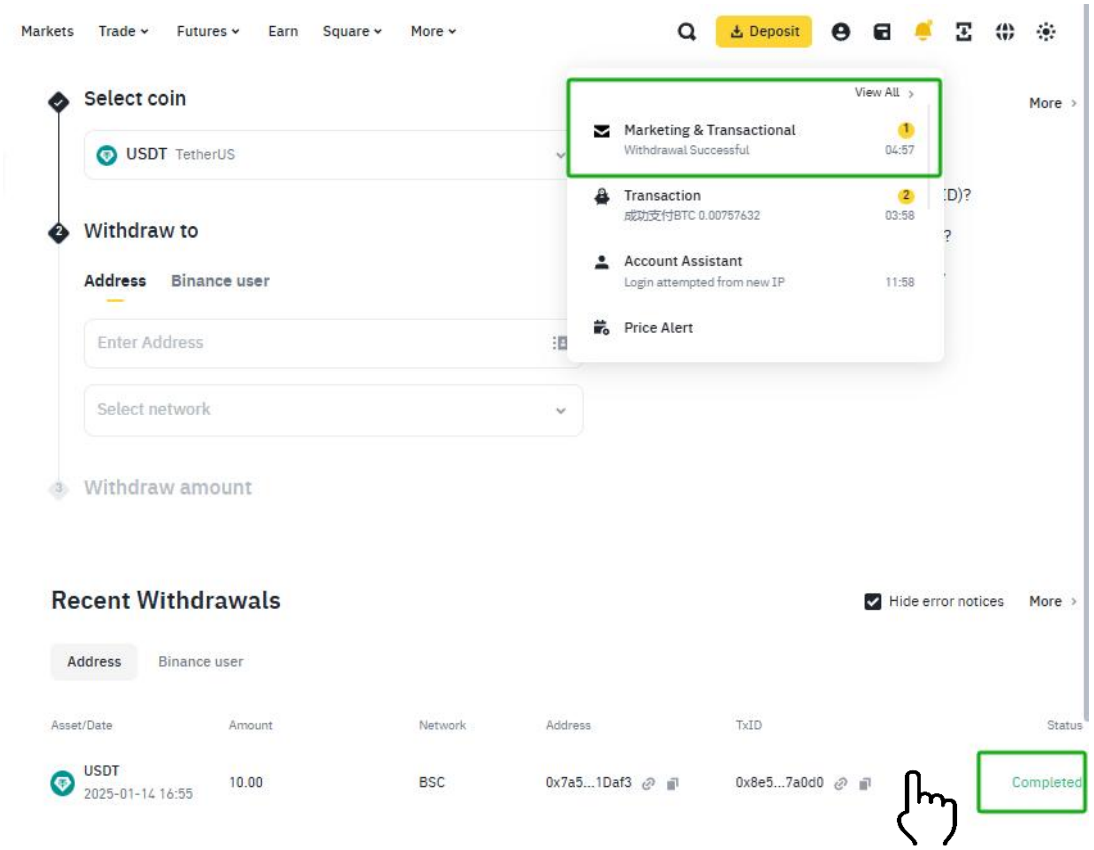
Waiting for Withdraw

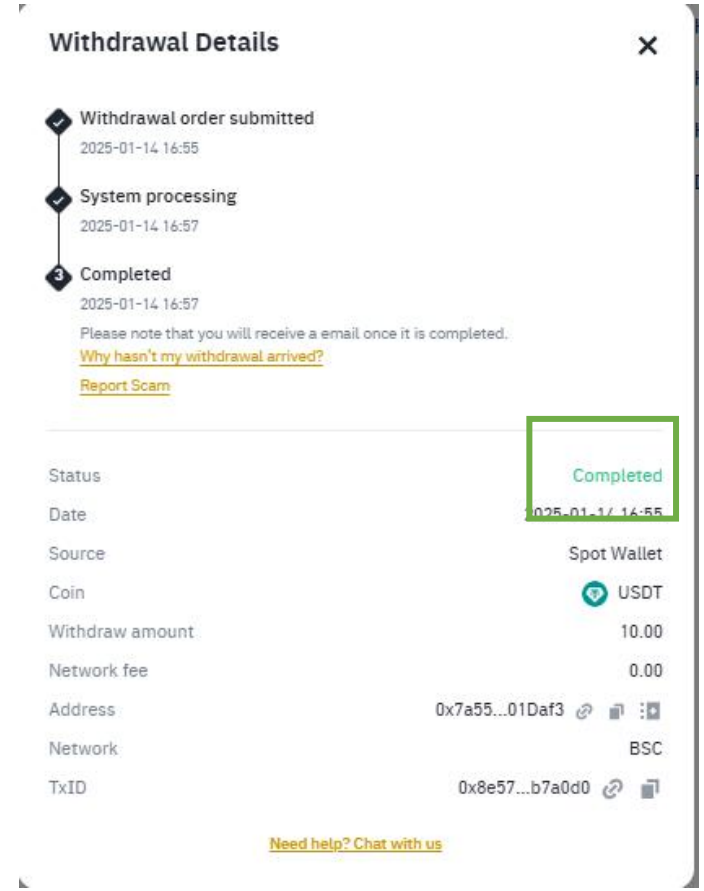




3. Withdraw Cryptocurrencies from CEX

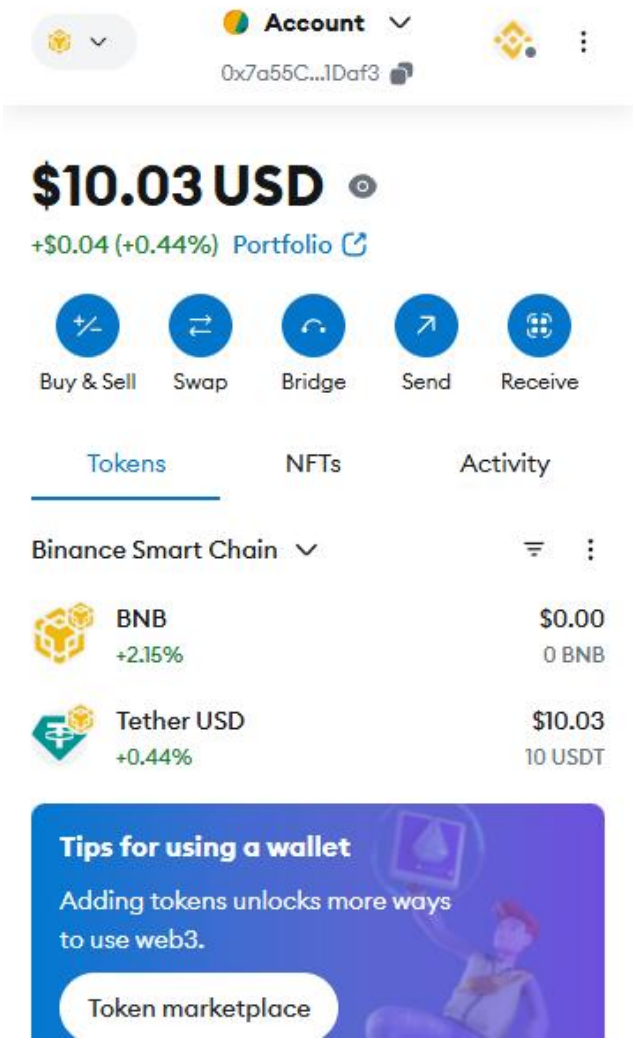
Check Withdraw Completed





3. Withdraw Cryptocurrencies from CEX

Check Your Wallet Balance



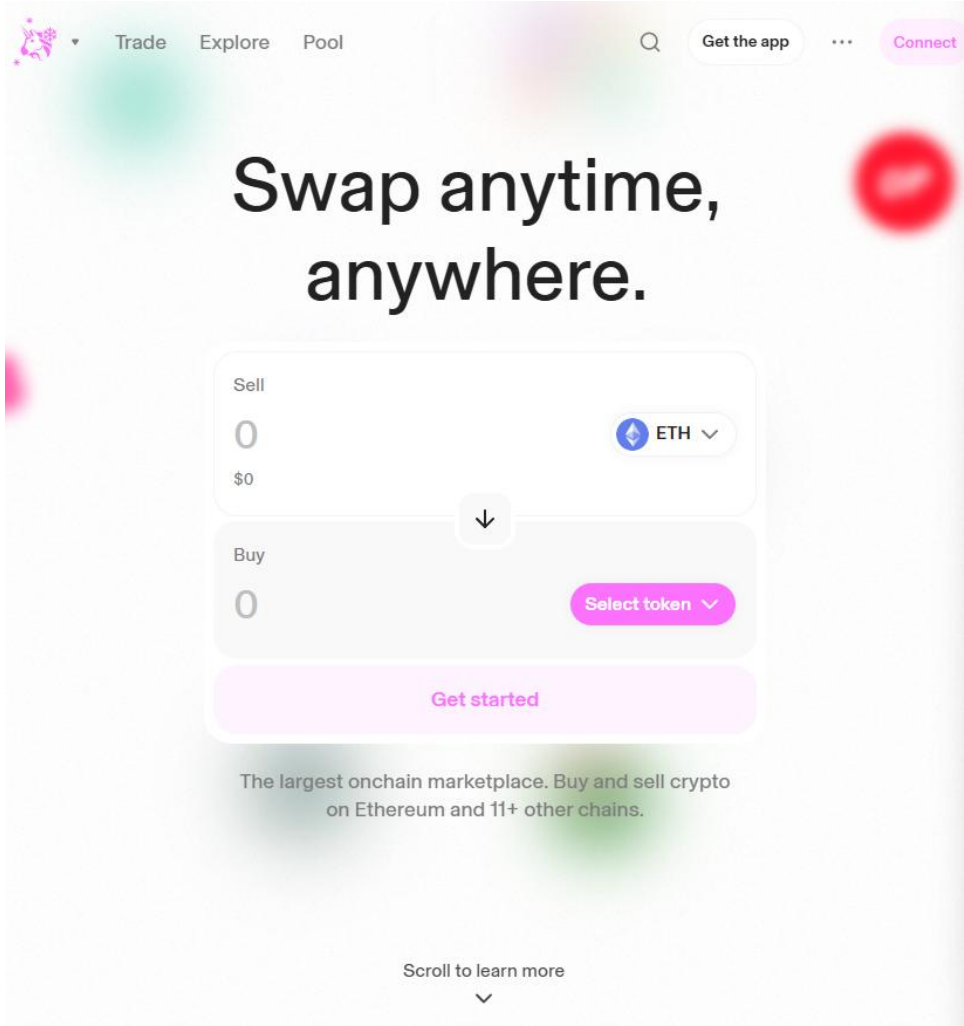
4. Swap Assets in a DEX

How to Access Uniswap

- **Search on Search Engines**

  1. Type " **uniswap** " in the search bar.
  2. Be cautious to select the official website:

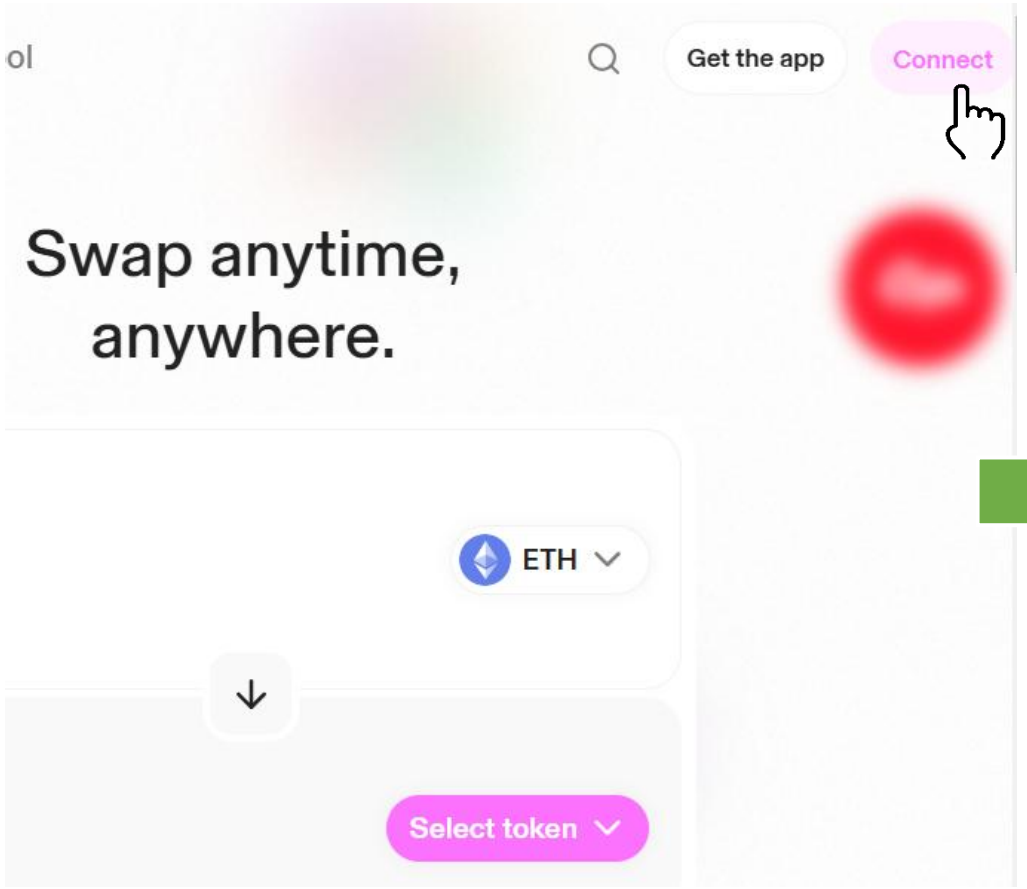
     https://app.uniswap.org/

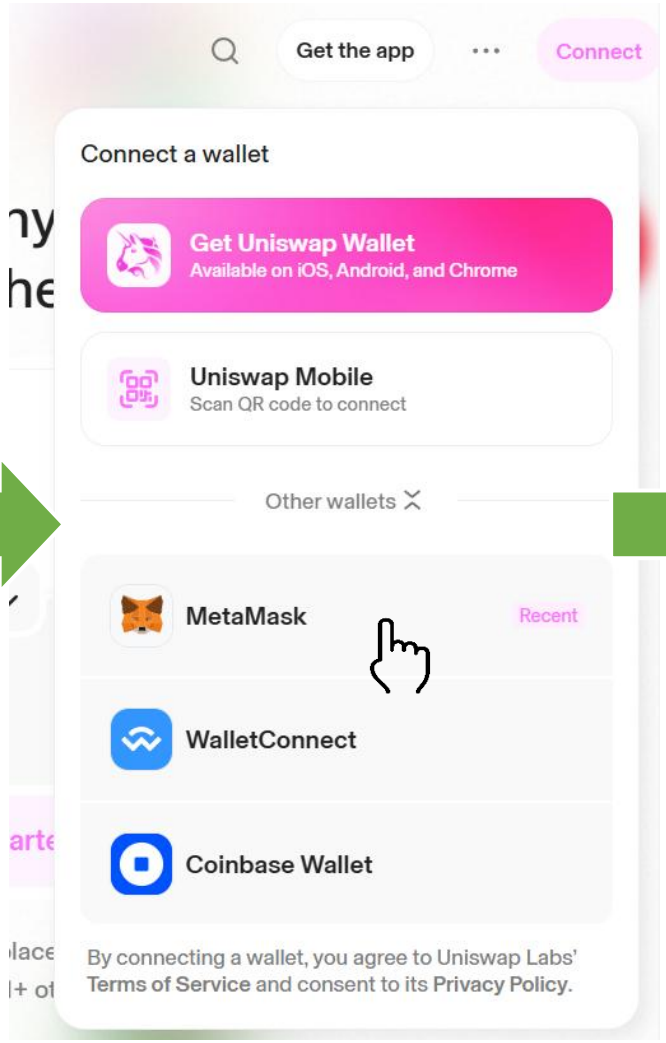


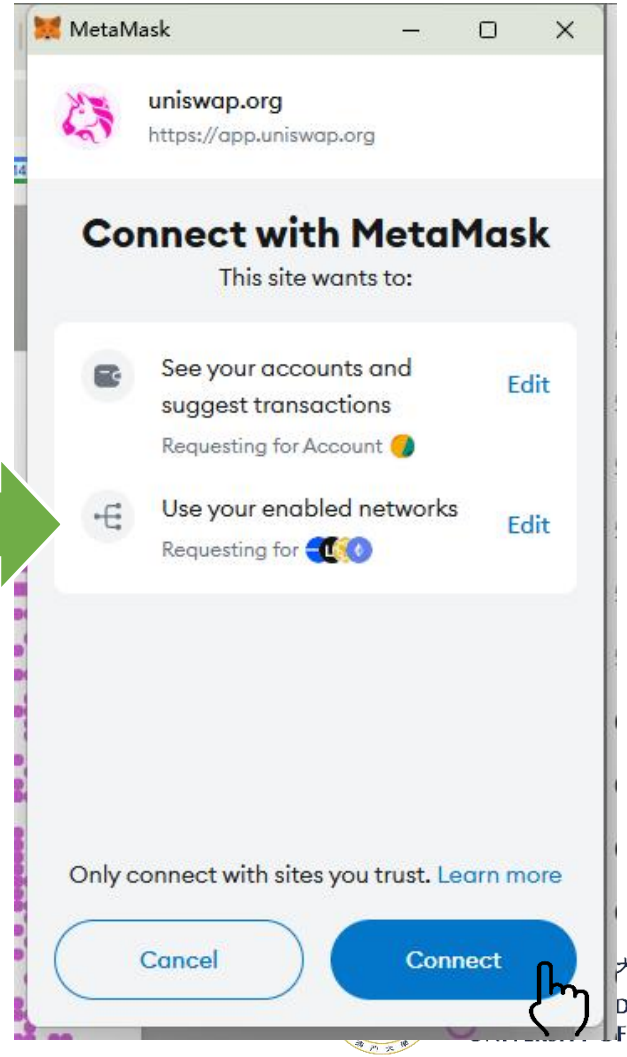
4. Swap Assets in a DEX

UNISWAP

Connect Your Wallet

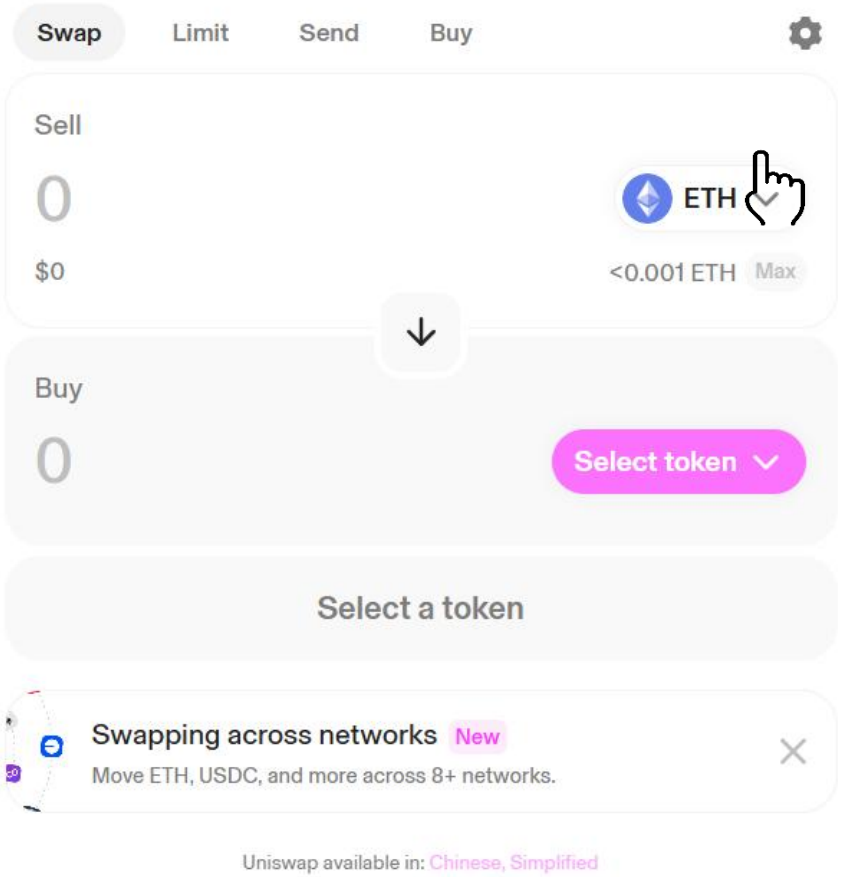


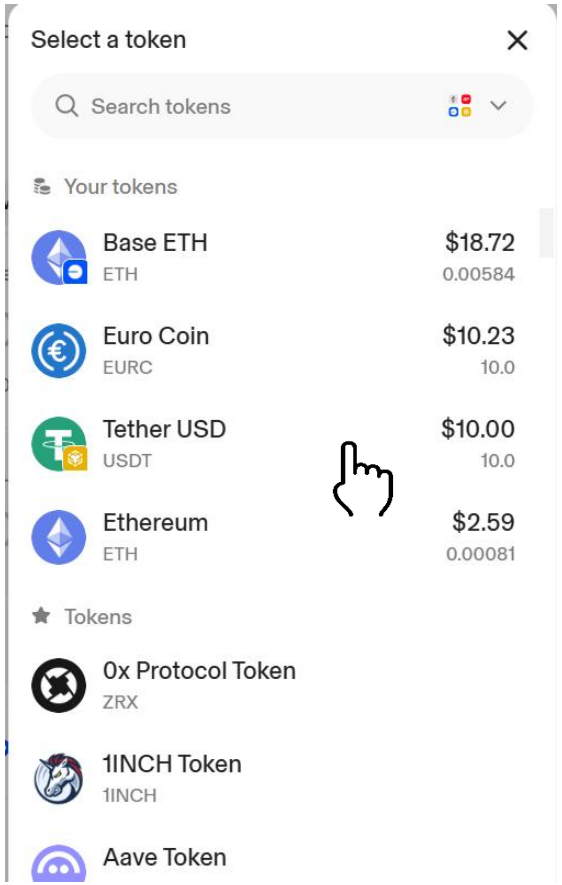




4. Swap Assets in a DEX

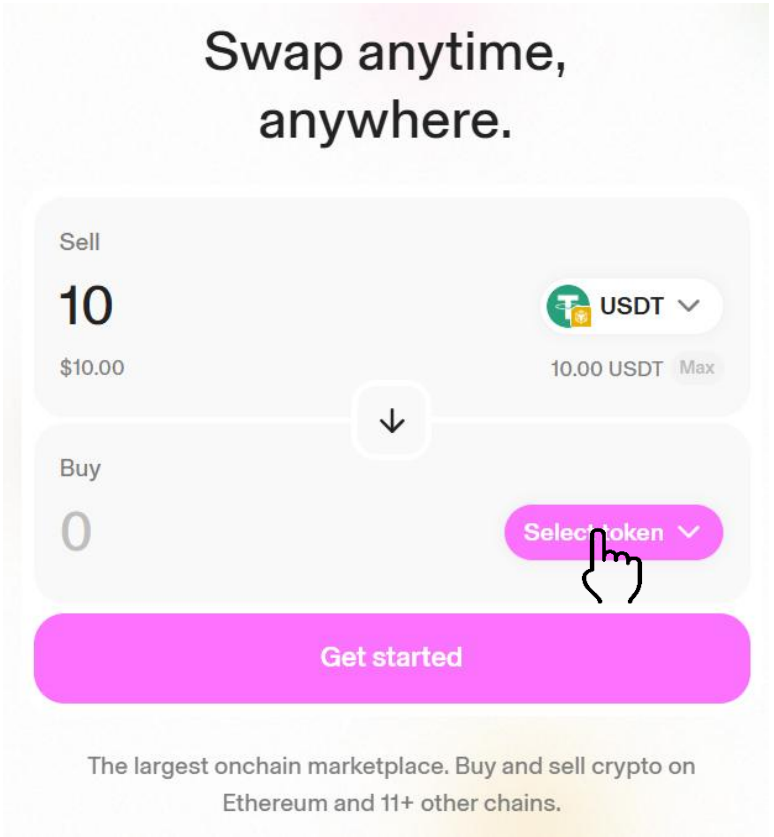
Choose Your Asset

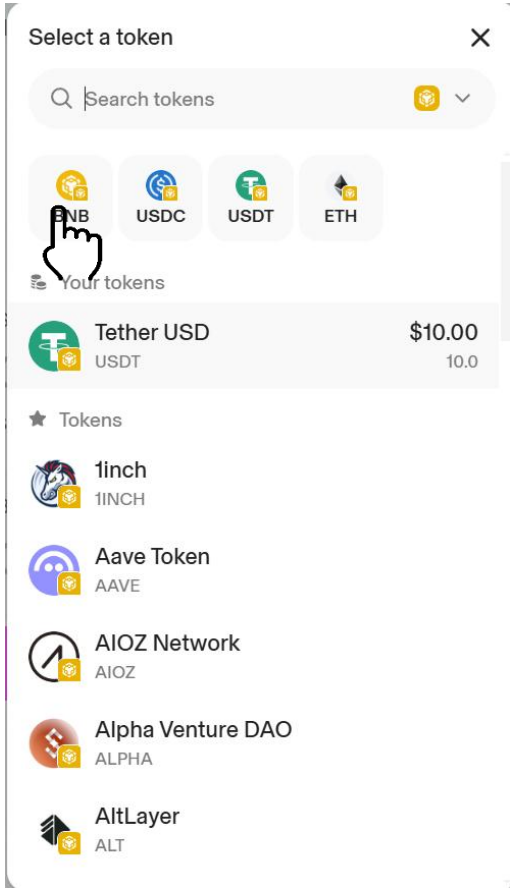


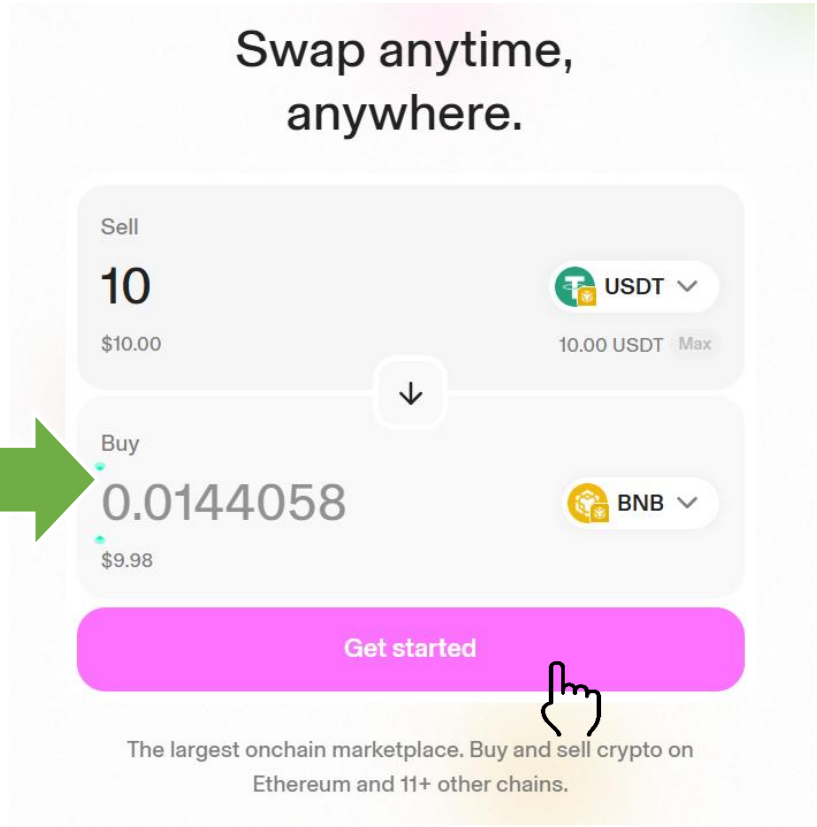


4. Swap Assets in a DEX

Choose the Cryptocurrency You Want to Swap





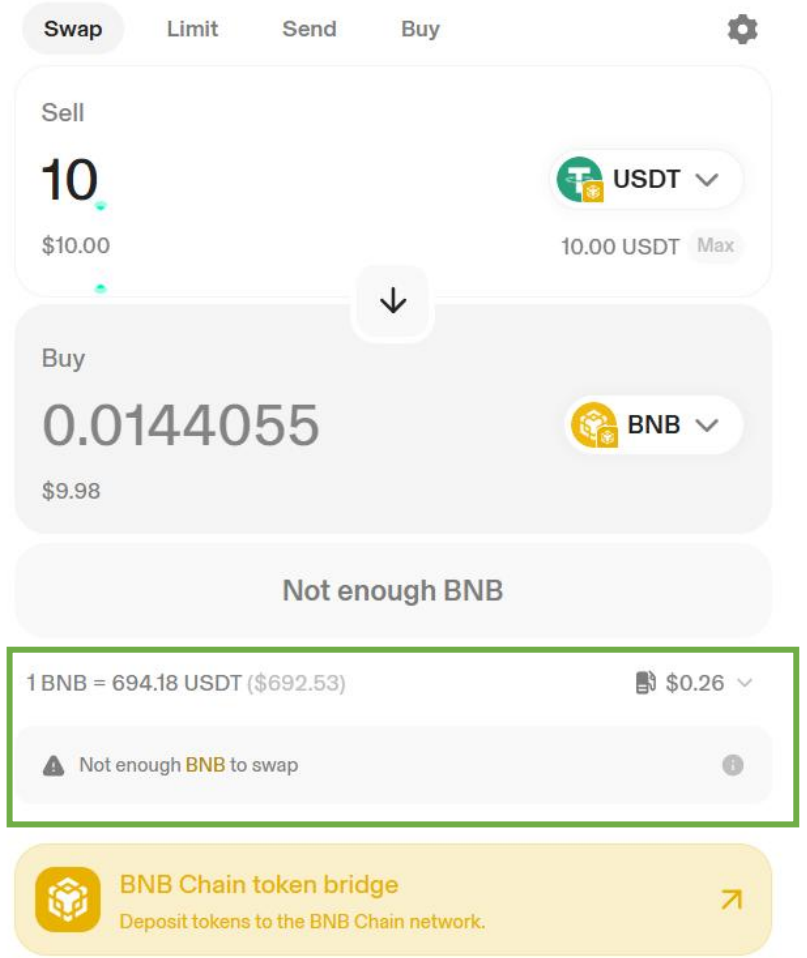


4. Swap Assets in a DEX

UNISWAP

Prepare gas fee:

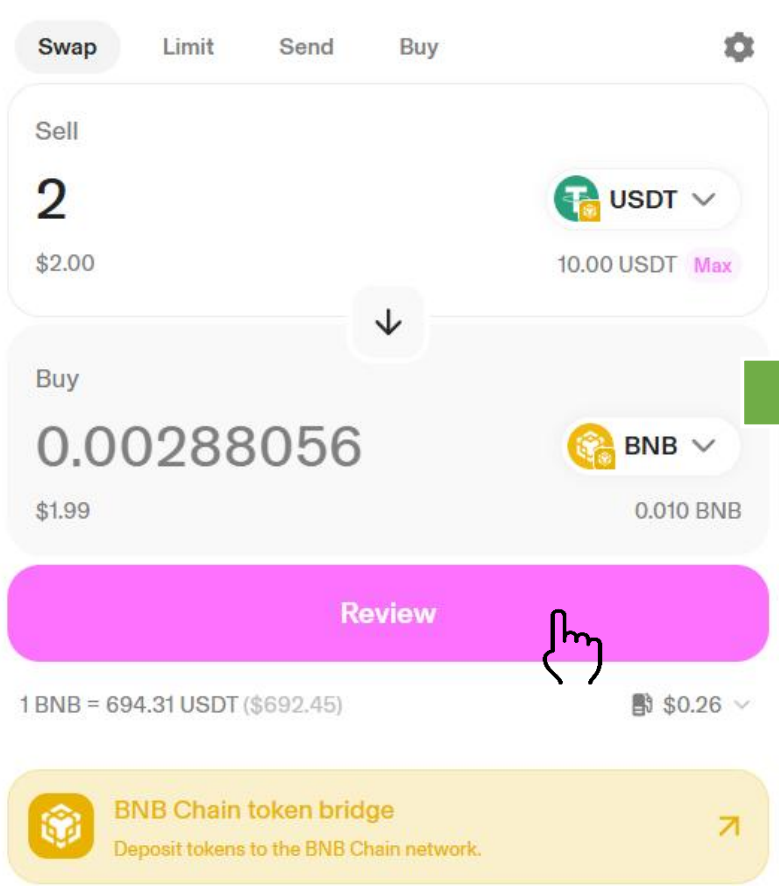
If you **don’ t have enough** native tokens to pay the gas fee, you need to go to **Binance** to buy the relevant tokens and then withdraw them to the on-chain wallet.

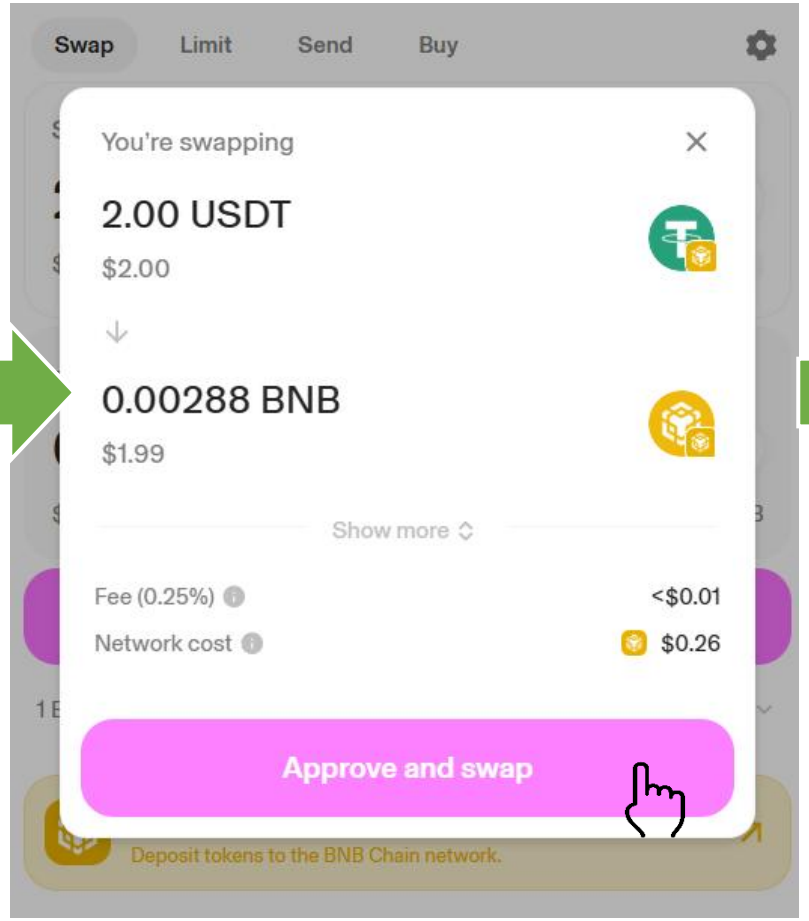


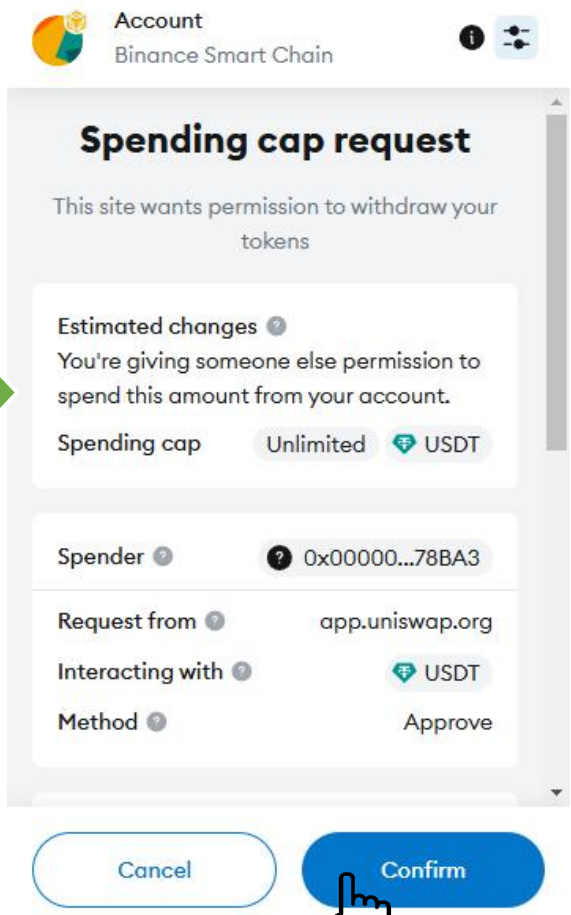
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

4. Swap Assets in a DEX

If You Have Prepared Gas Fee, Than You Can Swap





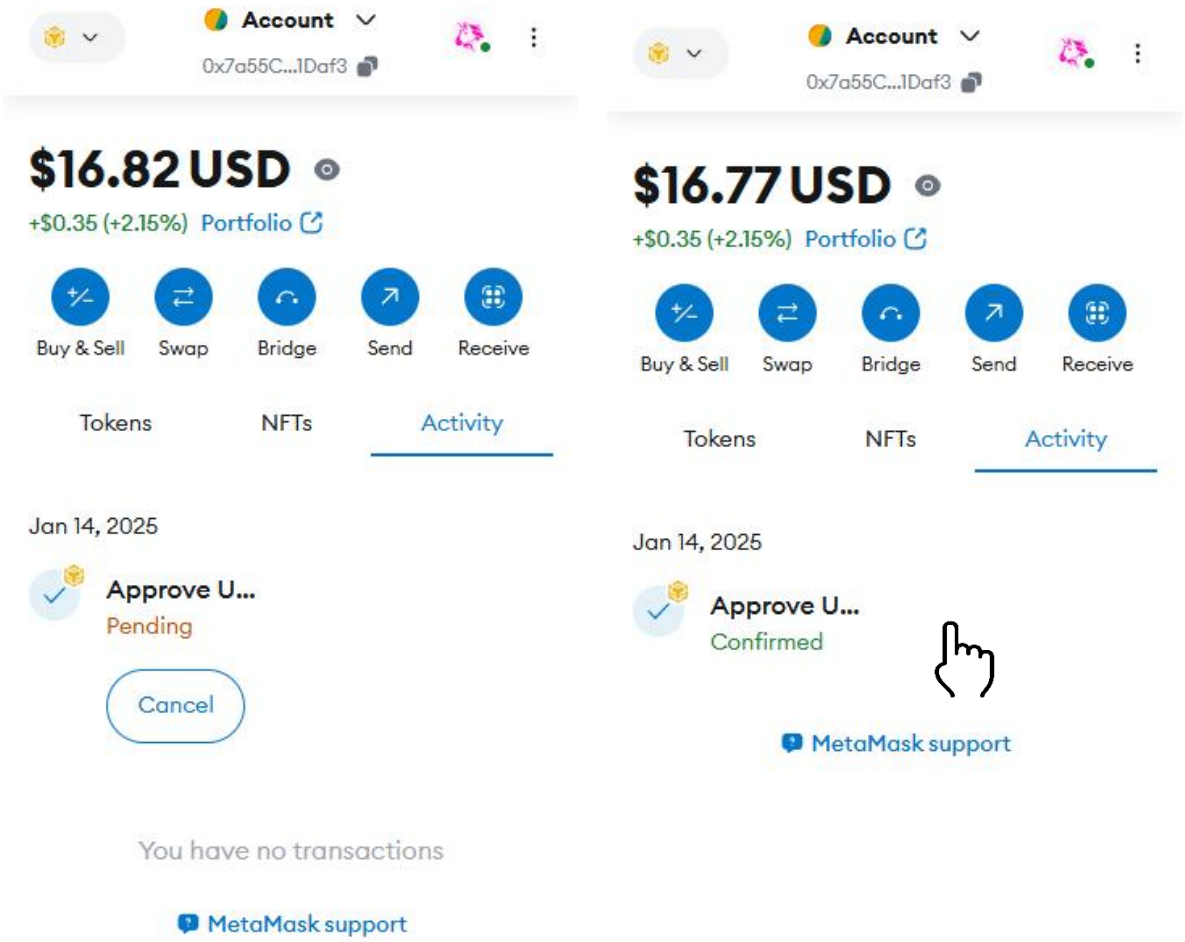


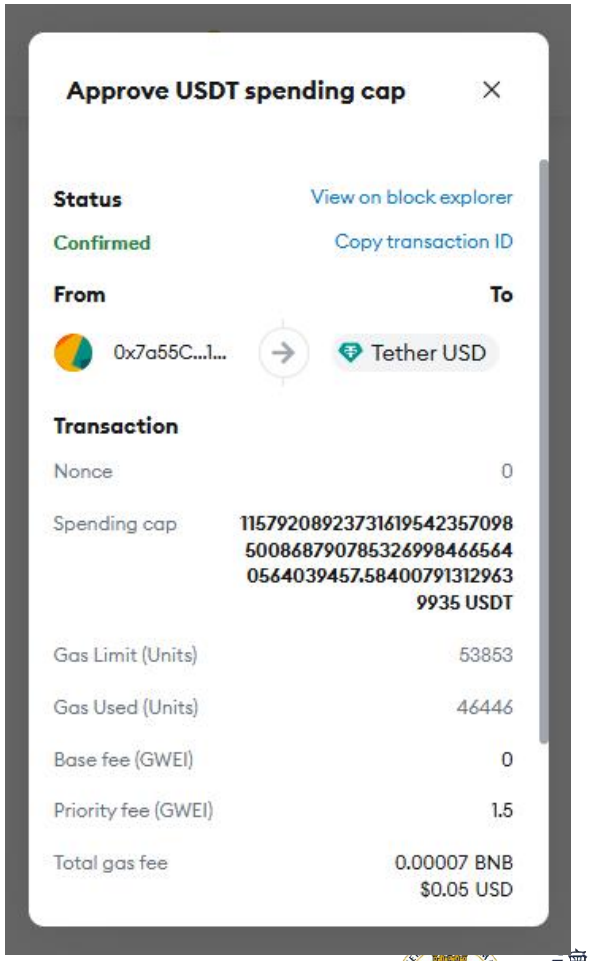
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

4. Swap Assets in a DEX

UNISWAP

Check Your Wallet: Waiting For Transaction Confirmed





澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

4. Swap Assets in a DEX

Check Your Wallet: Balance

